

CARLOS EDUARDO VAZQUEZ
GUILHERME SIQUEIRA SIMÕES

ENGENHARIA DE REQUISITOS



SOFTWARE ORIENTADO AO NEGÓCIO



ENGENHARIA DE REQUISITOS

CARLOS EDUARDO VAZQUEZ
GUILHERME SIQUEIRA SIMÕES

ENGENHARIA DE REQUISITOS

SOFTWARE ORIENTADO AO NEGÓCIO



Copyright© 2016 por Brasport Livros e Multimídia Ltda.

Todos os direitos reservados. Nenhuma parte deste livro poderá ser reproduzida, sob qualquer meio, especialmente em fotocópia (xerox), sem a permissão, por escrito, da Editora.

Editor: Sergio Martins de Oliveira

Diretora: Rosa Maria Oliveira de Queiroz

Gerente de Produção Editorial: Marina dos Anjos Martins de Oliveira

Revisão e copidesque: Camila Britto da Silva

Editoração Eletrônica: Abreu's System

Capa: Caio Cesar Vasconcelos

Arte final: Trama Criações

Produção de e-book: [S2 Books](#)

Técnica e muita atenção foram empregadas na produção deste livro. Porém, erros de digitação e/ou impressão podem ocorrer. Qualquer dúvida, inclusive de conceito, solicitamos enviar mensagem para editorial@brasport.com.br, para que nossa equipe, juntamente com o autor, possa esclarecer. A Brasport e o(s) autor(es) não assumem qualquer responsabilidade por eventuais danos ou perdas a pessoas ou bens, originados do uso deste livro.

BRASPORT Livros e Multimídia Ltda.

Rua Pardal Mallet, 23 – Tijuca

20270-280 Rio de Janeiro-RJ

Tels. Fax: (21)2568.1415/2568.1507

e-mails: marketing@brasport.com.br

vendas@brasport.com.br

editorial@brasport.com.br

www.brasport.com.br

Filial SP

Av. Paulista, 807 – conj. 915

01311-100 São Paulo-SP

Agradecimentos

Agradecemos a todos que nos ajudaram na elaboração deste livro. Em especial, gostaríamos de agradecer às seguintes pessoas: Caroline Messias Domiciano, Ewerton Daniel de Lima, Fabio Sibille Cabral, Franco De Biase Carreira, Keila Barbara Costa, Keila Eller Malta, Larissa Angélica Siqueira Nunes, Leonardo Kelly do Nascimento, Ludimila Nunes Gomes Nascimento, Nelson Camilo Orduz Illidge, Priscila dos Santos Araujo e Rodrigo Sergio de Santos Souza.

Durante o desenvolvimento deste trabalho, ambos tivemos que dedicar muito tempo – já restrito em função de nossas constantes viagens – e, por isso, sacrificamos grande parte do período que teríamos disponível às nossas famílias. Andrea Meira e Caio, Carla Faleiro, Clara e Isabela, agradecemos profundamente a vocês pela paciência, compreensão e pelas ausências presentes ou não.

Nossa gratidão, também, a Edilson Eloy dos Santos e Tatiane Faro Direne, que nos auxiliaram com suas revisões e cujos comentários contribuíram para a produção de um conteúdo de maior valor; a Everton Schonardie Pasqual, que teve um importante papel ao nos apresentar o “ovo de Colombo” do qual boa parte dos problemas de medição advêm por conta da falta de conhecimento sobre Engenharia de Requisitos; a Paulo de Tarso França, que nos apresentou o desafio de prover um conjunto de políticas de qualidade que permitissem avaliar a produção da Engenharia de Requisitos no contexto da contratação de fábricas de projeto; a Filipe Leyser e Willian Francisco da Silva, pelo desafio de organizar a Engenharia de Requisitos de maneira lógica e iterativa; a Victor Manaia Gonçalves Chaves, que nos levou a evoluir o modelo de gestão da Engenharia de Requisitos; e a Walter Lourenço Ferreira, pela avaliação de pacotes de software, o que contribuiu em diversos pontos desta obra.

Seria impossível nomear todas as pessoas que, direta ou indiretamente, auxiliaram neste processo. Foram muitos aqueles que participaram dos treinamentos realizados pelos autores ao longo dos anos e que culminaram na produção desta obra. Sem os questionamentos e os problemas apresentados, jamais poderíamos ter desenvolvido uma visão tão ampla e real dos problemas e soluções relacionados à Engenharia de Requisitos discutidos neste livro. Se você estiver lendo este agradecimento, saiba que ele também é para você: muito obrigado.

Os autores

Sobre os Autores

Carlos Eduardo Vazquez (carlos.vazquez@fattocs.com) é um profissional de TI com mais de vinte anos de experiência em desenvolvimento, manutenção e gestão em software de aplicação e de sistemas direcionando a tecnologia às necessidades das pessoas. Ele acredita que as tecnologias de medição de software e a medição do tamanho funcional em particular (como os pontos de função definidos pelo IFPUG – *International Function Point Users Group*, NESMA – *Netherlands Software Metrics Association* ou COSMIC – *Common Software Measurement International Consortium*) são ferramentas fundamentais para alcançar esse objetivo.

Desde 1991 é um usuário da análise de pontos de função do IFPUG, tendo treinado turmas no assunto a partir de 1993. Em 1996, foi um dos primeiros brasileiros a ser certificado como especialista em pontos de função (CFPS – *Certified Function Point Specialist*) pelo IFPUG. Repetiu o feito em 2012, sendo pioneiro como detentor da certificação COSMIC.

Em 1998, lecionou como professor substituto na Universidade Federal do Espírito Santo (UFES) e fundou a Fatto Consultoria e Sistemas. Em 2001, escreveu o livro “Análise de Pontos de Função: medição, estimativas e gerenciamento de projetos de software”, atualmente em sua 13ª edição, com mais de 12.000 exemplares vendidos e uma das principais referências no assunto no Brasil. Coordena a pesquisa e o desenvolvimento do conteúdo para os serviços educacionais na Fatto, atuando como instrutor e facilitador em turmas abertas ao público e fechadas para empresas. Também é responsável pela consultoria gerencial em TI, liderando um time de especialistas em métricas de software. É também engenheiro de requisitos certificado pelo *International Requirements Engineering Board* (IREB).

Guilherme Siqueira Simões (guilherme.simoess@gmail.com) é graduado em Ciência da Computação pela Universidade Federal do Espírito Santo

(UFES), pós-graduado em Gestão Empresarial também pela UFES, especialista em pontos de função (CFPS) certificado pelo IFPUG e pelo COSMIC, gerente de projetos (PMP – *Project Management Professional*) certificado pelo *Project Management Institute* (PMI) e engenheiro de requisitos (CPRE-FL – *Certified Professional for Requirements Engineering – Foundation Level*) certificado pelo *International Requirements Engineering Board* (IREB). Possui mais de vinte anos de experiência em desenvolvimento de sistemas e é, atualmente, sócio da Fatto Consultoria e Sistemas, onde atua como consultor e instrutor em serviços e cursos de medição, estimativas e requisitos de projetos de software. Atuou no desenvolvimento de toda a linha de serviços da Fatto e treinou centenas de profissionais da América Latina em requisitos e pontos de função. Participou da equipe de tradução para o português das versões 4.2 e 4.3 do *Manual de Práticas de Contagem*, do IFPUG. É ainda coautor do livro “Análise de Pontos de Função: medição, estimativas e gerenciamento de projetos de software”.

Sobre a Empresa



A FATTO Consultoria e Sistemas atua, há quase vinte anos, oferecendo serviços de consultoria e treinamento nas áreas de estimativas, medição e requisitos de software. Sua proposta é ajudar seus clientes a obter maior visibilidade do desempenho de seus processos de software, contribuindo nas tomadas de decisões e no alinhamento da gestão da Tecnologia de Informação (TI) aos objetivos estratégicos de seus negócios.

Mais de 15 mil alunos no Brasil e em países da América Latina já participaram dos treinamentos oferecidos pela FATTO, amplamente reconhecida pela excelência das suas capacitações. Além dos cursos, a empresa oferece também serviços de *mentoring*, auxiliando o cliente recém-treinado a colocar em prática o conteúdo aprendido em curso, o que permite agilizar o aprendizado e o retorno do investimento no treinamento.

Sua equipe de consultores e instrutores, com larga experiência e conhecimento em Engenharia de Software, Gerenciamento de Projetos e Governança de TI, está apta para ajudar o cliente a ganhar produtividade, ter mais visibilidade, controle e alavancar resultados dos seus projetos.

Referência brasileira no segmento, a FATTO investe e participa ativamente na comunidade de TI, desenvolvendo atividades de pesquisa, elaboração de conteúdos, publicação de artigos, participação em congressos e seminários. A meta é estar sempre em busca de soluções para o cliente enfrentar os desafios do futuro com eficiência e prontidão, esteja onde estiver.

Faça contato e conheça os nossos serviços. Acesse www.fattocs.com.

Sobre o Livro

Este livro é direcionado tanto para estudantes com interesse em desenvolvimento de sistemas quanto para profissionais envolvidos em projetos de software que desejam melhorar suas habilidades com requisitos. Buscamos escrever tanto para aqueles que atuam do lado do cliente do projeto (gestores, analistas de negócio, gerentes de sistemas) quanto do lado do fornecedor (analistas de requisitos, analistas de sistemas, analistas de testes, gerentes de projetos e desenvolvedores).

Propõe-se a apresentar a importância da Engenharia de Requisitos, seus conceitos, atividades e diversas técnicas úteis, de forma que esse conhecimento possa ser aplicado a qualquer metodologia de desenvolvimento de software disponível no mercado. Cada metodologia dita os momentos em que ocorre o trabalho de requisitos, a ordem das atividades, a relação com as outras disciplinas da engenharia de software, as técnicas a se empregar e os artefatos a serem produzidos. Acreditamos que este conteúdo seja útil para aqueles que trabalham tanto com metodologias ágeis quanto com as metodologias tradicionais. Tanto para um processo iterativo-incremental quanto para um processo sequencial (ou cascata). Tanto para projetos grandes quanto para pequenos. Tanto para desenvolvimento de novos produtos de software quanto para manutenção de softwares legados.

A publicação é fruto da experiência profissional dos autores e da pesquisa bibliográfica que teve como insumo principal o Corpo de Conhecimento da Análise de Negócio (BABOK®) do Instituto Internacional de Análise de Negócio (IIBA). Identificamos a Análise de Negócio, como definido pela IIBA, como a ligação crucial do negócio para o desenvolvimento de software. Ela é mais abrangente que a Engenharia de Requisitos; possui objetivos mais amplos.

Ao longo das diversas ações de consultoria e na condução de treinamentos, percebemos que a Análise de Negócio ser abordada em toda a sua extensão por um único perfil profissional é, ainda, uma visão de futuro.

Por isso, sentimos a necessidade de aprofundar os tópicos da Análise de Negócio referentes à Engenharia de Requisitos, mas sem nunca perder de vista as interfaces e a manutenção de um vínculo de comunicação contínuo relacionando o projeto de software à motivação para o conjunto de mudanças no qual ele está inserido e às mudanças nesse domínio.

Foi um processo gradual, e fomos incorporando ao conteúdo desta obra a nossa experiência profissional em projetos de software; o *feedback* recebido nas diferentes ações de treinamento e consultoria; as práticas de gerenciamento de projetos usadas no Scrum; o corpo de conhecimento de gerência de projetos (*PMBOK® Guide*) do PMI (*Project Management Institute*), o Processo Unificado da Rational da IBM (RUP), o guia prático do PMI para os praticantes da Análise de Negócio; os modelos de maturidade para desenvolvimento e aquisição CMMI e MPS.BR; e também a visão de vários outros autores do tema, incluindo os livros de Karl Wiegers e Ian Sommerville. Buscamos também cobrir toda a ementa da certificação CPRE-FL (*Certified Professional for Requirements Engineering – Foundation Level*), do IREB (*International Requirements Engineering Board*).

Buscamos escrever esta obra de maneira que o leitor se sinta confortável em lê-la tanto de maneira sequencial quanto por capítulos separados. Estruturamos o assunto de modo que a abordagem sequencial seja a mais didática possível para aqueles que estão tendo o primeiro contato com o tema.

Tão importante quanto o conteúdo teórico são os exercícios e os estudos de casos propostos, onde o leitor terá a oportunidade de praticar e refletir de maneira crítica sobre os temas apresentados. No final de cada capítulo há um conjunto de exercícios sobre o tema apresentado.

No **Capítulo 1 (Engenharia de Requisitos)** é apresentado o que é a Engenharia de Requisitos (ER), o porquê do termo “engenharia”, como a ER se contextualiza dentro da Engenharia de Software e como ela se insere em um processo de software. Esclarecemos a diferença entre disciplina e

fase do projeto e como a ER se apresenta em projetos com estratégia sequencial e iterativo-incremental.

O **Capítulo 2 (Requisito)** apresenta a definição de requisitos da IEEE, comentando a importância de cada parte da definição. Também define o que é especificação de requisitos, seu objetivo, qual seu público-alvo, qual o nível de detalhe adequado para uma especificação e critérios de qualidade.

Já o **Capítulo 3 (A Importância da Engenharia de Requisitos)** aborda as consequências de um trabalho de requisitos mal feito, os riscos para o projeto, quanto retrabalho isso pode ocasionar e o seu peso dentro da meta de qualidade em software.

O **Capítulo 4 (Dificuldades Comuns com Requisitos)** relata as dores mais comuns de quem está envolvido com o trabalho de requisitos. Comentamos sobre dificuldades relatadas na literatura e também vivenciadas por diversos clientes nossos. Para cada dificuldade colocamos algumas propostas de técnicas ou abordagens que ajudam a enfrentá-la.

No **Capítulo 5 (Tipos de Requisitos, Restrições e Premissas)** são apresentadas a definição de premissas e restrições e a classificação dos requisitos em diferentes tipos, exemplificando e mostrando a importância de cada um desses conceitos para o desenvolvimento de software.

O **Capítulo 6 (Atividades da Engenharia de Requisitos)** trata das macroatividades da ER, mas não apenas isso: mostra também como estudos anteriores ao projeto (ex.: análise de viabilidade) exercem um papel fundamental em facilitar ou dificultar o trabalho de requisitos.

No **Capítulo 7 (Elicitação de Requisitos)** é explicado o que é a elicitación de requisitos, seu objetivo, suas etapas, como ela se relaciona com as demais atividades da ER e também apresenta várias técnicas que podem ser empregadas nessa atividade.

O **Capítulo 8 (Análise de Requisitos)** explica o que é a análise de requisitos, seu objetivo e suas etapas. Aborda a especificação, a importância da modelagem, a organização dos requisitos e os pontos de vista funcional, estrutural e comportamental sobre os requisitos. Analisa também a verificação e validação de requisitos como papel fundamental na garantia da qualidade do trabalho de requisitos.

Por fim, o **Capítulo 9 (Gerência de Requisitos)** elucida o que é a gestão de requisitos, seu objetivo, suas etapas, como ela se relaciona com as demais atividades da ER e também apresenta várias técnicas que podem ser empregadas nessa atividade.

O **Glossário** contém uma breve definição dos termos e siglas mais importantes usados na ER e também de todas as técnicas apresentadas no livro.

O **Anexo** apresenta um estudo inspirado em um caso real, com várias das dificuldades comuns em especificações de requisitos. Esses casos são usados como prática de várias das técnicas apresentadas no livro.

As respostas dos exercícios propostos podem ser encontradas em <http://fattocs.com/pt/recursos/livro-requisitos.html>.

Sumário

1. Engenharia de Requisitos

1.1. Definição de Engenharia de Requisitos

1.2. Por que usar “engenharia” em Engenharia de Requisitos?

1.3. Engenharia de Requisitos como parte da Engenharia de Software

1.4. Engenharia de Requisitos em diferentes estratégias de desenvolvimento

1.5. O ambiente da Engenharia de Requisitos

1.6. O papel do analista de requisitos

1.7. Exercícios

2. Requisito

2.1. Uma palavra, muitos significados

2.1.1. Necessidade

2.1.2. Propriedade

2.1.3. Especificação

2.2. Definição de requisito

2.2.1. Resumo da definição

2.3. Especificação de requisitos

2.3.1. Especificação de requisitos para quem?

2.4. Nível de detalhe da especificação

2.4.1. Significado de nível de detalhe

2.4.2. Delimitar o escopo

2.4.3. Descrever um item no escopo

2.4.4. Mapear os requisitos no design ou implementação

2.4.5. Equívoco comum ao detalhar

2.5. Critérios para nível de detalhe da especificação

2.5.1. Desenvolvimento interno ou externo

2.5.2. Equipe dispersa ou agrupada geograficamente

2.5.3. Casos de testes baseados nos requisitos

2.5.4. Grau de incerteza das estimativas

2.5.5. Rastreabilidade dos requisitos

2.5.6. Envolvimento dos clientes

2.5.7. Conhecimento da equipe no domínio

2.5.8. Trabalho com características similares a anteriores

2.5.9. Desenvolvimento concorrente de procedimentos

2.5.10. Solução usará pacote

2.5.11. Expectativa de rotatividade de mão de obra

2.6. Critérios de qualidade da especificação

2.6.1. Correta

2.6.2. Completa

2.6.3. Clara (não ambígua)

2.6.4. Consistente

2.6.5. Modificável

2.6.6. Priorizada

2.6.7. Verificável (ou testável)

2.6.8. Rastreável

2.6.9. Onde usar tais critérios de qualidade?

2.7. Exercícios

3. A Importância da Engenharia de Requisitos

3.1. Motivação para a Engenharia de Requisitos

3.2. Impactos negativos da falha em requisitos

3.2.1. Sonda espacial Mars Climate Orbiter

3.2.2. Míssil antibalístico Patriot

3.2.3. Foguete Ariane 501

3.2.4. HAL 9000

3.2.5. Arquivo virtual do FBI

3.3. Uma tentativa de melhoria de processo

3.3.1. Principal motivo para o fracasso de projetos

3.3.2. Uma das principais causas de defeitos em software

3.3.3. Custo do reparo de defeitos

3.4. Especificações de requisitos como um ativo

3.5. Reflexão: onde a indústria de software investe energia

3.6. Exercícios

4. Dificuldades Comuns com Requisitos

4.1. Comunicação

4.2. Acesso às partes interessadas

4.3. Indecisões/Indefinições do usuário

4.4. Requisitos implícitos ou não ditos

4.5. Mudanças

4.6. Conflitos

4.7. Resistência à mudança

4.8. Parte interessada não domina seu negócio

4.9. Parte interessada não lê a especificação de requisitos

4.10. Partes interessadas insaciáveis com requisitos

4.11. Consistência

4.12. Necessidades vagas

4.13. Priorização

4.14. Conclusão

4.15. Exercícios

5. Tipos de Requisitos, Restrições e Premissas

5.1. Domínio do problema

5.1.1. Qual é a sua importância?

5.2. Requisitos (ou necessidades) de negócio

5.2.1. O que são?

5.2.2. Qual é a sua importância?

5.2.3. Critérios de qualidade

5.2.4. Quando são elaborados?

5.2.5. Papel do analista de requisitos

5.2.6. Onde ficam registrados?

5.3. Restrições e premissas

5.3.1. Restrições

5.3.1.1. Papel do analista de requisitos

5.3.1.2. Qual é a sua importância?

5.3.1.3. Restrição de negócio

5.3.1.4. Restrição técnica

5.3.2. Premissas

5.3.2.1. Qual é a sua importância?

5.3.2.2. Papel do analista de requisitos

5.4. Partes interessadas

[5.4.1. O caminho a partir dos requisitos de negócio](#)

[5.4.2. O que são?](#)

[5.4.3. Clientes e usuários × partes interessadas](#)

[5.4.4. Autoridade e responsabilidade](#)

[5.4.5. Qual é a sua importância?](#)

[5.5. Requisitos das partes interessadas](#)

[5.5.1. Onde ficam registrados?](#)

[5.6. Requisitos da solução](#)

[5.6.1. Qual é a sua importância?](#)

[5.6.2. Processo geral de desenvolvimento dos requisitos](#)

[5.7. Requisitos de transição](#)

[5.7.1. Qual é a sua importância?](#)

[5.7.2. Papel do analista de requisitos](#)

[5.8. Requisitos de software: solução + transição](#)

[5.8.1. Onde são armazenados?](#)

[5.9. Requisitos funcionais](#)

[5.9.1. Onde são armazenados?](#)

[5.9.2. Papel do analista de requisitos](#)

[5.9.3. Nível de granularidade](#)

[5.9.4. Requisitos funcionais com objetivo de usuário](#)

[5.9.4.1. Qual é a sua importância?](#)

[5.9.5. Requisitos funcionais com objetivo agregador](#)

[5.9.5.1. Qual é a sua importância?](#)

[5.9.5.2. Papel do analista de requisitos](#)

[5.9.5.3. Requisitos de negócio × requisitos agregadores](#)

[5.9.6. Requisitos funcionais com objetivo de subfunção](#)

[5.9.6.1. Qual é a sua importância?](#)

[5.9.6.2. Papel do analista de requisitos](#)

5.10. Requisitos não funcionais

5.10.1. Diferença entre requisitos não funcionais e restrição técnica

5.10.2. Qual é a sua importância?

5.10.3. FURPS+

5.10.4. ISO/IEC 25010

5.10.5. Papel do analista de requisitos

5.11. Requisitos inversos

5.11.1. Qual é a sua importância?

5.12. Requisitos de projeto e requisitos de qualidade

5.13. Exercícios

6. Atividades da Engenharia de Requisitos

6.1. Um único tema, diferentes pontos de vista

6.2. Primeiro marco: definição da necessidade

6.2.1. Atividades da Engenharia de Requisitos

6.3. Segundo marco: consenso sobre o escopo

6.4. Terceiro marco: detalhamento dos requisitos

6.5. Técnicas para obter consenso do escopo

6.5.1. Técnica: declaração de problema

6.5.2. Modelagem de escopo (modelagem ambiental)

6.5.3. Técnica: diagrama de contexto

6.5.3.1. Entidades externas

6.5.3.2. Depósitos de dados

6.5.3.3. Conclusão

6.5.4. Técnica: diagrama de casos de uso

6.5.5. Técnica: modelo de processo de negócio

6.6. Como saber com quem conversar

6.7. Exercícios

7. Elicitação de Requisitos

7.1. Definindo a elicitación de requisitos

7.2. Atividades da elicitación

7.2.1. Preparación para elicitación

7.2.2. Execução da elicitación

7.2.3. Documentación dos resultados da elicitación

7.2.4. Confirmação dos resultados da elicitación

7.3. Técnica: análise de documentos

7.3.1. O que é a análise de documentos

7.3.2. Como realizar a análise de documentos

7.3.2.1. Preparación

7.3.2.2. Execução

7.3.2.3. Finalização

7.3.2.4. Vantagens

7.3.2.5. Desvantagens

7.3.2.6. Conclusão

7.4. Técnica: glossário

7.4.1. Introdução

7.4.2. O que é o glossário

7.4.3. Onde encontrar

7.4.4. Quando começar

7.4.5. Como elaborar um glossário

7.4.6. Importância como produto

7.4.7. Importância como processo

7.4.8. Conclusão

7.5. Técnica: observação (etnografia)

7.5.1. O que é observação

7.5.2. Como realizar a observação

7.5.2.1. Preparação

7.5.2.2. Execução

7.5.2.3. Finalização

7.5.3. Vantagens

7.5.4. Desvantagens

7.5.5. Conclusão

7.6. Técnica: entrevista

7.6.1. O que é a entrevista

7.6.2. A primeira impressão é a que fica

7.6.3. Diretrizes para o entrevistador

7.6.3.1. Seja um bom ouvinte

7.6.3.2. Vá de coração aberto; livre-se de preconceitos

7.6.3.3. Busque os fatos, mas também opiniões

7.6.4. Como realizar a entrevista

7.6.4.1. Preparação

7.6.4.2. Preparação do roteiro de questões

7.6.4.3. Formato da entrevista

7.6.4.4. Forma de registro

7.6.5. Execução

7.6.6. Finalização

7.6.6.1. Vantagens

7.6.6.2. Desvantagens

7.6.7. Conclusão

7.7. Técnica: pesquisa/questionário

7.7.1. O que é a pesquisa/questionário

7.7.2. Como realizar a pesquisa

7.7.2.1. Preparação

7.7.2.2. Execução

7.7.2.3. Finalização

7.7.3. Vantagens

7.7.4. Desvantagens

7.7.5. Conclusão

7.8. Exercícios

8. Análise de Requisitos

8.1. Um problema: a descoberta tardia de que ainda falta muito

8.2. Visão geral da análise de requisitos

8.2.1. O que é análise de requisitos?

8.2.2. Por que análise de requisitos?

8.2.3. Como realizar a análise de requisitos?

8.3. Organizar a partir do exame, decomposição e síntese

8.3.1. Exame para identificar ou descrever tarefas

8.3.1.1. Tarefas já existentes no fluxo operacional

8.3.1.2. Inovação no fluxo operacional

8.3.1.3. Diferentes objetivos de informação

8.3.2. Decomposição da informação

8.3.3. Síntese da informação

8.4. Modelar e usar modelos para refinar a informação

8.4.1. O que é modelagem?

8.4.2. Por que modelagem?

8.4.3. Como realizar a modelagem?

8.4.3.1. Modelos preexistentes

8.4.3.2. Modelos a elaborar

8.4.3.3. Tipos de modelo quanto à forma

8.4.3.4. Tipos de modelo quanto à informação

8.4.3.5. Seleção de modelos

8.4.3.6. Seleção de modelos e as restrições organizacionais

8.4.3.7. Seleção de modelos e fatores humanos

8.4.3.8. Metodologia

8.5. Especificar para documentar os requisitos

8.5.1. O que é especificação?

8.5.2. Por que especificação?

8.5.3. Quando elaborar uma especificação?

8.5.4. Como elaborar uma especificação?

8.6. Verificação de requisitos

8.6.1. O que é verificação?

8.6.2. Quem realiza a verificação?

8.6.3. Por que verificação?

8.6.4. Quando realizar a verificação?

8.6.5. Como realizar a verificação?

8.6.5.1. Verificação de um modelo em especial e sua integração com os demais

8.6.5.2. Verificação da definição do escopo na lista de requisitos funcionais

8.6.5.3. Verificação da descrição do requisito funcional – o detalhamento do escopo

8.6.5.4. Técnicas úteis à verificação

8.7. Validação de requisitos

8.8. Técnica: histórias do usuário

8.8.1. Por que histórias do usuário?

8.8.2. Como elaborar uma história do usuário?

8.8.2.1. INVEST

8.8.2.2. CCC

8.8.2.3. Dividindo histórias do usuário

8.8.2.4. Épicos

8.8.2.5. Temas

8.9. Técnica: modelagem de processos

8.9.1. O que é um processo?

8.9.2. O que é modelagem de processo?

8.9.3. Diagrama, mapa e modelo de processos

8.9.4. Por que modelagem de processos?

8.9.5. Como realizar a modelagem de processos?

8.9.5.1. Representações de processos de negócio e a Engenharia de Requisitos

8.10. Técnica: decomposição funcional

8.10.1. Como aplicar a decomposição funcional?

8.10.2. Por que aplicar a decomposição funcional?

8.11. Técnica: modelagem de domínio

8.11.1. Como realizar a modelagem de domínio?

8.11.2. Conceito de negócio

8.11.3. Checklist para organizar o modelo de domínio

8.11.4. Relacionamentos entre conceitos

8.11.5. Representações para o modelo de domínio

8.11.6. Por que realizar modelagem de domínio?

8.12. Técnica: modelagem de casos de uso

8.12.1. O que é um caso de uso?

8.12.2. Diagrama de casos de uso

8.12.2.1. Associação entre um ator e o caso de uso

8.12.2.2. Outros tipos de relacionamento

8.12.3. Especificação dos casos de uso

8.12.3.1. Cenários

8.12.3.2. Como identificar um caso de uso?

8.12.4. Por que casos de uso?

8.13. Técnica: listas de verificação

[8.13.1. O que são listas de verificação?](#)

[8.13.1.1. Como usar listas de verificação?](#)

[8.13.2. Por que listas de verificação?](#)

[8.14. Técnica: prototipação](#)

[8.14.1. O que é prototipação?](#)

[8.14.2. Como aplicar a prototipação?](#)

[8.14.2.1. Prototipação de baixa fidelidade × alta fidelidade](#)

[8.14.2.2. Prototipação horizontal × vertical](#)

[8.14.2.3. Prototipação descartável × evolutiva](#)

[8.14.2.4. Riscos e cuidados](#)

[8.14.2.5. Protótipo × caso de uso](#)

[8.14.3. Por que prototipação?](#)

[8.15. Exercícios](#)

[9. Gerência de Requisitos](#)

[9.1. Gerência de requisitos no CMMI-DEV](#)

[9.1.1. Ciclo de vida da gerência de requisitos](#)

[9.2. Plano de gerenciamento de requisitos](#)

[9.2.1. Quem é responsável pela gerência de requisitos?](#)

[9.3. Gestão de mudanças](#)

[9.4. Obter aprovação sobre os requisitos](#)

[9.4.1. Como apresentar os requisitos para aprovação?](#)

[9.5. Técnica: controle de questões](#)

[9.5.1. O que é o controle de questões](#)

[9.5.2. Elementos do controle de questões](#)

[9.6. Rastreabilidade de requisitos](#)

[9.6.1. O que é a rastreabilidade de requisitos](#)

[9.6.2. Benefícios da rastreabilidade](#)

[9.6.3. Tipos de rastreabilidade](#)

[9.6.3.1. Rastreabilidade horizontal](#)

[9.6.3.2. Rastreabilidade vertical](#)

[9.6.3.3. Pré e pós-rastreabilidade](#)

[9.6.4. Matriz de rastreabilidade](#)

[9.7. Priorizar requisitos](#)

[9.7.1. Técnica: timeboxing/budgeting](#)

[9.7.2. Técnica: votação](#)

[9.7.3. Técnica: análise Moscow](#)

[9.7.3.1. Diretrizes para a priorização Moscow](#)

[9.8. Gerência de requisitos depende de ferramenta?](#)

[9.9. Exercícios](#)

[Bibliografia](#)

[Glossário](#)

[Anexo. Estudo Preliminar SRRO – Sistema de Registro de Responsabilidade de Obras](#)

[A.1. Objetivo](#)

[A.2. Justificativa](#)

[A.3. Prazo de entrega](#)

[A.4. Valor contratado](#)

[A.5. Responsáveis pelo projeto](#)

[A.6. Objeto](#)

[A.7. Quantidade estimada de pontos de função](#)

[A.8. Recebimento](#)

[A.9. Documento de Visão](#)

1. Engenharia de Requisitos

“Quando eu uso uma palavra’, disse Humpty Dumpty em um tom um tanto quanto insolente, ‘isso significa apenas aquilo que eu escolho que ela signifique – nem mais, nem menos’.”

Lewis Carroll Ao buscar maior conhecimento sobre determinado assunto, um dos primeiros passos é entender o que seu nome significa. Nesse sentido, este capítulo tem como objetivos: definir o que é a Engenharia de Requisitos, o porquê do termo “engenharia”, como ela se contextualiza dentro da Engenharia de Software e como se insere em um processo de desenvolvimento de software.

O trecho de “Alice no País do Espelho”, de Lewis Carroll, que abre este capítulo, já foi usado na discussão de questões de estado na Inglaterra e em mais de 250 decisões judiciais nos Estados Unidos.

Isso é relevante, pois, se todos agissem como Humpty Dumpty, seria a instalação do caos. Então, é necessário estabelecer o que as palavras significam (ou pelo menos dar início a esse processo).

A citação, além de cumprir o propósito de abrir o capítulo, é muito pertinente à produção da Engenharia de Requisitos (ER), como será abordado ao longo deste capítulo.

1.1. Definição de Engenharia de Requisitos

A Engenharia de Requisitos pode ser definida como uma disciplina da Engenharia de Software que consiste no uso sistemático e repetitivo de

técnicas para cobrir atividades de obtenção, documentação e manutenção de um conjunto de requisitos para software que atendam aos objetivos de negócio e sejam de qualidade.

O que são esses objetivos de negócio e o que vem a ser qualidade de requisitos são assuntos que serão explorados no Capítulo 2.

1.2. Por que usar “engenharia” em Engenharia de Requisitos?

Um neologismo é a utilização de novas palavras, compostas a partir de outras que já existem (em um mesmo idioma ou não), ou a atribuição de novos significados (ou sentidos) a palavras que já existem. O mundo atual está cheio de neologismos, com os mais diferentes propósitos. Por exemplo, ao se procurar por uma alternativa à aquisição de um carro zero quilômetro, não se encontra um carro usado; e sim um “seminovo”. Ao ligar para a central de atendimento em busca de ajuda para resolver um problema, não se fala com uma atendente (isso, claro, quando se consegue falar com uma pessoa); e sim com uma “consultora em serviços de telefonia”.

Os exemplos anteriores buscam através de um novo nome valorizar o objeto discutido. Então cabe aqui a pergunta: seria o termo Engenharia de Requisitos uma tentativa de apropriar-se do prestígio de uma profissão (“a engenharia”) para valorizar o assunto, sem que isso esteja associado ao real significado da palavra “engenharia”?

Não se trata de uma avaliação legal, para efeitos do exercício profissional ou a constituição de empresas, trata-se de uma avaliação do significado em termos gerais. Para isso será usada uma definição do termo encontrada na Wikipédia (acesso em ago. 2015): *A engenharia é a ciência, a arte e a profissão de adquirir e de aplicar os conhecimentos matemáticos, técnicos e científicos na criação, aperfeiçoamento e implementação de utilidades, tais como materiais, estruturas, máquinas, aparelhos, sistemas ou processos, que realizem uma determinada função ou objetivo.*

A Engenharia de Requisitos está intimamente ligada à aquisição e aplicação de conhecimento para a criação, o aperfeiçoamento e a implementação de

sistemas de informação.

Um complemento útil ao entendimento do que seja engenharia é uma definição de um processo geral para ela. O Museu de Ciências de Boston, nos Estados Unidos, desenvolveu o programa Engenharia é Elementar (*Engineering is Elementary*), que tem por objetivo motivar estudantes da primeira à oitava série a aplicar o que sabem sobre ciências e matemática. A Figura 1.1, adaptada deste programa, ilustra as cinco etapas desse processo, o ordenamento entre elas e a sua natureza cíclica.

A primeira etapa do processo é “pergunte” e busca identificar qual o problema, o que outros fizeram no sentido de resolvê-lo e quais as restrições que se aplicam. Em seguida, “imagine” quais são algumas soluções, pense em alternativas, escolha a melhor solução. Então, “planeje” desenhando um diagrama e preparando uma lista do que precisa; “crie” seguindo seu plano e teste os resultados. Por fim, “melhore” discutindo o que funciona, o que não funciona e o que poderia ser melhor, modifique o seu projeto para melhorá-lo e teste novamente.

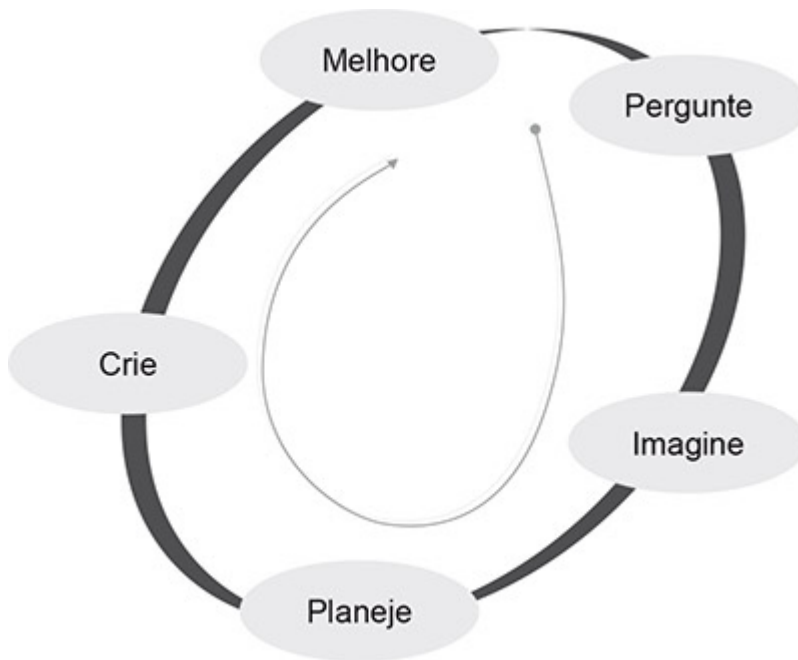


Figura 1.1. Um processo geral de engenharia explicado de forma simples para jovens.

A Engenharia de Requisitos está completamente alinhada a esse processo geral. Em um primeiro momento, pode-se pensar que ela se restringe apenas

às primeiras etapas presentes no processo; contudo, isso se revela falso quando se explora melhor um dos principais benefícios da Engenharia de Requisitos: habilitar o entendimento – de forma contínua – das necessidades do cliente para entregar uma solução que atenda aos seus objetivos de negócio, que são dinâmicos e mutáveis.

A Engenharia de Requisitos está inserida neste livro em um contexto onde os requisitos para o software estão contidos em uma solução mais abrangente que pode ter requisitos além dos requisitos para o software. Ainda assim, o foco deste livro concentra-se na aplicação da Engenharia de Requisitos para o desenvolvimento de software.

1.3. Engenharia de Requisitos como parte da Engenharia de Software

A Engenharia de Requisitos se insere no âmbito da engenharia de software, independentemente de qual a referência em sua definição. A ISO (2010) define a engenharia de software como: *(1) a aplicação sistemática de conhecimento tecnológico e científico, métodos e experiência ao projeto, implementação, teste e documentação de software; (2) a aplicação de uma abordagem sistemática, disciplinada quantificável ao desenvolvimento, operação e manutenção de software; ou seja, a aplicação de engenharia ao software.*

Destaca-se a palavra “disciplinada” no sentido de que o conhecimento e as habilidades relativos às tarefas da engenharia de software são organizados em disciplinas ou áreas de conhecimento. Não há um consenso ou um referencial único sobre quais sejam essas disciplinas. Há modelos de referência que direta ou indiretamente cumprem esse papel – os mais relevantes, por exemplo, são: ➤

- o Processo Unificado da Rational/IBM (RUP); ➤

- o Corpo de Conhecimento da Engenharia de Software do IEEE (SWEBOK, acrônimo do inglês *Software Engineering Body of Knowledge*); ➤

as áreas de processo do modelo de maturidade CMMI (*Capability Maturity Model – Integration*) do SEI (*Software Engineering Institute*).

Se o processo unificado for usado como uma referência, uma disciplina é definida como um meio de criar categorias de atividades baseadas em similaridades de interesses e cooperação no esforço de trabalho. Nele, há dois grupos de disciplinas – as disciplinas de engenharia: ➤

Modelagem de negócio.



Engenharia de requisitos.



Análise e projeto.



Implementação.



Engenharia de testes.



Implantação.

E as disciplinas de apoio:



Gerência de configuração e mudança.



Gerência de projetos.



Ambiente.

No SWEBOK, são dez áreas de conhecimento (abreviadas como *KA* – *Knowledge Area*), onde a primeira é requisitos de software, conforme apresentado a seguir: ➤

Requisitos de software.



Projeto (*design*) de software.



Construção de software.



Testes de software.



Manutenção de software.



Gerência de configuração de software.



Gerência de engenharia de software.



Processo de engenharia de software.



Métodos e ferramentas de engenharia de software.



Qualidade de software.

Enquanto o RUP e o SWEBOK abordam a Engenharia de Requisitos como uma única disciplina ou área de conhecimento, no CMMI utiliza-se o termo “área de processo” e o tema é abordado em duas delas.

Uma área de processo é um agregado de práticas relacionadas que, quando implementadas de maneira coletiva, satisfazem um conjunto de objetivos considerados importantes para realização de melhorias.

A seguir são relacionadas as duas áreas de processo do CMMI diretamente associadas à Engenharia de Requisitos, conjuntamente com suas abreviaturas: ➤

Desenvolvimento de requisitos (RD – *Requirements Development*).



Gestão de requisitos (REQM – *Requirements Management*).

Não importa o referencial utilizado para definir a engenharia de software, o assunto requisitos é fundamento para todo o trabalho das demais disciplinas. E o trabalho descrito nesses modelos reflete um tipo de especialização funcional. Este trabalho especializado deve ser organizado por uma estratégia de desenvolvimento, como ilustrado na Figura 1.2, antes de poder entregar software funcionando.

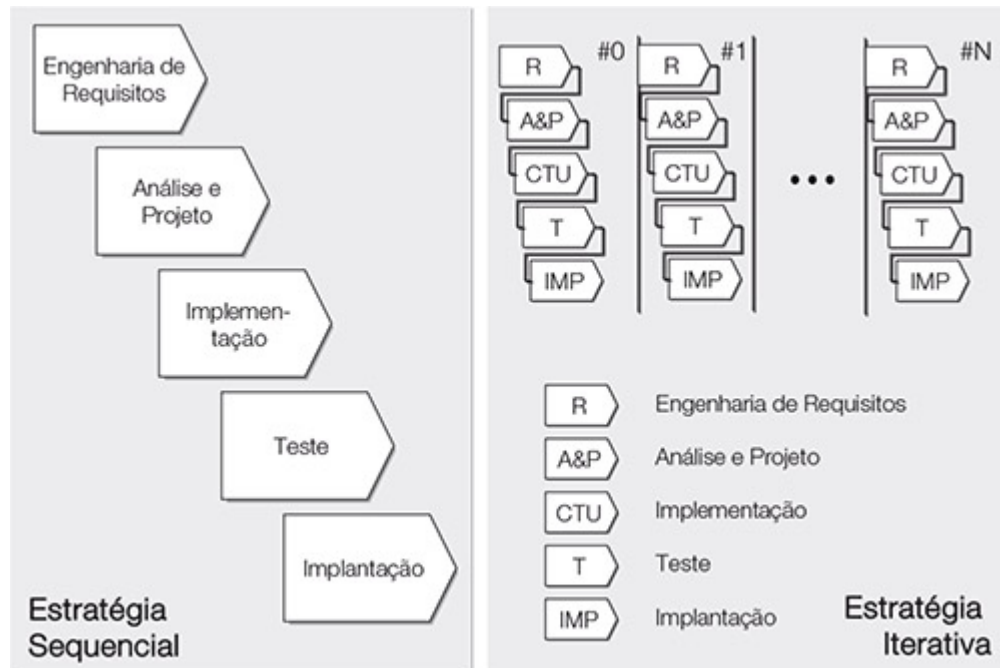


Figura 1.2. Diferentes estratégias de desenvolvimento organizam de diferentes maneiras o trabalho descrito nas disciplinas.

1.4. Engenharia de Requisitos em diferentes estratégias de desenvolvimento

Talvez a falta de conhecimento ou habilidade em usar adequadamente esses modelos de referência tenha conduzido a uma utilização equivocada destes em projetos reais, principalmente em organizações públicas ou empresas privadas de grande porte. Nessas organizações, é comum manter o desenvolvimento sob uma forte orientação ao planejamento e acomodar pouco espaço para a mudança, mesmo em cenários onde não há necessariamente uma grande complexidade técnica ou gerencial.

Isso fomenta um ciclo mais longo de retroalimentação entre a equipe de desenvolvimento e seus clientes. Dessa forma, a identificação e a correção de desvios acontecem quando passado muito tempo e investido mais trabalho que o necessário. Alguns desses desvios são consequências da duração excessiva de cada ciclo, o que potencializa que o desenvolvimento seja afetado por mudanças no ambiente de negócio. Outros desvios são

tratados como mudanças, quando na verdade são resultados de problemas de comunicação das mais diversas origens.

A Engenharia de Requisitos acaba sendo percebida como vilã nessa dinâmica, e muitos profissionais acabam por confundir o termo “requisito” com “documentação” (assunto explorado no Capítulo 2); ou, então, com uma fase de acompanhamento do progresso de projetos, tamanha a orientação ao planejamento.

Em termos práticos, acaba-se por determinar um ordenamento do trabalho muito próximo à estratégia sequencial, como ilustrado na Figura 1.3. Atribui-se ao padrão militar para o desenvolvimento de software do Departamento de Defesa dos Estados Unidos a popularização dessas estratégias como uma “cascata”, ainda que em seu segundo parágrafo afirme (DOD, 1985): *O desenvolvimento de software é em geral um processo iterativo, no qual uma iteração do ciclo de desenvolvimento de software acontece uma ou mais vezes ao longo de cada fase do ciclo de vida do sistema.*

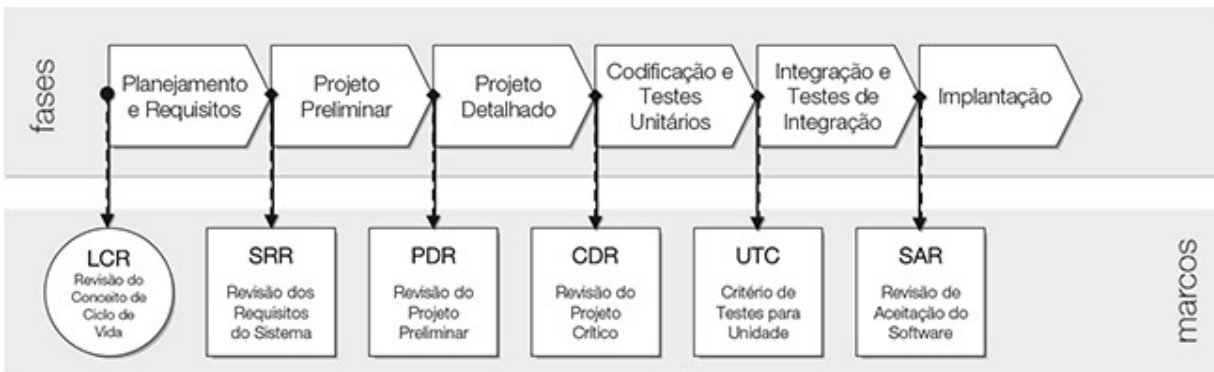


Figura 1.3. A estratégia sequencial para ordenamento do trabalho de desenvolvimento.

Limitar a atividade de Engenharia de Requisitos a uma única fase do projeto de desenvolvimento do software ou limitar o significado do termo “requisito” a apenas “documentação” é possível; porém, equivocado considerando as exigências e expectativas no mundo atual. Não é razoável nesse contexto desconsiderar as necessidades do cliente também como requisitos.

Essa visão (e implementação) equivocada dos modelos de referência citados levou a movimentos como o Manifesto Ágil e ao surgimento de propostas

como a *Extreme Programming* (XP) e o Scrum, por exemplo.

Organizar o trabalho espalhando as atividades de requisitos ao longo de todo o desenvolvimento produz melhores resultados. Colocar maior ênfase inicialmente no entendimento dos objetivos para o desenvolvimento e das suas restrições. Concomitantemente, explorar a abrangência do produto, especificando as principais atividades a serem informatizadas ou automatizadas. Neste caso, não se identificam todas as questões ou respondem-se àquelas já identificadas; já se sabe quais macrofunções serão tocadas pelo desenvolvimento; contudo, ainda não se sabe – especificamente – quais atividades em particular.

Em seguida e gradualmente, intercalam-se atividades da Engenharia de Requisitos na exploração da profundidade do produto, detalhando o seu comportamento esperado e preenchendo as lacunas deixadas anteriormente, com atividades de projeto, implementação e testes ao longo de ciclos curtos de desenvolvimento, por exemplo, de duas a quatro semanas.

A Figura 1.4 ilustra esse cenário com a inter-relação entre: ➤

- as fases de Iniciação, Elaboração, Construção e Transição utilizadas no planejamento e monitoramento do progresso, onde se deve avaliar a continuidade ou interrupção do desenvolvimento; ➤

- as iterações (ou ciclos), que incluem atividades das disciplinas/processos de Engenharia de Requisitos (R), Análise e Projeto (A&P), Implementação (CTU), Testes (T) e Implantação (IMP), realizadas em maior ou menor nível conforme o momento em que o desenvolvimento se encontra; e ➤

- os marcos com os objetivos de informação que devem ser alcançados para se considerar uma fase como concluída.

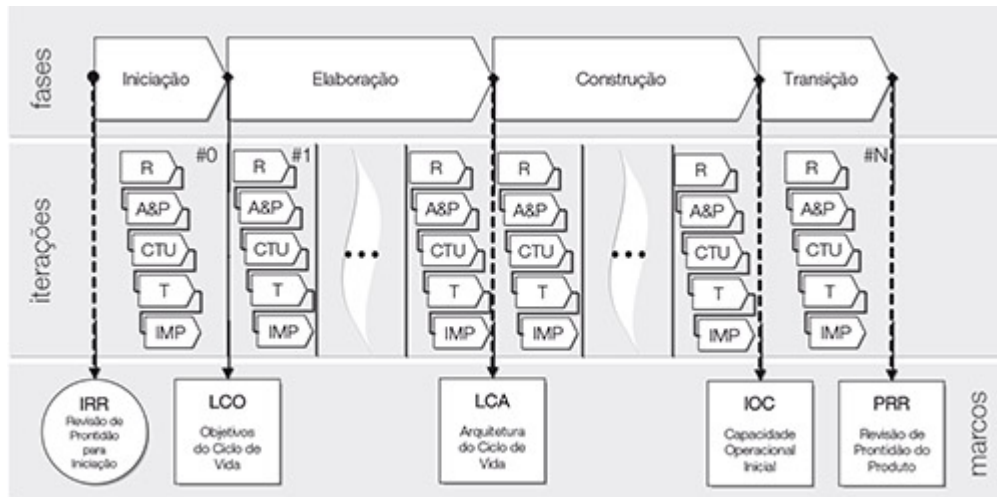


Figura 1.4. A estratégia iterativa para ordenamento do trabalho de desenvolvimento.

Algo que falta representar na Figura 1.4 é o maior ou menor nível de esforço relativo a cada disciplina. Por exemplo, a disciplina de Engenharia de Requisitos demanda mais esforço nas primeiras iterações, enquanto o oposto ocorre com atividades de implementação, onde basicamente é exercitada em alguma prova de conceito ou protótipo de itens com maior risco para o desenvolvimento.

Essa lacuna é preenchida pelo “gráfico das baleias”, que resume o processo unificado e apresentado na Figura 1.5. Esse apelido se dá pelo fato de as curvas de esforço de cada disciplina se assemelharem ao perfil de baleias.

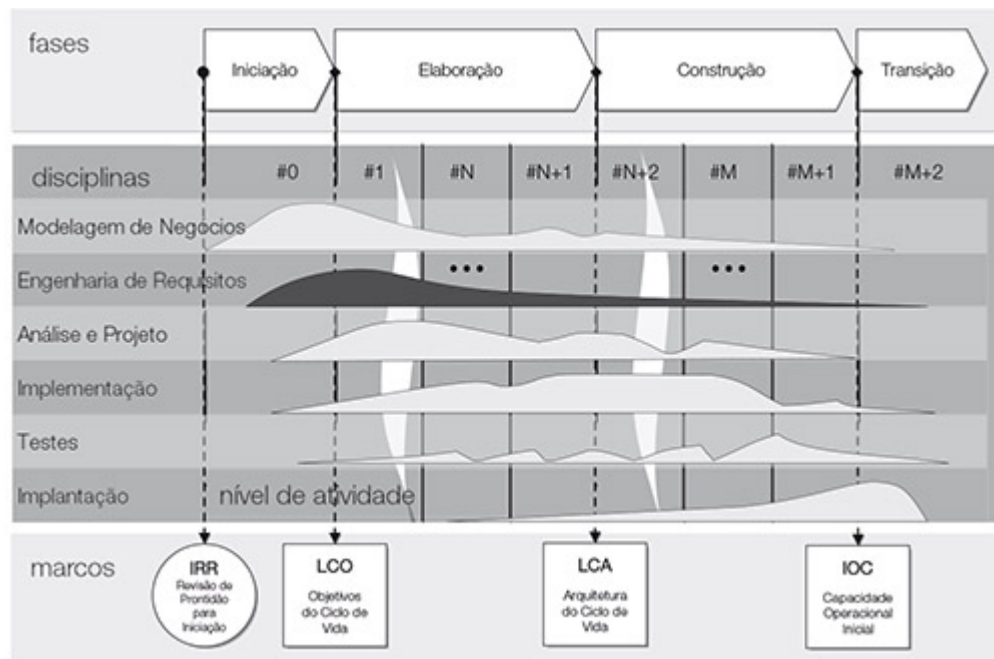


Figura 1.5. Comparativo do nível de esforço entre as disciplinas da engenharia de software.

Como foi observado na Figura 1.4, o desenvolvimento iterativo não pressupõe que todo trabalho de uma disciplina deve estar concluído para, só então, se trabalhar em outras disciplinas.

A fase de Iniciação, na maior parte das vezes, corresponde a um único ciclo. Nela, se consome em média 38% do esforço total investido em atividades da Engenharia de Requisitos.

É o tipo de atividade de maior concentração percentual quando comparado aos demais tipos de atividade necessários ao desenvolvimento. Estes, por exemplo, abordam decisões associadas à análise e ao projeto e implicam em alto risco para o desenvolvimento. O uso de uma nova tecnologia ou de níveis de serviço além dos usuais exige uma prova de conceito que requer trabalho de análise e projeto, implementação e testes.

Difícilmente algum código-fonte será entregue como parte de produto final nessa iteração #0. Se há código, então ele provavelmente está associado a uma prova de conceito (PoC) para validar uma premissa de arquitetura. Em momentos subsequentes, deve-se buscar como objetivo resultados que sirvam como incrementos para o produto final.

Essa primeira fase tem a variabilidade do esforço percentual em relação ao desenvolvimento como um todo bastante acentuada; entre uma faixa de 2% a 15%, sendo que a média de 5% é usualmente adotada para fins de planejamento. A Figura 1.6 complementa essa informação e ilustra a distribuição percentual média do esforço incluindo todas as fases (BOEHM, 2000).

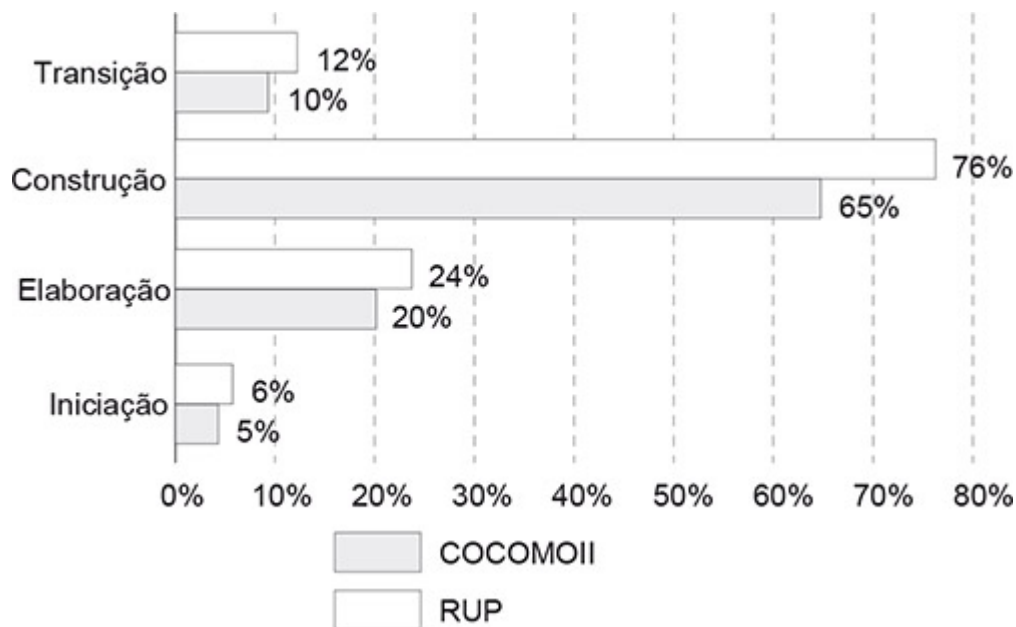


Figura 1.6. Distribuição do percentual de esforço médio entre as fases.

Considerando esses números, as atividades de requisitos nessa primeira fase correspondem a cerca de 2% do esforço total do desenvolvimento.

Durante os ciclos correspondentes à fase de Elaboração, o esforço médio investido nas atividades de requisitos, em comparação com as outras disciplinas, corresponde a 18% do esforço total. Durante a fase de Construção, 8%, e durante a fase de Transição, 4%.

No desenvolvimento completo, as atividades da Engenharia de Requisitos respondem por 15% do total, de acordo com Gartner (2010); 11%, de acordo com Boehm (2000); e entre 6% e 13%, dependendo da categoria de indústria (sistemas do usuário final; sistemas de informação gerencial; *outsourcing*; sistemas comerciais; sistemas militares; sistemas integrados de hardware e software; web), de acordo com Jones (2007). A Tabela 1.1 resume os dados do COCOMO II, que disponibiliza os percentuais de esforço investidos na Engenharia de Requisitos por fase.

Tabela 1.1. Comparativo do esforço médio investido entre as atividades da Engenharia de Requisitos e as demais disciplinas de acordo com a pesquisa do COCOMO II.

	(% do esforço investido na fase				% do esforço investido no desenvolvimento
Disciplina	Iniciação	Elaboração	Construção	Transição	
Engenharia de Requisitos	38%	18%	8%	4%	11%
Outras	62%	82%	92%	96%	89%

Um plano de iteração deve capturar a distribuição do trabalho pela descrição de uma matriz onde o tempo, as fases e as disciplinas se integram. Ele não deve ser objeto de um único planejamento inicial. Ao final de cada ciclo, esse plano deve ser atualizado indicando o que foi realizado e a visão atual para uma nova distribuição do trabalho para as próximas iterações. A falta da visão dessa necessidade por um caso de desenvolvimento único para cada projeto ou de mecanismos para sua implementação está entre os principais fatores para muitas iniciativas de adoção de estratégias iterativas falharem.

Os autores participaram da definição de um conjunto completo de instrumentos para operacionalizar o planejamento e acompanhamento do progresso no desenvolvimento que permite maior agilidade, mantendo as exigências corporativas para sua integração com a governança corporativa, com base na Engenharia de Requisitos e na Análise de Pontos de Função. O conteúdo completo está disponível em Itaipu (2011) e um resumo da apresentação realizada em um evento internacional está disponível no blog mantido pelos autores (FATTO, 2014).

Independentemente do posicionamento entre as várias opções do desenvolvimento sequencial ao desenvolvimento iterativo e incremental, sempre haverá necessidade da Engenharia de Requisitos, por mais “ágil” que seja a iniciativa. Especificamente para este trabalho, o interesse é que o conhecimento associado à Engenharia de Requisitos possa ser usado em projetos e organizações independentemente de quais estratégias de desenvolvimento sejam utilizadas.

1.5. O ambiente da Engenharia de Requisitos

A Figura 1.7 coloca a Engenharia de Requisitos no seu contexto e as principais inter-relações que os responsáveis por ela mantêm com os clientes e com a equipe de desenvolvimento.

A Engenharia de Requisitos facilita a interação com o cliente em termos de identificar e entender suas necessidades e na obtenção de um acordo acerca da solução que será entregue. Ela descreve e integra tarefas, técnicas, orientações, papéis e responsabilidades em fluxos de trabalho que: ➤

têm início com o entendimento da necessidade do cliente; e ➤
passam pelo acordo sobre a solução que será construída.

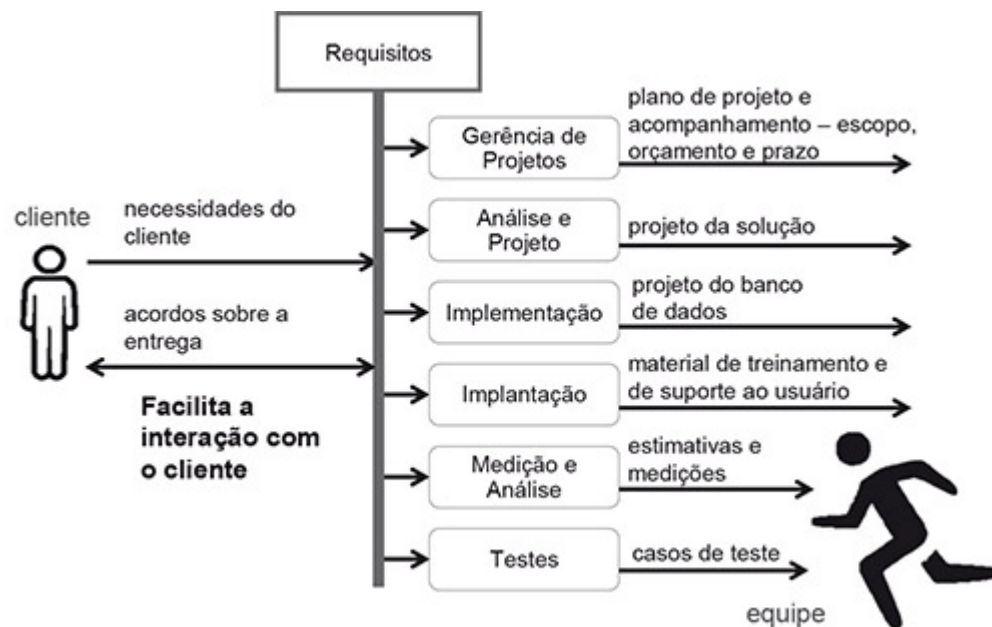


Figura 1.7. O contexto da Engenharia de Requisitos.

Ela produz insumos para uso por uma variedade de outras disciplinas da Engenharia de Software. Como resultado das tarefas de Engenharia de Requisitos, são fornecidos insumos para as disciplinas de: ➤

Análise e Projeto: na elaboração do projeto da solução.

Implementação: no projeto de banco de dados.



Gerência de Projetos: no planejamento de projetos e no seu acompanhamento quanto a escopo, orçamento e prazos.



Implantação: na confecção de material de treinamento e de suporte ao usuário.



Medição e Análise: para a produção de estimativas e medições.



Testes: com a documentação de casos de testes.

Observe que não há nesse contexto a atividade de programação propriamente dita ao citar a disciplina de implementação. Isso porque há necessidade de atividades complementares de *design*, decisões sobre os componentes de software mais adequados para alocar o comportamento descrito nos requisitos. Fazer diferente implica em exigir de um mesmo profissional habilidades de *design* e programação. Isso não implica dizer que os programadores sejam proibidos de ter acesso aos requisitos; mas que há necessidade de trabalho anterior de outra especialidade.

1.6. O papel do analista de requisitos

De acordo com o portal de empregos Catho (acesso em jul. 2015), o analista de requisitos é aquele que: *realiza o levantamento de requisitos e especificação de projetos de TI, desenvolvendo soluções para processos, mapeamento e análise de negócio. Elabora a documentação técnica de especificação de requisitos de softwares e status report para gestão de projetos.*

Ele também relaciona essa profissão às carreiras de analista de negócios de TI, analista de sistemas e analista de informações e complementa o perfil com algumas estatísticas sobre quem desempenha esse papel: ➤

35% têm pós-graduação.



37% têm graduação em Sistemas de Informação.



levaram 1 ano e 11 meses até chegar nesse cargo (a partir do cargo anterior).



56% têm inglês intermediário.

Esse ponto de vista, bastante simples, resume adequadamente uma visão pragmática daquele que desempenha as atividades da Engenharia de Requisitos.

O IREB (2014) nomeia o responsável pelo trabalho de engenharia de requisitos como engenheiro de requisitos e define sua função como alguém que, em colaboração com os interessados ao projeto elicita (levanta), documenta, valida e gerencia requisitos.

1.7. Exercícios

1. Cite exemplos de como a Engenharia de Requisitos está presente em cada uma das cinco etapas de um processo geral de engenharia, como apresentado na Figura 1.1.
2. Explique como o exercício da disciplina de Análise e Projeto e da disciplina de Implementação interagem com o exercício da disciplina de Requisitos na iniciação de um projeto de software. Procure fazer isso relacionando decisões sobre um assunto que influenciam a condução do outro e dando exemplos práticos.

3. O que diferencia a estratégia sequencial e iterativa-incremental do ponto de vista da Engenharia de Requisitos?
4. Nos projetos em que participou, você sabe dizer aproximadamente quanto o trabalho de requisitos corresponde ao esforço total do projeto? Em caso positivo, que atividades você percebe como mais trabalhosas?
5. Qual é a diferença entre “disciplina de requisitos” e “fase de requisitos”?
6. Por que o trabalho de requisitos pode acontecer também em etapas mais avançadas do projeto de software, como a construção e a transição?

2. Requisito

“Os planos não são nada. O planejamento é tudo.”

Dwight D. Eisenhower Este capítulo aborda os diferentes significados do termo requisito e a importância prática de saber reconhecer sua abrangência para o desempenho da Engenharia de Requisitos. Descreve minuciosamente o significado associado à especificação de requisitos, identificando os usuários típicos, os fatores que implicam em um maior ou menor grau de detalhamento e os critérios de qualidade que permitem verificá-la e validá-la.

2.1. Uma palavra, muitos significados

É comum encontrar, nos treinamentos promovidos pelos autores, profissionais de software (experientes e novatos) que consideram o termo “requisito” sinônimo de “documentação com as especificações de requisitos”. Eles não percebem o desenvolvimento dos requisitos como parte integrante da produção de software. A documentação é fruto da necessidade do registro dos resultados do trabalho intelectual, para que a informação não se perca e possa ser confirmada e compartilhada posteriormente.

O tratamento do trabalho de requisitos como sinônimo de documentação falha quando se avalia com maior profundidade a definição de qualidade, como presente em uma variedade de modelos de referência, como a ISO 9000, o *PMBOK® Guide* e o *BABOK®*, todas baseadas em Crosby (1979): “qualidade é aderência aos requisitos”.

Nessa leitura restrita do que seriam requisitos, conclui-se que o produto é de qualidade se suas características estão compatíveis com sua especificação; mesmo que não atenda à necessidade que se propõe. O produto nessa condição não pode ser rejeitado; não deve ser corrigido. Os custos adicionais para atender à necessidade a que o produto se propôs a atender em nada têm a ver com baixa qualidade. Essa leitura na indústria de software tem muito apelo e justifica o investimento na produção de especificações como um fim em si mesmo (gerar papel).

Essa interpretação, muito restrita, entra em colapso quando a própria Engenharia de Requisitos e seus produtos devem ser avaliados quanto à qualidade. Portanto, a definição de requisitos deve ser mais abrangente do que apenas a documentação com as especificações de requisitos.

O objetivo deste tópico é apresentar essa definição de requisitos mais ampla e como ela deve ser usada na Engenharia de Requisitos.

2.1.1. Necessidade

Avaliar a qualidade das atividades e dos produtos da Engenharia de Requisitos está relacionado à definição de qualidade de Juran (1988): “qualidade é a adequação ao uso”.

Adequação ao uso é cumprir com o seu propósito; daí o requisito estar relacionado às necessidades. Muitos desejam a casa própria, mesmo que ela ainda não exista; mesmo assim, a necessidade de segurança existe. O exemplo de um “desejo” é útil, apesar das organizações terem suas necessidades estabelecidas de maneira diferente dos indivíduos. Isso ajuda a entender que o requisito não se restringe à documentação com sua especificação, mas, antes e em seu berço, refere-se às diferentes necessidades que devem ser atendidas. Requisito é então: *(1) Uma condição ou capacidade necessária por um usuário para resolver um problema ou alcançar um objetivo (ISO/IEC/IEEE, 2010).*

2.1.2. Propriedade

Uma vez que alguém adquire sua tão desejada casa, pode-se constatar que esta possui uma área, uma quantidade de quartos, garagem, jardim, quintal

etc.

O termo requisito também se aplica nesse contexto em que se avalia um produto concreto pelas suas características reais. Mais especificamente, em termos de capacidades ou condições que devem ser atingidas, como o código de posturas no município em que está localizado. Daí a segunda parte da definição de requisitos: *(2) Uma condição ou capacidade que deve ser atingida ou possuída por um sistema ou componente de um sistema para satisfazer um contrato, padrão, especificação ou outro documento formalmente imposto (ISO/IEC/IEEE, 2010).*

2.1.3. Especificação

Seja o requisito uma necessidade a ser satisfeita (1) ou uma propriedade de um produto existente (2), ambos os casos podem ter suas capacidades especificadas em um documento. Voltando ao exemplo do imóvel, uma planta baixa, como ilustrada na Figura 2.1, pode representar tanto o imóvel que se deseja construir quanto um já existente.

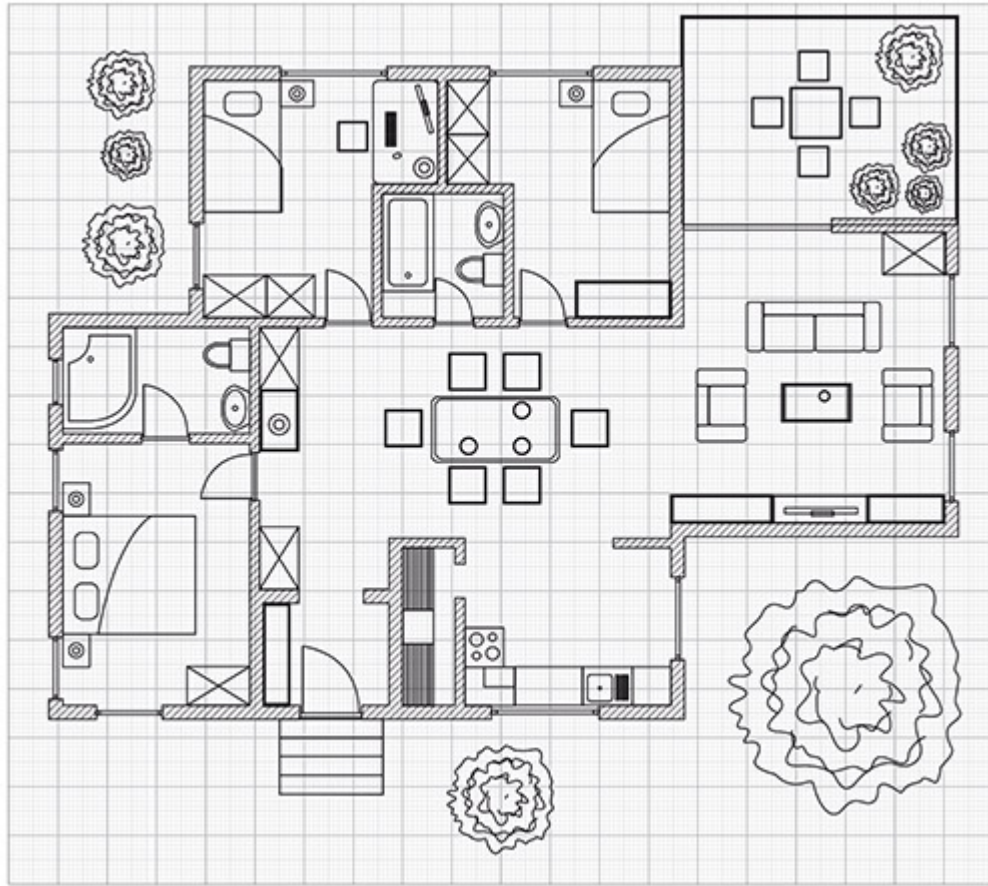


Figura 2.1. Requisitos como documentação de necessidade ou propriedade (crédito: Shutterstock).

Esta é a terceira parte da definição de requisitos: *(3) Uma representação documentada de uma condição ou capacidade como em (1) ou (2) (ISO/IEC/IEEE, 2010).*

Para diferenciar o significado (3) dos significados (1) e (2), convencionou-se o termo “especificação de requisitos” para a representação documentada, em lugar de somente “requisitos”.

2.2. Definição de requisito

A importância dessa definição se torna visível quando, por exemplo, se definem modelos de negócio para a contratação de serviços de desenvolvimento e manutenção de sistemas. Caso essa contratação inclua no escopo de atuação do fornecedor as atividades de desenvolvimento e

gestão de requisitos, é fundamental destacar a primeira parte da definição, onde se coloca ênfase na necessidade e permite que qualidade seja interpretada como adequação ao uso.

É comum encontrar profissionais que, inicialmente, comentam que esse não é o seu caso; que as atividades da Engenharia de Requisitos são todas realizadas pelo cliente quando da emissão de um pedido de proposta e que essa distinção não é tão relevante.

Ao serem questionados sobre quais produtos entregam como especificações, verifica-se que fornecem um escopo definido como macrofunções, embutindo vários itens a definir, por exemplo, quais tarefas dos fluxos operacionais no negócio em específico serão total ou parcialmente informatizadas; qual o comportamento que se espera do software em cada uma dessas tarefas; e quais as restrições que se aplicam ao software no desempenho de suas funções.

Resolver esses itens a definir exige obter informações adicionais por meio de atividades de levantamento junto às partes interessadas; organizá-las e validá-las por meio de atividades de análise. Sem isso, estará se delegando essas decisões ausentes em requisitos para profissionais que estão atuando em outras disciplinas (arquitetura, implementação, testes etc.) e sem necessariamente a competência ou autoridade para o exercício dessas atividades.

Quando esses modelos de negócio são baseados na remuneração em pontos de função (muito comum no mercado brasileiro), a medição baseada apenas em especificações pode provocar a inflação nos resultados porque, ainda que se trabalhe com especificações aprovadas (a terceira parte da definição de requisitos), elas podem não estar em sintonia com a primeira ou segunda parte da definição de requisitos.

2.2.1. Resumo da definição

A definição de requisito discutida neste tópico é fundamental para entender e aplicar a definição de qualidade usada em tantos modelos de referência. Isso porque, considerando que requisito seja também resolver um problema ou atingir um objetivo, aderência aos requisitos já engloba a adequação ao

uso. Sem o conhecimento prévio dessa definição de requisitos, a história é bastante diferente.

Cabe, uma vez explicada passo a passo a definição de requisitos da ISO/IEC/IEEE (2010), apresentá-la em seu inteiro teor:

1. Uma condição ou capacidade do sistema, solicitada por um usuário, para resolver um problema ou atingir um objetivo.
2. Uma condição ou capacidade que deve ser atendida por uma solução para satisfazer um contrato, especificação, padrão ou quaisquer outros documentos formais impostos.
3. Documentação da representação das condições ou capacidades apresentadas nos dois itens anteriores.
4. Uma condição ou capacidade que deve ser alcançada ou possuída por um sistema, produto, serviço, resultado ou componente para satisfazer um contrato, padrão, especificação ou outro documento formalmente imposto. Requisitos incluem as necessidades quantificadas e documentadas, desejos e expectativas do patrocinador, clientes e outras partes interessadas.

Como o leitor pode perceber, a definição (4) consolida, clarifica e complementa as três anteriores que lhe deram origem.

2.3. Especificação de requisitos

A especificação de requisitos é um contrato entre clientes e equipe de desenvolvimento. Ela deve esclarecer aos clientes o que será entregue como produto do trabalho da equipe de desenvolvimento. Esses clientes devem ser capazes de compreender a mensagem e fornecer *feedback* sobre eventuais falhas na especificação, para que estas sejam corrigidas de imediato, antes que trabalho errado seja produzido mais tarde no projeto. O objetivo é que os clientes aprovem de forma consciente a especificação para que o projeto possa seguir com menos riscos de entregar um produto que não os satisfaça.

Logo, o objetivo principal da especificação é documentar de forma fiel e completa todas as necessidades dos clientes e obter um aceite (aprovação)

sobre o que se está propondo entregar em termos de produto. Além disso, a especificação tem por objetivo permitir que a equipe de desenvolvimento consiga compreender exatamente o que os clientes desejam. Ela é, portanto, um importante instrumento de comunicação entre clientes e equipe de desenvolvimento.

Frequentemente, a especificação de requisitos não é um único documento, mas uma composição de vários tipos de documentos. Sejam apresentados como partes da especificação ou como documentos em separado, é comum que uma especificação de requisitos abranja: ➤

Visão geral: cita os objetivos do projeto, principais partes interessadas, um escopo preliminar com uma breve descrição das funções que o sistema deverá desempenhar (exemplo: documento de visão).



Glossário: definição dos termos técnicos (do negócio), sinônimos e acrônimos (siglas) usados ao longo do documento.



Modelos do sistema: mostram o relacionamento entre os componentes do sistema e entre o sistema e seu ambiente (exemplos: diagrama de contexto, diagrama de caso de uso, modelo de processo etc.).



Lista de requisitos funcionais: descreve tarefas e serviços que serão fornecidos pelo sistema aos seus usuários (exemplo: lista de casos de uso, histórias do usuário). Inclui também as interfaces externas do software.



Lista de requisitos não funcionais: descreve as restrições impostas sobre o software e as relaciona aos requisitos funcionais.



Especificação detalhada de requisitos: detalha os requisitos funcionais (por exemplo, especificações de caso de uso, regras de negócio).

Quando se trabalha com metodologias ágeis, uma construção que agrega especificações de requisitos na forma de histórias do usuário é o *Product Backlog*. Ele representa um estoque de requisitos especificados como histórias do usuário e que estão pendentes de incorporação ao produto de software. Cabe destacar que o *Product Backlog* não se limita a conter requisitos e inclui também elementos de *design* entre seus itens.

2.3.1. Especificação de requisitos para quem?

O público-alvo leitor da especificação de requisitos pode ser bem variado. Mas pode-se resumi-lo em dois tipos de leitores principais: clientes e membros da equipe de desenvolvimento. Em relação aos clientes, quase nunca isso abrange uma única pessoa ou somente o usuário final. Um software costuma ter vários tipos de usuário. Um projeto tem interessados em várias unidades organizacionais da empresa. Logo, o desafio é construir uma visão da especificação de requisitos que seja compreendida por essas várias pessoas presentes no lado cliente da história.

Pode-se afirmar com tranquilidade que o processo de elaboração da especificação de requisitos obriga aos vários interessados a refletir de maneira mais rigorosa sobre suas necessidades antes que o trabalho no projeto avance demais, o que reduz o retrabalho posterior de desenho, construção e testes. É parte do processo de elaboração da especificação uma avaliação de qualidade (vide verificação e validação no Capítulo 8), visando identificar omissões, inconsistências e falhas de entendimento.

Já para a equipe de desenvolvimento, são vários papéis interessados. O gerente de projetos usa a especificação como base para o planejamento do projeto e seu acompanhamento. O arquiteto de software usa-a para poder

elaborar a arquitetura do software. Administradores de banco de dados projetam o banco de dados usando a especificação como referência. Analistas de métricas e estimativas têm os requisitos como matéria-prima principal do seu trabalho. Testadores precisam da especificação para poder elaborar casos de teste ou mesmo executar os testes. Documentadores precisam da especificação para produzir manuais, material de treinamento, ajuda *on-line etc.*

Quando um programador usa a especificação como uma referência fundamental na codificação, há duas possibilidades. A primeira é a especificação de requisitos ter sido bem-feita. Neste caso, ele está desempenhando o papel de um projetista, já que a especificação não deve considerar em sua elaboração elementos de projeto necessários à codificação. A segunda possibilidade é a especificação de requisitos ter sido malfeita. É o caso de ela incluir decisões de *design*, como, por exemplo, quais os padrões de projeto (*design patterns*) utilizados para atender às necessidades de informação dos usuários.

Além disso, a especificação cumpre um papel que vai além do projeto: ela serve de base para o trabalho de manutenção futura no sistema, depois que este for entregue.

Se a especificação de requisitos é um contrato entre clientes e desenvolvedores, então ambos obrigatoriamente serão os seus leitores. Porém, o foco maior deve ser sempre no cliente. O documento de requisitos, que é orientado ao cliente, também é útil para a equipe de desenvolvimento. Mas o inverso não é verdade.

Se um dos objetivos da especificação é obter aceite dos clientes sobre o que será entregue no projeto, toda informação apresentada na especificação deve usar a linguagem de negócio dos clientes, não a de tecnologia. Deve-se evitar que a especificação entre em aspectos de implementação do software. Terminologia de TI e enfoque na implementação do software são barreiras à comunicação com os clientes, prejudicando o *feedback* deles sobre o produto que se propõe a entregar.

Nem todas as necessidades de informação da equipe de desenvolvimento serão supridas pelo documento de requisitos. Portanto, outros documentos

de caráter mais técnico poderão ser elaborados, mas sem que a implementação contamine o documento de requisitos.

A experiência dos autores ao analisar projetos em diversas empresas ao longo dos anos constatou que um erro muito comum é elaborar a especificação de requisitos tendo em mente a equipe de desenvolvimento como sua única leitora, chegando ao ponto de incluir trechos de código-fonte! Ou seja, um documento muito útil para o programador, mas sem muita utilidade para os clientes, pois estes dificilmente conseguirão compreender sua mensagem.

2.4. Nível de detalhe da especificação

A falha em não detalhar de maneira suficiente as informações na especificação de requisitos pode levar a interpretações equivocadas ou à criação de um número elevado de premissas. Por outro lado, decidir por detalhar demais a especificação de requisitos pode ser um elemento paralisante do projeto (a especificação nunca é finalizada), além de oneroso. O desafio é encontrar o equilíbrio do nível de detalhe adequado da especificação de requisitos para o seu projeto.

2.4.1. Significado de nível de detalhe

A discussão sobre o nível de detalhe na especificação de requisitos deve ser precedida pelo significado que se pretende dar ao termo. Os níveis de detalhe podem ser subdivididos conforme três objetivos diferentes: ➤

Delimitar o escopo preliminar e definir o escopo final da solução.



Descrever o funcionamento e as restrições de um item no escopo.



Mapear os requisitos para *design* ou implementação.

Dando sequência ao exemplo de reformular a sinalização urbana, várias decisões são tomadas priorizando as regiões com maior potencial para que se atinja o objetivo de melhorar o fluxo de automóveis. Uma dessas regiões, presente no escopo inicialmente definido em termos mais gerais, já possui informação que permite discernir os trechos de rua relevantes (Mapa 2.2). Com isso, permite-se aumentar o nível de detalhe do escopo.

Finalmente, em um terceiro momento, a especificação em sua última versão descreve todo o escopo no nível de detalhe das tarefas do usuário. Todas as decisões sobre o escopo estão tomadas e sabe-se com precisão quais atividades do processo serão incluídas como parte da solução.

Concluindo a comparação com o uso dos mapas, decidem-se quais trechos de rua devem sofrer intervenção. Em se tratando da região metropolitana de São Paulo, o escopo, nesse nível de detalhe, não caberia nesta página.



Mapa 2.2. Visão detalhada de parte da região metropolitana de São Paulo (crédito: adaptado de Google Maps).

O nível de detalhe, no sentido de qualificar o escopo de uma maneira mais geral ou mais específica, busca explorar a abrangência da solução. Para

realizar a transição para outro tipo de nível de detalhe, é melhor usar outra metáfora.

Observe a ilustração do lago (Figura 2.2). Ela permite depreender a visão de sua extensão, mas não de sua profundidade; o mesmo acontece com os requisitos que podem estar detalhados quanto ao seu escopo no nível de atividade (ou de trechos de rua para reformulação de sinalização urbana); contudo, sem explicar o que se espera da solução quanto a uma atividade em particular (ou o que deve ser feito em um determinado trecho de rua).



Figura 2.2. Vista da extensão de um lago (crédito: WolfmanSF, <https://goo.gl/ttZETv>).

2.4.3. Descrever um item no escopo

O conceito de nível de detalhe pode também indicar o grau em que o comportamento da solução e as restrições que se aplicam estão descritos para cada tarefa individualmente. Nesse sentido, a especificação pode variar desde o ponto em que não há nada descrito (apenas se sabe o que faz parte do escopo), até uma descrição completa com todos os passos que descrevem a interação do usuário com a solução; o armazenamento e recuperação de dados; e as regras e restrições que se aplicam.

Nível de detalhe, nesta acepção, refere-se a explorar a solução e as restrições a que está sujeita no âmbito de uma tarefa em particular. Comparando a imagem com a abrangência do lago, agora o interesse está em entender a sua profundidade. O mesmo lago usado no exemplo anterior

está agora ilustrado (Figura 2.3) com uma perspectiva também de sua profundidade.

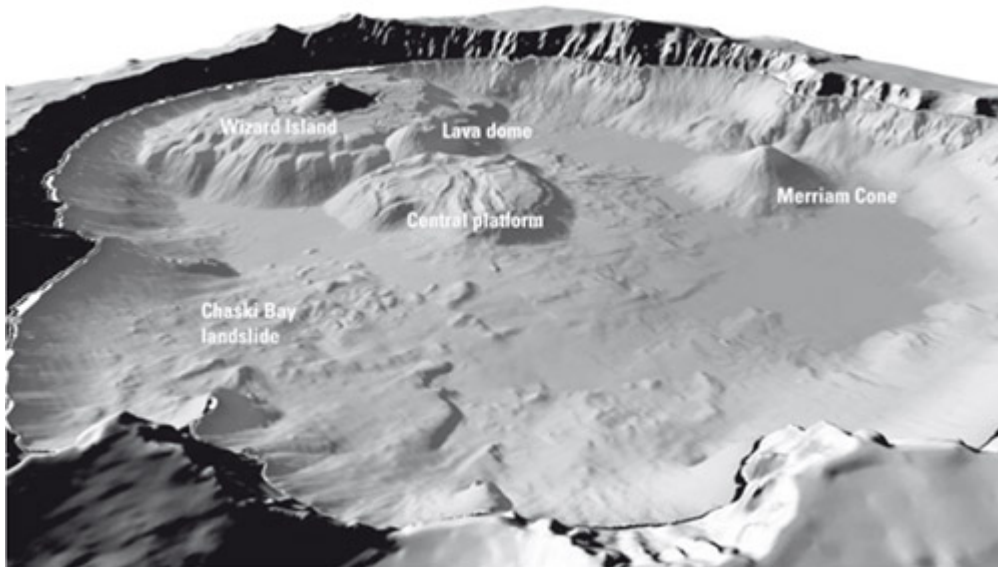


Figura 2.3. Visão também da profundidade de um lago (crédito: United States Geological Survey).

Em resumo, a Engenharia de Requisitos utiliza-se inicialmente de um escopo descrito de uma maneira geral. Isso em função do nível de incerteza associado à solução, que dá espaço ainda para diversas decisões sobre a consolidação desse escopo (quais tarefas serão afetadas pela solução informatizada, por exemplo) e sobre a descrição de cada item nesse escopo (qual o comportamento que se espera da solução ao interagir com seus usuários, por exemplo). A dinâmica dessa evolução é representada pela Figura 2.4.



Figura 2.4. Evolução dos níveis de detalhe conforme evolui o desenvolvimento dos requisitos.

2.4.4. Mapear os requisitos no *design* ou implementação

O mapeamento dos requisitos para uma determinada arquitetura ou sua implementação em uma linguagem de programação, ainda que seja um significado válido para mais detalhe, deve ser evitado no âmbito da Engenharia de Requisitos. Fazer isso significa sair do escopo da disciplina de requisitos e entrar no escopo de outra disciplina (por exemplo, *design*). Um exemplo é alocar o comportamento descrito nos requisitos em um: ➤

- componente de software na camada de apresentação, o intercâmbio de dados com o usuário; ➤

- componente de software na camada de persistência, o armazenamento e recuperação de dados; ➤

- sistema gerenciador de regras de negócio (*Business Rule Management System* – BRMS), as regras.

Decisões prematuras de arquitetura ou construção podem levar a uma solução final não tão boa.

2.4.5. Equívoco comum ao detalhar

É um equívoco comum pensar que quanto mais detalhada a especificação de requisitos, melhor. Como um contrato entre cliente e desenvolvedores, o nível de detalhe deve ser o que melhor consiga promover a comunicação de ambas as partes.

Se você observar os contratos que firmou – muitos sem nem mesmo notar –, perceberá vários bem enxutos, ocupando somente uma folha, e outros bem extensos, de dezenas ou mesmo centenas de folhas. Há contratos que representam negócios de pouco valor (ingresso de cinema) e outros de muito valor (financiamento de uma casa). Então, quanto mais significativo o valor envolvido, mais detalhado tende a ser o contrato. Mas este não é o único fator a influenciar o detalhamento.

Quando se compra um carro, há opção de ir a uma loja e financiá-lo. Isso irá gerar um contrato com várias páginas. No entanto, se o dono da loja for

um amigo de longa data, é possível que a transação aconteça com base somente em um acordo verbal (que não deixa de ser um contrato). O valor do item é o mesmo nas duas situações.

Sendo relacionamentos pessoais, o que determina o maior ou menor detalhamento do contrato é o grau de confiança entre as duas partes. A Engenharia de Requisitos é aplicada em ambientes corporativos que transcendem interesses de um indivíduo e não deveriam ter suas decisões baseadas somente em termos de confiança e desconfiança. Imagine a prestação de contas de um executivo ao conselho de administração sobre o fracasso de um projeto apresentando como explicação o fato de que a empresa contratada traiu sua confiança.

Nesse paralelo feito entre os contratos por indivíduos e os contratos no plano de corporações, confiança surge como uma representação de risco. Logo, quanto menor o risco avaliado na relação entre cliente e desenvolvedores, menor a necessidade de detalhamento da especificação de requisitos. Quanto maior o risco avaliado na relação entre as partes, maior a necessidade de detalhamento.

O Manifesto Ágil (BECK et al., 2001) prega: “colaboração com o cliente mais que negociação de contratos” e no seu Princípio 4: “pessoas de negócio e desenvolvedores devem trabalhar diariamente em conjunto por todo o projeto”. São diretrizes que favorecem a construção de confiança entre as partes, e com isso reduzem a necessidade de detalhamento da especificação.

Um documento que melhor permite depreender a importância da tradução de confiança em um plano pessoal para riscos em ambiente com exigências de transparência e governança corporativa é o Acórdão nº 2.314/2013 do plenário do Tribunal de Contas da União (TCU, 2013). Ele reúne os riscos elencados em três grupos distintos (processos, produtos e pessoas) e não se limita aos riscos cuja materialização foi verificada nas auditorias, como, por exemplo, o risco da rotatividade de pessoal verificado em um ambiente que fomenta a pessoalidade.

É importante que se reflita sobre quão detalhada será a especificação de requisitos, pois há custo e tempo envolvidos nisso. Detalhar desnecessariamente significa desperdício de recursos no projeto. Ausência

de detalhes pode significar interrupção de atividades seguintes do projeto e introdução de erros por falta de informação – enfim, retrabalho.

2.5. Critérios para nível de detalhe da especificação

Este é um assunto tão crítico que deve ser pensado em uma abrangência não só do projeto em questão, mas dos processos; seja em políticas de qualidade para o desenvolvimento interno de software, seja em modelos de negócio para a contratação de serviços no mercado (desenvolvimento externo).

Caso não haja uma definição prévia do nível de detalhe nos processos, o gerente de projeto deve estabelecer essa política para o projeto a partir de potenciais elementos de risco. Se os detalhes sobre os requisitos não forem especificados pelos analistas de requisitos, em algum momento serão definidos por outros. Em última instância, os desenvolvedores tomarão essas decisões, pois o software não admite ambiguidade. Quanto menos detalhes, maior a autonomia dos desenvolvedores nessas decisões e vice-versa. Por isso, há que se balancear custo e riscos.

Antes de definir o nível de detalhe em que a especificação de requisitos deve ser elaborada, deve-se: ➤

definir com clareza o contexto em que ela será usada; ➤

decidir sobre quais tipos de informação devem estar presentes; ➤

avaliar os riscos conforme os fatores que podem implicar em mais ou menos detalhe (vide tabela seguinte).

A seguir apresenta-se uma lista (não ordenada) de fatores que podem influenciar na escolha do nível de detalhe adequado.

Tabela 2.1. Fatores para o nível de detalhe da especificação de requisitos.

Nº	Menos detalhe na especificação	Mais detalhe na especificação
1	Desenvolvimento interno	Desenvolvimento externo
2	Equipe agrupada	Equipe dispersa

Nº	Menos detalhe na especificação	Mais detalhe na especificação
3	Não elabora casos de testes ou a elaboração ocorre em paralelo aos requisitos	Casos de testes elaborados a partir da especificação
4	Estimativas menos precisas	Estimativas mais precisas
5	Baixa rastreabilidade de requisitos	Alta rastreabilidade de requisitos
6	Alto envolvimento dos clientes	Baixo envolvimento dos clientes
7	Alto conhecimento da equipe sobre o negócio	Baixo conhecimento da equipe sobre o negócio
8	Precedentes existentes	Sem precedentes
9	Desenvolvimento concorrente de procedimentos operacionais	Desenvolvimento com procedimentos operacionais já definidos e maduros
10	Uso de pacotes na solução	Solução não adotará pacotes
11	Baixa expectativa de rotatividade de mão de obra	Alta expectativa de rotatividade de mão de obra

2.5.1. Desenvolvimento interno ou externo

Quando o desenvolvimento do projeto é realizado internamente na empresa, a equipe do projeto e os clientes são colegas de trabalho e compartilham interesses comuns. Em geral, muitos já se conhecem antes do projeto e podem ter até uma relação próxima. Além disso, normalmente trabalham no mesmo espaço físico, o que facilita o encontro e diminui a necessidade de formalismo nas interações. Ou seja, a interação face a face é mais fácil e frequente.

Para o desenvolvimento externo do projeto, a equipe e os clientes são de empresas distintas e têm interesses distintos. Não são colegas de trabalho. Se o desenvolvimento ocorre no regime de fábrica de software, boa parte da equipe do projeto tem contato limitado com os clientes. Clientes e equipe do projeto estarão longe fisicamente. Dependendo da fábrica de software, boa parte da equipe do projeto pode estar localizada em outra cidade, outro país ou outro continente (a Índia, com sua presença forte em serviços de *outsourcing*, é o exemplo clássico). Com menos possibilidade de interação

face a face, é necessário mais detalhamento dos requisitos para minimizar eventuais problemas de comunicação.

2.5.2. Equipe dispersa ou agrupada geograficamente

De certa forma, este item tem relação com o anterior. Só que, no caso anterior, a ênfase foi no desenvolvimento interno ou externo como fator que pode favorecer ou não a proximidade entre clientes e desenvolvedores.

No entanto, neste item o que se considera é a dispersão entre os membros da equipe do projeto. Hoje o trabalho remoto é uma realidade comum em muitas empresas. Várias trabalham seus projetos com times virtuais. Há vantagens na abordagem do trabalho remoto – por exemplo, mais facilidade de obter pessoas mais competentes para a equipe. No entanto, a falta de convivência entre os membros da equipe é um fator que dificulta a formação do senso de equipe entre as pessoas e uma barreira de comunicação em potencial.

O *PMBOK® Guide* apresenta o agrupamento dos membros da equipe em um mesmo espaço físico como uma estratégia de aprimoramento de sua capacidade de atuação como equipe. Isso promove a comunicação face a face, o que dispensa tanto detalhamento na especificação. Em caso de dúvida, basta perguntar ao colega do lado. O Manifesto Ágil também reforça isso no seu Princípio 6: “o método mais eficiente e eficaz de transmitir informações para e entre uma equipe de desenvolvimento é através de conversa face a face”.

2.5.3. Casos de testes baseados nos requisitos

A princípio este item soa estranho. Normalmente o que seria comum imaginar são os casos de testes sendo elaborados a partir da especificação de requisitos. Porém, nem sempre essa é a estratégia adotada no desenvolvimento do projeto. Há casos onde os critérios de aceitação já estão definidos e vão orientar os testes. Neste caso, é o trabalho de requisitos que usa essas informações como matéria-prima do seu trabalho.

Há casos também onde a equipe de testes participa do trabalho de requisitos para a elaboração dos casos de testes. Nesses casos, tanto o analista de

testes quanto o analista de requisitos estão usando a mesma fonte de informação; só que, enquanto o primeiro elabora os casos de testes, o segundo elabora a especificação de requisitos. Não há ordem de precedência entre os dois trabalhos.

Quando a especificação de requisitos é matéria-prima para a equipe de testes elaborar seus casos de testes e depois executá-los, certamente há mais necessidade de detalhes na especificação. A elaboração dos casos de testes a partir da especificação é uma ferramenta muito poderosa de verificação de requisitos, ajudando a melhorar a sua qualidade.

Os autores já presenciaram o desenvolvimento do software descolado da elaboração de suas especificações de requisitos em órgãos do governo. Enquanto o fornecedor mobilizava uma equipe para produzir documentação (em tese, a especificação de requisitos) para atender às exigências de entrega do contrato, outros desenvolvedores atuavam simultaneamente direto com os usuários para obter as informações para a construção do software.

Neste caso, a documentação produzida não agregava valor algum além da conformidade às exigências do contrato. O trabalho em sua elaboração era meramente burocrático porque não necessariamente havia nexos entre as especificações de requisitos e o produto propriamente dito. O verdadeiro trabalho de requisitos acontecia na equipe que atuava diretamente com os usuários.

2.5.4. Grau de incerteza das estimativas

Toda estimativa envolve um grau de incerteza por definição. A incerteza nas estimativas está diretamente relacionada ao grau de maturidade e estabilidade dos requisitos. A maturidade refere-se à relação entre decisões sobre requisitos pendentes e que já foram tomadas; estabilidade refere-se à probabilidade de mudanças em requisitos já definidos.

Quanto mais cedo se necessita de uma estimativa, menor a chance dos requisitos estarem maduros ou estáveis. Quando um projeto está em análise de viabilidade, é pouco provável que haja espaço para uma especificação de requisitos detalhada. Para esses propósitos, normalmente trabalha-se com

uma estimativa de ordem de grandeza baseada nos requisitos existentes cujo nível de detalhe em geral é associado a uma declaração de escopo preliminar. No entanto, se há a necessidade de se obter uma estimativa definitiva, haverá a exigência de mais informações e detalhes na especificação de requisitos.

2.5.5. Rastreabilidade dos requisitos

Alguns tipos de projeto implicam em alto grau de evidências que relacionam os requisitos entre si e entre artefatos de outras disciplinas na engenharia de software; por exemplo, sistemas de missão crítica. Para a indústria aeronáutica, existem certificações para software de missão crítica. Um caso é o padrão DO-178C, que exige rastreabilidade ao ponto que cada linha de código-fonte esteja diretamente rastreada para um requisito e um caso de teste. Logo, nesses casos há a necessidade de requisitos mais detalhados.

2.5.6. Envolvimento dos clientes

O grau de envolvimento dos clientes no projeto é um fator crítico para o seu sucesso. E também influencia o nível de detalhe dos requisitos. Para o cliente que está altamente envolvido e participativo no desenvolvimento do projeto, uma consequência natural é a construção de uma relação de confiança mais forte com a equipe de desenvolvimento. Nesse ambiente, também a comunicação face a face entre as duas partes será mais frequente. Tudo isso favorece para que não seja necessária uma especificação tão detalhada. E este é o objetivo buscado no Princípio 4 do Manifesto Ágil (BECK et al., 2001): “pessoas de negócio e desenvolvedores devem trabalhar diariamente em conjunto por todo o projeto”.

2.5.7. Conhecimento da equipe no domínio

O provérbio “para bom entendedor, meia palavra basta” encaixa-se bem aqui. Quando a equipe do projeto tem experiência de outros projetos no negócio do cliente, muitos detalhes são desnecessários de se especificar. O que não pode ocorrer (mas infelizmente ocorre com frequência) é a equipe

do projeto pensar que já tem toda a solução para o cliente sem antes ouvi-lo atentamente sobre suas necessidades e as de seu negócio.

No desenvolvimento interno, é comum que a mesma equipe execute vários projetos para uma mesma área de negócio. Então, com o passar do tempo essa equipe acumula conhecimento sobre esse domínio, podendo conversar no mesmo nível de conhecimento do negócio com os clientes. Há profissionais que passam a conhecer tanto do negócio do cliente que deixam suas posições em projetos de software e passam a atuar como gestores no negócio.

No desenvolvimento externo, há fornecedores que atuam exclusivamente com projetos em áreas de negócio muito específicas (ou verticais): bancária, telecomunicações, energia, mineração *etc.* Isso também favorece que mantenham uma equipe com experiência no negócio, facilitando a comunicação com os clientes.

Há também fornecedores que atuam sem tanta especialização, desenvolvendo projetos para qualquer área de negócio em que surgir uma oportunidade. Nesse caso, hoje uma equipe pode estar atuando em um projeto da área de saúde, mas depois seus membros podem ser alocados em projetos de outras áreas – por exemplo, logística. Portanto, os responsáveis pelo trabalho de requisitos nessas equipes devem estudar previamente o negócio do cliente para conseguir um nivelamento mínimo para a comunicação eficaz. Ainda assim, será necessária uma especificação mais detalhada para que toda a equipe consiga compreender bem o que deve ser construído e entregue.

O modelo de estimativa COCOMO II coloca que a experiência do analista com a aplicação e o domínio em que ela se insere tem efeitos multiplicativos na produtividade, que pode implicar desde uma redução na produtividade em 22% ao seu aumento em 19% em relação a uma experiência de um ano com a aplicação. Ainda que o estudo não entre no mérito das causas subjacentes dessa constatação, pode-se atribuir esses ganhos de produtividade à maior agilidade associada ao desenvolvimento e à gestão dos requisitos.

2.5.8. Trabalho com características similares a anteriores

Nas situações em que há precedentes existentes ao projeto que será desenvolvido, a necessidade de detalhamento torna-se menor. Precedentes podem ser projetos muito similares que já foram desenvolvidos e que podem servir como fonte de informação para o projeto que será desenvolvido. Exemplos: ➤

Reengenharia de um sistema existente (*refactoring*): desenvolver novamente um sistema legado em uma nova tecnologia. Neste caso, o próprio legado ou sua documentação pode suprir muito dos detalhes necessários dos requisitos para o novo projeto.



Manutenções similares em sistemas distintos: um caso é um projeto para adequar o sistema de poupança de um banco para uma determinação do Banco Central, porém já se sabe que uma manutenção similar já foi feita para o sistema de contas correntes. Assim, muitas das decisões adotadas na primeira manutenção podem servir para o novo projeto.

2.5.9. Desenvolvimento concorrente de procedimentos

Há casos em que o desenvolvimento dos requisitos está inserido em um ambiente em que, além do software, há o desenvolvimento de novos procedimentos operacionais. Nesses casos, é possível perceber de antemão que o projeto estará sujeito a uma grande quantidade de solicitações de mudança em requisitos. Isso porque não há ainda um processo de negócio definido ou a sua definição ainda não está madura; restam várias decisões a serem tomadas e estas ainda estão em discussão junto aos gestores das áreas envolvidas.

Nesses casos, tentar produzir uma especificação mais detalhada será um desafio. Manter essa documentação detalhada atualizada e consistente após cada mudança implica em muito retrabalho. Então, cabe a pergunta: para

que detalhar algo ainda volátil que sofrerá mudanças pouco depois? Se não houver razão forte o suficiente para isso, busque uma especificação menos detalhada.

2.5.10. Solução usará pacote

Se o projeto tem por objetivo oferecer como parte da solução a adoção de um pacote de software disponível no mercado, o trabalho de requisitos não precisa se aprofundar em tantos detalhes acerca da profundidade e deve colocar o seu foco no escopo. Se o produto já existe, não faz sentido especificar detalhes de suas características. Isso porque o que se espera é que a organização cliente adeque os seus procedimentos às melhores práticas embutidas no pacote. A eventual necessidade de customização do pacote deveria ser restrita a até 15% das suas funcionalidades.

Os requisitos referentes à avaliação de um pacote devem permitir avaliar as necessidades de sua adequação às necessidades do cliente e o grau de manutenção que este pacote deve sofrer. A Figura 2.5 ilustra a comparação entre os requisitos do produto e do negócio indicando a interseção entre os dois conjuntos.

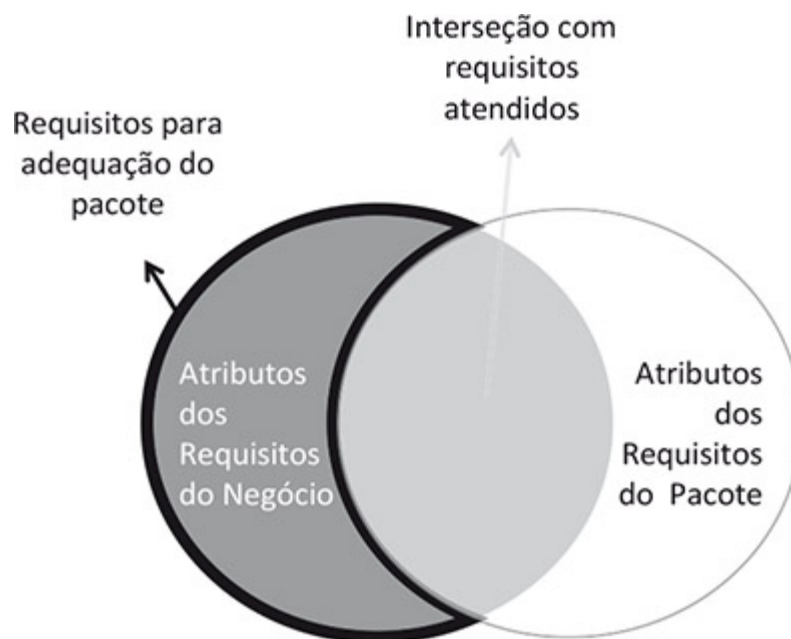


Figura 2.5. Visão geral do posicionamento dos requisitos na avaliação de pacotes.

Quanto à lacuna existente entre os requisitos do negócio e do pacote, há diferentes soluções que podem mudar as práticas da organização para se adequar às melhores práticas presentes no pacote ou modificar as práticas do pacote para adequá-las às práticas da organização. Para este último item, ele consiste de (Figura 2.6): ➤

Configuração: é a adequação do pacote por meio de instrumentos nativos que permitem adequar o funcionamento ao requisito do cliente sem necessidade de desenvolvimento ou codificação adicional.



Customização: são mudanças em código-fonte para adequar funcionalidade que não está disponível por meio de configuração.



Manutenção na base instalada: são adaptações nos sistemas existentes para acomodar o intercâmbio de dados com o pacote em avaliação.

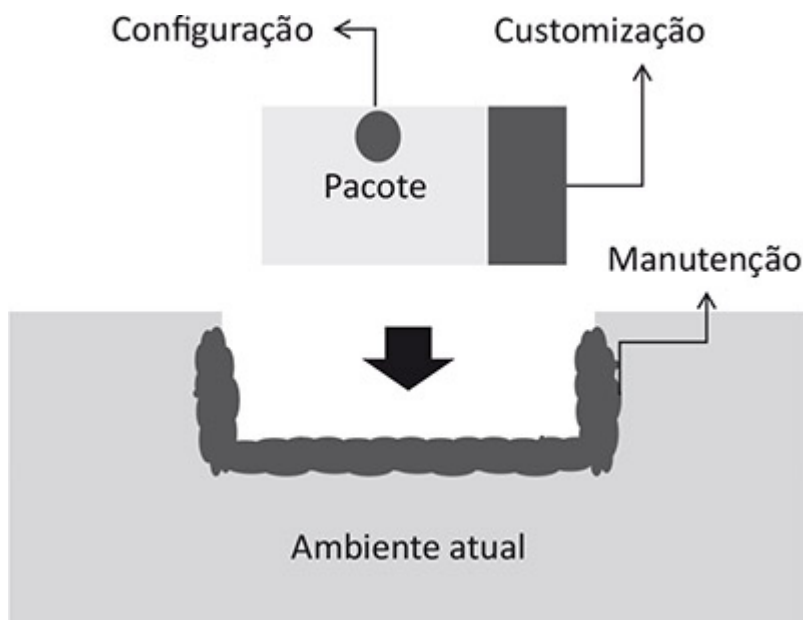


Figura 2.6. Tipos de ação relacionados à adequação do pacote à organização.

Dubey (2009) recomenda que cerca de 80% da funcionalidade do pacote deve estar aderente aos procedimentos operacionais atuais na organização ou pela sua adequação às melhores práticas embarcadas no software. Também coloca que não mais que 15% sejam objeto de customização. Os requisitos para avaliação de pacotes devem estar em um nível de detalhe suficiente para suportar decisões como essas.

2.5.11. Expectativa de rotatividade de mão de obra

Rotatividade de mão de obra é um grande desafio para a gestão do conhecimento nas empresas. Sempre que uma pessoa da equipe sai, algum conhecimento se perde, em maior ou menor grau. A documentação de requisitos torna-se então uma ferramenta importante para a retenção desse conhecimento.

Considerando que há empresas que vivenciam a rotatividade de pessoal com mais intensidade que outras, cabe ao gerente de projetos em última instância avaliar a probabilidade de perda ou troca de pessoal na equipe do projeto e também no lado do cliente, para definir um nível de detalhe na documentação que consiga ajudar a mitigar esse risco.

Como regra geral, quanto maior a rotatividade de pessoal, maior o detalhamento. Menos rotatividade, menos necessidade de detalhes.

2.6. Critérios de qualidade da especificação

A primeira questão que deve ser abordada quando se fala da qualidade da especificação de requisitos é: não existe especificação de requisitos perfeita! Este é o item 10 das “verdades universais” sobre requisitos de software de Wiegers (2006). A busca pela perfeição é infinita, e os projetos têm recursos limitados, principalmente tempo. Então o analista de requisitos deve ser extremamente objetivo e ter foco para trabalhar na elaboração de uma especificação de requisitos que seja boa o suficiente para o contexto no qual o projeto será desenvolvido.

É a velha máxima em ação: “o ótimo é inimigo do bom”. Uma boa especificação de requisitos deve conseguir atender a dois objetivos: ➤

Ajudar os clientes a descrever com exatidão o que desejam obter do projeto.



Ajudar os desenvolvedores a entender exatamente o que os clientes querem.

Uma consequência disso é que a especificação de requisitos deve evitar abordar questões relativas ao *design* ou implementação do software. Embora possam ser questões que ajudem os desenvolvedores, elas prejudicam o entendimento pelos clientes. Questões relativas à implementação devem ser abordadas por outras disciplinas da engenharia de software e nos seus artefatos específicos.

Outro problema de abordar temas relativos à implementação na especificação de requisitos é que isso acaba forçando decisões prematuras de implementação que podem impor restrições desnecessárias à solução. Mais à frente essas decisões podem se revelar não sendo as melhores, mas os arquitetos e desenvolvedores que estão usando a especificação como fundamento não terão a liberdade de propor abordagens melhores.

Deve-se destacar que há decisões anteriores ao *design* e à implementação que, se tomadas, devem, sim, fazer parte da especificação de requisitos. Por exemplo: hoje existem dezenas de navegadores de internet. Deixar para os responsáveis pelo *design* ou implementação a decisão sobre suportar todos (o que implica um maior custo) ou suportar apenas um (o que restringe a base de usuários) não é possível porque não é a eles que competem essas decisões de custo ou de mercado.

Os critérios de qualidade mais comuns para uma boa especificação de requisitos são fornecidos pela norma IEEE 830, que destaca as seguintes características: ➤

Correta.



Completa.

☐

Clara.

☐

Consistente.

☐

Modificável.

☐

Priorizada.

☐

Verificável (ou testável).

☐

Rastreável.

2.6.1. Correta

Uma especificação de requisitos é correta quando cada requisito especificado ajuda a satisfazer ao menos uma necessidade ou demanda legítima do negócio presente nos objetivos do projeto. O vínculo (ou rastreabilidade) dos requisitos na especificação para as necessidades de negócio facilita a verificação se todos os requisitos estão corretos. Um requisito presente na especificação que não tem relação alguma com nenhuma necessidade de negócio é um requisito incorreto, ainda que esteja muito bem redigido ou modelado.

Uma especificação correta não sofre de dois problemas graves na gestão de escopo de um projeto, *scope creep* e *gold plating*, pois não conterà

requisitos supérfluos.

2.6.2. Completa

Todos os elementos significativos do contexto de interesse (o domínio do problema) devem ser descritos. Exemplos: ➤

Funcionalidades, aspectos de qualidade, restrições de projeto e interfaces externas.



Definição de todo comportamento de resposta para cada tipo de entrada possível para o software.



Rótulos e referências para todas as figuras, tabelas e diagramas presentes na especificação.

O vínculo dos requisitos da especificação para requisitos de mais alto nível também ajuda a detectar partes incompletas da especificação. Na prática, nenhum requisito de mais alto nível (o último nível são os requisitos de negócio) deveria ficar sem ao menos um requisito correspondente em uma especificação mais detalhada.

Toda especificação com partes “a definir” é incompleta. Isso é aceitável enquanto a especificação encontra-se em elaboração. Porém, é inadmissível quando a especificação é considerada finalizada. A Figura 2.7 ilustra dois diferentes conjuntos com os requisitos corretos e os requisitos especificados. A interseção entre os dois representa o subconjunto dos corretos e especificados; os demais requisitos especificados são supérfluos. Quanto ao restante dos requisitos corretos (mas não especificados) podem indicar omissões ou apenas uma questão de priorização.

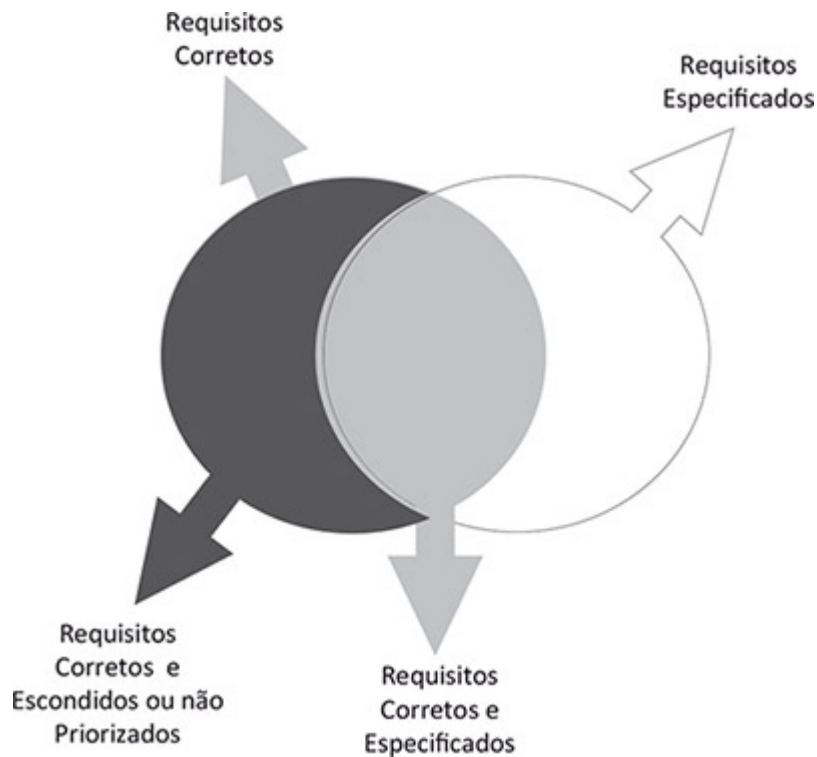


Figura 2.7. A área escura indica os requisitos corretos, porém não especificados (incompletos). A área em branco são os requisitos especificados e supérfluos (incorretos).

Jones (2012) comenta que quanto maior o tamanho do software, menos completa é a sua especificação.

Tabela 2.2. Tamanho e completeza da especificação de requisitos de acordo com o tamanho do sistema.

Tamanho do software (em pontos de função)	Páginas de requisitos por ponto de função	Total de páginas de requisitos	Grau de completeza da especificação
100	1,15	115	95%
1.000	0,75	750	80%
10.000	0,60	6.000	60%

2.6.3. Clara (não ambígua)

Uma especificação de requisitos é clara quando não possui (ou minimiza) ambiguidade. Em termos práticos, ela possui uma única interpretação para todo o público-alvo.

Na maioria das vezes, usa-se a linguagem natural para descrever requisitos; porém, ela é inerentemente ambígua. Isso exige atenção e disciplina do autor da especificação, para que escreva pensando com a cabeça de cada leitor do seu público-alvo, de tal forma que todos consigam um entendimento homogêneo.

É comum haver termos com mais de um significado. Isso faz parte da nossa linguagem e do vocabulário de muitas áreas de negócio. Nessa situação o uso do glossário no projeto, definindo os termos e cada um dos seus significados para o domínio do projeto, torna-se uma ferramenta fundamental para diminuir a ambiguidade.

O extremo oposto de uma especificação de requisitos clara são os horóscopos (sem querer ofender ou polemizar com os que acreditam em astrologia).

No instante em que este tópico era escrito, consultou-se um portal sobre o horóscopo de um dos autores, que dizia: “nesta fase de mudanças importantes em sua carreira, você deve ficar atento a novos projetos que podem trazer novas oportunidades para grandes passos. Deixe o Universo mostrar o caminho que deve seguir”. Você não sabe qual o signo do autor ou quando isso aconteceu; mas é bem possível que, independentemente do seu signo, tenha considerado a previsão válida.

Uma das coisas que torna o tema horóscopo atraente para muitas pessoas é exatamente a alta possibilidade de a pessoa ler uma previsão e “enxergar-se dentro dela”. São textos muito genéricos e ambíguos, que comportam uma variedade muito grande de interpretações.

E há muitos analistas de requisitos escrevendo especificações que se parecem muito com horóscopos! Porém, para o desenvolvimento de software isso é um problema grave, pois criam nas partes interessadas (inclusive a própria equipe de desenvolvimento) diversas interpretações do que o software fará. Ocorre que o software a ser entregue terá um comportamento específico, ou seja, atenderá a uma única interpretação dos requisitos. Wiegers (2006) chama a atenção para isso no item 9 de suas “verdades universais” sobre requisitos de software: “o requisito pode ser vago, mas o produto será específico”. Como várias interpretações são

geradas e somente uma será entregue, cria-se um problema grave de satisfação com o cliente.

Uma especificação de requisitos é na essência um texto como outro qualquer, portanto as boas práticas de redação aplicam-se aqui também. No entanto, percebe-se uma deficiência significativa na formação dos novos profissionais de TI quanto à competência de redação. Sem que o analista tenha uma boa habilidade de escrita, é difícil produzir uma especificação de qualidade.

Além disso, podem-se destacar outras dicas para uma especificação de requisitos mais clara: ➤

Usar modelos e diagramas para compor sua especificação. Além de oferecer uma informação mais amigável para leitura (imagem em lugar de texto), cada tipo de modelo adota uma estrutura formal para sua notação que também elimina ambiguidade.



Identificar de maneira única cada requisito. O mais comum é criar um atributo para o requisito, o seu identificador, que cumpre esse papel. Sem uma identificação única de cada requisito, qualquer referência a um determinado requisito pode suscitar entendimento de uma referência a outro requisito similar.



Adotar uma estrutura padrão e de fácil leitura para sua especificação. Assim, partes interessadas que já participaram de outros projetos terão familiaridade com o documento de requisitos. E o que é uma estrutura de fácil leitura? É aquela que permite a leitura seletiva do conteúdo que é relevante para determinada parte interessada, por exemplo. Se todas as partes interessadas necessitam ler toda a especificação para conseguir obter uma informação específica, significa que ela está mal estruturada.



Minimizar o uso de cláusulas condicionais complexas e sentenças longas.



Não assumir que o leitor tenha conhecimento sobre o domínio.



Usar terminologia consistente e que seja familiar às partes interessadas que revisarão ou utilizarão os requisitos.



Evitar termos indeterminados, como “etc.”, “entre outros”, “talvez”, “em geral”, “se possível”, “poderia”. Quando não houver alguma definição pendente da informação, prefira “a definir”.



Evitar termos subjetivos ou vagos, como “pequeno”, “grande”, “amigável”, “flexível”, “portável”, “razoável”, “bom”, “ruim”, “intuitivo”.



Organizar os requisitos em grupos e subgrupos de forma a refletir relações causais ou temporais entre eles, ou por temas com afinidade, como em um livro. Exemplo: organizar os requisitos de acordo com uma sequência de processamento – recepção de imagem, processamento de imagem, exibição de imagem e tratamento de imagens de baixa qualidade.



Use exemplos! Exemplos são ótimas formas de esclarecimento. Ajudam a tornar visíveis ambiguidades (quando a definição e o

exemplo conflitam) e oferecem uma forma mais fácil de entender uma definição.

Cuidado ao descrever requisitos usando quantificadores universais (usualmente identificados nas palavras como: todos, nenhum, algum, nunca, sempre), pois nem sempre são usados corretamente. Por exemplo, veja o requisito “o sistema deve enviar uma notificação ao administrador para todas as tentativas de login sem sucesso, com todos os dados relativos a esta tentativa”. Será que a notificação deve ocorrer para todas as tentativas? Isso não poderia resultar em centenas ou milhares de notificações ao administrador em um dia? Quais são os dados que devem ser enviados especificamente quando se diz “todos”? Quanto ao local de origem da tentativa, bastaria informar o endereço IP ou deve-se informar a possível região geográfica?

2.6.4. Consistente

Especificação consistente é aquela que não possui contradições entre suas partes, seja no mesmo nível ou em níveis diferentes. Exemplos de contradições: ➤

Há conflitos temporais entre requisitos: REQ-03 indica que o evento A precede o evento B; REQ-12 indica que os eventos A e B são simultâneos.



Dois requisitos utilizam nomes diferentes para o mesmo objeto do mundo real.



Um requisito tem o objetivo de manter um cadastro de clientes e outro requisito cita que dados de clientes deverão sempre ser consultados em outro sistema.

Quando há mais de uma pessoa elaborando a especificação, a chance de inconsistência aumenta bastante. E é também muito comum a

inconsistência surgir de solicitações de mudanças mal assimiladas na especificação de requisitos. Para minimizar a possibilidade da introdução de contradições na especificação ao modificar requisitos, é importante outra característica de qualidade na especificação, que é ser modificável.

2.6.5. Modificável

Esta característica significa que modificações podem ser feitas de maneira fácil, completa e consistente sem comprometer a estrutura e o estilo da especificação. Isso minimiza a possibilidade de erros serem cometidos na especificação por conta de uma mudança malfeita (partes que deveriam ser modificadas, mas foram esquecidas).

Quer ver um exemplo de especificação que peca nesse quesito? Chega uma solicitação de mudança no projeto determinando que o sistema a ser desenvolvido disponibilize cupons de desconto como mais uma forma de pagamento. O analista de requisitos ao tomar conhecimento disso leva sua mão à testa: “puxa, vou ter que alterar uns vinte casos de uso!”.

Quando a especificação não permite mudanças com facilidade, há um esforço significativo do analista em “varrer” a documentação procurando que pontos precisam ser ajustados para adaptá-la à mudança solicitada. Aqueles que têm um olhar de “águia” vão conseguir identificar todos os pontos (ou quase todos, e este é o perigo) que precisam ser alterados. Mas existe um universo significativo de pessoas que não possuem esse olhar tão detalhista e que vão deixar passar despercebidas diversas partes que precisariam ser alteradas, deixando dessa forma a especificação inconsistente.

Algumas dicas para uma especificação modificável são: ➤

- Ter uma organização coerente e fácil de usar que inclua, por exemplo, uma tabela de conteúdo, um índice remissivo, glossário e um controle de alterações.



- Evitar redundância (ou seja, não repetir informação nem descrever o mesmo requisito mais de uma vez).



Expressar cada requisito separadamente em vez de combinado com outros requisitos.



Estabelecer a rastreabilidade entre os requisitos (isso é tema de um tópico do Capítulo 9).

Vale comentar que o glossário ajuda a evitar redundância na especificação, concentrando a definição de termos em um só lugar e facilitando futuras mudanças na especificação.

Para o problema apresentado no exemplo do início deste tópico, a dificuldade poderia ser evitada se, ao especificar os casos de uso, o analista percebesse que há passos ou regras comuns a vários casos de uso (relativos à forma de pagamento) e que poderiam ser fatorados em um caso de uso específico ou um documento de especificação de regras. Neste caso a mudança afetaria somente essa parte da especificação. Fica mais simples de atualizar e com menos risco de deixar algo inconsistente para trás.

2.6.6. Priorizada

A especificação é priorizada quando cada requisito recebe um valor de importância relativa com base em um ou mais critérios – por exemplo: risco, valor para o negócio e custo. O objetivo principal da priorização é assegurar que o esforço do projeto será focado nos requisitos mais críticos, reduzindo assim os riscos do projeto. De forma mais específica, permite: ➤

- que se identifiquem os requisitos que devem ser analisados primeiro; ➤

- que se planeje quais requisitos serão implementados primeiro; ➤
- estimar quanto tempo ou atenção serão destinados aos requisitos.

A tarefa de priorização será abordada com mais ênfase no Capítulo 9.

2.6.7. Verificável (ou testável)

A especificação verificável é aquela na qual algum método (de custo-benefício aceitável) pode ser estabelecido para objetivamente demonstrar que a solução satisfaz cada requisito especificado. Se não for possível definir um método para verificar o requisito, então ele deve ser removido ou revisado. Exemplos: ➤

O requisito “A interface de usuário deve ser amigável” não é verificável, pois é subjetivo; já estabelecer que “oito em cada dez usuários da área consegue usar o produto sem treinamento prévio” é.



O requisito “Estabelecer que 95% das transações devem ser processadas em menos de 1 segundo cada” é verificável, pois está enunciado em termos mensuráveis.



O requisito “O sistema não deve travar nunca” não é verificável, pois demanda toda a eternidade para ser testado.



O requisito “O sistema deve rodar em todos os navegadores web do mercado” também não é verificável. Neste caso o aspecto é mais com relação ao custo-benefício. Seria necessário conhecer todos os navegadores disponíveis, o que por si só demandaria um esforço alto de pesquisa, considerando ainda que outros navegadores podem estar sendo criados enquanto o projeto está em andamento. O esforço de teste do sistema em todas as versões de todos esses navegadores pode significar muito mais que o esforço planejado para todo o resto do projeto.

No Capítulo 8 será abordada a atividade de verificar requisitos, que cumpre um importante papel na garantia da qualidade da especificação de requisitos.

2.6.8. Rastreável

Uma especificação rastreável é aquela que estabelece relação entre requisitos, suas origens e produtos derivados. Isso torna a especificação mais modificável, mais fácil de verificar se está correta e completa, além de facilitar a análise de impacto das mudanças.

A rastreabilidade auxilia a verificar a conformidade do produto com os requisitos, seja identificando requisitos que estão faltando (especificação incompleta) ou sobrando (especificação incorreta). A rastreabilidade também ajuda a identificar se todos os objetivos de negócio estão cobertos pelos requisitos e produtos gerados, prevenindo insatisfações das partes interessadas.

Ela também auxilia na gerência de riscos. Requisitos com muitas relações possuem riscos maiores. Isso pode ser mitigado de várias formas – uma delas alocando um profissional mais experiente para tratá-los, por exemplo.

O tema rastreabilidade será abordado com mais ênfase no Capítulo 9.

2.6.9. Onde usar tais critérios de qualidade?

Esses critérios de qualidade são úteis como um referencial para o analista de requisitos durante a elaboração da especificação. Estando atento a eles ao especificar, certamente conseguirá produzir um trabalho de melhor qualidade.

Mas a aplicação mais direta desses critérios é nas atividades de verificação e validação de requisitos, que são tratadas no Capítulo 8. Estas são as atividades que em última instância visam assegurar a qualidade do trabalho de requisitos.

2.7. Exercícios

1. Os seguintes documentos podem estar contemplados em uma especificação de requisitos, exceto: a)
Documento de arquitetura.
b) Documento de visão.

- c) Especificação de caso de uso.
 - d) Glossário.
 - e) Projeto físico do banco de dados.
 - f) Diagrama de contexto.
2. A elaboração da especificação de requisitos deve dar enfoque a algum tipo de leitor específico? Em caso positivo, qual? Justifique.
 3. Cite três fatores que podem influenciar o nível de detalhe da especificação de requisitos.
 4. Os requisitos são um contrato entre cliente e desenvolvedores. Tendo isso como base, é correto afirmar que:
 - a) Quanto mais detalhada a especificação de requisitos, melhor ela é.
 - b) Quanto menor a relação de confiança entre as partes, mais enxuto pode ser esse contrato.
 - c) Quanto menor o nível de confiança entre as partes, maior a necessidade da formalização deste contrato.
 - d) Independentemente do nível de confiança, um contrato verbal não deve ser “assinado”.
 - e) Mais importante que ser detalhado é o contrato ser claro e de fácil compreensão pelas partes.
 5. Assinale os casos em que se necessitaria de uma especificação de requisitos mais detalhada.
 - a) Primeiro projeto com um cliente até então desconhecido.
 - b) Cliente sem tempo para participar de reuniões do projeto.
 - c) Expectativa de manter todos os membros da equipe até o final do projeto.
 - d) Parte da equipe trabalhando em outra unidade da empresa.
 - e) Equipe desenvolve projetos para a área de negócio do projeto há cinco anos.
 6. Ao se trabalhar com metodologias ágeis, a especificação de requisitos torna-se dispensável?

3. A Importância da Engenharia de Requisitos

“Se você não identifica os requisitos certos, não importa
quão bem você execute o resto do projeto.”

Karl Wieggers “Não há nada tão inútil quanto fazer eficientemente
o que não deveria ser feito.”

Peter Drucker

Este capítulo apresenta elementos que justificam os investimentos na Engenharia de Requisitos, principalmente em função dos desperdícios que sua aplicação pode evitar e do maior alinhamento do projeto às necessidades de negócio que ela promove. Apresentam-se evidências de como a negligência em atividades pouco onerosas provocam prejuízos monumentais e de que a negligência na definição do “contrato” entre quem tem necessidades de negócio e quem provê soluções em software está mais próxima a uma regra que a uma exceção.

3.1. Motivação para a Engenharia de Requisitos

A Engenharia de Requisitos é uma das disciplinas fundamentais à engenharia de software e que provê informações para maioria das demais disciplinas. Isso por si só já seria motivo para ressaltar sua importância.

Este capítulo apresenta os resultados de pesquisas que embasam de forma quantitativa esse ponto. Em resumo, o que se busca demonstrar é que negligenciar a disciplina de requisitos traz como consequências: atrasos no cronograma e custo adicional, nível alto de defeitos no software entregue e, principalmente, a entrega de um software que não satisfaz plenamente as necessidades do cliente.

É possível que o leitor discorde de alguns dos números apresentados, afinal são trabalhos de pesquisa sobre dados de amostras de projetos e empresas, não de todo o universo. Porém, não se deve ficar preso aos números em si, mas aos fenômenos destacados. Certamente o leitor com alguma experiência em projetos perceberá que tais fenômenos estão presentes também em sua realidade, variando apenas na intensidade apontada pelas pesquisas.

Apontam-se também algumas constatações que demonstram o quanto a ER é negligenciada pelo mercado e pelo meio acadêmico na formação de novos profissionais para desenvolvimento de software.

3.2. Impactos negativos da falha em requisitos

Antes de abordar os resultados de pesquisas que reforçam a importância da Engenharia de Requisitos, ilustram-se a seguir alguns casos famosos de falhas em software ou em projetos relacionadas de alguma forma à deficiência no exercício da Engenharia de Requisitos. Isso também ajudará a sensibilizar o leitor sobre sua importância. É possível que você tenha conhecimento de outras histórias – neste caso, fale com os autores. Interessa-nos conhecer mais “causos” sobre o tema.

3.2.1. Sonda espacial Mars Climate Orbiter

A *Mars Climate Orbiter* (MCO) foi uma sonda espacial norte-americana cujo objetivo primário era o estudo do clima marciano. Foi lançada em dezembro de 1998, alcançando Marte nove meses e meio depois. Porém, ao

entrar na órbita de Marte, a MCO foi destruída na atmosfera devido a um erro de cálculo nessa manobra (Figura 3.1).

Somente a perda da sonda resultou num prejuízo de US\$ 125 milhões à NASA (agência espacial norte-americana). Isso sem considerar ainda os custos de desenvolvimento do foguete para o lançamento, do próprio lançamento da sonda e da operação da missão.

Uma investigação apurou que a causa primária do erro na trajetória de entrada em Marte da sonda foi um software desenvolvido por um fornecedor que produzia resultados no sistema imperial britânico (*pounds-seconds*). Isso estava em desacordo com a exigência da NASA, que esperava as medidas no sistema métrico universal (*newton-seconds*). Quando os resultados foram usados por outro software (este desenvolvido pela própria NASA), isso resultou no erro do cálculo da rota de entrada na atmosfera, causando sua destruição.

Não se sabe afirmar se a NASA forneceu as especificações detalhadas ao fornecedor (o que neste caso remete a um erro de especificação devido à inconsistência na interface entre os dois softwares) ou se o fornecedor participou do desenvolvimento dos requisitos para o software (neste caso pode ter havido uma falha de levantamento).

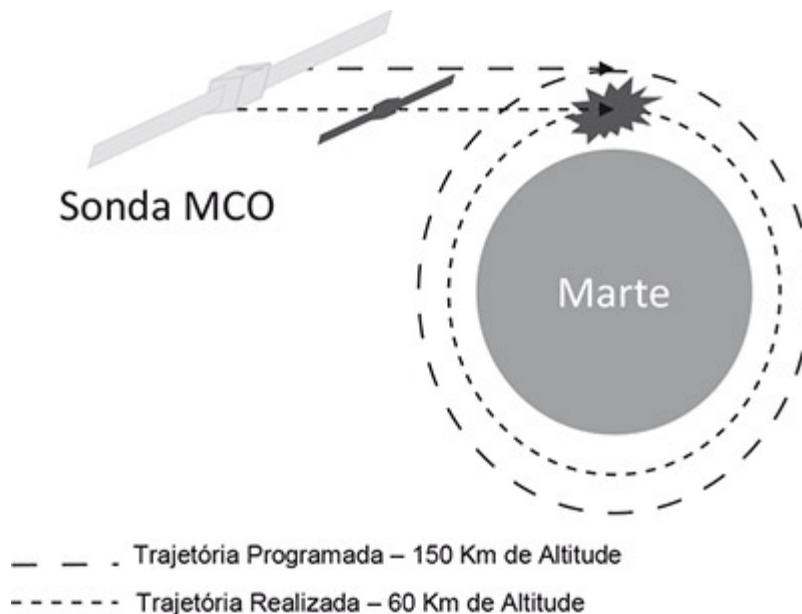


Figura 3.1. O controle da Terra enviou instruções de correção de rota para o MCO na unidade *pounds-seconds* e o software da MCO esperava dados na unidade

***newton- -seconds*. Isso antecipou sua entrada na atmosfera de Marte, causando sua destruição.**

3.2.2. Míssil antibalístico *Patriot*

Durante a Guerra do Golfo na década de 1990, a coalizão de forças liderada pelos EUA usou um sistema de defesa com mísseis antibalísticos *Patriot* (Figura 3.2). Em 25 de fevereiro de 1991 este sistema falhou ao não interceptar um míssil *Scud* lançado pelo Iraque. O míssil iraquiano matou 28 militares americanos e feriu outros 98. Posteriormente descobriu-se que a falha estava no software do míssil.

O software original que calculava a trajetória do *Patriot* utilizava dados dos sinais de radar e tinha que trabalhar com precisão de frações de segundo. Para lidar com mísseis mais modernos de alta velocidade foi criada uma sub-rotina para obter de forma mais precisa (mais casas decimais) as informações do relógio do sistema. Porém, esta sub-rotina não foi utilizada em todas as partes necessárias do software. A consequência disso é que o acúmulo de falhas de precisão, depois de certo tempo, tornava o sistema de interceptação do míssil ineficaz. Não foi apenas uma falha de programação, mas uma falha de avaliação do impacto da mudança.

Aparentemente essa deficiência do sistema era conhecida e uma solução de contorno bastante comum em software foi adotada: “reiniciar o sistema sempre que possível” para zerar os acúmulos de imprecisão. Lembre-se que isso ocorreu na década de 1990, e atualizar o software dos equipamentos em campo de guerra não seria trivial. Infelizmente o termo “sempre que possível” da solução dada é vago. Quando o míssil iraquiano foi lançado, o sistema do *Patriot* estava rodando sem ser reiniciado há mais de cem horas, e o acúmulo de imprecisões havia deixado o sistema defasado em um terço de segundo, o que impossibilitou a interceptação.



Figura 3.2. O software responsável pela trajetória do míssil *Patriot* tinha um defeito que causava atraso no seu relógio. A solução de contorno especificada foi reiniciar o sistema “sempre que possível” (crédito: Shutterstock).

3.2.3. Foguete Ariane 501

Em 4 de junho de 1996, o foguete Ariane 501 (da agência espacial europeia) foi lançado da base Kourou, na Guiana Francesa, e aproximadamente 37 segundos após a decolagem desviou-se bruscamente do curso previsto, partiu-se e explodiu (Figura 3.3). Este foi o primeiro lançamento da série Ariane 5, consumiu dez anos de desenvolvimento e teve um custo de US\$ 7 bilhões. A perda do foguete e sua carga somou um prejuízo de mais de US\$ 370 milhões.

A comissão de investigação apontou que a causa da falha foi a perda total de informações de orientação do foguete após a sequência de ignição principal. Essa perda de informação foi provocada por erros de

especificação e *design* do software do sistema de referência inercial. Este software foi reutilizado da série Ariane 4. Porém, os foguetes dessas séries possuíam algumas características distintas, enquanto o Ariane 5 foi projetado para levar mais carga, o que implica em padrões distintos de trajetória e velocidade. O erro foi não adaptar o software do Ariane 4 aos requisitos distintos do Ariane 5.



Figura 3.3. O foguete Ariane 501 explodiu 37 segundos após lançamento por falha na especificação do software do seu sistema de referência inercial. Veja em: https://youtu.be/gp_D8r-2hwk (crédito: Shutterstock).

3.2.4. HAL 9000

Este é um exemplo da ficção. Na obra “2001: uma odisseia no espaço”, o computador de bordo da espaçonave, chamado HAL 9000, tenta matar toda a tripulação. Na obra seguinte, “2010: odisseia dois”, descobre-se que esse comportamento do computador ocorreu porque ele foi programado para cumprir dois requisitos conflitantes: divulgar plenamente suas informações e manter segredo para a tripulação do verdadeiro objetivo da missão.

Portanto, a solução encontrada por HAL para cumprir os dois requisitos foi eliminar a tripulação.

3.2.5. Arquivo virtual do FBI

No início do ano 2000 o FBI (a polícia federal norte-americana) iniciou o desenvolvimento de um software chamado *Virtual Case File* (VCF), uma espécie de arquivo virtual dos processos de investigação conduzidos pelo FBI que permitiria compartilhamento de informações de diferentes casos entre os agentes. O planejamento original previa uma duração de três anos para o projeto.

Após os ataques terroristas de 11 de setembro de 2001, o FBI recebeu fortes críticas por não ter antecipado o ataque, dadas as evidências que foram descobertas depois. A falha foi não ter conseguido estabelecer a relação entre as várias evidências disponíveis. O VCF, que ajudaria em parte nesse objetivo, passou a ser uma das prioridades máximas do FBI: seus requisitos aumentaram, assim como prazos e custos.

Porém, depois de cinco anos de desenvolvimento e de custos incorridos de US\$ 170 milhões, o projeto foi abortado com o sistema ainda em desenvolvimento. Investigações posteriores constataram várias causas para o fracasso do projeto, dentre elas: ➤

Mudanças frequentes em requisitos. O fornecedor contratado para o desenvolvimento alegou que o FBI adotou a tática de tentativa e erro em várias decisões e que a filosofia dos responsáveis era “vou saber o que eu quero depois de ver o produto”.



Alta rotatividade da gerência (o que também contribuiu para as mudanças frequentes em requisitos).



Aumento não controlado do escopo, com requisitos sendo acrescentados mesmo com o projeto já atrasado.

3.3. Uma tentativa de melhoria de processo

Ganho de produtividade é o sonho de toda empresa, e com as que desenvolvem software não é diferente. Visando isso, a diretoria de Tecnologia da Informação de certa empresa investiu fortemente na aquisição de ferramentas para desenvolvimento de software do topo do mercado e também na capacitação de sua equipe técnica nessas ferramentas. Para alguns, investiu ainda na certificação profissional das ferramentas. Depois de certo tempo, ficou perceptível que a equipe passou a trabalhar com mais desenvoltura com as ferramentas e o quão ágil era agora para criar coisas novas. A empresa estava orgulhosa de sua “tropa de elite” para desenvolvimento de software.

No entanto, para as demais áreas clientes não houve benefícios perceptíveis. A insatisfação com os projetos entregues pela TI ainda era alta. Os prazos e orçamentos continuavam a ser excedidos. Uma investigação foi feita para tentar descobrir as causas dos problemas. Uma primeira constatação foi descartar o ferramental e o pessoal não qualificado tecnicamente como causa do problema. Na visão dos clientes da TI, o problema era que ela não conseguia entender suas necessidades e sempre entregava algo que não lhes atendia. Na visão dos desenvolvedores, o problema era dos clientes, que nunca sabiam ao certo o que desejavam, o que os obrigava a refazer um mesmo trabalho várias vezes.

Em resumo, velocidade não era o problema, pois a equipe técnica conseguia produzir com rapidez. O problema era de direção: a equipe não conseguia produzir a coisa certa. E a direção correta era descoberta por tentativa e erro. Cada tentativa era até rápida para se produzir, mas o ciclo todo até chegar à tentativa definitiva era o que interessava ao cliente – e continuava demorado. O trabalho da equipe técnica podia ser descrito em uma frase: “entra lixo, sai lixo. Mas, uau, com que velocidade!”.

Faltava a essa empresa a atenção devida à Engenharia de Requisitos. Ela é a responsável por construir essa ponte entre as necessidades do cliente e uma solução que o satisfaça de forma eficaz.

3.3.1. Principal motivo para o fracasso de projetos

O *Project Management Institute* (PMI) apresenta uma constatação preocupante: 47% dos projetos que fracassam são causados por uma deficiência no exercício da Engenharia de Requisitos (PMI, 2014). Esse estudo não é restrito a projetos de software, mas a projetos de forma geral. É possível que o resultado seja ainda pior para o contexto de projetos de software.

Quando um projeto de engenharia civil falha na Engenharia de Requisitos, é mais difícil ocultar: uma construção, seja ela uma ponte, um edifício, uma usina ou uma estrada, é visível, e, se for refeita, ainda que parcialmente, o fato será percebido por muitos. Para um projeto de software, as falhas de requisitos também vão gerar retrabalho, porém isso não costuma ser tão visível. Se um programa precisar ser reescrito, possivelmente somente os membros da equipe técnica irão ter ciência disso.

Somente o fato de ser o maior motivo isolado de fracassos em projetos já justificaria a importância da Engenharia de Requisitos. E isso deveria resultar em um investimento adequado em termos de planejamento para melhorar o seu exercício. Porém, o mesmo estudo constata que apenas 20% das organizações pesquisadas possuem alta maturidade com a Engenharia de Requisitos. Esse mesmo estudo também revela que organizações com baixo desempenho na Engenharia de Requisitos desperdiçam quase dez vezes mais dinheiro em projetos que as de alto desempenho. Ou seja, enquanto as organizações não evoluírem sua maturidade com requisitos, a possibilidade de fracassos nos projetos continuará sendo significativa.

E por que deficiência na Engenharia de Requisitos potencializa a chance de fracasso do projeto? Por vários motivos, como os descritos a seguir.



Estimativas de custo com um nível de incerteza compatível com seu propósito têm como matéria-prima requisitos de qualidade. Aliás, requisitos de qualidade podem mostrar que o projeto não é viável, não justificando seu investimento. Nem todas as necessidades de negócio devem ser atendidas a qualquer custo e nem todas aquelas que originalmente se pretende atender por meio de um novo produto de software precisam ser resolvidas dessa forma. Muitos projetos

fracassam porque são iniciados com a premissa de custo subestimada, embasada em um escopo que desconsidera áreas funcionais, macroprocessos e pessoas que são afetadas pelo software em desenvolvimento.



A entrega, ao final do projeto, de um produto que não satisfaça o cliente. Ainda que o projeto tenha sido concluído no prazo e no orçamento previsto, este será um fracasso.



O projeto incorporar ao produto requisitos que não deveriam estar no escopo. São requisitos que consomem recursos do projeto e concorrem com requisitos necessários. Promovem o risco e comprometem a capacidade de entregar um produto que atenda às necessidades de negócio que motivaram seu desenvolvimento, observando as premissas de custo e tempo que o justificaram em primeiro lugar.



O produto ser entregue sem cumprir com requisitos necessários e que sequer foram identificados. Isso pode resultar em um produto inútil ou em retrabalho para adequá-lo aos requisitos essenciais e que não foram descobertos.



Falhas na comunicação dos requisitos ao longo do projeto, potencializando a sua não compreensão e, como resultado, a entrega de um produto deficiente cuja adequação exige retrabalho.



Há requisitos que apenas são descobertos com o uso de protótipos; principalmente de usabilidade. Contudo, há mudanças que são falta de cuidado da equipe de desenvolvimento em entender e desenvolver as necessidades do cliente. São mudanças que desperdiçam recursos e que poderiam ser evitadas se as necessidades fossem mais bem compreendidas inicialmente e se tivesse existido cuidado na sua comunicação ao longo do desenvolvimento do produto; por exemplo, com uma especificação adequada a esse propósito.

3.3.2. Uma das principais causas de defeitos em software

Para o público leigo, em geral, um produto é de qualidade quando não apresenta defeitos. O mesmo vale para software. O critério para avaliação de qualidade mais comumente utilizado, não só pelos usuários do software como também pela equipe técnica de desenvolvimento, são os defeitos percebidos no produto.

O que se espera, portanto, para que seja possível considerar um software como tendo qualidade? Seguindo a definição de qualidade da ISO 9000, conclui-se que software de qualidade é aquele cujas características presentes atendem aos seus requisitos.

No entanto, é frequente a queixa dos usuários de software sobre falhas no seu funcionamento. Os defeitos podem se originar em qualquer fase do ciclo de vida do projeto. O mais comum é que erros sejam inseridos durante a construção do software. Ou seja, o produto construído tem um comportamento diferente do especificado.

Porém, uma parte significativa dos defeitos tem origem nos requisitos. Como assim? Qualidade não é cumprir com requisitos? Como seria possível o requisito ter defeito? Se o software funciona de acordo com o requisito, isso não deveria ser considerado defeito!

Se isso parece confuso, lembre-se que a definição de requisito apresentada no Capítulo 2 possui três partes. Faz parte da Engenharia de Requisitos identificar “as capacidades necessárias por um usuário para resolver um

problema” (parte 1 da definição) e criar uma “representação documentada” (parte 3 da definição).

Qualquer falha em identificar essas capacidades, ou qualquer falha em documentar essas capacidades, traduz-se então em um defeito introduzido pelo exercício da Engenharia de Requisitos, pois resultará em produto com comportamento indesejado pelo cliente.

Observe que prover documentação em excesso é uma falha em documentar; prover documentação descrevendo algo que apenas se poderá confirmar após prototipação é uma falha em documentar. Ou seja, avaliar se houve uma falha na documentação requer avaliar os fatores discutidos no Capítulo 2.

Logo, não se pode dizer que o software é de qualidade simplesmente porque é aderente a uma especificação, se esta especificação não representa corretamente as necessidades do usuário. Os desenvolvedores implementarão tudo o que foi dito na especificação de requisitos, ou o que conseguiram entender dela. Ocorre que tal especificação nem sempre reflete fielmente as necessidades dos usuários. Como evitar esse problema? Veja os tópicos de verificação e validação no Capítulo 8.

Capers Jones (2013) apresenta que, nos EUA, um em cada cinco defeitos em potencial são originados de requisitos, ou seja, 20% do total. Dean Leffingwell (1997) argumenta que, em projetos mais complexos, erros em requisitos são os mais comuns. Pohl (2011) afirma que erros em requisitos podem corresponder a até 60% do total de erros no projeto, isso considerando também como erro requisitos que faltaram ser implementados no produto. E este tipo de erro só é detectado em etapas mais avançadas do trabalho, não raro na homologação do produto.

Na Tabela 3.1, Jones (2012) apresenta os defeitos em potencial em software, segregados por fonte do defeito e estratificados por ordem de grandeza do sistema. O tamanho do software está expresso em pontos de função, que é uma unidade de medida de software na sua perspectiva funcional (VAZQUEZ, 2013).

Tabela 3.1. Defeitos em potencial por tamanho de sistema e origem.

Origem do defeito	Tamanho do sistema (em pontos de função – PF)		

	100	1.000	10.000
	Defeitos potenciais (<i>bugs</i> /PF)		
Requisitos	0,75	1,00	1,25
Arquitetura	0,10	0,25	0,50
Projeto (<i>design</i>)	1,00	1,25	1,50
Código-fonte	1,70	1,75	2,00
Material de testes	1,50	1,85	2,00
Documentação	0,65	0,70	0,75
Base de dados	2,00	2,75	3,00
Website	1,50	1,75	2,00
Total	9,20	11,30	13,00

Na Tabela 3.1 os defeitos em requisitos têm incidência menor que os dos outros autores citados. Os defeitos com origem no código-fonte, base de dados ou website têm incidência maior – no entanto, são os defeitos mais fáceis de eliminar. Já os defeitos originados em requisitos são mais difíceis de eliminar com os métodos tradicionais: teste e análise estática. Isso porque, uma vez que uma especificação tenha sido aprovada com um requisito defeituoso, os casos de testes elaborados a partir dessa especificação tenderão apenas a confirmar seu comportamento equivocado.

O mesmo autor também destaca que as mudanças em requisitos tendem a ter uma densidade de defeitos maior que os requisitos originais, o que não chega a ser tanto uma surpresa, pois as mudanças costumam ser tratadas em geral de forma apressada. Isso também faz com que esses defeitos sejam de mais difícil remoção, justamente por muitas vezes “pularem” etapas do controle de qualidade do projeto. A título de ilustração desse fenômeno, um crescimento de 10% no escopo do projeto implica em um aumento de 12% na quantidade de defeitos.

3.3.3. Custo do reparo de defeitos

A indústria de software tem um potencial muito grande para explorar ganho de eficiência. De acordo com Boehm (2001), de 40% a 50% do esforço em projetos de software são gastos em retrabalho para a correção de defeitos. Um dos fatores que contribui para essa ineficiência é que, ao se corrigir um defeito, há de 20% a 50% de chance de se introduzir outro defeito no software (BROOKS, 1995).

Boehm (2001) também aponta que o esforço para encontrar e corrigir um defeito após a entrega do software pode frequentemente custar com vezes mais do que corrigir este defeito enquanto ainda se está elaborando requisitos (Figura 3.4). E, como vimos nos casos citados no início do capítulo, erros detectados com o software em produção podem causar prejuízos várias ordens de grandeza maiores que o próprio custo de correção do defeito.

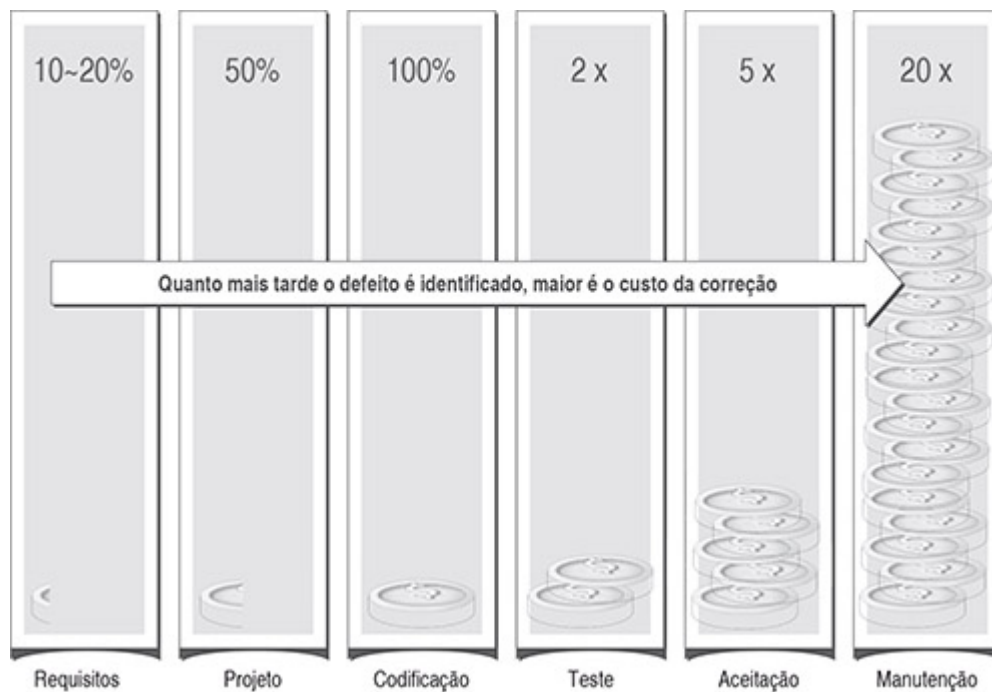


Figura 3.4. Custo relativo para correção de defeitos conforme o ponto onde é identificado.

Pohl (2011) argumenta ainda que a maior parte dos erros originados em requisitos é detectada em fases adiantadas do projeto. Isso é coerente com a experiência dos autores, acrescentando que uma parte significativa desses erros é identificada quando da homologação ou validação pelo cliente.

A meta deveria ser fazer certo da primeira vez. Logo, minimizar erros nas etapas iniciais do projeto é onde se consegue mitigar com mais intensidade o retrabalho. O método de tentativa e erro, além de ser mais caro e demorado, gera insatisfação dos clientes. Logo, evoluir na qualidade do trabalho de requisitos impactará positivamente em custo, prazo e satisfação do projeto.

3.4. Especificações de requisitos como um ativo

Em muitas empresas o entendimento quanto ao funcionamento de um sistema legado depende de uma investigação de seu código-fonte. E o código-fonte é expresso em uma linguagem de programação de computadores, passível de compreensão quase sempre apenas por desenvolvedores, e muitas vezes somente por aqueles especializados naquela linguagem específica. Dependendo de um programador ler código-fonte para saber como o software funciona representa uma sobrecarga de trabalho significativa ao negócio.

Além disso, há riscos tanto na tradução do código-fonte para o entendimento do desenvolvedor quanto na transmissão desse entendimento do desenvolvedor aos responsáveis pela gestão do negócio. Também há um esforço analítico relevante do desenvolvedor, para além da tradução, juntar os diversos componentes de implementação nos quais uma única função no negócio se subdivide. Esse tipo de cenário representa uma sobrecarga muito grande às atividades de manutenção de software. Ou seja, mexer em um produto em que não se sabe direito como funciona, além de ser mais demorado, possui mais chance de erros.

O trabalho de requisitos envolve identificar, criar e estruturar o conhecimento. E, em tese, todo esse conhecimento pode ser guardado na cabeça de alguém. Porém, a memória humana é falha e as pessoas são perecíveis. Logo, é importante que esse conhecimento seja persistido (documentado). Neste caso, as especificações de requisitos funcionam como um ativo valioso na organização: representam esse conhecimento que foi elaborado. Elas ajudam a minimizar o impacto de rotatividade de

pessoal na equipe do projeto. Toda vez que alguém sai da equipe, uma parte do conhecimento se perde. No entanto, se a especificação de requisitos existe e possui um nível de detalhe adequado, tal perda é atenuada.

3.5. Reflexão: onde a indústria de software investe energia

Pelo exposto até este ponto, já se percebe o quão importante é a Engenharia de Requisitos. O próprio fato de você ser leitor deste livro já pressupõe que você julga o tema importante. Porém, cabe refletir um pouco sobre a relevância que a indústria e a academia dão a esse tema.

Do ponto de vista da formação acadêmica dos novos profissionais em desenvolvimento de software, é comum que a Engenharia de Requisitos seja apresentada como parte da disciplina de engenharia de software e não como disciplina específica. Isso resulta quase sempre em uma visão superficial do tema.

Para muitas empresas, o esforço de capacitação da equipe técnica é em sua maioria dedicada a ferramentas, às vezes até em ferramentas de gestão de requisitos. Mas sem uma base sólida dos conceitos fica difícil tirar o melhor proveito da ferramenta.

Para alguém interessado no tema, qual a oferta de livros sobre o assunto? Talvez o leitor já tenha feito essa pesquisa, mas, caso não tenha feito, recomendamos a experiência. Pesquise em uma livraria virtual qualquer livros sobre “requisito”. O resultado não será volumoso, contam-se nos dedos de uma só mão as opções disponíveis. Se incluirmos livros de engenharia de software, que tratam o tema em um capítulo, consegue-se aumentar, mas não muito, as opções. Se a pesquisa for por livros em inglês, daí há mais opções.

Mas compare o resultado dessa pesquisa com uma pesquisa sobre livros de uma determinada linguagem de programação (por exemplo: Java, .NET, PHP). Serão centenas de opções retornadas. Mesmo em inglês, a discrepância de livros sobre requisitos e sobre linguagem de programação é gigantesca.

Isso ajuda a perceber a inversão de valores praticada no mercado. Muitos vendem a ideia de que a ferramenta X ou Y resolverá seus problemas em desenvolvimento de software. É compreensível, pois há empresas vendendo esse ferramental, que movimenta altas somas. Porém, para aqueles que já têm um tempo de estrada no mercado, não é difícil perceber que o desenvolvimento de software vive de muitos modismos. As ondas passam, ferramentas vêm e vão. E os problemas continuam: projetos entregando produtos que não satisfazem o usuário.

Portanto, a maioria aprende sobre a Engenharia de Requisitos na prática, participando de projetos, percebendo o que funciona, o que não funciona (errando) e como superar essas dificuldades em futuros projetos. É um jeito caro de aprender.

3.6. Exercícios

1. Nos projetos de sua empresa que apresentam problemas ou fracassam, qual é o percentual deles que têm como uma das causas a deficiência na Engenharia de Requisitos? Cite alguns exemplos.
2. Qual é o percentual de esforço aproximado dedicado à disciplina de Engenharia de Requisitos nos projetos de sua empresa? Esse valor é adequado? Em que momento do projeto (primeiro, segundo ou último terço) esse esforço é mais intenso?
3. Do total de solicitações de mudanças de requisitos ao longo dos projetos em que você já participou, qual é o percentual aproximado de solicitações que poderiam ter sido evitadas se um trabalho melhor de requisito houvesse sido feito?
4. Qual é o percentual aproximado de defeitos detectados nos projetos em que participou que têm sua origem em uma falha do trabalho de requisitos?
5. É possível calcular o retorno sobre o investimento na melhoria das práticas de Engenharia de Requisitos de uma empresa? Explique.

4. Dificuldades Comuns com Requisitos

“As pessoas não sabem o que querem,
até mostrarmos a elas.”

Steve Jobs O objetivo deste capítulo é apresentar as dificuldades mais comuns ao se aplicar a Engenharia de Requisitos. Não há a pretensão de esgotar todas as dificuldades que se pode enfrentar; tampouco se pretende definir uma ordem de importância ou frequência nessa relação.

Para várias dessas dificuldades são apontadas alternativas de solução, sendo que um dos principais objetivos deste livro é fornecer o ferramental para lidar melhor com elas. A intenção é também aumentar a consciência sobre essas questões de forma que o leitor consiga se preparar para vencê-las (ou contorná-las) no seu trabalho de requisitos.

4.1. Comunicação

É razoável supor que a maioria das dificuldades associadas à Engenharia de Requisitos esteja direta ou indiretamente relacionada à comunicação. E uma das dificuldades mais comuns ao trabalho de requisitos é conseguir comunicar-se de forma clara, seja com as partes interessadas, seja com os demais membros da equipe do projeto.

A ambiguidade está presente na nossa linguagem do dia a dia. E se isso já nos causa transtornos na vida pessoal (Quadro 4.1), que dirá

profissionalmente. O analista de requisitos deve estar sempre atento à ambiguidade nas mensagens – tanto das que envia quanto das que recebe. Quando ela é percebida de imediato, resolvê-la custa quase nada. Porém, quando ela não é percebida de imediato, os danos vão se acumulando e crescendo à medida que o tempo passa. Isso porque a parte interessada e o analista de requisitos seguirão com seus trabalhos seguros de que a mensagem foi compreendida.

Esse desencontro muito provavelmente somente virá à tona quando o produto final for entregue ou estiver em teste de aceitação do usuário. O mesmo fenômeno da ambiguidade repercute na especificação de requisitos ou nela é originado. Em ambos os casos, aumenta-se a chance de uma compreensão equivocada pelos demais profissionais atuando no desenvolvimento e, conseqüentemente, de uma construção de produto que não atende ao seu propósito.

Uma solução que diminui o problema da ambiguidade é atentar para o fato de que a comunicação não se constitui simplesmente da mensagem. Considerar apenas a mensagem é sinal de grande imaturidade ou de falta de atenção e interesse ao interpretá-la. Deve-se, portanto, conhecer aos demais elementos na comunicação e considerá-los para que a comunicação seja efetiva: ➤

Emissor: alguém que emite a mensagem. Pode ser uma pessoa, um grupo, uma empresa, uma instituição.



Receptor ou destinatário: a quem se destina a mensagem. Pode ser uma pessoa, um grupo ou mesmo um animal, como um cão, por exemplo.



Código: a maneira pela qual a mensagem se organiza. O código é formado por um conjunto de sinais, organizados de acordo com determinadas regras, em que cada um dos elementos tem significado em relação com os demais. Pode ser a língua, oral ou escrita, gestos,

código Morse, sons *etc.* O código deve ser de conhecimento de ambos os envolvidos: emissor e destinatário.



Canal de comunicação: meio físico ou virtual que assegura a circulação da mensagem, por exemplo, ondas sonoras, no caso da voz. O canal deve garantir o contato entre emissor e receptor.



Mensagem: é o objeto da comunicação, constituída pelo conteúdo das informações transmitidas.



Referente: o contexto, a situação à qual a mensagem se refere. O contexto pode se constituir na situação, nas circunstâncias de espaço e tempo em que se encontra o destinador da mensagem. Pode também dizer respeito aos aspectos do mundo textual da mensagem.

Denomina-se ruído os elementos que perturbam e que dificultam a compreensão pelo destinatário, como, por exemplo, o barulho ou mesmo uma voz muito baixa. O ruído pode ser também de ordem visual, como borrões, rabiscos *etc.* Sempre há algum ruído, por isso, ele não deve ser desconsiderado ao avaliar qualquer comunicação.

Na piada usada como exemplo de problemas na comunicação no Quadro 4.1, percebe-se que o destinatário não se preocupou com o referente daquela mensagem, ou mesmo com o seu emissor.

Minha esposa disse: — Amor, vá ao mercado e compre uma caixa de leite. Se eles tiverem ovos, traga seis.

Eu voltei com seis garrafas de leite.

Ela disse:

— Por que você comprou seis garrafas de leite?

Respondi:

— Porque eles tinham ovos!

Quadro 4.1. Piada popular na internet, de autoria desconhecida, que ilustra a ambiguidade na comunicação.

Outra dificuldade relativa à comunicação é manter sintonizados todos os envolvidos na Engenharia de Requisitos, sejam membros da equipe de requisitos, sejam as partes interessadas. O número de canais de comunicação em potencial entre essas pessoas é dado pela Fórmula 4.1, que apresenta os caminhos de comunicação (p) em função da quantidade de pessoas (n).

$$p = \frac{n \times (n - 1)}{2}$$

Fórmula 4.1. Caminhos de comunicação (p) em função da quantidade de pessoas (n).

Isso significa que, para um conjunto de dez pessoas, são 45 canais de comunicação possíveis. Para 11 pessoas, isso já sobe para 55. Aumentou-se em um indivíduo e isso implicou em um acréscimo de dez caminhos.



Figura 4.1. Progressão do número de caminhos de comunicação.

A solução que a humanidade criou para lidar com essa complexidade foi a hierarquia. Com ela as comunicações são centralizadas em certos elementos que têm a autoridade ou a responsabilidade de decidir pelo seu encaminhamento para a pessoa indicada e observando uma cadeia de comando predeterminada.

Com a informatização e automação de processos de negócio, as hierarquias hoje são menos rígidas que no passado; contudo, ainda não é possível adotar plenamente o potencial de uma rede como a representada na Figura 4.1.

Provavelmente nem todas as pessoas do grupo terão comunicação com todas as demais. Mas, ainda assim, a complexidade é uma preocupação relevante. O analista de requisitos deve ter a preocupação de levantar esses caminhos.

Decisões de requisitos que forem tomadas por uma parte do grupo necessitam ser compartilhadas com outros, que podem contestar, complementar ou mesmo desafiar aquelas decisões. Isso é fundamental para que o trabalho de aprovação dos requisitos depois seja rápido e sem muitas surpresas.

Para enfrentar essas dificuldades de comunicação, é fundamental que o analista de requisitos desenvolva suas habilidades de comunicação (verbal, não verbal, escuta e escrita) e também o relacionamento interpessoal. Essas habilidades são muito mais importantes, por exemplo, do que dominar técnicas de modelagem ou ferramentas CASE (*Computer-Aided Software Engineering*). Porém, tais habilidades raramente são abordadas na formação dos profissionais de tecnologia. Ou seja, há uma lacuna grave na formação acadêmica de quem se envolverá no trabalho de requisitos.

4.2. Acesso às partes interessadas

Muitas vezes o trabalho de requisitos acontece sem que seja permitido ao analista de requisitos interagir diretamente com algumas partes interessadas. Seja por falta de disponibilidade de tempo ou interesse da parte interessada, por questões políticas, sociais, culturais ou qualquer outro motivo.

Por exemplo, certa vez um profissional, membro de uma organização militar, relatou que, quando era necessário levantar requisitos com pessoas de patente superior à sua, era mais difícil obter acesso a elas do que quando as pessoas eram de patente igual ou inferior à sua. Outro caso interessante foi de um profissional que trabalhou em um projeto onde um desembargador era a parte interessada principal, mas este só dirigia a palavra à sua secretária ou a outros desembargadores e juízes. Embora pitorescos, esses exemplos citados não são tão comuns. A dificuldade de acesso mais comum é quando a parte interessada é de fora da organização (clientes, fornecedores, parceiros, aliados etc.).

Nestes casos, normalmente se designa um intermediário para cumprir o papel da parte interessada. Este deveria ser alguém que conseguisse a representar bem, transmitindo fielmente suas necessidades e expectativas. Por exemplo, a área de marketing costuma representar a voz do cliente para as demais áreas da empresa. Porém, conduzir o trabalho de requisitos através de um intermediário é sempre um risco maior. Se esse intermediário falhar no seu papel, certamente o produto entregue ao final do projeto terá sérios problemas.

Outro caso de dificuldade de acesso à parte interessada é quando ela não deseja se envolver tanto no trabalho de requisitos. Em geral, alega-se “falta de tempo” para participar de reuniões. Pode-se dividir esse cenário em dois casos.

O primeiro é aquele em que a parte interessada necessária ao trabalho de requisitos não é a demandante direta do projeto, ou não se beneficiará de forma direta dos resultados dele. Por exemplo, o departamento de RH empreende um projeto que precisa da participação do departamento financeiro. Como somente o RH usufruirá benefícios do projeto, o financeiro tratará a sua demanda de participação como baixa prioridade dentro de suas atividades. Às vezes a situação é ainda mais delicada – por exemplo, quando a parte interessada está fora da empresa (cliente, fornecedor, parceiro). Nesse caso, é mais difícil ainda ter alguém interno na empresa com autoridade para exercer alguma obrigação sobre a parte interessada.

O segundo caso é aquele em que a parte interessada é a demandante direta do projeto. Isso é um pouco mais complicado de entender, pois como é possível alguém demandar um projeto e não ter tempo para se dedicar a ele? É ilógico, mas acontece. Isso pode ocorrer porque aquela parte interessada está acostumada a delegar para a TI a definição de requisitos relativos ao seu negócio. É um desvio de função da TI, mas que infelizmente ocorre bastante. Ou também pode ser o caso de a parte interessada solicitar o projeto, mandar a TI executá-lo e exigir ser acionado somente quando o projeto estiver concluído, para então avaliar o que ficou bom e o que não ficou – e aí sim começar a se envolver. Isso significa retrabalho alto para conseguir chegar a uma solução que o satisfaça. Muitas vezes isso ocorre porque as pessoas de fora da TI não conhecem o processo de

desenvolvimento de sistema e não sabem da importância de sua participação nesse processo. Um esclarecimento nesse sentido será benéfico para melhorar seu engajamento.

Curioso é que muitas empresas desejam embarcar na onda dos projetos ágeis adotando uma nova metodologia, mas ignorando por completo a sua filosofia. O Princípio 4 do Manifesto Ágil (BECK et al., 2001) prega: “pessoas de negócio e desenvolvedores devem trabalhar diariamente em conjunto por todo o projeto”. Portanto, faz parte da filosofia ágil obter uma cultura de participação intensa das partes interessadas. Logo, vencer essa dificuldade implica em uma mudança de cultura que extrapola os limites da área de tecnologia; deve abranger toda a empresa.

Quando a falta de tempo para se reunir é uma desculpa para alguém não se envolver no projeto, deve-se buscar outras formas de levantar informação além de entrevistas. Outras opções são as técnicas de observação, análise de documentos e questionário descritas no Capítulo 7. Ou também pode-se tentar encontrar outra pessoa com mais disponibilidade ou interesse e que possa fornecer as mesmas informações. Quase sempre há essa opção.

Se o analista de requisitos não tiver poder ou autoridade para vencer essa barreira de acesso às partes interessadas, é fundamental a participação do gerente de projetos (ou alguém com mais autoridade) para ajudar na busca de uma solução que minimize esses riscos no trabalho de requisitos. O ideal é trabalhar para que a presença do analista seja desejada por aqueles que podem criar essa barreira.

4.3. Indecisões/Indefinições do usuário

A dificuldade de o usuário em saber o que quer, ou, então, de se expressar corretamente quanto ao que deseja, é uma percepção de praticamente todos os analistas de requisitos com alguns anos de experiência. Varia a intensidade com que ocorre, desde a parte interessada que não sabe expressar sua necessidade até aquela que expressa a necessidade errada.

A princípio, pode-se pensar que a culpa por essa dificuldade é do usuário; afinal, seria sua obrigação ter uma visão clara da necessidade. Ledo engano. Fosse assim, o trabalho do analista de requisitos se resumiria quase que ao

de um tomador de notas. E o grande valor que o analista de requisitos pode agregar com o seu trabalho é justamente conseguir compreender corretamente as necessidades do usuário, ainda que este, a princípio, não saiba bem o que quer. Faz parte da Engenharia de Requisitos provocar e questionar as partes interessadas até o ponto em que se consiga uma percepção clara das necessidades. A postura passiva não condiz com o que se espera de um analista de requisitos.

Para essas partes interessadas, deve-se também buscar um método mais adequado de obtenção de informação. Entrevistas, por exemplo, talvez não sejam tão produtivas. A observação (vide Capítulo 7) pode funcionar melhor. A prototipação (vide Capítulo 8) também é outra ferramenta muito valiosa. Aqueles que não sabem o que querem, quando apresentados a uma proposta mais concreta de solução (o protótipo), saberão dizer com segurança ao menos o que não querem.

4.4. Requisitos implícitos ou não ditos

Caso comum na experiência profissional de muitos analistas de requisitos é achar que fez um bom trabalho de requisitos: ouviu as partes interessadas, documentou e confirmou suas necessidades, especificou uma solução, apresentou e a validou com todos e obteve sua aprovação. O desenvolvimento do projeto seguiu até a entrega do produto (ou parte dele) para homologação. Durante a homologação a parte interessada começa a relatar diversas necessidades não atendidas pelo produto, o que em princípio toma o analista de requisitos de surpresa, pois tais necessidades nunca haviam sido explicitadas antes. Quando o analista confronta a parte interessada sobre essas necessidades agora colocadas e que nunca foram solicitadas, ouve uma frase clássica, mais ou menos assim: “claro que não falei sobre isso, pois para mim estava implícito que isso estaria contemplado. É óbvio que o produto deveria ter isso!”.

Quem errou neste caso? Muitos podem argumentar que errou quem não pediu as coisas de forma explícita. Alguns analistas chegam ao ponto de mostrar a especificação assinada pela parte interessada, em uma forma de demonstrar que essas necessidades nunca estiveram no escopo. Pensando de

forma prática, encontrar um culpado para o erro ajuda pouco. Qualquer que seja o culpado, o cliente já ficou insatisfeito ao perceber que a solução não lhe atende. Tentar atribuir a culpa ao próprio cliente só irá piorar a situação. O papel do analista de requisitos é descobrir as necessidades das partes interessadas, estejam elas à tona (explícitas) ou mais ocultas (implícitas). É ilusão achar que o trabalho de requisitos se restringe ao que está explícito. Seria muito mais fácil assim, porém não funciona.

Haveria alguma forma do analista tomar conhecimento dessas necessidades implícitas? Não há nenhuma ferramenta ou técnica que garanta que a especificação de requisitos está completa. Por isso, a dificuldade em lidar com necessidades implícitas. O que se pode fazer é buscar estratégias para minimizar essas surpresas. A mais eficaz delas é conhecer bem o negócio da parte interessada. Quando o analista de requisitos interage com a parte interessada com baixo conhecimento do negócio, sua capacidade de visualizar o “óbvio” fica bastante comprometida. À medida que aumenta seu conhecimento, começa a perceber o que não foi dito, mas está implícito.

A observação (vide Capítulo 7) é muito útil também, tanto para ganhar conhecimento do negócio quanto para perceber o que não foi dito. A prototipação (vide Capítulo 8) é útil para antecipar a descoberta dos requisitos implícitos. Ao ter contato com o protótipo, a parte interessada provavelmente comentará a ausência de alguma característica que implicitamente ela esperava obter.

4.5. Mudanças

Uma reclamação muito comum entre alguns analistas de requisitos é a frequência alta de mudanças em requisitos. E, de fato, toda mudança acarreta algum ônus ao trabalho já feito. Mudanças deveriam existir para aumentar (ou preservar) o valor do projeto. Porém, é importante entender a natureza da mudança para identificar se esse fenômeno é um problema que deve ser tratado ou não. Há mudanças que poderiam ser evitadas. Por exemplo, mudanças que visam corrigir a solução proposta. Isso é muito comum quando não se faz um bom trabalho inicial de requisitos. Muitas vezes falta proatividade ao analista de requisitos para provocar as partes

interessadas a revelar mais informações, em vez de manter um comportamento passivo, apenas escutando quem frequentemente não sabe muito bem o que quer. Neste caso, o projeto começa com o escopo mal compreendido e ao longo da sua execução lacunas e erros vão sendo descobertos, e mudanças terão que ser feitas para corrigir o rumo.

Há também as mudanças que fogem ao controle do trabalho do analista de requisitos. São aquelas originadas no ambiente de negócio no qual a solução irá operar. Por exemplo: alteração de normas regulatórias, lançamento de um produto novo de um concorrente, mudanças de estratégia da organização. E muitas mudanças são benéficas ao projeto, pois aumentam o seu valor agregado. Mas, mesmo ainda dentro desse cenário de mudanças inevitáveis, é possível prever que algumas acontecerão. Por exemplo, a troca de um gestor por fim de mandato. É muito frequente que o novo gestor tenha outras prioridades, e isso acarretará em mudanças no projeto. Planejar o projeto para finalizar antes da troca da gestão é uma forma de minimizá-las.

Este é o tipo de mudança para o qual o analista de requisitos deve estar preparado para trabalhar. É ilusão querer produzir uma especificação de requisitos que não será alterada. O trabalho de requisitos deve ser feito para que as mudanças possam ser aceitas sem traumas ou dificuldades. Faz parte da vida.

No entanto, quanto maior o tamanho do projeto, maior tende a ser sua duração e, conseqüentemente, maior a incidência de mudanças. A Tabela 4.1, apresentada por Jones (2012), mostra uma ideia geral do crescimento dos requisitos com a duração do projeto.

Tabela 4.1. Duração do projeto × crescimento de requisitos.

Tamanho do projeto (em pontos de função)	Duração do projeto (meses)	Taxa de crescimento mensal dos requisitos	Crescimento mensal dos requisitos (em pontos de função)	Crescimento total do projeto (em pontos de função)
100	4,50	1,00%	1,00	2,25
1.000	15,00	1,25%	12,50	93,75
10.000	44,00	1,25%	125,00	2.750

4.6. Conflitos

Quanto maior a quantidade de partes interessadas em um projeto, maior o potencial de conflitos de interesses, necessidades e informações. Os conflitos podem ser de várias naturezas: de ordem pessoal, profissional, político, familiar *etc.* Entender sua origem ajuda na solução. Resolver conflitos, em princípio, não é obrigação do analista de requisitos, porém esse tipo de habilidade é importante para que se possa dar celeridade ao trabalho, sem depender da intervenção de outros. E, mais importante que isso, antes de solucionar conflitos, o analista deve ter cuidado ao desenvolver seu trabalho para não criar conflitos entre e com as partes interessadas desnecessariamente. Podem-se listar alguns conflitos comuns:



Requisitos de partes interessadas que são inviáveis de atender simultaneamente. Por exemplo, dois departamentos têm interesse em ter alçada de aprovação sobre operações do negócio de forma exclusiva. Ou um departamento deseja restringir determinados dados somente ao seu domínio e outro deseja que estes sejam públicos.



Informações inconsistentes sobre como funciona um processo de negócio. As explicações de duas partes interessadas sobre o mesmo processo não se encaixam.



Solicitações de partes interessadas fora do escopo do projeto.



Partes interessadas que são adversárias, antipáticas ou hostis umas às outras.



Falta de sintonia entre as áreas de negócio envolvidas. Cada área está preocupada apenas com o seu contexto e interesse, sem se importar com as demais. Isso muitas vezes resulta em um produto distante do que seria o ideal e que em geral implementa um processo ineficiente.

Diferentemente do programador, que interage a maior parte do tempo com o computador, o analista de requisitos precisa interagir boa parte do seu tempo com outras pessoas. Portanto, o trabalho de requisitos exige na maior parte do tempo uma boa habilidade de relações interpessoais. Não é exagero dizer que o analista de requisitos deveria incorporar um pouco das habilidades de psicólogo, diplomata e político para conseguir evitar e eliminar conflitos.

4.7. Resistência à mudança

Todo projeto de software implica em algum tipo de mudança para as partes interessadas. Novidades frequentemente geram temor e/ou preocupação nas pessoas. Manter a zona de conforto é a reação natural da maioria, por isso é frequente encontrar resistência a mudanças. Em geral, mudanças em projetos costumam trazer benefícios para todos. Porém, há casos onde, por falha em uma comunicação adequada no projeto, algumas partes interessadas podem criar receios sobre a mudança que chegará. Esse tipo de situação irá diminuir o nível de cooperação delas com o projeto. Há também os casos onde, de fato, a mudança pode trazer perdas para algumas partes interessadas. Por exemplo, o resultado do projeto pode eliminar postos de trabalho ou diminuir o poder de alguém em específico (ex.: compartilhando informação que antes era restrita). Neste caso, além da dificuldade de cooperação com elas, talvez ainda haja a intenção de sabotar o projeto.

O que fazer então? A primeira coisa que o analista deve buscar é compreender o impacto que o projeto acarretará a cada parte interessada. Para aquelas onde já se sabe que o impacto será negativo, deve-se buscar meios alternativos para obtenção de informação que não dependa da

interação direta com elas. Exemplos: identificação de outras pessoas, análise de documentos e observação (vide Capítulo 7). Para as partes interessadas que não serão impactadas negativamente, deve-se buscar uma comunicação adequada dos resultados que se pretende alcançar com o projeto e dos benefícios diretos que ele trará, de maneira a diminuir a barreira à cooperação. Quase sempre isso funciona muito bem.

4.8. Parte interessada não domina seu negócio

Apesar de ser uma situação estranha, é comum ouvir essa reclamação de muitos analistas de requisitos. Embora longe de ser desejável, isso pode ocorrer por vários motivos. Às vezes é o caso de um gestor recém-chegado àquela área de negócio e que ainda está se inteirando. É uma dificuldade em tese transitória, pois em breve aquela pessoa já dominará todo o conhecimento necessário para gerir a área. Há também o caso de a pessoa estar no cargo não pela sua competência ou experiência, mas por fatores políticos. E também há o caso de organizações governamentais que trocam boa parte das pessoas em cargos de gestão após uma eleição em que se muda o governo.

Resta então ao analista buscar conhecer bem o negócio, através de outros meios (estudo de documentação disponível ou identificar outras pessoas com mais conhecimento – quase sempre há), para conseguir fazer um bom trabalho. Há casos de analistas que passam a dominar tão bem o conhecimento de determinado negócio que são convidados a mudar de área, sair da TI e ir para a área de negócio.

Há também o caso das partes interessadas que delegam tanto a definição de requisitos à TI que deixam de dominar o seu negócio, ficando reféns do software: “trabalho desse jeito porque é o jeito que o sistema permite”, ou “não sei bem como funciona, informo tais dados e o sistema faz o resto”. Às vezes essa delegação de responsabilidade para a TI é muito conveniente, pois se o projeto não for um sucesso sempre é possível usá-la como bode expiatório: “foram eles que definiram o sistema”. Este é o típico problema

que demanda uma solução no âmbito da direção da organização, para que cada área assuma suas devidas responsabilidades.

4.9. Parte interessada não lê a especificação de requisitos

Como visto no Capítulo 2, a especificação de requisitos é o contrato da equipe de desenvolvimento com o cliente. Ela deve comunicar ao cliente tudo que será entregue (logicamente, satisfazendo todas as suas necessidades) e o cliente deve conseguir compreendê-la e dar sua aprovação para que o trabalho no projeto possa seguir adiante. A especificação de requisitos é o resultado de um trabalho que pode ter demorado dias, semanas ou meses. O que fazer quando o cliente não tem interesse em revisá-la para aprovação?

Isso representa um sério problema, pois seguir com o projeto sem nenhum *feedback* da parte interessada sobre a especificação representa um alto risco de construir um produto que não trará satisfação. Há casos em que a política é não seguir com o trabalho até a aprovação da especificação. Isso cria um impasse – ou, o que é muito comum, a parte interessada endossa a especificação sem revisá-la, para que o trabalho continue.

É importante tentar entender as razões pelas quais não há interesse das partes interessadas na especificação de requisitos. Alguns motivos possíveis são: ➤

Partes interessadas não compreendem a importância da especificação e entendem o papel de revisão como mera burocracia. Se for este o caso, significa que faltou à TI da empresa comunicar como é o seu processo de trabalho para entregar os projetos, qual a importância da especificação e da participação das partes interessadas, bem como as consequências negativas de ignorar a especificação (neste caso, é até possível recuperar problemas de projetos passados para citar como exemplo). Às vezes há motivo para que as partes interessadas interpretem a especificação como burocracia excessiva. Muitas empresas definem um nível de

documentação demasiado alto para os requisitos e que não agrega valor (ou não vale o esforço para tanto detalhe). Cabe então reavaliar o processo de desenvolvimento, para definir o que pode ser “enxugado” sem prejuízo aos projetos.



A estratégia adotada de apresentação da especificação para aprovação (vide Capítulo 9) pode estar errada. Não é razoável entregar dezenas de especificações de caso de uso para uma parte interessada, solicitar que sejam lidas minuciosamente e aprovadas. Muitas pessoas, ao depararem com centenas de páginas para leitura, podem ficar desanimadas, procrastinar, fazer uma leitura superficial ou mesmo endossar sem ler. E mais: será que é relevante a essas pessoas todos os casos de uso do projeto? Neste caso, o erro é apresentar muita informação detalhada sem conseguir que se valide e aprove sequer uma visão de mais alto nível da solução.



Parte interessada já está (ou acha que está) ciente do que será entregue e não vê necessidade de revisar a especificação, pois já sabe o que está contido ali. Isso pode ocorrer dada a intensidade com que a parte interessada foi envolvida no trabalho de elicitação de requisitos. Se de fato ela está bem informada, o conveniente seria repassar conjuntamente com ela a especificação, passando mais rápido pelas partes com as quais ela está alinhada e explicando com mais calma aquelas das quais ela pode não estar ciente.

4.10. Partes interessadas insaciáveis com requisitos

Aqueles com alguma experiência notam que os requisitos possuem uma propriedade parecida com os gases: inicialmente não têm forma e volume bem definidos e tendem a ocupar todo o espaço disponível (neste caso,

recursos do projeto). Essa característica expansiva dos requisitos torna a gestão do escopo de projetos um desafio.

E por que os requisitos se expandem? Porque as partes interessadas sempre têm necessidades a serem satisfeitas; elas nunca se esgotam. Além disso, quando se interage com essas partes no trabalho de requisitos, é comum que elas explicitem todas as suas necessidades, inquietudes e desejos, sem se importar em saber se tudo isso será atendido pelo projeto que está sendo desenvolvido (a expectativa delas é que seja). Nesse momento, se o analista não estiver atento, começa a incorporar ao projeto todos os requisitos que vão surgindo durante a interação.

Porém, quase sempre não é objetivo do projeto resolver todos os problemas daquela parte interessada. Os objetivos do projeto costumam ser bem específicos, e muitas das demandas daquele interessado podem não estar alinhadas aos objetivos do projeto. Ou seja, estão fora do escopo. Deveriam ser atendidas por outro projeto. Ou, se a parte interessada tiver poder para tanto, modificam-se os objetivos do projeto para que ele incorpore o atendimento a aquelas demandas.

Outra explicação para a característica expansiva dos requisitos é que, em muitas empresas, desenvolver software é de graça! Como assim? Na verdade, o desenvolvimento de software é uma atividade cara. Porém, para os clientes internos do desenvolvimento de software, o custo é zero: basta pedir que a TI faz. Não há um custo sendo alocado ao orçamento da área solicitante, o custo de toda a TI é compartilhado por toda a empresa.

Este caso é como um condomínio onde a conta de água é rateada entre os apartamentos, independentemente do consumo individual. A preocupação em economizar ou fazer uso racional do recurso é desestimulada. Quando cada apartamento arca diretamente com o custo do seu consumo individual, o uso mais racional faz com que o consumo global do condomínio seja sensivelmente menor.

O mesmo vale para requisitos. Quando se paga pelo recurso consumido, o cliente passa a planejar melhor a demanda que irá solicitar para desenvolvimento. E também pensará melhor antes de pedir uma mudança, afinal nem todas valem o custo--benefício. Isso, é claro, quando o custo é diferente de zero.

4.11. Consistência

Um bom trabalho de requisitos tem que resultar em uma especificação de requisitos que seja uma história bem contada: com início, meio e fim bem definidos e com todas as peças se encaixando e transmitindo com clareza a mensagem. Ocorre que o trabalho de requisitos vai sendo desenvolvido de forma evolutiva; a especificação é criada e vai recebendo incrementos, melhorias e correções ao longo do tempo. O conhecimento do autor da especificação vai mudando (aumentando) durante esse processo e partes da especificação elaboradas ao início podem já ter uma perspectiva diferente de partes elaboradas posteriormente.

Isso pode resultar em uma especificação final de requisitos com inconsistências, por exemplo: termos sendo usados em partes da especificação e outros termos usados em outras partes, todos se referindo ao mesmo assunto. Isso prejudica a qualidade da especificação e possibilita surgimento de erros em atividades seguintes do projeto por falhas de interpretação do leitor.

Essa dificuldade aumenta quando se considera que solicitações de mudanças em requisitos ocorrem ao longo do trabalho. Ou seja, partes da especificação que estavam já consolidadas deverão ser alteradas. Se a mudança impactar várias partes da especificação, mas houver falha na identificação de todos os pontos impactados da especificação, esta será alterada de forma incompleta e inconsistências surgirão.

Outro complicador é quando há mais de um autor para a especificação de requisitos. Para projetos de maior porte, principalmente, é bem provável que a especificação seja resultado de um trabalho coletivo, e não individual. Neste caso, há que se ter uma metodologia de trabalho bem alinhada entre os autores para que se consiga produzir uma especificação que conte uma história de forma harmônica, e não um “Frankenstein”, com pedaços desproporcionais e mal encaixados.

4.12. Necessidades vagas

Quando se inicia um trabalho de requisitos, normalmente se encontra um cenário onde as necessidades apresentadas são mais ou menos vagas. É raro encontrar (mas não impossível) as necessidades iniciais das partes interessadas sendo expostas de forma clara e específica. Que bom seria se esta fosse a regra! Mas será que se fosse assim haveria demanda para um analista de requisitos?

Transformar essas necessidades vagas em necessidades claras e específicas é responsabilidade do analista de requisitos. É trabalhoso conseguir isso. Pode ser bem demorado, inclusive. Mas o trabalho de requisitos não se conclui enquanto houver um nível alto de incerteza sobre as necessidades a serem atendidas.

Encontrar uma especificação de requisitos, em tese finalizada, mas com requisitos vagos é quase sempre um sintoma de preguiça ou desatenção do autor. E, conseqüentemente, sua utilidade fica comprometida.

4.13. Priorização

A priorização tem como função assegurar que os recursos do projeto sejam focados nos itens mais relevantes. Daí a importância de, na especificação, diferenciar cada requisito em termos de importância, dentre dezenas ou centenas de outros requisitos.

A responsabilidade por definir a prioridade do requisito deveria ser da parte interessada, facilitada pelo gerente de projetos. O analista de requisitos participa da discussão de priorização para ajudar no processo, mostrando eventuais impactos das decisões e esclarecendo melhor cada requisito, se necessário.

A grande dificuldade é ter as partes interessadas assumindo a responsabilidade da priorização. Muitos se omitem nessa questão, o que implica em delegar a terceiros (em geral, a equipe de desenvolvimento) a prioridade que será dada a cada requisito. Outros não se omitem e deixam explícito que tudo é prioridade. Mas, na prática, isso não é priorizar.

Certa vez o amigo de um dos autores participou de um projeto onde uma das responsabilidades era dar sustentação a um sistema legado que seria

descontinuado. Todo dia era relatado ao menos um problema desse sistema, que tinha que ser investigado e solucionado. Os problemas apareciam com mais velocidade que a capacidade de solucioná-los. E a maioria dos problemas era relatada pela área como sendo de solução urgente.

Em pouco tempo este amigo estava com uma lista de problemas a solucionar que ocupava uma página e perdido sobre qual item atacar primeiro. Procurou então o gerente da empresa para que este o orientasse sobre o que solucionar primeiro. O gerente, solícito, se prontificou a ajudar e começou a percorrer os olhos pela lista, comentando: — Este primeiro item é urgente! Este segundo também é. O terceiro é muito importante. Este quarto é extremamente crítico. Este aqui é prioritário (...).

E o gerente seguiu percorrendo toda a lista, falando adjetivos diferentes, mas querendo dizer que tudo deveria ser resolvido o quanto antes. Resultado: este amigo saiu da conversa do jeito que entrou e foi solucionar os itens na ordem que lhe era mais conveniente.

A maioria de nós, quando confrontados com a necessidade de priorizar, tem grande dificuldade. Priorizar é uma escolha difícil. Significa deixar algo para trás, o que deixa muita gente intranquila. Uma analogia que representa bem essa dificuldade de escolher para priorizar é a de uma criança em uma loja de brinquedos. Ela vê vários que lhe agradam, toma-os pela mão e tenta carregar todos que consegue, até que o pai lhe apresenta a difícil questão: “filho, papai só pode comprar um brinquedo, escolha apenas um para levar”. Nesse momento tem início uma negociação demorada e tensa. Se o pai escolher por conta própria qual brinquedo levar, há grande chance de o filho ficar insatisfeito.

4.14. Conclusão

Como comentado no início, as questões apresentadas até aqui resumem boa parte das dificuldades com o trabalho de requisitos, mas não exaure a discussão. Certamente o leitor que tem alguma vivência em projetos pode relatar nuances distintas para os itens apresentados, bem como dificuldades que não foram apresentadas neste capítulo. Comente com os autores sobre essas experiências (via endereços de e-mail presentes no início do livro).

4.15. Exercícios

1. Liste as cinco dificuldades mais frequentes que você já encontrou para realizar seu trabalho na Engenharia de Requisitos. Não se limite aos itens apresentados neste capítulo.
2. Das opções seguintes, assinale seu nível de impacto sobre a Engenharia de Requisitos.

	Impacto baixo	Impacto médio	Impacto alto
Rotatividade de desenvolvedores			
Rotatividade de gestores de negócio			
Mudança de versão da linguagem de programação			
Nível de maturidade da gestão organizacional do cliente			
Uso da ferramenta CVS para gestão de configuração de código-fonte			

3. Comente a frase: “boa parte dos problemas com requisitos é causada pelo próprio cliente”.

5. Tipos de Requisitos, Restrições e Premissas

“— Você me diria, por favor, qual caminho eu devo seguir a partir daqui?
— Isso depende bastante de para onde você quer ir.
— Eu não me importo muito onde.
— Então não interessa qual caminho você vá.”

Lewis Carroll (Alice no País das Maravilhas) “O mapa não é o território.”

Alfred Korzybski O objetivo deste capítulo é apresentar uma estrutura de classificação para os requisitos, explicando quais necessidades de informação caracterizam cada um desses tipos, sua finalidade e importância, e contextualizando isso no domínio do problema e da solução. Além disso, também se explica o que são restrições e premissas, a diferença entre esses fundamentos para o conceito de requisito e como eles se relacionam ao trabalho do analista de requisitos. A Figura 5.1 ilustra essa estrutura de classificação de requisitos.

5.1. Domínio do problema

O domínio do problema é onde surge o motivo para a mudança facilitada pela Engenharia de Requisitos. É a partir desse ponto que se deve buscar entendimento antes de desenvolver uma solução, sob o risco de ter a solução perfeita para o problema errado. Ter informações sobre isso é importante porque “deve-se conhecer a meta antes de conhecer o percurso” (Jean Paul Sartre).

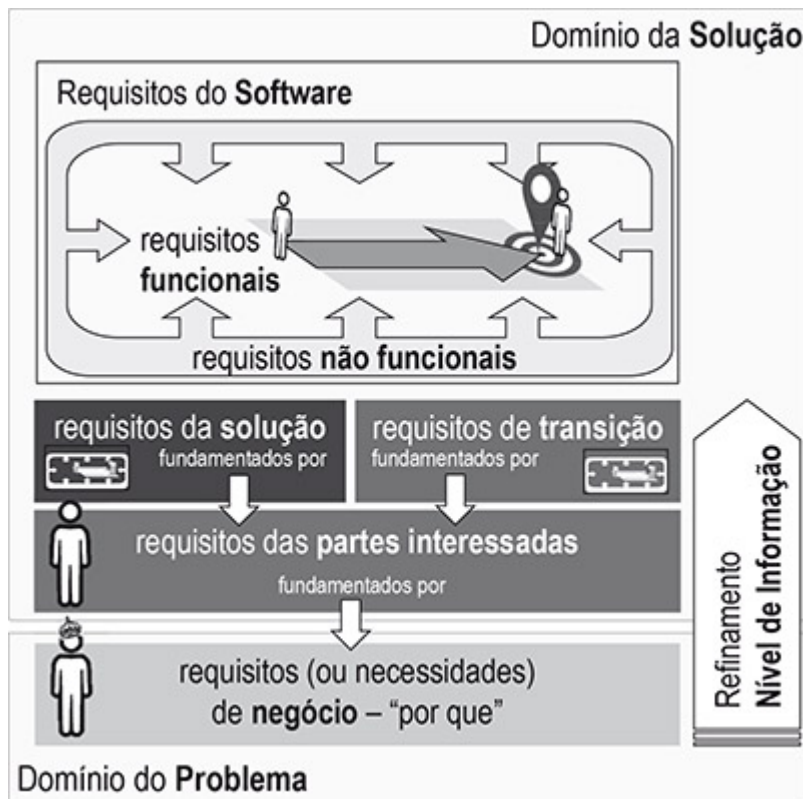


Figura 5.1. Os diferentes tipos de requisitos abordados pela Engenharia de Requisitos.

Para entender a importância do domínio do problema, cabe uma pergunta importante para toda a Engenharia de Requisitos: o que motiva uma organização a agir?

O domínio é uma área em análise. Ela corresponde às fronteiras de unidades organizacionais ou mesmo à organização como um todo; as partes interessadas chave e suas interações com os recursos compreendidos dentro das fronteiras.

Se a pergunta fosse o que motiva uma pessoa a agir e estabelecer novos objetivos, então uma resposta válida seria medo e esperança. Para as respostas, o imaginário popular criou uma simbologia associada ao chicote e à cenoura, ainda que no mundo atual entenda-se que isso não seja o suficiente (Figura 5.2).



Figura 5.2. A oportunidade ou o problema como motivação (crédito: Shutterstock).

Quando se coloca a mesma questão em uma perspectiva organizacional, deve-se antes definir onde as decisões sobre os objetivos que uma organização persegue são tomadas. Esse espaço é o chamado domínio do problema. O domínio do problema compreende uma área – ou mais – sujeita à análise. É compreendido por três itens, como indicado na Figura 5.3.



Fronteiras que delimitam cada uma dessas áreas. Sempre que se fala em uma fronteira, costuma-se pensar em um território com um governo que estabelece regras de convívio (leis) válidas para aquele território. No contexto da Engenharia de Requisitos, esses territórios são organizações, como empresas e órgãos públicos. Eles também podem ser uma ou mais áreas funcionais de uma organização, por exemplo: coordenação de contratos, departamento de marketing, diretoria financeira, equipe de manutenção *etc.*



Por trás de qualquer governo sempre há pessoas no comando. As partes interessadas chave são os agentes responsáveis por estabelecer e perseguir objetivos usando os recursos compreendidos nessas fronteiras. Exemplo: diretores, gestores, conselhos de administração, responsáveis técnicos *etc.*



Por fim, as partes interessadas chave possuem autoridade e responsabilidade que estabelecem como elas interagem com os recursos contidos nas fronteiras, como: decisões, resoluções, normas e políticas.

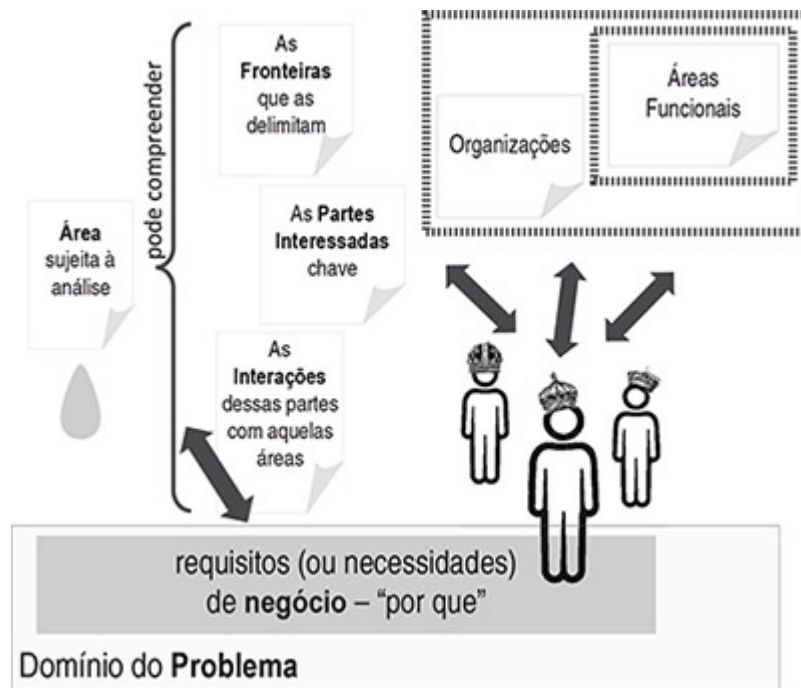


Figura 5.3. O domínio do problema em termos de seus elementos componentes. É nesse espaço que nascem os requisitos (ou necessidades) de negócio.

5.1.1. Qual é a sua importância?

O domínio do problema delimita o escopo inicial de uma solução em termos de áreas funcionais ou processos de negócios. É nele que estão as partes interessadas chave, que são o ponto de partida para toda a atividade da Engenharia de Requisitos. É nesse espaço que se encontram as informações sobre as relações de autoridade e responsabilidade que devem orientar o analista de requisitos a diferenciar o que é uma opinião do que é um fato.

5.2. Requisitos (ou necessidades) de negócio

Requisitos (ou necessidades) de negócio são declarações de mais alto nível de objetivos, metas ou necessidades da organização. Eles descrevem as razões pelas quais um projeto foi iniciado, as metas que o projeto deve atingir e as métricas que serão utilizadas para aferir o seu sucesso. Requisitos de negócio descrevem necessidades da organização como um todo e não de grupos ou partes interessadas.

5.2.1. O que são?

As necessidades do negócio são problemas a resolver – resultados de algum medo no paralelo traçado com indivíduos – por exemplo, um concorrente passa a oferecer ao mercado um novo produto com potencial de reduzir a sua base de clientes. Elas também podem ser oportunidades a aproveitar – como a esperança de alcançar algum benefício – por exemplo, entrar em novo mercado até então não explorado. Os exemplos citados são bastante abrangentes, mas isso não precisa ser assim. Eles podem ser bem menos abrangentes, como um chefe de departamento que vê a oportunidade de reduzir falhas na inspeção por meio de um melhor acesso à atualização da documentação dos procedimentos operacionais padrão.

O objetivo de resolver problemas ou aproveitar oportunidades é manter as condições atuais ou alterá-las para um novo cenário em que se deseja operar. Um exemplo associado à manutenção das condições atuais é o esforço com as adequações necessárias para se continuar operando em um determinado ambiente (“manutenção da liderança de mercado”), e um exemplo associado à alteração das condições atuais é a iniciativa para oferecer um novo serviço.

As necessidades de negócio representam os objetivos que uma área busca alcançar. São as métricas que servirão de base para avaliar o grau de sucesso ao final de iniciativas para o seu atendimento. Por exemplo, reduzir em 50% o custo atual do processo de arrecadação de contas de energia elétrica da concessionária do serviço.

5.2.2. Qual é a sua importância?

Com o conhecimento das necessidades de negócio, há mais liberdade para imaginar possíveis soluções. É comum a solicitação da área de negócio chegar na forma de um pedido para colocar o que é feito atualmente em uma planilha eletrônica em um sistema de informação. Sem conhecer a motivação por detrás da planilha e do pedido, é possível entregar o que foi pedido. Isso não tem o mesmo efeito de entregar o que é necessário.

Conhecer de forma clara as necessidades de negócio fornece importantes referências sobre o valor para o negócio das solicitações recebidas e, com isso, é possível mais facilmente estabelecer prioridades.

Iniciativas mais longas podem envolver o esforço de trabalho de muitas pessoas ao longo de meses ou anos. Facilmente, cenários assim são propícios a desvio dos objetivos estabelecidos. Ao interagir com os usuários, eles vão apresentando várias necessidades. Só que o analista de requisitos deve sempre buscar estabelecer uma ligação entre essas necessidades e os requisitos de negócio. Se não há essa ligação, significa que a necessidade está fora do escopo do projeto e não deve ser atendida, pois todos os requisitos da solução devem estar relacionados ao menos a um dos requisitos de negócio do projeto. Tal ligação, ou rastro, será tratada com mais ênfase ao apresentar o tema da rastreabilidade no Capítulo 9.

Revisitar as necessidades de negócio e os objetivos associados em marcos de validação intermediária permite avaliar se o caminho percorrido não se desviou do alvo e se ele representa a melhor solução.

5.2.3. Critérios de qualidade

As necessidades de negócio estão em um alto nível, no sentido de não entrar nos detalhes sobre a solução. As solicitações de negócio com as intenções iniciais costumam conter informações vagas e um tanto genéricas. Ainda que elas se mantenham em um alto nível, em seu desenvolvimento final elas não devem se manter vagas; devem se tornar específicas o suficiente para nortear o desenvolvimento dos requisitos e a sua posterior validação. O objetivo deste tópico é apresentar critérios de qualidade que podem ser usados para ajudar nesse refinamento.

Malcolm Forbes disse: “é tão mais fácil sugerir uma solução quando você não sabe demais sobre o problema”. Sem conhecer de maneira específica os critérios que permitam avaliar o sucesso da solução, muito maior é o universo de escolhas possível. Contudo, será que com isso a necessidade será atendida?

Um exemplo de declaração genérica e nebulosa de uma representação preliminar das necessidades de negócio é “aumentar a satisfação do cliente com o serviço de atendimento”.

Deve-se, portanto, estabelecer metas para o desenvolvimento das solicitações com as intenções iniciais em informações que possam ser usadas ao longo do projeto; sem que com isso se distancie das necessidades de negócio.

Se as necessidades de negócio representam objetivos a serem alcançados, então é possível usar o conhecimento desenvolvido na concepção da Administração Por Objetivos (APO). Trata-se de um processo de entendimento dos objetivos de uma organização, de maneira que a administração e os funcionários desempenhem as suas funções de acordo com os objetivos traçados e que os compreendam.

A APO introduziu o método SMART para avaliar a validade dos objetivos. Ele pode ser aplicado para avaliar se o estudo do domínio do problema produziu representações das necessidades de negócio que permitam o trabalho prosseguir em direção aos requisitos da solução.

Saiba mais!

O método SMART tem o objetivo de avaliar a validade de objetivos, que devem ser:

S – eSpecífico: descreve algo que apresenta um resultado observável.

M – Mensurável: são resultados passíveis de acompanhamento e medição.

A – Alcançável: as necessidades de negócio consideram a viabilidade do investimento.

R – Relevante: elas estão alinhadas com a visão, a missão e os objetivos-chave da organização.

T – Tempestivo: os objetivos definidos têm uma janela de tempo definida que é consistente com as oportunidades ou os problemas associados.

Ainda que haja críticas quanto à eficácia da APO nos seus objetivos na administração de empresas, o método SMART é uma boa ideia para apoiar a definição das necessidades de negócio para os propósitos da Engenharia de Requisitos.

O analista de requisitos não costuma ter poder para conduzir o trabalho em direção à declaração de uma necessidade de negócio que seja alcançável, relevante ou tempestiva; contudo, ele sempre deve garantir os dois primeiros critérios: específica e mensurável.

A intenção inicial de “aumentar a satisfação do cliente com o serviço de atendimento” não representa um resultado observável e não pode ser medido de maneira objetiva. Ao provocar de maneira proativa as partes interessadas chave em busca desses objetivos, poderiam ser lançadas perguntas: o que é a satisfação do cliente? Como medi-la? Qual a meta de aumento?

E se descobre que na verdade o cliente tem um problema que é um índice alto de reclamações devido à espera para atendimento. E que o seu desejo seria diminuir esse tempo, para que essa fonte de insatisfação fosse eliminada.

Neste caso, ainda se poderia perguntar qual o tempo de espera que seria tolerável, e então o requisito seria reformulado como: “reduzir o tempo de espera no atendimento a, no máximo, trinta segundos”.

5.2.4. Quando são elaborados?

Os requisitos de negócio são elaborados em atividades de análise de negócio, anteriores à criação do projeto. Normalmente são atividades que

estão fora do escopo de atuação da área de TI, porém é comum que a TI apoie algumas dessas atividades. Nas empresas é comum nomeá-las como: pré-projeto, anteprojeto, estudo de viabilidade, análise de viabilidade, análise de negócio *etc.*

5.2.5. Papel do analista de requisitos

Quando há atividades sérias de planejamento prévio, as intenções iniciais já são desenvolvidas em necessidades de negócio que observam as metas definidas pelo SMART. Porém, frequentemente é difícil encontrar as intenções iniciais observando naturalmente essas metas.

Os requisitos de negócio deveriam ser elaborados pelos próprios responsáveis do negócio interessados no projeto. O papel do analista de requisitos é refiná-los (se necessário), identificando questões que – uma vez respondidas – permitam melhor compreensão das intenções iniciais, ainda que sem buscar alternativas de solução. Por exemplo, considerando o requisito de negócio do estudo de caso do anexo ao final do livro: “agilizar a coleta de informações de obras de médio e grande porte”, algumas perguntas podem ser elaboradas para entender o domínio do problema e representar as necessidades de negócio de maneira adequada, como: ➤

Quais são as principais dificuldades na fiscalização e que dificultam a coleta de dados?

☐

Qual é o tempo médio gasto no preenchimento manual dos formulários?

☐

Qual é a sua ideia de agilidade de coleta de dados proposta?

☐

Agilizar em quantos % o processo de coleta de informação?



Como se mede a coleta de informações?

Esse assunto será discutido mais profundamente nos capítulos 6 e 7.

5.2.6. Onde ficam registrados?

Os requisitos de negócio costumam estar presentes em documentos que justifiquem o projeto e são originalmente elaborados pelas partes patrocinadoras da iniciativa em solicitações informais ou formais, como, por exemplo: ➤

Casos de negócio (*business case*).



Estudos de viabilidade.



Anteprojetos.



Termos de abertura de projetos.

A partir desses documentos ou a partir do levantamento e da análise juntos às partes interessadas chave (quando de solicitações informais), o analista de requisitos tipicamente elabora um documento de visão.

Esses mesmos documentos também cumprem o papel de capturar as partes interessadas, restrições e premissas, todos conceitos discutidos nos tópicos seguintes.

5.3. Restrições e premissas

O objetivo deste tópico é definir e exemplificar restrições e premissas de forma que possam ser identificadas e tratadas de maneira adequada ao

longo do desenvolvimento dos requisitos. Ambas são informações relevantes para o sucesso do desenvolvimento do projeto e que devem ser descobertas, analisadas e tratadas ao estudar o domínio do problema. Elas são requisitos? Não, mas afetam diretamente a sua definição.

5.3.1. Restrições

Do ponto de vista da gestão de projetos, restrições são limitações às opções para executar o projeto. Para o enfoque da Engenharia de Requisitos, restrição é qualquer limitação às possíveis soluções de software em desenvolvimento. Neste caso, o enfoque é nas limitações ao produto a ser entregue e não ao projeto em si. Não representam diretamente requisitos, mas induzem à definição de requisitos específicos. As restrições podem ser de origem de negócio ou técnicas e afetam o desenho da solução, sua construção, testes, validação e implantação. São aspectos da situação atual ou do estado futuro planejado que não podem ser mudados. Ou seja, restrições são impostas, não se negociam.

5.3.1.1. Papel do analista de requisitos

Muitas restrições já são definidas na criação do projeto; outras são descobertas ao longo do trabalho de requisitos. No entanto, é importante que toda restrição identificada seja validada, pois muitas vezes ela é falsa. E isso pode levar o projeto para uma proposta de solução que não é a mais adequada.

Por exemplo, há uma restrição de prazo muito apertado no projeto para que ele atenda a uma determinação legal. Devido ao prazo exíguo, a solução proposta pode ser a mais rápida de se construir, mesmo que não seja a mais elegante ou a que resolva o problema de forma definitiva. No entanto, ao se validar a razão dessa restrição de prazo, pode-se descobrir que para atender à questão legal na data estipulada não é necessário que todo o projeto esteja concluído, somente uma parte das funcionalidades é que precisa estar operando na data específica. Neste caso, então, há a possibilidade de elaborar uma solução mais bem-acabada, com um prazo de entrega proporcionalmente mais longo, porém considerando no plano maior do projeto uma entrega intermediária para atender à data crítica.

5.3.1.2. Qual é a sua importância?

Quando as restrições não são identificadas tempestivamente, então soluções que nem deveriam ser cogitadas passam a ser, e vão desperdiçando tempo e recursos do projeto. Não raro, são as restrições que direcionam o trabalho no sentido da maior ou menor abrangência dos requisitos da solução e o grau de qualidade exigido. Por exemplo, uma data fixa de entrega pode implicar em concessões quanto ao escopo e às exigências de usabilidade.

5.3.1.3. Restrição de negócio

As restrições de negócio refletem limites sobre: ➤

Recursos orçamentários ou de tempo: por exemplo, a diretoria limitou o orçamento da solução em até um milhão de reais ou a data da implantação deve ser em até um mês antes do próximo Natal.



Disponibilidade dos envolvidos no projeto: por exemplo, limitações para que apenas uma pessoa de cada departamento seja envolvida no projeto ou para que certas partes não sejam afetadas pela solução.



Restrições baseadas nas habilidades da equipe de projeto e das partes interessadas: por exemplo, não há instrumentos para avaliar o progresso e fundamentar pagamentos intermediários para viabilizar o uso de abordagens ágeis de desenvolvimento alinhado às exigências de governança corporativa em vigor.



Outras restrições organizacionais: por exemplo, o uso de impressão digital não é possível para autenticação de usuário, pois alguns trabalhadores não a possuem nítida o suficiente devido à atividade desempenhada.

5.3.1.4. Restrição técnica

As decisões de negócio no mundo atual impõem várias restrições associadas à tecnologia. São restrições estabelecidas ainda no domínio do problema e que não devem ser modificadas ao longo do desenvolvimento da solução. Uma restrição técnica pode incluir qualquer decisão prévia de arquitetura que possa impactar o projeto da solução. São exemplos de restrições técnicas: ➤

Linguagem de programação. Exemplo: o componente de software da solução deve ser desenvolvido em C#.NET. Ou seja, mesmo que exista uma equipe experiente em Java e bibliotecas nesta linguagem que acelerariam o trabalho, tal opção não é viável devido à restrição.



Plataformas e produtos específicos de software e hardware. Exemplo: o projeto é criado já com a decisão tomada de que a parte móvel da aplicação utilizará o coletor da marca Symbol versão 09 e que o software deve funcionar no Internet Explorer versão 10.x sem necessidade de *plugins*. É muito comum o cliente colocar restrições nesse sentido, principalmente pensando em aproveitar o que já existe disponível na organização.



Tamanho de mensagens. Exemplo: as mensagens XML que transitarão via MessFlow não podem exceder a 20 KB. Este tipo de restrição pode surgir em função de disponibilidade de banda de rede na qual o software irá operar.



Espaço ocupado pelos componentes de software da solução. Exemplo: o tamanho de cada componente de software da solução executando na parte móvel não deve exceder 5 MB. Para software que operará em dispositivos específicos, isso pode ser um tipo de restrição comum.



Número máximo e espaço ocupado pelos arquivos de dados.

Exemplo: o número máximo de arquivos abertos na parte móvel da aplicação em um mesmo momento não deve ultrapassar 50 arquivos e a soma do espaço ocupado pelos arquivos não deve exceder 1 MB. Idem ao item anterior e muito comum em software embarcado.



Limitações quanto ao uso de processador ou memória. Exemplo: os componentes de software a ser executado no validador não devem exceder 64 KB.

5.3.2. Premissas

Premissas são suposições, informações que se acreditam como verdadeiras, mas que ainda não foram confirmadas. Muitas vezes essas premissas são criadas pelo próprio analista de requisitos, para que se possa seguir adiante com a elaboração dos requisitos.

Alguns exemplos de premissas:



O serviço de cálculo do valor residual do contrato estará disponível no sistema CFF em sessenta dias. Esta premissa indica que o sistema CFF fornecerá em uma data futura uma informação necessária para o nosso sistema que está em desenvolvimento. Se isso não se confirmar até o final destes sessenta dias, o prazo do nosso projeto poderá ser diretamente impactado.



Os usuários são fluentes no idioma inglês e têm acesso à internet em banda larga. Esta premissa leva à decisão de que o software não seja multi-idioma (apenas inglês), pois reduz custo, e que se pode trabalhar a interface de usuário com recursos de multimídia

(imagem, vídeo e som) de alta qualidade, pois há disponibilidade de banda de rede dos usuários. Se durante o projeto descobre-se que essa informação não é verdadeira, todo o projeto de interface e eventualmente a arquitetura terão que ser reformulados para dar suporte a mais de um idioma e para operar com menos capacidade de banda de rede.



O acesso às transações do software é apenas via navegador Mozilla Firefox. Esta premissa pode levar à decisão de usar recursos específicos deste navegador. Se mais tarde descobrir-se que haverá usuários acessando o sistema via outros navegadores, uma parte significativa do trabalho poderá ser refeita.



Todos os clientes possuem CPF. Esta premissa pode levar à decisão de usar o CPF como chave de acesso do cliente ao sistema. Se mais tarde se descobre que há clientes que não têm CPF, uma parte significativa do processo de autenticação do cliente poderá ser refeita.

Muitas premissas têm relação com dependências que vão sendo criadas no projeto com o trabalho de outras equipes. Neste caso, é mais fácil identificar a premissa como uma “promessa”. Por exemplo, a equipe do sistema CFF prometeu entregar o serviço de cálculo de saldo em sessenta dias para uso pelo nosso projeto. Ao especificarmos o software do nosso projeto, consideramos que ele usará este serviço, que estará disponível. No entanto, o que acontecerá se ao final dos sessenta dias descobrirmos que o serviço prometido não está disponível?

5.3.2.1. Qual é a sua importância?

Caso as premissas não se confirmem, riscos irão se materializar para o projeto. Portanto, devem ser tratadas como riscos a serem gerenciados. Para isso, necessitam ser documentadas e comunicadas aos responsáveis pela

gestão de riscos, para que estes possam incluí-las em seu planejamento e controle.

Há premissas que não têm relação direta com o trabalho de requisitos, mas com outras questões do projeto. Nesses casos, provavelmente não haverá participação do analista de requisitos na sua identificação. Por exemplo, o patrocinador informou que o projeto pode ser planejado com a disponibilidade de recursos à razão de cem mil reais por mês para sua execução, porém isso ainda não foi homologado pela diretoria.

Um contraexemplo de premissa é quando já existe certeza de uma informação, então ela não deve ser considerada premissa. Não há mais risco dela não se confirmar. Se a informação de que todos os clientes possuem CPF é uma certeza, então qualquer decisão pode ser tomada com base nessa informação, sem risco de que o fundamento da decisão desmorone.

Tanto as premissas quanto as restrições são definidas no domínio do problema e a diferença entre elas é que a restrição reflete uma decisão final já tomada e a premissa é uma decisão em primeira instância que pode, ou não, se confirmar.

5.3.2.2. Papel do analista de requisitos

A gestão de riscos não é responsabilidade do analista de requisitos. Porém, durante o trabalho de requisitos, ele deve estar atento para a percepção de premissas como tal, ou mesmo o seu estabelecimento e obtenção de consenso sobre ela. Isso porque muitas vezes é necessário assumir premissas para que o desenvolvimento da solução possa prosseguir.

Portanto, o analista de requisitos deve trabalhar de forma colaborativa com o responsável pela gestão de riscos, informando-o de cada nova premissa identificada ou criada. O ideal seria tentar confirmar imediatamente toda premissa, mas nem sempre isso é possível ou está sob o controle do analista de requisitos.

5.4. Partes interessadas

O objetivo deste tópico é definir o que são as partes interessadas. Seu significado costuma ser de entendimento geral, mas formalismo, contudo, é

necessário para os fins específicos da Engenharia de Requisitos.

Isaac Asimov, um dos mais celebrados autores de ficção científica de todos os tempos, disse: “pessoas que pensam saber tudo são um grande incômodo para aqueles entre nós que sabem”. Muitos podem pensar saber de tudo ou, pelo menos, se sentir assim.

Essa visão do mundo é pertinente ao analista de requisitos? Não. Ele não deve se colocar na posição de dono da verdade. Deve reconhecer que são as partes interessadas que detêm o conhecimento e que cabe a ele facilitar o processo de revelação e organização desse conhecimento.

5.4.1. O caminho a partir dos requisitos de negócio

Uma vez entendidos o domínio do problema e as necessidades de negócio, há início a exploração do domínio da solução. No domínio do problema, as necessidades de negócio estabelecem resultados a serem alcançados sem detalhar ou estabelecer quais caminhos seguir para que sejam atendidas. A identificação de uma necessidade, a decisão sobre qual o caminho percorrer e a realização da jornada correspondente requerem uma série de etapas, como as ilustradas na Figura 5.4.

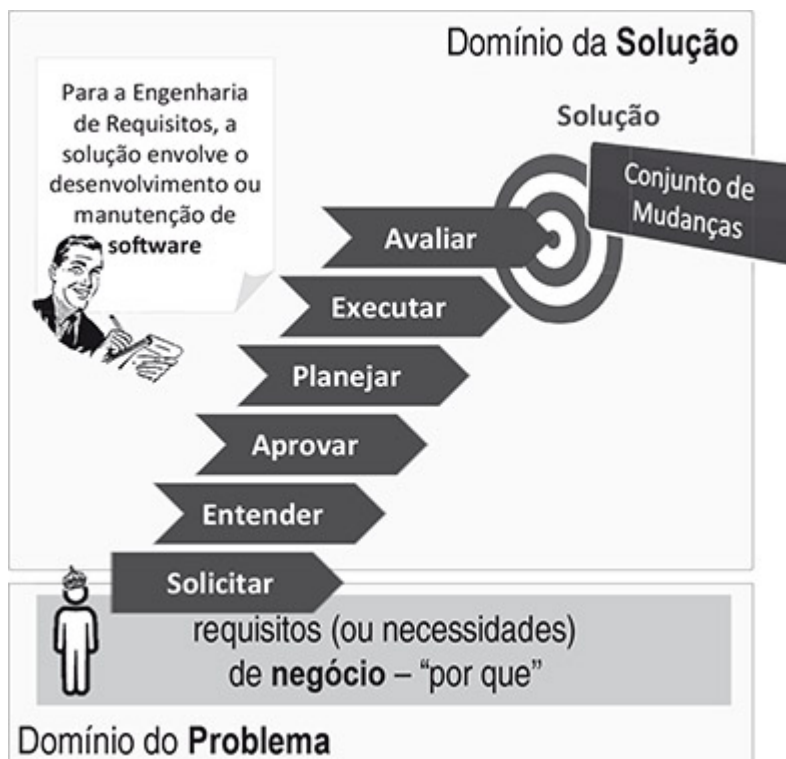


Figura 5.4. Etapas da identificação das necessidades de negócio em direção à solução.



Solicitar: as necessidades identificadas no negócio originam solicitações para seu atendimento e têm um processamento associado a elas.



Entender: as solicitações devem ser entendidas e alternativas associadas desenvolvidas para apreciação.



Aprovar: as alternativas devem ser avaliadas e a melhor solução encaminhada para sequência em seu atendimento.



Planejar: a sequência no atendimento requer mobilizar recursos e estabelecer metas em um plano para a execução da alternativa escolhida.



Executar: o plano deve ser executado, o progresso monitorado e ajustes realizados conforme a sua evolução.



Avaliar: os resultados entregues devem ser avaliados para confirmação dos objetivos propostos e adoção das medidas corretivas, se necessário.

Quando se fala em atendimento às necessidades, alternativas de solução e resultados subsequentes, o que se busca como objetivo é um conjunto de

mudanças à situação atual. A esse conjunto de mudanças se dá o nome de solução, que muitas vezes é implementada por software, enfoque deste livro. E essas mudanças são promovidas por projetos.

O foco deste tópico é discutir os agentes que têm o conhecimento necessário para evoluir na progressão entre essas etapas ao longo do desenvolvimento de requisitos intermediários, necessários à especificação dos requisitos da solução a partir das necessidades de negócio.

5.4.2. O que são?

As necessidades de negócio normalmente são fruto de uma resolução coletiva que reflete os desejos da área em análise como um todo. A jornada a partir das necessidades de negócio em direção à solução exige desenvolvê-las junto aos indivíduos responsáveis pela identificação e seleção da melhor solução dentre os diferentes caminhos em potencial. Esses indivíduos são as partes interessadas, que podem: ➤

- afetar a solução de alguma forma; ➤
- ser afetados de alguma forma por ela.

Ainda que na qualificação anterior o foco esteja na solução, observe que, independentemente dela, as partes interessadas compartilham as mesmas necessidades de negócio.

5.4.3. Clientes e usuários × partes interessadas

Os termos “cliente” e “usuário” são muito comuns de usar para se referir aos indivíduos que deverão ser envolvidos no trabalho de requisitos como fontes de informação. E, de fato, clientes e usuários são casos específicos de partes interessadas. Mas o conceito de parte interessada é mais amplo. Quando se refere ao cliente, o senso comum leva a imaginar que é quem paga pelo projeto. Quando se refere ao usuário, o senso comum leva ao indivíduo que usará o produto a ser desenvolvido pelo projeto. E são bastante razoáveis esses entendimentos. Porém, para a maioria dos projetos, uma parte significativa dos requisitos será obtida com partes interessadas que não são clientes ou usuários.

Portanto, será utilizado ao longo deste livro o termo “parte interessada” no lugar de cliente ou usuário (a não ser que efetivamente esteja se referindo a quem usa o sistema, e, então, o termo “usuário” será utilizado em especial). Os modelos de maturidade para desenvolvimento de software CMMI e MPS.BR usam o termo “fornecedores de requisitos”, que é o mesmo sentido dado para “parte interessada” neste livro.

5.4.4. Autoridade e responsabilidade

As partes interessadas – para os objetivos da Engenharia de Requisitos – são pessoas envolvidas no desenvolvimento dos requisitos. Por exemplo, no desenvolvimento de uma nova solução de autoatendimento bancário, identificam-se inicialmente como partes interessadas: ➤

- os usuários de serviços bancários, sejam eles clientes do banco ou não; ➤

- os operadores da retaguarda do serviço (ex.: abastecimento de dinheiro, recolhimento de envelopes etc.).

Nem sempre a parte interessada é uma pessoa em particular que se convida a participar do desenvolvimento dos requisitos. Por exemplo, existe um grupo difuso de milhões de pessoas na condição de usuárias de serviços bancários, o que inviabiliza que essas partes interessadas sejam todas ouvidas no desenvolvimento dos requisitos. Elas são então representadas por alguém do banco, seja um gestor de canais eletrônicos ou de marketing, que cumprirá o papel desta parte interessada junto ao analista de requisitos.

De forma análoga, há vários operadores de retaguarda atuando nos distintos pontos de autoatendimento. Talvez interagir com todos seja inviável; logo, designar um representante (ou amostra) selecionado deste grupo para interagir no levantamento de requisitos é fundamental.

Seja a parte interessada diretamente envolvida ou uma parte interessada representada, aquele envolvido no desenvolvimento dos requisitos deve ter autoridade e responsabilidade compatíveis com as decisões necessárias à Engenharia de Requisitos.

5.4.5. Qual é a sua importância?

A identificação das partes interessadas é fator crítico de sucesso para o trabalho de requisitos. Uma parte interessada não identificada irá significar requisitos sendo descobertos tardiamente no projeto ou omitidos, gerando provavelmente grande retrabalho mais à frente no projeto ou mesmo o seu fracasso.

5.5. Requisitos das partes interessadas

A função das partes interessadas no desenvolvimento dos requisitos é fornecer informações e *feedback* quanto à compreensão destes. A essas informações se dá o nome de requisitos das partes interessadas. O objetivo deste tópico é entender o que são esses requisitos e as suas particularidades.

Há um comentário de Miles Davis que descreve a importância de entender e saber tratar os requisitos das partes interessadas: “se você entendeu tudo o que eu disse, você deve ser eu”. Ainda que sejam requisitos intermediários, é a partir deles que surgem oportunidades para identificar conflitos a serem resolvidos e oportunidades de consolidar requisitos semelhantes.

Os requisitos das partes interessadas são os resultados intermediários do desenvolvimento das necessidades de negócio em direção à especificação da solução e devem ser fundamentados por esses últimos. Eles: ➤

- são obtidos junto às partes interessadas; ➤
- descrevem suas necessidades de informação para o desempenho de suas tarefas; ➤
- representam a visão da parte interessada sobre a sua interação com a solução; ➤
- são produtos das atividades de elicitação de requisitos (descrita no Capítulo 7); ➤
- são insumos das atividades de análise de requisitos (descrita no Capítulo 8).

5.5.1. Onde ficam registrados?

Como resultado das atividades de levantamento de informações junto às partes interessadas e da volatilidade da memória humana, devem ser criados

registros que documentem as informações levantadas e as decisões tomadas nas mais diferentes formas. Esses registros podem ser: ➤

gravações;



notas;



atas.

Durante a condução do evento de levantamento, pode ser adequado registrar: a)

apenas os pontos-chave em notas; b)

a gravação em áudio e/ou vídeo de todo o evento; ou ainda c)

a elaboração de uma ata.

O termo ata tem sido usado com um significado diferente do associado à atividade de secretariar uma reunião registrando todos os atos e fatos ocorridos naquele evento. Ele se aproxima de um documento de entendimento ou memória de levantamento, elaborados após o evento de levantamento.

Explorar as condicionantes para essas opções será objeto de discussão em cada técnica apresentada no Capítulo 7. O tópico sobre o nível de detalhe dos requisitos do Capítulo 2 também é pertinente, ao descrever os requisitos das partes interessadas.

O exemplo na Figura 5.5 ilustra um documento com a memória de levantamento. Ao longo dos capítulos 7 e 8 o leitor receberá orientação prática, principalmente sobre como elaborar melhor a pauta que servirá como fio condutor para os assuntos tratados nas atividades de levantamento e análise de requisitos.

1. Identificação			
Detalhamento sobre o processo proposto de arrecadação			Nº 004
Data reunião	Início	Término	Local

03/02/2018	09:00	11:45	Sala de reunião 109
Participante	P/A	Grupo/Empresa	E-mail
<i>P = Presente, A = Ausente</i>			
2. Pauta			
a) Levantamento de informações sobre a arrecadação de contas			
Assuntos tratados			
a) A arrecadação de contas será realizada por agentes arrecadadores (farmácias, loterias e pequenas lojas) e nem sempre com acesso à internet. b) Os agentes arrecadadores que não possuem acesso à internet realizarão a transferência dos dados de arrecadação ao final do dia por meio de malote. c) O controle da taxa de arrecadação devida ao agente depende da forma como este transmite os dados e do volume de arrecadação. d) O valor arrecadado pelos agentes será depositado na conta da controladoria, e os dados serão transmitidos conjuntamente com os dados da arrecadação.			
3. Pendências			
Descrição	Responsável	Critic.	Limite
<i>Criticidade: A = alta, M = média ou B = baixa</i>			
4. Histórico do documento			
Versão	Autor	Data	Observações
1.0	Carlos Eduardo Vazquez	04/02/2018	Versão inicial

Figura 5.5. Exemplo de documento com requisitos das partes interessadas.
Fonte: FATTO Consultoria e Sistemas.

5.6. Requisitos da solução

Os requisitos das partes interessadas originalmente contêm conflitos a resolver, lacunas de informação, oportunidades de racionalização, enfim, representam informações ainda não estruturadas e devem ser organizados em especificações antes de serem usados no desenvolvimento do software. Ao produto desse trabalho de resolução dos conflitos, eliminação das lacunas de informação e aproveitamento das oportunidades de racionalização é dado o nome de requisitos da solução. E esses requisitos são materializados em uma especificação de requisitos. Os requisitos da solução são produto do trabalho da análise de requisitos, descrita no Capítulo 8.

Os requisitos da solução descrevem suas características de forma a satisfazer os requisitos de negócio e os requisitos das partes interessadas. De forma análoga aos requisitos das partes interessadas, cabe manter a rastreabilidade entre os requisitos da solução e os requisitos anteriores que o fundamentam; o Capítulo 9 explica o tema da rastreabilidade e orienta em sua utilização.

5.6.1. Qual é a sua importância?

As especificações com os requisitos da solução capturam todas as decisões tomadas sobre o escopo e o comportamento esperado para o software em desenvolvimento de tal forma que não se percam nem sejam esquecidas. Caso isso acontecesse, então haveria retrabalho de algum profissional responsável por outra disciplina da engenharia de software para revisar assuntos já discutidos e decididos.

O próprio processo de elaborar as especificações antecipa a descoberta da necessidade de novas decisões a partir de requisitos das partes interessadas em desenvolvimento. Isso evita assumir premissas desnecessárias em atividades subsequentes à Engenharia de Requisitos.

Com as especificações é possível validar o entendimento da solução entre os responsáveis pelo desenvolvimento e as demais partes interessadas antes que trabalho desnecessário e mais oneroso seja despendido. Neste processo

há identificação de especificações incompletas, inconsistentes e aquelas que não atendem às necessidades de negócio. Melhor identificar isso neste momento do que no teste de aceitação do usuário!

5.6.2. Processo geral de desenvolvimento dos requisitos

De maneira resumida, um processo para o desenvolvimento dos requisitos consiste em três passos, ilustrados na Figura 5.6: ➤

Organizar os requisitos das partes interessadas em um escopo da solução: foco na abrangência e sem profundidade ainda.



Desenvolver os requisitos das partes interessadas, resolver conflitos, eliminar redundâncias, superar lacunas de informação para rever o escopo e detalhar os requisitos da solução.



Fechar um pacote de requisitos com a profundidade suficiente para encaminhar um conjunto de especificações para atualizar a arquitetura; implementar e testar as unidades que compõem o software; e assim por diante, conforme o processo em que a Engenharia de Requisitos se insere.

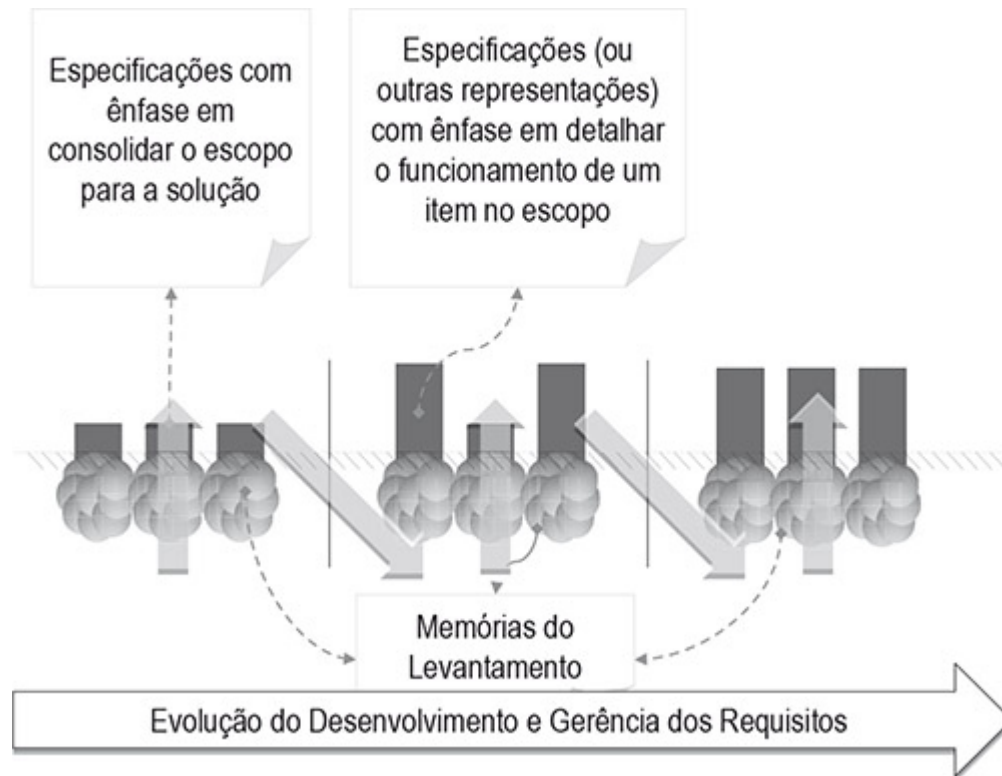


Figura 5.6. Uma visão geral das etapas do desenvolvimento dos requisitos das partes interessadas (os globos abaixo) para as especificações (os paralelepípedos acima).

5.7. Requisitos de transição

Imagine que na construção de um edifício seja necessário construir um vestiário e um refeitório para os operários. Ao final da obra, o vestiário e o refeitório serão desmontados, pois não farão parte do prédio propriamente dito – seu uso foi necessário apenas durante a construção. No caso da Engenharia de Requisitos, esses seriam os requisitos de transição.

Eles cumprem o papel de permitir que a nova solução que será desenvolvida possa entrar em operação plena, da forma desejada pelo cliente. Quando não há uma solução atual, então eles estão ausentes e trata-se de uma solução de inovação total. Eles não podem ser definidos até que o desenho da solução seja finalizado e são descartados após a solução ser entregue por completo. Sua identificação e seu desenvolvimento requerem as mesmas tarefas e técnicas dos requisitos da solução. Também são

materializados em uma especificação de requisitos, fruto de atividades de análise de requisitos.

Exemplos de requisitos de transição resultantes dessa análise não se limitam ao software e podem incluir elementos como: ➤

- os dados de agentes arrecadores e de clientes com seus respectivos históricos serão migrados dos fichários hoje mantidos manualmente para o novo sistema informatizado; ➤

- os agentes arrecadores serão treinados na utilização do novo sistema antes de sua entrada em produção; ➤

- no primeiro mês de operação da nova solução em produção, a solução atual continuará em funcionamento em paralelo e será descontinuada a partir desse período; ➤

- os dados de contrato serão migrados do sistema legado para o ERP, após filtragem e ajustes manuais.

Desses exemplos, para o foco do trabalho do analista de requisitos, devem-se segregar os que apenas envolvem o desenvolvimento de software dos que envolvem ações em outras esferas (por exemplo, gestão de projetos).

Mas, em resumo, os exemplos mais comuns de requisitos de transição são migrações de dados do sistema velho para o novo sistema que está sendo desenvolvido.

5.7.1. Qual é a sua importância?

A motivação para haver um tipo de requisito para descrever a transição para a nova solução é propiciar a melhor organização das especificações na gestão de requisitos. Tal organização permite uma melhor gestão do conhecimento, comunicação e medição funcional do projeto de software.

A gestão do conhecimento (ou pelo menos a sua retenção) exige a segregação dos requisitos da solução, que continuam válidos após a entrega, dos requisitos de transição, que duram o tempo de vida do projeto. Essa segregação também facilita o entendimento entre a equipe de desenvolvimento e seus clientes, melhorando com isso a comunicação entre essas partes. Por fim, os requisitos de transição são insumos para

identificação de funcionalidades de conversão na análise de pontos de função.

Para muitos projetos a maior complexidade está justamente na transição e não no produto a ser construído. Imagine uma companhia aérea que deseja trocar seu sistema de vendas de passagens. Será que seria aceitável a estratégia de implantar o novo sistema solicitando aos usuários do departamento comercial que digitassem no novo sistema todas as passagens vendidas no sistema velho e que ainda não foram usadas? Ou que a venda de passagens fosse suspensa por algumas horas enquanto os dados do sistema velho são transferidos para o novo sistema? Provavelmente não. O mais plausível é que o cliente deseje uma transição na qual a sua operação de vendas não seja interrompida por nenhum momento. E isso já faz com que a transição a ser elaborada deixe de ser trivial.

5.7.2. Papel do analista de requisitos

A identificação dos requisitos de transição deve ser precedida da avaliação de dois cenários: o cenário-alvo, necessário à nova solução em funcionamento (*to-be*), e o cenário atual, indicando a situação atual como está (*as-is*). A partir da comparação desses dois cenários, identificam-se itens de ação que devem ser atendidos (*to-be*) para que a solução entre em plena operação.

5.8. Requisitos de software: solução + transição

Os requisitos de software são compostos pelos requisitos da solução (o produto a ser entregue) e pelos requisitos da transição (se houver). Ambos são compostos por requisitos funcionais e requisitos não funcionais, como ilustrado na Figura 5.7.

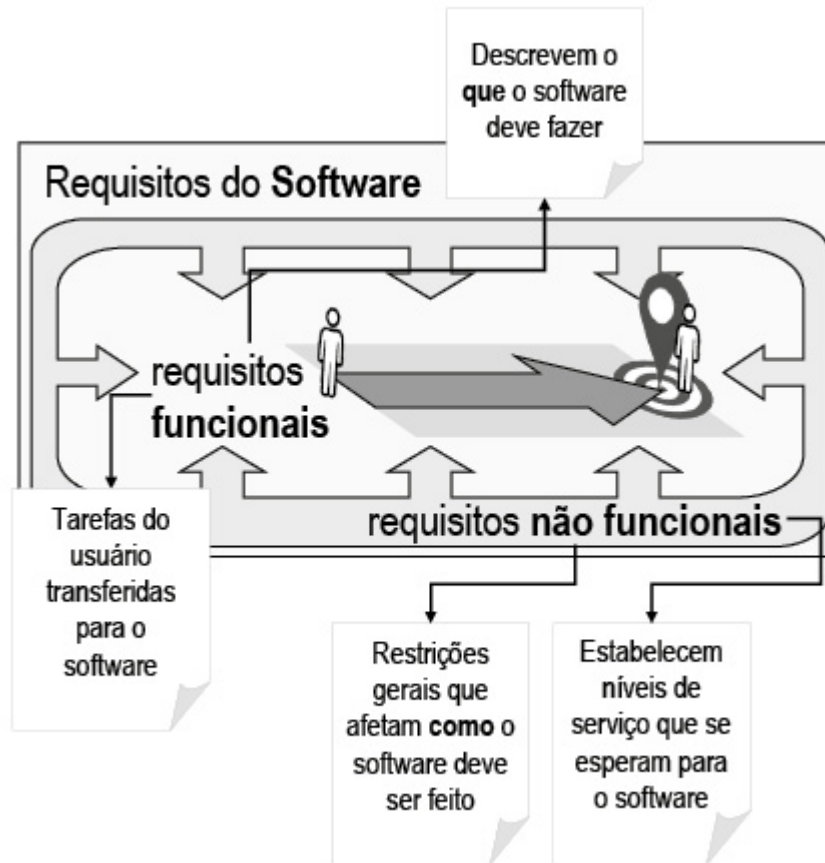


Figura 5.7. Os requisitos de software definidos por requisitos funcionais e não funcionais.

Resumidamente, os requisitos funcionais descrevem o que o software faz, considerando uma perspectiva de tarefas e serviços de seus usuários em específico. Os requisitos não funcionais descrevem qualidades que o produto de software deve observar para ser efetivo e limitações gerais ao funcionamento dos requisitos funcionais estabelecidos para o software.

5.8.1. Onde são armazenados?

Os requisitos de software são mantidos em diversos tipos de artefatos, que variam conforme a metodologia de desenvolvimento de software usada pela organização. Por exemplo: ➤

Documento de visão.

Lista de requisitos.

☐

História de usuário.

☐

Caso de uso.

☐

Modelos.

☐

Layout de telas e relatórios.

☐

Especificação funcional.

☐

Especificação complementar.

☐

Glossário.

5.9. Requisitos funcionais

Os requisitos funcionais descrevem o comportamento que o software deve ter em termos das tarefas ou serviços do usuário. Isso se opõe à descrição do desenho da arquitetura da solução ou de sua implementação em uma plataforma tecnológica usando determinadas linguagens de programação. Um requisito funcional não é e nem substitui uma especificação de

programas, de componentes ou coisa similar. Vale ressaltar que os requisitos funcionais não descrevem o desenho da arquitetura de solução, mas são profundamente afetados por ela.

Não se deve supor que o trabalho de arquitetura se inicia somente após o trabalho de requisitos. Decisões arquiteturais de alto risco devem ser tomadas o mais cedo possível, e determinados requisitos funcionais em um cenário de arquitetura passam a ser requisitos não funcionais em outro. Por exemplo, o controle de acesso de usuários corporativos. Dependendo das decisões arquiteturais tomadas no início do ciclo de vida, o controle de acesso pode ser parte da especificação funcional ou então parte de uma infraestrutura comum de suporte que aborda requisitos não funcionais de segurança.

O comportamento esperado pelo software é descrito em um requisito funcional se refere ao intercâmbio de informações entre o usuário, o software sendo descrito e os meios de armazenamento até que um objetivo específico seja alcançado. Esse objetivo específico – um objetivo do usuário – é concluir a tarefa sob sua responsabilidade de tal maneira que seus resultados possam ser usados como insumos em outras tarefas por usuários com outras responsabilidades ou em outros momentos – seja por um usuário com as mesmas responsabilidades ou não. A Figura 5.8 ilustra essa relação entre os requisitos funcionais.

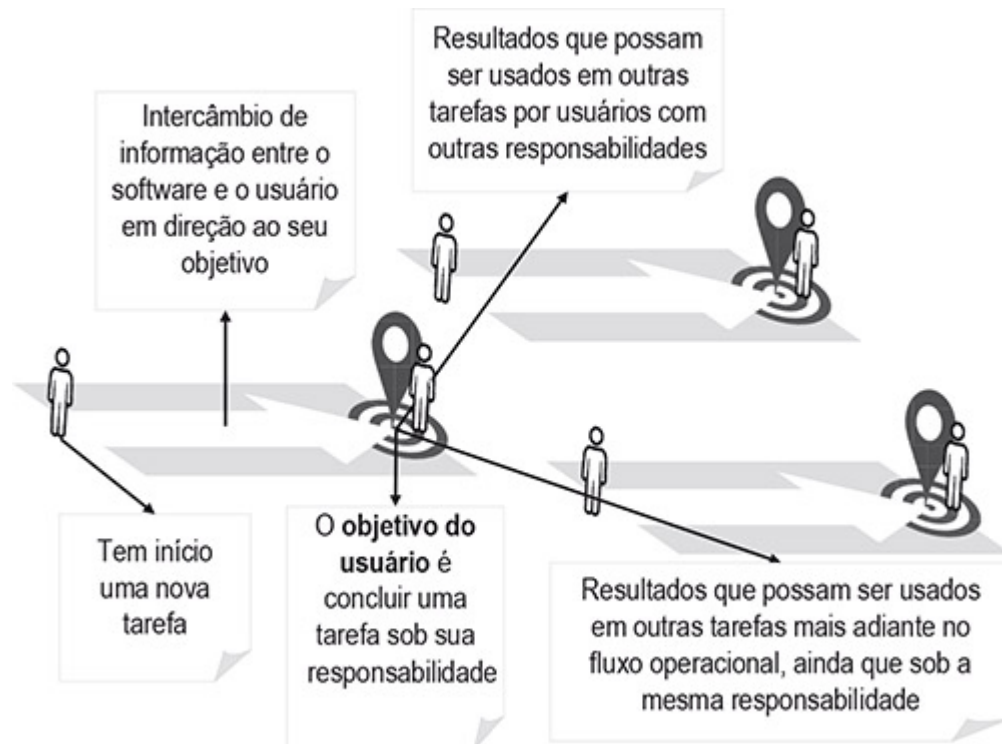


Figura 5.8. O que são os requisitos funcionais e a natureza da inter-relação entre eles.

5.9.1. Onde são armazenados?

Os requisitos funcionais podem estar presentes como parte de vários artefatos, como: documento de visão, listas de requisitos, histórias do usuário, especificações de casos de uso e modelos de processo. Esses tipos de artefatos serão abordados no Capítulo 8.

Para fins de simplificação, se denominará especificação funcional um documento que contenha requisitos funcionais, ainda que estes estejam em processo de desenvolvimento e não em seu estágio final.

5.9.2. Papel do analista de requisitos

São exemplos de requisitos em especificações funcionais descrições como:

1. Efetuar a gestão dos cursos.
2. Emitir o certificado de participação do aluno no curso.
3. Garantir que somente alunos com frequência superior a 75% possam emitir seu certificado.

Todos esses exemplos são de requisitos funcionais, contudo apenas o segundo está especificamente associado a uma única tarefa ou serviço do usuário.

Isso é muito comum. Boa parte do trabalho do analista é refinar requisitos mais abrangentes, como o apresentado no primeiro exemplo, ou consolidar fragmentos, como o apresentado no terceiro exemplo, até que se chegue ao nível adequado da especificação.

Ou seja, durante o desenvolvimento dos requisitos o esperado é que haja nas especificações funcionais elementos que tenham diferentes granularidades. A seguir explica-se melhor o que seja nível de granularidade no âmbito de uma especificação funcional.

5.9.3. Nível de granularidade

O nível de granularidade é a maior ou menor abrangência na descrição do comportamento esperado para o software em uma especificação funcional. Essa abrangência está relacionada ao tipo de objetivo associado ao requisito presente nessa especificação. A Figura 5.9 ilustra a relação entre esses objetivos e o nível de granularidade. Será usada uma classificação de três níveis de granularidade proposta por Cockburn (2000) para casos de uso e generalizada pelos autores para requisitos funcionais.

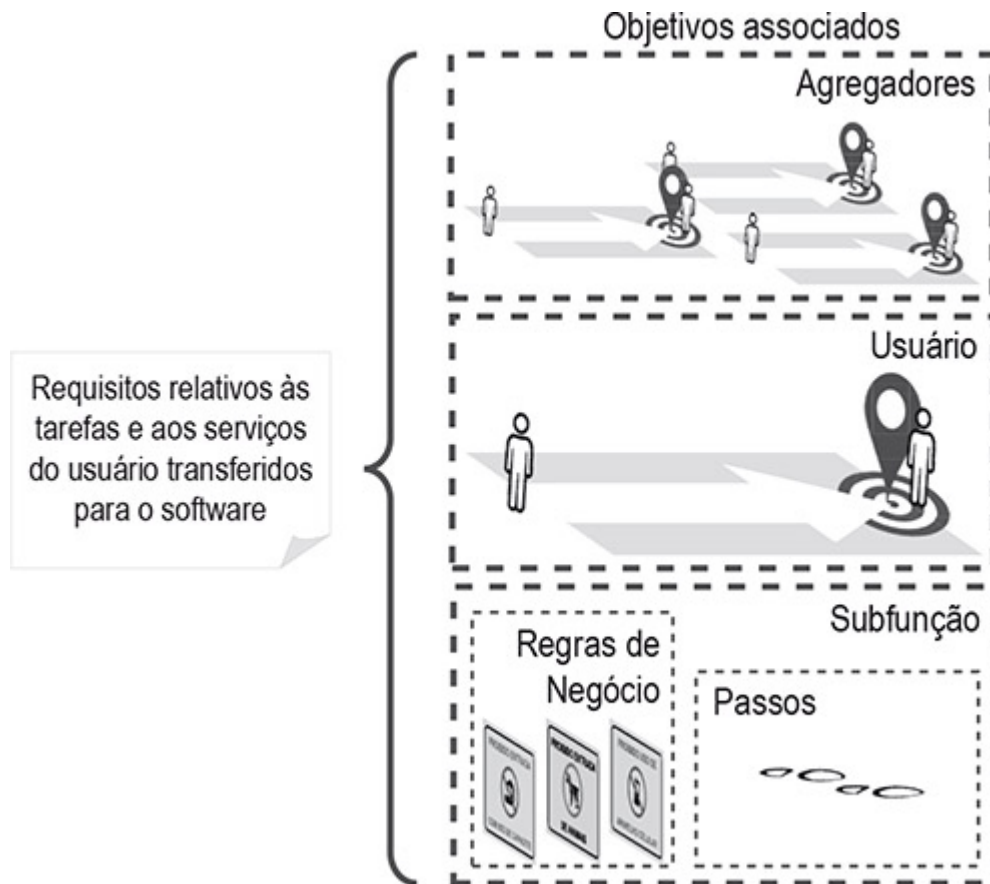


Figura 5.9. Diferentes níveis de granularidade em requisitos presentes em uma especificação funcional e sua relação com os objetivos associados.

5.9.4. Requisitos funcionais com objetivo de usuário

Se nada for dito ao contrário quando se refere a um requisito em uma especificação funcional, então se trata do próprio requisito funcional como descrito anteriormente. É o requisito funcional especificado: ➤

no nível de uma única tarefa sob a responsabilidade de um único indivíduo; ➤

em um determinado momento no qual o usuário tem tudo o que precisa para que a tarefa seja concluída até o limite de sua responsabilidade no fluxo operacional em que ela se insere.

Ao final da tarefa, o usuário cumpre seu objetivo, fica satisfeito, não há nada mais a se fazer. Se um trabalho envolve mais de um indivíduo, é porque há mais de uma tarefa presente.

São exemplos de requisitos funcionais identificados neste nível: ➤

Efetuar baixa do título na relação de contas a receber.



Emitir carta de renovação de contrato.



Emitir certificado de participação do aluno no curso.

Em todos os exemplos, uma vez que determinado requisito tenha sido concluído, tudo que deveria ser feito em resposta a um evento externo foi feito.

5.9.4.1. Qual é a sua importância?

A importância de saber identificar um requisito funcional neste nível é poder delinear o escopo do software em desenvolvimento de maneira inequívoca; sem dúvidas quanto à sua abrangência. O escopo descrito por meio de requisitos funcionais neste nível pode mais facilmente ser validado e compreendido.

Este é o único nível de descrição de processos que pode ser padronizado e é, por consequência, aquele utilizado por todos os métodos de medição do tamanho funcional padronizados pela norma ISO/IEC 14143.

Por ser passível de padronização, o requisito funcional especificado neste nível pode ser verificado de maneira objetiva quanto à conformidade com políticas de qualidade previamente estabelecidas. O requisito funcional estar especificado neste nível é o critério de qualidade presente em diversas metodologias. Ele é o nível em que um caso de uso deve ser identificado e elaborado antes de ser empacotado e encaminhado para projeto e implementação; o mesmo é válido para uma história de usuário.

5.9.5. Requisitos funcionais com objetivo agregador

São requisitos que agregam vários objetivos de usuários individuais em uma única especificação de alto nível. Quanto maior o nível, mais gerais

são seus objetivos – e para que um objetivo de maior nível seja alcançado, outros de menor nível devem ser alcançados.

Referem-se, portanto, a objetivos mais gerais e estão em um nível de abrangência associado a objetivos colaborativos; aos processos de negócio de alto nível. Não se referem a uma única tarefa ou serviço. Resumem um conjunto de tarefas de um ou mais usuários.

A Figura 5.10 ilustra o objetivo de “Controlar Aditivos” como um conjunto de objetivos de menor nível subordinados.

Para simplificar, considera-se o nome “requisitos agregadores” para denominar os elementos na especificação de requisitos funcionais associados a um objetivo agregador.

São exemplos de requisitos agregadores: ➤

Controlar fluxo de caixa.



Gerir relacionamento com clientes.



Efetuar gestão dos cursos.

Quais são especificamente as tarefas associadas a esses requisitos? Talvez sejam óbvias para o leitor da especificação (e por isso não são detalhadas) ou ainda não sejam conhecidas. Contudo, neste último caso, sabe-se que há tarefas que devem ser sobre esse requisito, pois se decidiu que elas são parte do escopo do software em desenvolvimento.

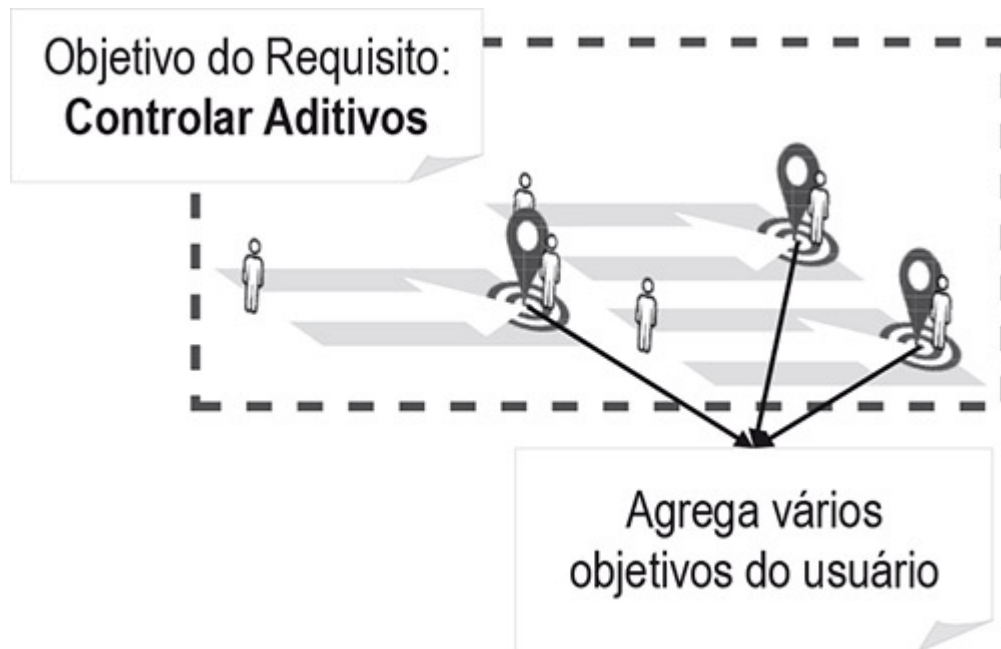


Figura 5.10. Controlar aditivos como um objetivo agregando vários objetivos subordinados.

5.9.5.1. Qual é a sua importância?

O propósito de descrever um requisito agregador é resumir um conjunto de tarefas do usuário em um único item quando ainda não se sabe exatamente quais delas existem, quais existirão ou quais delas serão objeto de informatização ou automação no escopo do projeto em desenvolvimento.

5.9.5.2. Papel do analista de requisitos

Em momentos preliminares do desenvolvimento, talvez boa parte dos requisitos identificados para compor a especificação funcional sejam requisitos agregadores. Isso se dá porque várias decisões sobre o escopo final do software em desenvolvimento ainda estão pendentes. É um item a ser definido; geralmente, rotulado como TBD (*to be defined*).

Identificar requisitos agregadores não se trata de uma estratégia de desenvolvimento. Os requisitos que estão no nível adequado para um requisito funcional devem ser identificados e descritos independentemente do momento em que se esteja no ciclo de vida.

Ao longo do desenvolvimento dos requisitos, o analista deve refinar os requisitos neste nível agregador no sentido de identificá-los e descrever de

maneira mais específica até que se chegue ao nível adequado a um requisito funcional.

No entanto, alguns requisitos funcionais possuem um comportamento padronizado que dispensam um detalhamento como o proposto aqui. Um exemplo são os requisitos associados à manutenção cadastral e, normalmente, denominados CRUD (*Create, Read, Update, Delete*).

5.9.5.3. Requisitos de negócio × requisitos agregadores

Um cuidado para o qual se deve atentar é diferenciar um requisito agregador de uma necessidade de negócio. Afinal, enquanto a necessidade de negócio está no domínio do problema com todo um universo de soluções em potencial, o requisito agregador tem o objetivo de descrever a solução escolhida. Diferenciá-los é importante porque os seus usos são distintos. Não fazer isso significa confundir a solução com o problema.

Para ilustrar um e outro, por exemplo, considere os dois casos ilustrados na Figura 5.11: “Controlar Aditivos” e “Diminuir o Risco Operacional”.



Figura 5.11. Comparativo entre requisito agregador e requisito de negócio.

A descrição de “Controlar Aditivos” corresponde a um requisito agregador porque agrega vários objetivos em um nível de objetivo de usuário. E não há pistas de um problema ou oportunidade de negócio. Ainda que ao longo da Engenharia de Requisitos ele deva ser refinado, já está decidido que ele faz parte do domínio da solução. Há, todavia, uma série de decisões ainda em aberto para o atendimento das necessidades de negócio.

Já a descrição associada a “Diminuir o risco operacional com serviços contratados mas não formalizados” indica uma necessidade de negócio, pois descreve um problema. Ela foi o ponto de partida para uma série de decisões que, nesse estágio do desenvolvimento, culminou com o estabelecimento do objetivo de “Controlar Aditivos” como parte do escopo para a solução da organização estar exposta a riscos operacionais.

Portanto, não confunda o problema com a solução. Entregar um software ao cliente é a parte mais fácil do trabalho. Entregar um software que o satisfaça é que é a questão-chave. E para isso é necessário ter a visão clara do problema.

5.9.6. Requisitos funcionais com objetivo de subfunção

Análogos aos requisitos agregadores, mas em sentido inverso, são fragmentos que compõem uma especificação funcional em um nível inferior ao nível dos objetivos do usuário: são passos e regras. Ou seja, isoladamente não atendem a um objetivo do usuário. Por simplificação, esses fragmentos de requisitos são denominados especificações funcionais de subfunções.

Os passos descrevem o intercâmbio de dados em ambas as direções entre o usuário e o software; e entre este último e os requisitos de armazenamento. Para um exemplo de requisito de subfunção como conjunto de passos, podemos citar a autenticação de um cliente em um serviço de autoatendimento para realizar transações bancárias. Este é um excelente exemplo de uma sequência de passos que deve ser especificada em separado dos requisitos funcionais que a inserem. Não há, neste cenário, a figura de um *login* onde o usuário se identifica e, feito isso, pode realizar transações para as quais esteja autorizado. Cada tipo de transação individual (saque de conta corrente, transferência, pagamento, investimento etc.)

requer o mesmo conjunto de passos para a identificação do cliente, que poderiam ser: ➤

Verificar se o cartão existe, se está vigente e se não está bloqueado.



Verificar se a operação desejada é compatível com o tipo do cartão.



Verificar se a senha informada está correta.



Incrementar erros de senha se a senha informada for incorreta.



Zerar erros de senha se a senha informada for correta.

Ainda que isoladamente essa sequência de passos não atenda aos critérios para um requisito funcional, sua especificação como um requisito no nível de subfunção é apropriada.

As regras normalmente estão ligadas a leis que regem o negócio e descrevem de forma complementar o funcionamento de seus processos – por isso, muitas vezes são chamadas de regras de negócio. Elas descrevem políticas corporativas, a legislação pertinente, as regulamentações governamentais e os padrões de mercado aos quais a solução deve se subordinar. Elas não estão especificamente ligadas à solução (e, portanto, ao software), mas ao domínio do problema em que elas se inserem. Elas existem independentemente do software; do processo de negócio estar automatizado ou não. As regras de negócio deveriam também ser tratadas como um ativo organizacional e não apenas como parte do projeto ou do produto de software. Além disso, o software pode ter regras que não são do negócio, mas relativas ao seu modo específico de funcionamento; estão ligadas ao software em específico.

São exemplos de regras de negócio:



Somente alunos com frequência igual ou superior a 75% podem emitir seu certificado.



Somente clientes com idade superior a 18 anos podem ser correntistas.



Somente compras acima de R\$ 500,00 podem ter pagamento parcelado.



Crianças de até 23 meses transportadas no colo do responsável não pagam passagem.

5.9.6.1. Qual é a sua importância?

No caso das regras de negócio, é comum que elas sejam compartilhadas entre diferentes requisitos funcionais, inclusive alocados a produtos de software diferentes. Portanto, especificá-las de forma independente dos requisitos funcionais em que se aplicam contribui para uma melhor gestão de requisitos (facilidade de modificação e principalmente de reutilização).

Uma tendência quando se utilizam ferramentas de BRMS (*Business Rules Management System*) é que, além de especificar as regras de negócio em separado dos requisitos funcionais, estas sejam mantidas e implementadas fora de uma mesma aplicação de software em particular.

Já no caso de uma sequência de passos, só se justifica a sua especificação em separado como uma subfunção quando há um comportamento compartilhado por vários requisitos funcionais. Isso torna os documentos de requisitos mais facilmente adaptáveis a mudanças, pois diminui redundância de informação.

5.9.6.2. Papel do analista de requisitos

O analista de requisitos, quando obtém a informação das partes interessadas como subfunções, deve saber identificá-las como tal para investigar em quais requisitos funcionais essas subfunções se inserem. Por outro lado, quando ele for o autor das especificações funcionais, deve identificar as oportunidades como as descritas para uma melhor gestão de requisitos.

“Isso não é um requisito, é uma regra de negócio!”. Um comentário muito comum, principalmente entre os profissionais responsáveis pela medição do tamanho funcional de software, é que determinado requisito em uma especificação funcional não é um requisito, mas uma regra de negócio.

O comentário trata-se na verdade de um jargão. Afinal, uma regra de negócio é um requisito! A intenção por trás do comentário é indicar que aquela regra de negócio não é um requisito funcional no nível de objetivo do usuário como descrito neste texto e, portanto, trata-se de uma subfunção que não deve ser tratada como uma função.

5.10. Requisitos não funcionais

Este tópico trata dos requisitos não funcionais que descrevem limitações de ordem geral aos requisitos funcionais e, nisso, complementam a especificação do software. Afinal, ele não deve apenas funcionar; deve funcionar bem. Ao descrever o que seja esse “funcionar bem”, os requisitos não funcionais acabam por estabelecer níveis de serviço esperados para o funcionamento do software.

Enquanto os requisitos funcionais referem-se especificamente a uma tarefa do usuário, os requisitos não funcionais indicam restrições de ordem geral que abordam aspectos relativos: ➤

Ao ambiente: como interoperabilidade, segurança, privacidade, sigilo.



À **organização**: por exemplo, locais para operação, hardware alvo, aderência a padrões.



À **implementação**: como plataforma de software, hardware, linguagem de programação.



À **qualidade**: por exemplo, a facilidade de uso, a confiabilidade, a eficiência, a portabilidade e a facilidade de manutenção.

Essa relação é apenas ilustrativa de alguns tipos de requisitos não funcionais e não busca ser uma lista completa.

A Figura 5.12 representa o papel dos requisitos não funcionais de estabelecer níveis de serviço que direcionam como será o projeto da arquitetura que suporta as camadas de software voltadas ao atendimento dos requisitos funcionais.

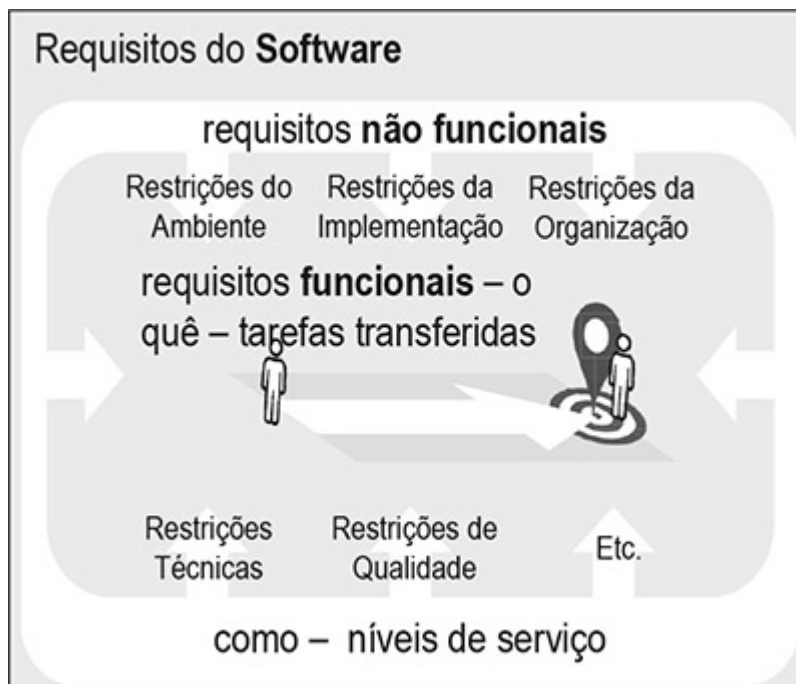


Figura 5.12. Relação entre requisitos não funcionais e requisitos funcionais.

Um aspecto que facilita a identificação dos requisitos não funcionais é que eles costumam ser constantes entre os projetos ou mudam pouco de um projeto para outro na mesma organização. Por isso, mesmo em fases bem iniciais de um projeto, se consegue ter uma boa visibilidade sobre os aspectos não funcionais necessários ao software.

Embora não seja uma regra, uma característica que costuma diferenciar os requisitos funcionais dos não funcionais é que os não funcionais costumam se manifestar de forma geral sobre o software, e os funcionais de forma específica. Por exemplo, facilidade de uso do software é um tipo de requisito não funcional, e uma definição sobre essa questão em geral irá valer para todo o software. Já os requisitos funcionais tratam de tarefas específicas que o software realiza, eles não costumam se espalhar por todo o produto.

Outro raciocínio útil para essa distinção, e consequência do parágrafo anterior devido a esse comportamento geral, é que o requisito não funcional frequentemente é alocado à parte arquitetural do software que dará suporte às funcionalidades.

Para organizações que possuem uma metodologia madura de desenvolvimento de software, a identificação de boa parte dos requisitos não funcionais é também facilitada, pois muitos aspectos técnicos e de qualidade dos projetos são padronizados pela metodologia através de guias específicos. Por exemplo: guia de usabilidade, guia de segurança, guia de portabilidade. Por isso os requisitos não funcionais de um projeto tendem a ser em menor número que os requisitos funcionais, o que facilita sua gestão.

Veja alguns exemplos de padrões corporativos do governo federal brasileiro que definem diversos aspectos relativos aos requisitos não funcionais para seus softwares desenvolvidos: ➤

Padrões Web em Governo Eletrônico (e-PWG):

<http://epwg.governoeletronico.gov.br/cartilha-usabilidade>



Modelo de Acessibilidade de Governo Eletrônico (e-Mag):

<http://emag.governoeletronico.gov.br/>



Arquitetura de Interoperabilidade de Governo Eletrônico (e-PING):

<http://eping.governoeletronico.gov.br/>

5.10.1. Diferença entre requisitos não funcionais e restrição técnica

O requisito não funcional é o resultado de uma elaboração (de escolhas) para uma solução em particular dentre várias possíveis. As restrições técnicas são limitações impostas externamente às possíveis soluções; não é algo passível de mudança na esfera do projeto, não se negociam. Essa diferença é importante do ponto de vista da gestão do projeto que entregará a solução. O requisito não funcional pode ser mudado; a restrição técnica, não. A restrição técnica induz a um requisito não funcional mais específico. A restrição está no domínio do problema e o requisito não funcional no domínio da solução.

Para esclarecer as diferenças entre ambos, seguem dois exemplos: ➤

Cenário 1: a restrição estabelece que a interface do software com o usuário deve ser baseada na web. Uma opção de interface ao estilo de uma aplicação Windows está descartada, mas outras são possíveis, desde que baseadas na web. O requisito não funcional consiste então em decidir sobre qual navegador a ser suportado, e é selecionado o Internet Explorer. As opções de usar o Chrome ou Firefox seriam válidas, mas não foram as escolhidas.



Cenário 2: a restrição estabelece que o navegador a ser suportado é o Chrome. Todas as outras opções de navegadores estão descartadas na seleção da solução. A possibilidade de escolha para o requisito não funcional é mais restrita neste caso e poderia envolver qual versão a suportar do navegador definido pela restrição.

Pode-se perceber, com esses cenários, que as restrições técnicas não deixam de ser também um subconjunto dos requisitos não funcionais, como ilustrado na Figura 5.13.

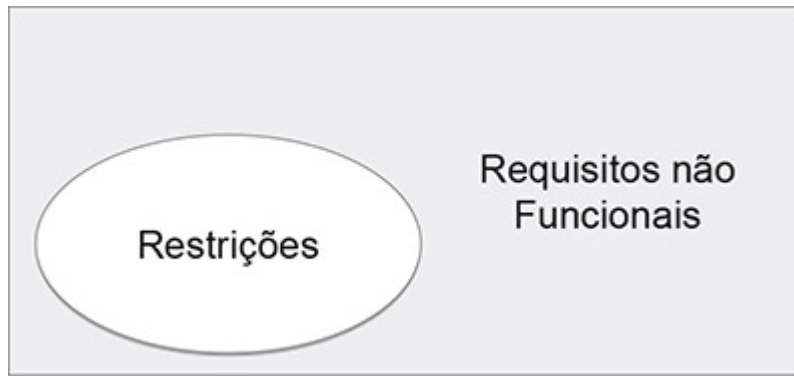


Figura 5.13. Restrições como um subconjunto dos requisitos não funcionais.

5.10.2. Qual é a sua importância?

Alguns requisitos não funcionais são óbvios, como haver mensagens que descrevam os erros em um vocabulário comum aos usuários do software especificado... mas será que Nelson Rodrigues estava certo e “só os profetas enxergam o óbvio”?

Várias decisões de *design* também são resultados dos níveis de serviço definidos nos requisitos não funcionais, e muitos projetos – principalmente quando contratados – fracassam por deixar de considerar os requisitos não funcionais.

Imagine um requisito funcional que descreva uma transação de saque em uma máquina de autoatendimento com todos os seus passos e regras de negócio associados implementados. Há um evento que provoca a realização desse requisito funcional – digamos, um cliente do banco com necessidade de dinheiro – e há um objetivo a ser atingido – digamos, ter o saldo atualizado e receber o dinheiro.

Um usuário, ao tentar fazer o saque, não consegue interagir com a máquina de autoatendimento. Ela parece pronta para receber transações, mas como tem apenas botões e não possui uma tela de toque, nada que faça provoca qualquer reação. Ele se desloca para outra máquina com tela de toque e durante uma primeira tentativa de saque o usuário viola uma regra de negócio. Em consequência, recebe a resposta: ORA-1403.

De alguma maneira, ele intui o que isso quer dizer (ele percebeu que havia digitado a conta errada). Corrige o problema e dá início a uma nova transação. Passadas três horas, 24 minutos e 19 segundos, ele consegue realizar o saque.

Observe que, mesmo que todos os requisitos funcionais sejam observados, a solução: ➤

é inflexível quanto ao tipo de máquina que deve ter interface por toque; ➤

as mensagens são obscuras; e ➤

o tempo de resposta é intolerável.

“Inflexível”, “obscuras” e “intolerável” são julgamentos... qual é a lei? Considerar como óbvio? Óbvio para quem? Como nem todos são profetas, melhor não se valer do óbvio!

Com o objetivo de sistematizar a identificação, o desenvolvimento e a validação dos requisitos não funcionais, há alguns padrões de classificação de requisitos que podem ajudar. É possível encontrar na literatura várias formas distintas de classificação de requisitos não funcionais. Nos dois itens a seguir são apresentados dois destes casos.

5.10.3. FURPS+

Um padrão bastante simples é denominado pelo acrônimo FURPS+, que se refere à: ➤

Functionality (funcionalidade): enfoca os requisitos funcionais.



Usability (usabilidade): trata da facilidade de uso do software e inclui fatores humanos, estética, consistência na interface do usuário, ajuda *on-line* e contextual, assistentes, documentação, materiais de treinamento.



Reliability (confiabilidade): trata de integridade, conformidade e interoperabilidade do software. Inclui aspectos de frequência e gravidade de falha, possibilidade de recuperação de falhas, previsibilidade, exatidão e tempo médio entre falhas.



Performance (desempenho): trata de velocidade, eficiência, taxa de transferência, tempo de resposta e uso de recursos.



Supportability (suportabilidade): trata de extensibilidade, adaptabilidade, manutenibilidade, compatibilidade, configurabilidade, escalabilidade, instalabilidade, localizabilidade (ex.: internacionalização) e testabilidade.



“+” refere-se a: ✓

Restrições de design (projeto): restringe algo sobre o projeto da arquitetura do software.



Restrições de implementação: restringe algo sobre a construção do sistema.



Restrições de interface: restrições de formatos, tempos ou outros fatores usados por tal interação.



Restrições físicas: limitações quanto ao hardware que deve ser suportado.

5.10.4. ISO/IEC 25010

Outro padrão bem mais abrangente é a norma ISO/IEC 25010, que define 31 subcategorias distribuídas em oito categorias de Qualidade do Produto de Software/Sistema (Figura 5.14). As categorias são: ➤

Adequação funcional: grau no qual um produto fornece funcionalidades que atendem às necessidades especificadas e implícitas quando utilizado em determinadas condições.



Eficiência no desempenho: desempenho relativo à quantidade de recursos usados em determinadas condições.



Compatibilidade: grau no qual um produto, sistema ou componente pode trocar informações com outros produtos, sistemas ou componentes; e também, ou alternativamente, executar as suas funções necessárias enquanto compartilha o mesmo ambiente de hardware e software.



Usabilidade: grau no qual um produto ou sistema pode ser usado por determinados usuários para alcançar determinados objetivos de maneira efetiva, eficiente e satisfatória em determinado contexto de uso.



Confiabilidade: grau no qual um sistema, produto ou componente executa determinadas funções em determinadas condições por um determinado período de tempo.



Capacidade de manutenção: grau de efetividade e eficiência com o qual um produto ou sistema pode ser modificado por aqueles com essa intenção.



Segurança: grau no qual um produto ou sistema protege informações e dados de tal forma que as pessoas ou sistemas tenham o grau de acesso apropriado aos seus perfis e níveis de autorização.



Portabilidade: grau de efetividade e eficiência com o qual um sistema, produto ou componente pode ser transferido de um equipamento, software ou de um ambiente operacional para outro.

Cada uma das oito categorias relacionadas, por sua vez, agrega uma série de subcategorias. Por exemplo, citou-se antes o tempo de resposta de 03:24:19. Um requisito não funcional que estabeleça que o tempo de resposta médio tolerável seja de dez segundos para transações interativas está inserida na categoria **eficiência do desempenho**, em específico na subcategoria **comportamento no tempo**, que é definido como grau no qual o tempo de resposta e de processamento e as taxas de vazão de um produto ou sistema, quando executando suas funções, atendem aos seus requisitos.



Figura 5.14. As categorias definidas na norma ISO/IEC 25010.

Observe que a norma cita “grau”. Portanto, há espaço para estabelecer não apenas o tempo médio de resposta tolerável, como também o desejável, como cinco segundos, por exemplo. Com isso, é possível estabelecer um sinal vermelho para o cenário em que esse tempo médio for superior aos dez segundos; um sinal amarelo, quando ele for superior a cinco segundos; e um sinal verde para cenários onde o tempo médio de resposta seja inferior ou igual a cinco segundos.

Outra situação ilustrada no exemplo refere-se à mensagem ORA-1403. Certamente, esse tipo de comunicação entre o software e o usuário dificulta a sua operação. A subcategoria **facilidade de operação** subordinada à categoria **facilidade de uso** é descrita como: ➤

Grau no qual um produto ou sistema tem atributos que o tornam fácil de operar e controlar.

Um exemplo de especificação de requisito não funcional abordando essa subcategoria poderia ser: ➤

O sistema deve apresentar mensagens de erro orientadas à tarefa, com sugestões ou instruções simples e construtivas para correção do erro, sempre que possível posicionando o cursor no campo objeto da mensagem e usando termos específicos e vocabulário neutro – não repreensivo.



As informações fornecidas pelo usuário e que não se relacionam diretamente ao problema indicado pela mensagem de erro não devem exigir nova digitação; a mensagem deve apontar o erro cometido ou a informação que falta.



Conjuntamente à mensagem de erro, o sistema deve apresentar um código do erro e um caminho para solução. O código não deve ser fornecido sozinho.



Uma mensagem em particular deve segregar – usando símbolos específicos para comunicar a informação ao usuário – entre os seguintes tipos de mensagem: ✓

Erro de negócio: mensagens que indicam uma violação de uma regra de negócio.



Erro de sistema: mensagens que indicam a materialização de condição provocada em camada de software anterior à aplicação (*framework*; software de gestão de mensagens; software de gestão de banco de dados; software de gestão de interface com o usuário; sistema operacional; *drivers* de dispositivos etc.).



Mensagens de confirmação: mensagens com finalidade de proteção proativa, evitando ações críticas do usuário por equívoco.



Mensagens de informação: mensagens emitidas durante um processamento em andamento e que não exigem especificamente ação por parte do usuário em resposta a sua emissão.

Dentro do cenário utilizado como base para explorar os requisitos não funcionais, restou um item ainda não abordado: o sistema apenas é compatível com determinados equipamentos (que tenham recursos de toque de tela). Essa qualidade é enquadrada na subcategoria **interoperabilidade** presente na categoria **compatibilidade**. Ela é definida como: ➤

Grau no qual dois ou mais sistemas, produtos ou componentes podem trocar informação e usar a informação que foi trocada.

Um exemplo muito comum de um requisito não funcional desse tipo – **compatibilidade** – é o suporte à diferentes *browsers* e versões de um navegador web (um dos clientes atendidos pela empresa dos autores empreendia sistematicamente desenvolvimento e testes para 33 combinações diferentes).

Este exemplo, assim como os anteriores, é interessante também para ilustrar o que se deseja dizer com “restrições de ordem geral”. Os tempos de resposta e o critério para comunicação de mensagens entre usuário e sistema não se restringem a uma tarefa ou a um objetivo em particular do usuário, mas se espalham por todas as transações interativas.

5.10.5. Papel do analista de requisitos

Observe que esses mesmos objetivos podem ser alcançados pela especificação de um projeto de arquitetura de solução; contudo, este não é o trabalho do analista de requisitos. Ele deve especificar os objetivos que

devem ser atingidos e deixar os profissionais responsáveis pelo projeto e pela implementação fazerem as escolhas para as quais foram treinados de forma a melhor atingir esses objetivos.

Da mesma maneira que ao longo deste texto uma série de critérios de qualidade foram apresentados no que tange à disciplina de requisitos, essas outras disciplinas na engenharia de software também devem ter os seus critérios de qualidade, que acabam por ser também requisitos não funcionais.

As categorias presentes na ISO/IEC 25010 e no FURPS+ são apresentadas com o objetivo de apoiar o analista de requisitos a atuar de maneira proativa para evitar os problemas adjacentes ao óbvio ilustrado na abertura deste tópico.

Na prática, é raro uma organização precisar se preocupar com todas essas categorias de requisitos não funcionais. Por exemplo, em determinada empresa se padronizou o desenvolvimento de projetos de software para uma única plataforma e linguagem de programação: *mainframe* IBM e COBOL. Logo, portabilidade não é uma questão relevante nos projetos dessa empresa. Já outra empresa tem seu foco no desenvolvimento para plataforma *mobile*, o que faz com que portabilidade seja tratada de forma prioritária nos seus projetos.

Portanto, os casos dos dois tópicos anteriores podem ser usados como inspiração para uma organização definir quais categorias de requisitos não funcionais são relevantes para seus projetos. Definida essa lista, que terá o papel de uma lista de verificação, o ideal é elaborar, para cada item, questões que devem ser abordadas durante o desenvolvimento de requisitos, facilitando, assim, sua elicitación nos projetos.

5.11. Requisitos inversos

Um requisito inverso é uma declaração, como uma proposição negativa, indicando o que o sistema ou projeto não irá fazer ou entregar. Ele tem a seguinte forma: ➤

Não deve <xxxx>.

Eles também são denominados como “fora do escopo”, “não escopo”, “escopo negativo”, “requisito negativo”, “limites do projeto” e “exclusões do escopo”. Os profissionais de testes não têm muita simpatia por esse tipo de requisito porque eles não são passíveis de teste. Afinal, tudo o que está fora da aplicação é o que a aplicação não faz e é impossível provar tudo o que a aplicação não faz.

5.11.1. Qual é a sua importância?

A intenção de apresentar este tipo de requisito como parte da estrutura de tipos de requisitos neste livro é pela sua função na clarificação do escopo. Imagine a situação onde há um requisito agregador em que não se sabe quais as tarefas que ele agrega; contudo, sabe-se que uma em particular estará fora. Aí entra o requisito inverso sem o lado negativo destacado anteriormente, afinal o requisito agregador cumpre o papel de delimitar o requisito inverso de forma que ele não se refira a tudo o que a aplicação não faz.

O requisito inverso, portanto, ajuda a gerenciar as expectativas das partes interessadas, evitando criar expectativas irreais e, com isso, possíveis reclamações futuras.

Ainda que os requisitos inversos ajudem na compreensão dos requisitos do projeto, deve-se priorizar sempre declarar explicitamente o que a aplicação faz. O uso de requisitos inversos deve ser feito com parcimônia. Ou seja, a atenção maior das especificações deve estar na definição do escopo e não na definição do não escopo.

5.12. Requisitos de projeto e requisitos de qualidade

A classificação de requisitos apresentada até este momento está de acordo com o proposto tanto pelo *PMBOK® Guide* quanto pelo *BABOK®*. Porém, vale comentar que o *PMBOK® Guide* cita dois outros tipos de requisitos: requisitos de qualidade e requisitos de projeto. A definição apresentada para esses tipos é relevante apenas para o trabalho de gerência de projetos e não

para o trabalho da Engenharia de Requisitos. No entanto, convém descrevê-los.

Os requisitos de projeto indicam ações, processos ou outras condições a que o projeto deve atender. Eles têm o seu foco em aspectos relativos à execução do projeto. Por exemplo: ➤

Seguir a Metodologia de Gestão de Projetos do SISP (MGP-SISP).



Formar a equipe com pelo menos 10% de pessoas portadoras de alguma deficiência.



Acionar o departamento de compras para todas as aquisições.

Os requisitos de qualidade descrevem a qualidade de uma entrega do projeto e não uma qualidade do produto de software associado ao projeto. Por exemplo, um cronograma tem como condição para ser aceito que cada atividade presente em sua estrutura analítica tenha um responsável. Ou seja, os requisitos de qualidade estão relacionados a critérios de aceitação dos entregáveis da gestão do projeto.

5.13. Exercícios

1. A partir da documentação fornecida no estudo de caso (anexo), identifique: ➤
Requisitos de negócio (problemas e/ou oportunidades a enfrentar).



Restrições de negócio.



Restrições técnicas.



Premissas.

2. Assinale a opção que menos representa a documentação de um requisito de parte interessada: a)
Ata de reunião.
 - b) Gravação de áudio da entrevista.
 - c) Especificação de caso de uso.
 - d) Gravação de vídeo da realização de uma tarefa do negócio.
 - e) Foto com anotações no quadro durante a reunião.
3. Assinale a opção que menos representa a documentação de um requisito da solução.
 - a) Ata de reunião.
 - b) Modelo de caso de uso.
 - c) Especificação de caso de uso.
 - d) Documento de visão.
 - e) Diagrama de contexto.
4. Assinale a opção que não representa um requisito de solução ou transição.
 - a) O sistema deve emitir diariamente relatório de vendas por região.
 - b) As faturas em aberto no sistema velho devem ser migradas para o novo sistema comercial.
 - c) Ao cadastrar um cliente, sua identificação deve ser validada no sistema CRM.
 - d) O sistema deve diminuir o tempo de tramitação do reembolso de despesas para dois dias úteis.
 - e) O sistema deve suportar idiomas inglês e espanhol.
5. Assinale a opção que melhor representa um requisito funcional: a)
O sistema tem por objetivo reduzir o custo de tramitação de processo em 20%.
 - b) Todas as transações do sistema devem responder ao usuário em no máximo 1 segundo.
 - c) O sistema deve ter interface com usuário responsiva a dispositivos móveis.
 - d)

O gestor de compras escolhe a proposta mais vantajosa para aquisição.

- e) A emissão de certidão deve ser permitida somente a funcionários com nível de chefia.
6. Quais os três níveis de granularidade em que se pode encontrar um requisito funcional?
- a) Segregador, usuário e subfunção.
 - b) Funcional, não funcional e transição.
 - c) Usuário, agregador e subfunção.
 - d) Não usuário, funcional e agregador.
 - e) Usuário, funcional e agregador.
7. O que é correto afirmar do requisito funcional de subfunção?
- a) Está no nível de processos de negócio.
 - b) Abrange passos e requisitos de negócio.
 - c) Pode representar um conjunto de passos.
 - d) É relativo a uma única tarefa do usuário.
 - e) Deve ser completo e não exigir passo anterior ou subsequente para que um indivíduo alcance o seu objetivo.
8. O que não é correto afirmar do requisito funcional de objetivo de usuário?
- a) Deixa o usuário satisfeito ao seu final.
 - b) Representa uma regra de negócio.
 - c) Está sob a responsabilidade de um único indivíduo.
 - d) Descreve uma história com o intercâmbio de informações entre o usuário e a solução em busca de um objetivo específico.
 - e) Permite de maneira objetiva avaliar o escopo em termos de tarefas que serão objeto de informatização ou automação, ao contrário de requisitos descritos em nível agregador, que não permitem com clareza identificar quais de suas partes serão ou não objeto de informatização ou automação.
9. Os requisitos descritos a seguir são correspondentes a um sistema de Segurança Portuária, que visa controlar a entrada e saída de pessoas e veículos na área portuária. Este sistema é necessário para atender ao Código Internacional para proteção de Navios e Instalações Portuárias (ISPS – *International Ship and Port Facility Security Code*). O Código ISPS estabelece determinadas regras que tornam

os navios e instalações portuárias mais seguras. É uma norma norte-americana que impõe a rastreabilidade da área portuária com intuito de evitar ataques terroristas. Dentre as medidas adotadas, pode-se destacar: a)

- Estabelecimento de maior controle de entrada e saída de pessoas e veículos nas instalações portuárias.
- b) Delimitação do perímetro do porto.
- c) Instalação de sistema de vigilância dos limites do perímetro do porto e do cais.
- d) Necessidade de cadastramento das pessoas e veículos que entram na instalação portuária.

Prescreve ainda o Código que um navio, antes de chegar ao porto, deve informar os últimos dez portos que visitou. Caso algum não seja certificado de acordo com o Código, poderão ser adotadas medidas adicionais de proteção, tais como inspecionar o navio, colocá-lo em quarentena etc., o que causará atraso na operação do navio, provocando sérios prejuízos. Tendo em vista que o comércio marítimo internacional é um setor altamente competitivo, os navios que o realizam passariam a evitar portos que não são certificados de acordo com o Código ISPS.

Para cada requisito funcional descrito a seguir do sistema de segurança portuária, marque a classificação adequada quanto ao seu nível de abrangência: ➤

Objetivo agregador: processo de negócio ou conjunto de tarefas.



Objetivo de usuário: uma tarefa específica.



Objetivo de subfunção: regras ou passos de uma tarefa.

Identificador	1
Controlar entrada e saída de pessoas e veículos nas instalações portuárias.	

☐ Objetivo agregador	☐ Objetivo de usuário	☐ Objetivo de subfunção
--	---	---

Identificador	2		
Agendar a liberação da entrada de um visitante.			
[] Objetivo agregador	[] Objetivo de usuário	[] Objetivo de subfunção	

Identificador	3		
Comandar a liberação da catraca para a entrada de um visitante.			
[] Objetivo agregador	[] Objetivo de usuário	[] Objetivo de subfunção	

Identificador	4		
Prover relatórios gerenciais para monitoramento de incidentes.			
[] Objetivo agregador	[] Objetivo de usuário	[] Objetivo de subfunção	

Identificador	5		
Apenas gerentes podem registrar um incidente ocorrido com atraso superior a 15 dias.			
[] Objetivo agregador	[] Objetivo de usuário	[] Objetivo de subfunção	

Identificador	6		
Ao registrar entrada de um veículo, validar existência da placa no Detran.			
[] Objetivo agregador	[] Objetivo de usuário	[] Objetivo de subfunção	

Identificador	7
Manter cadastro de portos certificados na norma ISPS.	
☐ Objetivo agregador	☐ Objetivo de usuário ☐ Objetivo de subfunção

10. Classifique cada requisito do sistema de segurança portuária descrito a seguir em: **a) Requisito de negócio:** metas e objetivos desejados pela organização, motivação do projeto (problemas ou oportunidades a enfrentar).

b) Requisito funcional (da solução): tarefas e serviços oferecidos pelo sistema (o que o sistema faz).

c) Requisito não funcional (da solução): aspectos de qualidade, técnicos, ambientais, padrões (como as funções irão ser entregues).

d) Requisito de transição: permite que a nova solução entre em operação plena.

Req. 1	Os funcionários só acessam a área portuária dentro do período da sua escala de trabalho.		
☐ Req. negócio	☐ Req. funcional	☐ Req. não funcional	☐ Req. transição

Req. 2	A identificação de visitantes e funcionários para acesso será realizada através de biometria (impressão digital).		
☐ Req. negócio	☐ Req. funcional	☐ Req. não funcional	☐ Req. transição

Req. 3	O tempo de resposta para liberar a catraca não deve exceder dois segundos.		
☐ Req. negócio	☐ Req. funcional	☐ Req. não funcional	☐ Req. transição

Req. 4	Os dados dos visitantes já cadastrados serão migrados para o novo sistema.

☐ Req. negócio	☐ Req. funcional	☐ Req. não funcional	☐ Req. transição
---------------------------------------	---	---	---

Req. 5	A empresa deve atender à norma de segurança ISPS (<i>International Ship and Port Facility Security Code</i>).		
☐ Req. negócio	☐ Req. funcional	☐ Req. não funcional	☐ Req. transição

Req. 6	Sempre que um visitante entrar na área portuária o sistema deve enviar um e-mail avisando o responsável sobre a visita. O e-mail deve ter o nome do visitante, data e hora da entrada.		
☐ Req. negócio	☐ Req. funcional	☐ Req. não funcional	☐ Req. transição

Req. 7	O período máximo permitido de indisponibilidade do sistema é de dez minutos/dia.		
☐ Req. negócio	☐ Req. funcional	☐ Req. não funcional	☐ Req. transição

Req. 8	O sistema deve minimizar a possibilidade de multas pela Autoridade Portuária por não comunicação imediata de incidentes.		
☐ Req. negócio	☐ Req. funcional	☐ Req. não funcional	☐ Req. transição

Req. 9	O sistema deve oferecer interface com usuário que suporte os idiomas: inglês e português.		
☐ Req. negócio	☐ Req. funcional	☐ Req. não funcional	☐ Req. transição

Req. 10	Os relatórios do sistema devem ser disponibilizados em formato PDF e HTML.		
☐ Req. negócio	☐ Req. funcional	☐ Req. não funcional	☐ Req. transição

6. Atividades da Engenharia de Requisitos

“A luz precede toda transição. Seja a luz no fim do túnel, pelas frestas nas portas ou no brilho de uma ideia, ela está sempre lá, anunciando um novo começo.”

Teresa Tsalaky (The Transition Witness) Este capítulo descreve uma visão da Engenharia de Requisitos a partir da divisão do trabalho em tarefas com um conjunto comum de habilidades e responsabilidades. Não se aborda a Engenharia de Requisitos inserida em um processo ou metodologia específica. Utiliza-se um processo genérico, definido em termos de diferentes objetivos de informação associados aos marcos que delimitam suas fases. Dessa forma, a informação apresentada não se limita necessariamente a uma metodologia, abordagem ou processo de desenvolvimento em particular.

Apresentam-se as atividades de gerência, elicitação e análise de requisitos e como elas interagem entre si. Destaca-se o estudo de viabilidade, anterior ao projeto e que cumpre o papel de ponte para o exercício de atividades da Engenharia de Requisitos, tipicamente ordenadas em uma espiral de construção do conhecimento, de aumento do nível de informação disponível. Toda espiral tem seu início a partir de algum ponto. Neste capítulo serão apresentadas técnicas de análise úteis para estabelecer esse fim.

6.1. Um único tema, diferentes pontos de vista

A produção da Engenharia de Requisitos pode ser descrita em diferentes perspectivas. A visão colaborativa, apresentada no Capítulo 1 e usada para descrever os processos de desenvolvimento, é uma delas. Nela, a ênfase está na descrição de um processo de referência geral que ordena atividades para que o trabalho possa ser subdividido entre diferentes profissionais ou pelos mesmos profissionais em diferentes momentos.

O profissional desempenhando uma dessas atividades da Engenharia de Requisitos produz resultados intermediários – na perspectiva do projeto – ao término de sua realização. Esses devem ser passíveis de avaliação de qualidade quanto a aspectos internos e externos, ainda que em caráter preliminar. O objetivo disso é identificar erros mais cedo para evitar a sua propagação e a intensificação de seus efeitos negativos conforme o trabalho avança. Uma vez aceitos, aqueles resultados intermediários servem como insumos para a realização de outras atividades.

O desenvolvimento de software termina quando o objetivo do processo é alcançado: um produto de software completo é entregue plenamente operacional e atende com sucesso às necessidades de negócio que motivaram a mudança da qual ele faz parte. Caso o desenvolvimento concluído seja referente à primeira versão do produto, então sua linha de base é estabelecida e tem início a sua manutenção. Caso o desenvolvimento tenha sido referente a uma manutenção em um produto com uma linha de base prévia, então é estabelecida uma nova linha de base.

O processo deve contar com orientação que descreva os insumos necessários, os passos que devem ser seguidos, as regras que se aplicam e os produtos que devem ser entregues. Essa orientação deve ser flexível o bastante para que o trabalho total possa ser subdividido em partes menores abordando ciclos de desenvolvimento, que costumam receber nomes como iterações, ondas, ciclos, *sprints* etc. A Figura 6.1 exemplifica um quadro (baseado no *Kanban*) que dá visibilidade à progressão no desenvolvimento observando um processo de referência.

à mudança. Isso, sem dúvida, afeta como os processos de desenvolvimento (Engenharia de Requisitos inclusive) são definidos e utilizados.

Descrever as atividades da Engenharia de Requisitos em uma perspectiva dissociada de um processo em específico e colocar a ênfase em melhores práticas facilita a assimilação do assunto. O interessado se acultura de maneira mais concreta, pela associação do tema ao desempenho de uma tarefa específica. Em contrapartida, ele não vê onde os resultados intermediários se encaixam em uma sequência mais abrangente, que entregue um resultado final de valor para o negócio, e com isso pode promover a perda de seu interesse.

Para mitigar esse risco, o assunto será exposto usando um processo genérico, que não se propõe a cumprir especificamente o papel de uma metodologia de desenvolvimento completa, mas antes servir como referência geral quando da discussão de uma atividade no âmbito da Engenharia de Requisitos.

Utiliza-se para isso o mapeamento baseado no COCOMO II, resultante da compatibilização de diferentes tipos de estratégias de desenvolvimento. Não se podem ignorar as repercussões do Manifesto Ágil no mercado atual. Esse mapeamento, portanto, também deve incluir o posicionamento dessa lógica ágil. O Scrum é usado como referência porque ele é uma das metodologias ágeis mais empregadas.

A Figura 6.2 apresenta esse mapeamento entre suas fases e os objetivos gerais de informação ao seu término, e com ênfase nos momentos de maior atividade da Engenharia de Requisitos em comparação às atividades nas demais disciplinas da engenharia de software.

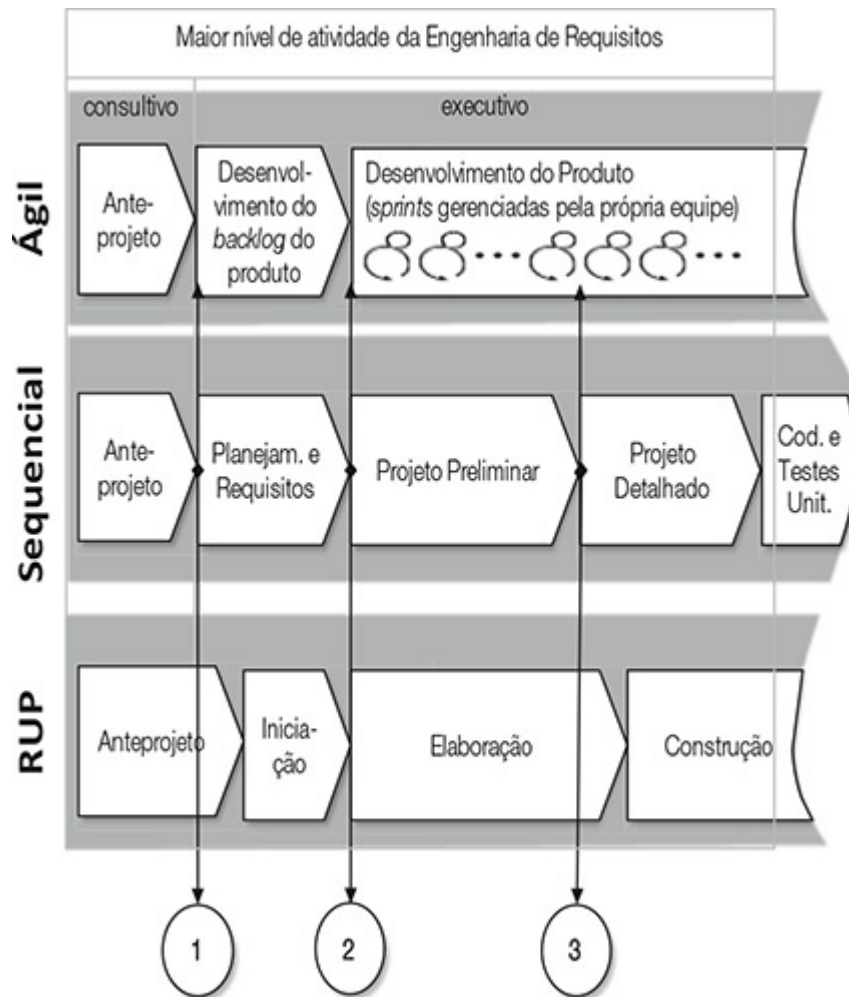


Figura 6.2. Compatibilização entre diferentes estratégias com base nos resultados que devem entregar no tempo na forma de um modelo geral de processo para referência.

Esse mapeamento destaca três marcos, que serão mais bem descritos ao longo deste capítulo, e o seu posicionamento em relação aos diferentes processos ou estratégias de desenvolvimento ilustradas; damos nomes a eles para facilitar a sua referência ao longo do texto:

1. Marco de Definição da Necessidade (DefNec).
2. Marco de Consenso do Escopo (ConEsc).
3. Marco de Detalhamento dos Requisitos (DetReq).

No processo geral utilizado, em momentos iniciais as atividades da Engenharia de Requisitos têm caráter consultivo, dada a responsabilidade

pelo processo nesse ponto ser associada à função de planejamento e não especificamente de desenvolvimento. Essas atividades assumem uma conotação executiva em seguida, pois se adentra no desenvolvimento propriamente dito, objeto da engenharia de software.

6.2. Primeiro marco: definição da necessidade

Na perspectiva da aquisição da integralidade do desenvolvimento, o trabalho anterior ao marco de Definição da Necessidade relacionado no processo geral de referência corresponde à preparação de uma solicitação de proposta. Um termo utilizado no âmbito da administração privada para esse fim é “solicitação de proposta” (*Request for Proposal* – RFP). Na administração pública, utiliza-se o termo “projeto básico” para documentar as conclusões desta primeira fase.

Independentemente de o desenvolvimento ser orientado à contratação ou ao desenvolvimento interno, esta primeira fase corresponde ao trabalho para elaboração de um anteprojeto ou estudo de viabilidade. O caso de negócio (*business case*) também é um produto típico. É um trabalho que também acontece quando da elaboração de um Plano Diretor de Tecnologia da Informação (PDTI).

Ainda que sejam documentos diferentes, eles capturam em comum: a)

- os requisitos de negócio – objetivos que o software candidato deve alcançar quando em capacidade operacional; b)
- as partes interessadas chave identificadas; e c)
- o escopo para o desenvolvimento do software, com as fronteiras entre ele e seu ambiente delineadas e um conceito de operação definido e aprovado.

Este último item deve estabelecer a segregação entre: o que será manual, o que será uma responsabilidade alocada ao software a desenvolver e, eventualmente, o que será alocado ao hardware (como é o caso de algumas soluções de criptografia ou automação bancária).

Nesta fase, os gestores de negócio, muitas vezes apoiados por profissionais da tecnologia da informação, analisam situações do negócio com seus problemas a resolver ou oportunidades a aproveitar. Todo desenvolvimento de sistemas deveria nascer após um estudo de viabilidade para que esse investimento esteja alinhado às decisões dos gestores da organização.

Quando o estudo de viabilidade é documentado adequadamente, possibilita informações que facilitam (e muito) o trabalho do analista de requisitos. Ele pode ser resumido em quatro tarefas principais para os objetivos da Engenharia de Requisitos: ➤

Definir as necessidades de negócio.



Identificar as partes interessadas.



Definir os casos de negócio.



Definir o escopo da solução.

A primeira delas é **definir as necessidades de negócio**. Nela se identificam e definem as necessidades de negócio como definidas no Capítulo 5. Toda mudança legítima, que surge em uma organização, é consequência disso.

A segunda é **identificar as partes interessadas**. Trata-se de identificar, ainda em caráter preliminar, as partes que serão afetadas ou afetam a mudança em vista. Neste caso, o que interessa ao analista de requisitos é identificar um subconjunto dessa lista: aqueles com influência sobre os requisitos do projeto.

A terceira tarefa é **definir os casos de negócio**. Eles declaram as metas-chave e as medidas de sucesso para um projeto. Também determinam se a solução proposta compensa o investimento. Abordam: ➤

benefícios quantitativos e qualitativos (sejam estes financeiros ou não); ➤

métodos e racionais utilizados para quantificar benefícios e custos; ➤
orçamento e fluxo de caixa esperado; ➤
tempo de retorno e lucro esperado; ➤
oportunidades e ameaças (riscos); ➤
restrições e premissas.

Por fim, a quarta tarefa é **definir o escopo da solução**. Seu objetivo é definir o conjunto de mudanças, recomendado em detalhes suficientes que permitam às partes interessadas compreender as novas capacidades do negócio e as versões que serão entregues a cada iteração do projeto. Deve-se organizar informação relativa às principais características e funções que devem ser incluídas e as interações que a solução proposta terá com pessoas e sistemas: ➤

As principais dependências técnicas e de negócio.



As unidades de negócios que serão envolvidas.



Os processos de negócios que serão melhorados (ou redesenhados) e seus proprietários.



Sistemas de informação e outras tecnologias que provavelmente serão afetadas.

Quando essas decisões estão tomadas e as informações sobre elas organizadas em documentos que permitem uma visão compartilhada entre os envolvidos, o marco de Definição da Necessidade é atingido.

É possível iniciar o desenvolvimento mesmo sem que esse marco tenha sido atingido. Ainda que seja possível, isso não quer dizer que seja desejável. A falta de entendimento organizacional dos objetivos do produto e o desenvolvimento concorrente de procedimentos operacionais diminuem a

produtividade no desenvolvimento. Os efeitos disso são exponenciais; e quanto maior o tamanho do desenvolvimento, maiores os impactos da falta dessas decisões e informações na velocidade do desenvolvimento. Se isso é inevitável, então esforço e orçamentos adicionais devem ser contingenciados para esse fim.

6.2.1. Atividades da Engenharia de Requisitos

A partir do marco da Definição da Necessidade no processo de referência descrito na Figura 6.2 tem início o desenvolvimento. Ao longo de toda a sua extensão, a Engenharia de Requisitos agrega três grupos de atividade contextualizados na Figura 6.3. São eles: ➤

Gerência de requisitos.



Elicitação de requisitos.



Análise de requisitos.

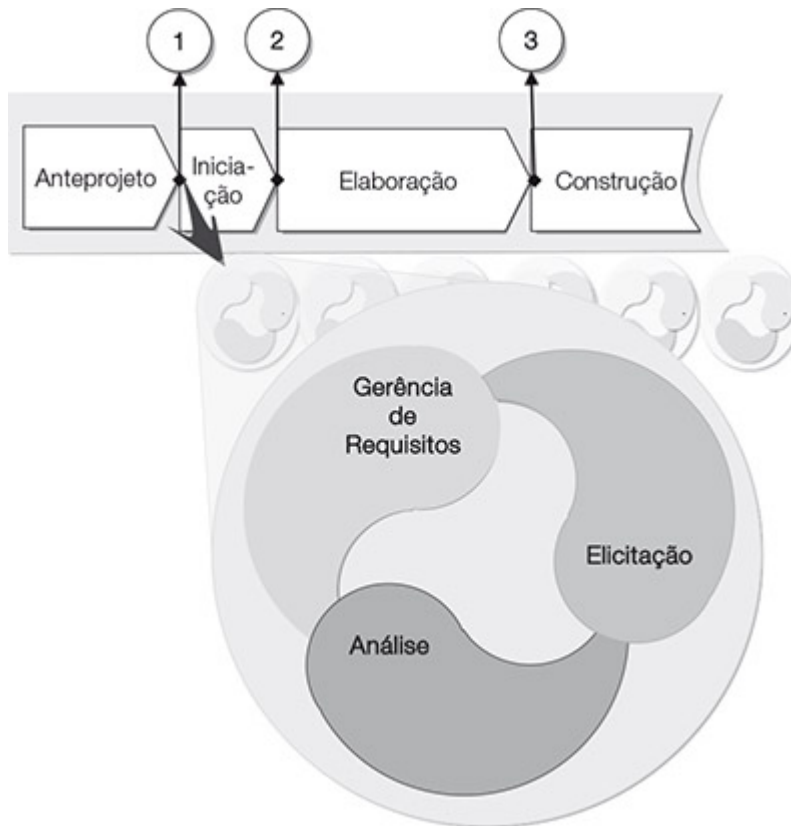


Figura 6.3. Tarefas da Engenharia de Requisitos.

As atividades de gerência de requisitos têm como principais objetivos: a)

Identificar a melhor forma de comunicar os requisitos e a maneira como será mantido o conhecimento obtido para uso futuro.

b) Administrar conflitos, problemas e mudanças a fim de garantir o acordo sobre o escopo da solução.

c) Priorizar requisitos.

O Capítulo 9 trata desse assunto em maior profundidade.

As atividades de elicitação de requisitos vão da seleção de técnicas de levantamento de dados à sua aplicação com o objetivo de identificar e entender os domínios envolvidos, os requisitos de negócio, as partes interessadas e seus requisitos, a solução e os requisitos de transição. Vai mais além da simples coleta de requisitos, pois permite identificar de maneira proativa requisitos adicionais não explicitamente fornecidos; requisitos com impacto no produto final e em vários produtos intermediários do desenvolvimento. Gera como resultados memórias de

levantamento. O Capítulo 7 explora em maior profundidade a elicitação de requisitos.

As informações resultantes das atividades de elicitação ainda escondem conflitos a serem resolvidos, oportunidades de racionalização a serem aproveitadas e exigem uma análise mais apurada, posterior ao evento que originou aquelas informações. Esse trabalho é o objeto da análise de requisitos cuja realização promove a documentação, organização, modelagem, classificação da informação em grupos coerentes, verificação e validação dos requisitos. O Capítulo 8 aborda em detalhes essas atividades.

6.3. Segundo marco: consenso sobre o escopo

O objetivo primário da realização de atividades da Engenharia de Requisitos durante a iniciação do desenvolvimento é entender o problema em que o desenvolvimento se insere e alinhar um escopo para o sistema a desenvolver.

Adicionalmente, desenvolvem-se também as informações disponíveis em busca do consenso sobre o conjunto correto de requisitos. Isso requer estabelecer uma visão compartilhada, ter registro das conclusões sobre essa visão compartilhada de forma que elas possam ser confirmadas e, claro, confirmá-las junto às partes com poder legítimo para esse fim.

Avaliar se o marco de **consolidação do escopo** no processo geral de referência foi atingido requer que se definam: ➤

- as necessidades de negócio em termos específicos e mensuráveis conjuntamente com as principais restrições e premissas do desenvolvimento; ➤

- as entidades mais importantes que interagem com o sistema (de maneira ativa ou passiva), cujo ponto de partida para identificação, tipicamente, são as partes interessadas e os sistemas de informação que provavelmente serão afetados pelo desenvolvimento; ➤

- os eventos mais importantes para os quais o sistema deve responder. Eles devem ser descritos entre uma e três linhas. Para os

considerados de alta prioridade pelos clientes, ou de alto risco pela equipe de desenvolvimento, deve-se capturar uma descrição passo a passo do fluxo básico de informações associadas a cada um deles. Recomenda-se aplicar a regra 80/20 para decidir quais são os elementos mais importantes da lista anterior. Esta regra, também conhecida como Princípio de Pareto, diz que 80% das consequências advêm de 20% das causas.



os termos importantes e um glossário analisado. Quando há relacionamentos específicos entre conceitos-chave cuja captura é essencial, esses devem estar documentados e analisados em um modelo de domínio complementar ao glossário.

6.4. Terceiro marco: detalhamento dos requisitos

A partir do marco de Consolidação do Escopo, o objetivo maior é criar uma arquitetura que sirva como linha de base para todo o desenvolvimento. Isso não é o objeto da Engenharia de Requisitos. Contudo, para que essa arquitetura seja desenvolvida, são necessários um exame dos requisitos mais significativos (aqueles que têm grande impacto na arquitetura do sistema) e uma avaliação dos riscos associados.

Portanto, em termos de Engenharia de Requisitos, o marco de Detalhamento dos Requisitos é atingido quando a visão e os requisitos do produto são estáveis. Todos os eventos para os quais o produto deve prover resposta e todas as entidades que interagem com ele devem estar identificados. As especificações – descrevendo em detalhes o comportamento esperado pelo produto – estão aproximadamente 80% concluídas e os 20% restantes estão em desenvolvimento. Os níveis de serviços que o produto deve entregar estão definidos. Há um acordo entre as partes interessadas sobre a adequação desses últimos aos propósitos a que o produto deve atender.

6.5. Técnicas para obter consenso do escopo

A seguir serão apresentadas técnicas úteis para atingir a consolidação do escopo.

6.5.1. Técnica: declaração de problema

O *canvas* de modelo de negócio (*Business Model Canvas*) foi criado com o objetivo de descrever um modelo de negócio em uma página. É uma ferramenta de empreendedorismo e estratégia que permite descrever, projetar, constar, inventar e alavancar um modelo de negócio. Desde então essa ideia de criar uma tela (“canvas”) para estruturar ideias tem proliferado na arena da gerência de projetos e também no desenvolvimento de sistemas. A declaração de problema é bem alinhada a essa ideia.

A declaração de problema cumpre o papel de prover um “canvas” genérico para apoiar a organização e validação do trabalho de refinamento das declarações genéricas e que costumam descrever as necessidades de negócio. Ao contrário de quando se inicia a partir de uma página em branco, há orientação prática e critérios de qualidade que são associados à estrutura da declaração de problema.

O analista preencher o modelo como um fim em si mesmo e transcrever as informações da manifestação original com a necessidade de negócio, sem se ater aos objetivos citados anteriormente – que definitivamente não são gerar outra folha de papel –, é o lado negativo de se utilizar um recurso como esse.

A declaração de problema declara a necessidade de negócio tendo como objetivo observar os critérios de qualidade SMART descritos no Capítulo 5. Ela identifica as partes interessadas chave e descreve, brevemente, o impacto das necessidades de negócio nas partes interessadas em geral.

A declaração de problema ajuda a estruturar a “fala do cliente”. Um exemplo é apresentado na Tabela 6.1.

Tabela 6.1. Exemplo das informações sobre o domínio do problema expressas na forma de uma declaração de problema.

Problema	O planejamento e o controle no atendimento ambulatorial em consultórios e clínicas médicas são feitos manualmente, o que os torna ineficientes e muito sujeitos a erros.
Afeta	Clientes, médicos e atendentes.
Impacto	Os clientes aguardam mais tempo que o necessário e têm com frequência suas consultas canceladas porque um mesmo horário com um determinado médico foi agendado para duas ou mais pessoas. O médico tem seu tempo subutilizado e o fluxo de caixa é prejudicado por problemas operacionais no faturamento junto aos planos de saúde. Os atendentes têm duplicidade na execução de tarefas em função de erros e há um maior investimento de tempo em seu treinamento, já que não há um sistema que os oriente nos processos sob a sua responsabilidade.
Uma solução para ser bem-sucedida deve	Impedir a sobreposição de agendas; apontar a identificação de horários disponíveis em tempo hábil para procurar eliminá-los e conforme um plano geral de atendimento definido para cada profissional; e propor agendas alternativas para utilização da disponibilidade de maneira proativa por parte do atendente. Reduzir a glosa na documentação enviada para faturamento contra as operadoras de planos de saúde ao patamar de 10% do volume total de guias.



Problema: a descrição do problema, mesmo que tenha esse nome, não necessariamente corresponde a um problema a resolver. Ele também pode indicar uma oportunidade a se aproveitar. O analista de requisitos deve procurar extrair toda informação acessória e descrever de maneira bem específica o assunto que provocou o projeto.



Afeta: uma das principais preocupações na Engenharia de Requisitos é identificar as pessoas certas. Daí, levantar junto a elas assuntos sobre os quais têm domínio formal ou informal. Ainda que não se identifiquem indivíduos em específico, identificar as funções no negócio que sofrem o impacto do problema ou possuem interesse

em aproveitar a oportunidade em estudo é um excelente primeiro passo.



Impacto: enquanto a descrição em “Problema” limita-se ao assunto que motivou a mudança em desenvolvimento, neste espaço são descritos os efeitos de não se resolver o problema ou não se aproveitar a oportunidade para as funções indicadas.



Uma solução bem-sucedida deve...: o melhor critério de qualidade para descrever a eficácia de uma mudança é que ela tenha cumprido com o seu propósito. Este espaço é onde se registra qual é esse propósito. Com isso, quando a mudança em desenvolvimento se tornar operacional, é possível comparar o que ela entrega em termos de resultados com os benefícios-chave descritos neste item.

Estes são os critérios de sucesso para avaliar os resultados quando entregues. O interesse é descrever os benefícios que devem ser alcançados com a mudança; não os meios para alcançá-los. Estes últimos são resultados de decisões tomadas ao longo de seu desenvolvimento. Limitar as opções quando o propósito é mapear as necessidades de negócio não traz benefício algum; pelo contrário.

6.5.2. Modelagem de escopo (modelagem ambiental)

O objetivo da modelagem de escopo é identificar limites apropriados para o sistema em desenvolvimento. Esses limites são definidos indicando basicamente o que está dentro e fora do sistema; o que é interno e externo ao sistema. Os meios mais comuns para esse fim são: ➤

Diagrama de contexto.



Diagrama de casos de uso.



Modelo de processo de negócio.

6.5.3. Técnica: diagrama de contexto

O diagrama de contexto (ilustrado na Figura 6.4) modela o ambiente no qual o sistema está inserido, representando-o como um processo único. Ele indica elementos externos com os quais o sistema interage. A natureza dessa interação é a do intercâmbio de informações consumidas ou produzidas; não é a do fluxo de materiais, de ordem física. O diagrama de contexto não entra no mérito de qual é o processamento associado aos fluxos de dados.

Permite a quem o consulta identificar os relacionamentos do sistema com outros processos, áreas funcionais, clientes, controladores e fornecedores, com isso delimitando em termos gerais o que é parte do escopo do sistema e o que está no âmbito de outras entidades. Ele não se propõe a descrever especificamente quais transações devem ser desenvolvidas. Essa informação ainda deve ser revelada a partir dos limites definidos na modelagem ambiental.

É um diagrama de fluxo de dados (DFD) desenvolvido em seu mais alto nível de abstração. O propósito do DFD neste texto é modelar o ambiente como descrito anteriormente; não é ser o ponto de partida para uma dinâmica de decomposição funcional crescente com o objetivo de produzir a especificação dos requisitos a partir dessa abordagem.

Os seus elementos são como:

- a) entidades externas; b) depósitos de dados, também externos; e c) fluxos de dados que indicam o intercâmbio de dados entre o sistema e esses dois últimos.

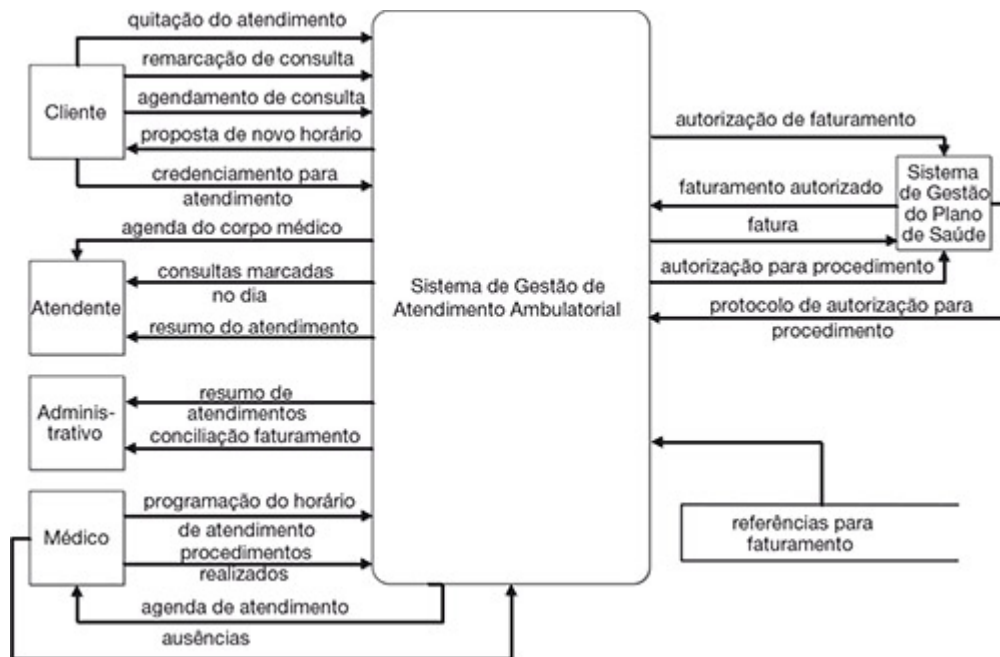


Figura 6.4. Diagrama de fluxo de dados nível zero como o diagrama de contexto.

6.5.3.1. Entidades externas

As entidades externas devem ter um comportamento próprio na perspectiva de quem responde pelo processo. Esse comportamento deve culminar com um estímulo da entidade externa para que o sistema faça alguma coisa ou com a resposta da entidade externa a um estímulo originado do sistema. Ele é o comportamento de: ➤

peessoas;



representações de organizações, processos de negócio ou áreas funcionais; ➤

outros sistemas de informação; ➤

dispositivos.

As entidades externas, quando interagem com o sistema, desempenham um papel. Por isso, o termo equivalente à entidade externa na UML é denominado “ator”. Destaca-se que o comportamento associado às entidades externas não deve ser relativo ao de um requisito de armazenamento.

As entidades externas têm por propósito classificar um grupo de pessoas (ou coisas) que interagem com o sistema de maneira formal ou informal. Aqueles classificados como tal compartilham: ➤

- a divisão do trabalho em diferentes funções; ➤
- um conjunto específico de habilidades e conhecimentos; ➤
- certas responsabilidades e relacionamentos.

Portanto, desenvolver o conhecimento sobre esses objetivos de informação associados às entidades externas deve ser um dos objetivos ao realizar as atividades de elicitación nessa fase do desenvolvimento. De fato, é condição necessária para avaliar o sucesso em alcançar o marco de Consolidação do Escopo.

A análise subsequente, baseada nas informações levantadas, não deve se limitar a elaborar o diagrama de contexto. Ele deve também organizar e documentar essas informações. Nisso, é possível identificar lacunas no desenvolvimento desses objetivos de informação e novas atividades de elicitación devem ser realizadas com o objetivo de superar essas lacunas.

O resultado desse trabalho facilita o exercício da Engenharia de Requisitos ao longo de todo o desenvolvimento.

6.5.3.2. Depósitos de dados

Os depósitos de dados (ou arquivos) no diagrama de contexto têm a função de representar requisitos de armazenamento sobre conceitos de negócio mantidos externamente que são apenas referenciados pelo sistema. Se houvesse atualização de dados pelo sistema, esses dados seriam considerados internos ao sistema.

Destaca-se que o termo “arquivo” nessa definição não se refere às soluções da tecnologia da informação que implementam requisitos de armazenamento externos em uma mídia física, como tabelas de banco de dados, arquivos em XML *etc.* O termo se refere a um conceito de negócio sobre o qual o sistema necessita recuperar dados. O conceito de negócio dá coesão a um conjunto de dados inter-relacionados que descrevem: a)

- sua estrutura; b)
- seu comportamento; e c)
- suas inter-relações.

A força que provê a coesão desse conjunto de dados em torno de um conceito de negócio é chamada de dependência funcional.

Os propósitos de um depósito de dados em um diagrama de contexto são: ➤

Complementar dados que são manipulados pelo sistema e que precisam de referências mantidas externamente para sua qualificação.



Validar dados transitados pelo sistema com base em dados recuperados externamente.



Subsidiar cálculos e a avaliação de condições processadas na aplicação a partir da recuperação de dados externos.



Organizar dados necessários ao processamento do sistema em unidades coesas observando critérios mantidos externamente.

6.5.3.3. Conclusão

O objetivo do diagrama de contexto é representar o ambiente no qual o software está inserido e seus principais fluxos de dados, sem abordar o seu processamento interno. Uma vez que se obteve consenso entre as partes interessadas sobre o contexto representado pelo diagrama, ao longo do trabalho de requisitos no projeto é pouco provável que se necessite atualizá-lo.

O surgimento de um novo ator ou algum fluxo de dados crítico que indique um novo macroprocesso até então externo ao sistema indica uma mudança nas premissas básicas acordadas para o projeto. Como parte do tratamento dessa mudança, a atualização do diagrama de contexto se justifica como meio para facilitar um novo consenso.

O objetivo é que, a partir do diagrama de contexto, possam ser mais facilmente identificadas as partes interessadas para elicitar requisitos e validar os resultados de sua análise, delimitando o escopo com base no trâmite de informações entre o ambiente interno do sistema e externo dos usuários.

Portanto, não há motivos para se ater estritamente ao vocabulário gráfico do DFD em qualquer das vertentes mais conhecidas (GANE-SARSON, 1979; DEMARCO, 1979). Na verdade, os autores deste livro consideram que seja até mais efetivo o uso de representações como as da Figura 6.5, onde não há tanta ênfase no detalhamento dos fluxos de dados e a principal intenção é destacar quais macroprocessos estão incluídos no escopo do desenvolvimento.

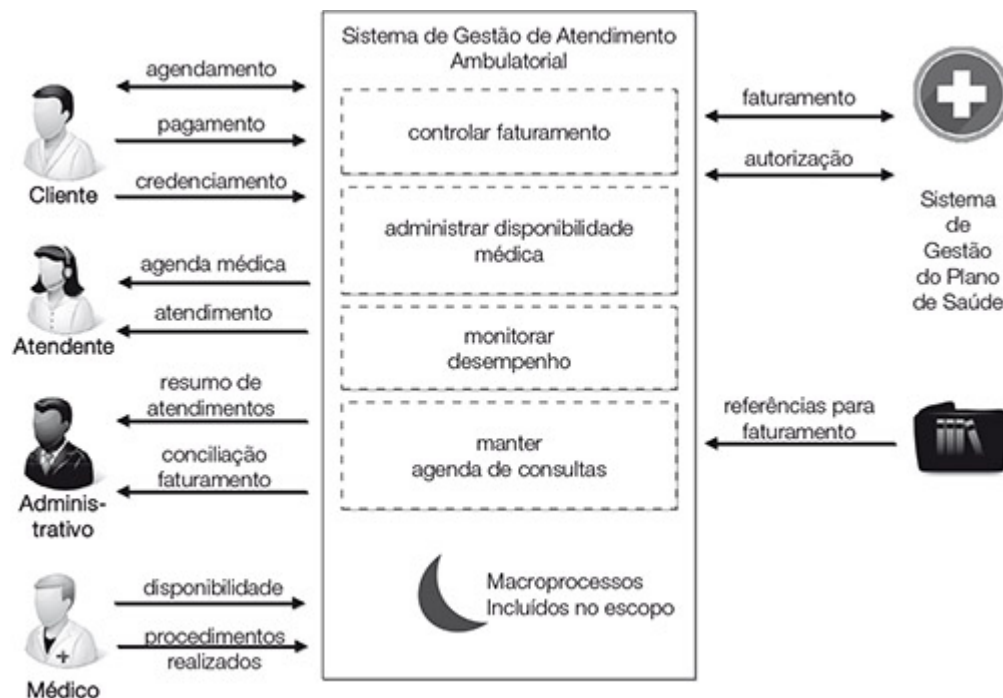


Figura 6.5. Um diagrama de contexto estilizado e com menos ênfase nos fluxos de dados específicos com o objetivo de ilustrar os macroprocessos incluídos no escopo.

6.5.4. Técnica: diagrama de casos de uso

Neste tópico, pretende-se discutir o papel de um diagrama de casos de uso para modelar o escopo no âmbito da iniciação de um desenvolvimento; não se pretende esgotar o tema, mais bem explorado no Capítulo 8.

O objetivo de um diagrama de casos de uso é descrever quem faz o quê no sistema. Ele representa visualmente: ➤

os eventos para os quais o sistema deve prover uma resposta: faz isso por meio da identificação de casos de uso do sistema por um ator, cada um associado a uma tarefa sob a responsabilidade do ator e que se beneficia dos resultados entregues pelo sistema; ➤

as entidades externas com as quais o sistema deve interagir: atores nos casos de uso do sistema; e ➤

a relação entre esses dois elementos.

O diagrama de casos de uso não entra em detalhes sobre: a)

as sequências de passos necessárias ou possíveis para prover uma resposta ao evento que o motivou; b)

as regras de negócio que se aplicam; ou c)

a sequência de casos de uso que deve ser realizada para se cumprir um processo de negócio.

A Figura 6.6 ilustra um diagrama de casos de uso.

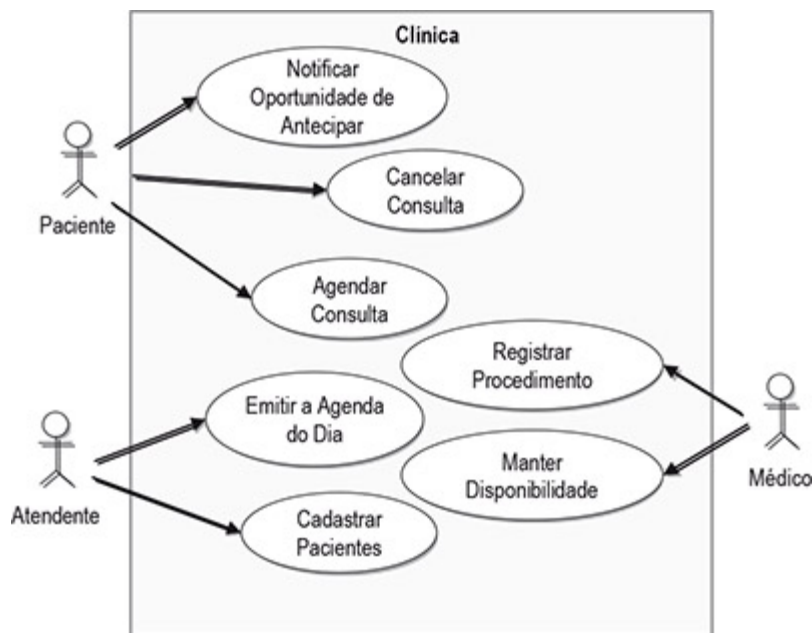


Figura 6.6. Recorte de um diagrama de casos de uso para uma clínica médica.

Fazendo um paralelo com o diagrama de contexto, o recorte parcial na Figura 6.5 ilustra que ambas as ferramentas têm o mesmo propósito de

indicar quem são os atores com os quais o sistema deve interagir. Enquanto o diagrama de contexto indica as principais funcionalidades do sistema por meio dos fluxos de dados, o diagrama de casos de uso faz isso com base na identificação de casos de uso do sistema por um ator. Diferentemente do diagrama de contexto, o diagrama de casos de uso não representa os requisitos de armazenamento externo do sistema.

Em seu estágio final, tipicamente próximo de ser alcançado o marco de **detalhamento dos requisitos** do processo de referência, os casos de uso do sistema estão no âmbito da execução de uma tarefa ou serviço pelo qual o ator é responsável. Ele então modela o escopo em um nível de detalhe no qual é possível apontar especificamente em quais tarefas o ator deve ter um caso de uso do sistema.

O mais provável é que, na ponte do anteprojeto até o marco de **consolidação do escopo**, os casos de uso do sistema identificados sejam macroprocessos que agregam objetivos colaborativos, compreendendo várias tarefas das quais ainda não se sabe especificamente quais serão incluídas no escopo e quais estarão fora dele. Então, em termos de propósito, os casos de uso do sistema identificados nesse ponto do desenvolvimento dos requisitos cumprem o mesmo propósito dos fluxos de dados em um diagrama de contexto.

Se o diagrama de casos de uso é utilizado para modelar o ambiente em um estágio inicial do desenvolvimento, então ele refletirá uma foto que ainda não indica especificamente em quais tarefas o ator deve ter um caso de uso do sistema. Portanto, ele deve ser encarado como tendo diferentes estados de evolução. A sua qualidade deve ser avaliada de acordo com o momento em que ele é produzido ou atualizado e os diferentes objetivos associados aos diferentes momentos no ciclo de vida do software. Ele deverá ser atualizado na medida em que as decisões sobre quais tarefas serão objeto de automatização ou informatização vão sendo tomadas e, portanto, novos casos de uso com o sistema são identificados.

A intenção não é recomendar que o trabalho de elicitação siga essa lógica de decomposição funcional descrita no parágrafo anterior. A intenção é destacar a necessidade de atualizar os diagramas de casos de uso na medida em que o desenvolvimento dos requisitos evolui.

6.5.5. Técnica: modelo de processo de negócio

O objetivo desta explicação está restrito aos propósitos do modelo de processo de negócio no âmbito da iniciação do desenvolvimento como meio para modelar o ambiente em que o sistema em produção se insere e ajudar na identificação de partes interessadas para dar sequência à elaboração dos requisitos.

Um modelo de processo de negócio implica na representação de um determinado estado do negócio (atual ou futuro) e dos respectivos recursos envolvidos, tais como pessoas, informação, instalações, automação, finanças e energia. Como é utilizado para representar com mais precisão o funcionamento daquilo que está sendo modelado, requer mais dados acerca do processo e dos fatores que afetam seu comportamento. A Figura 6.7 ilustra um modelo de processos de negócio.

É um modelo de alto nível para processos de negócio. Diz-se alto nível porque não tem por objetivo descrever o passo a passo de cada tarefa. Sua análise permite identificar quais atores desempenham quais tarefas e como elas se relacionam entre si em direção aos objetivos do processo de negócio. Ele não se limita a representar apenas as tarefas que contenham casos de uso do sistema pelo ator, incluindo em sua representação tarefas manuais. No âmbito do vocabulário da modelagem de negócio, uma tarefa manual é aquela que não é executada ou gerenciada por um sistema de informação automatizado que implementa um processo de negócio.

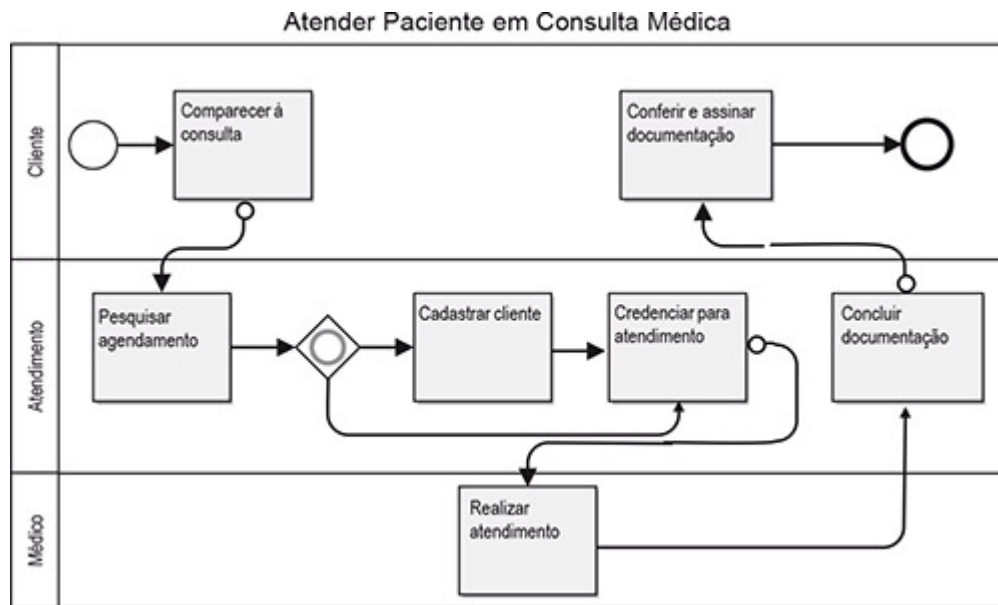


Figura 6.7. Exemplo de modelagem de processos de negócio.

O modelo de processo de negócio integra diversas áreas funcionais, sistemas e mesmo outras organizações que interagem com o processo. Ele permite identificar as responsabilidades desses diferentes elementos e, por isso, também é um modelo ambiental.

Os três tipos de elementos básicos de modelagem que merecem maior destaque na iniciação do desenvolvimento são: ➤

Atividade: representa o trabalho realizado em determinado momento do processo.



Fluxo: representa a sequência das atividades desde o início até a conclusão do processo.



Raia: representa quem é o responsável pelas atividades contidas nesta parte do processo.

6.6. Como saber com quem conversar

A identificação preliminar de partes interessadas acontece no anteprojeto e o seu refinamento é uma das primeiras ações para iniciação do desenvolvimento. A Engenharia de Requisitos aproveita-se dessa informação para identificar o subconjunto dessas partes relevantes para seus objetivos.

Ao elaborar ou analisar uma declaração de problema como da Tabela 6.1, deve-se atentar para quem é impactado pelo problema ou oportunidade associada à mudança. A declaração de problema apresenta uma função em geral e não um indivíduo em particular. Deve-se, portanto, descobrir junto às partes interessadas chave quem são os indivíduos que possuem a autoridade e que podem responder por essa função no desenvolvimento dos requisitos. Essa investigação (elicitação) não deve se restringir às funções destacadas na seção com o impacto; deve-se também avaliar as demais seções com o mesmo objetivo.

Todos os demais modelos apresentados neste capítulo também proveem insumos para orientar a investigação adicional sobre quem são os indivíduos que devem ser ouvidos ou outros documentos que devem ser analisados para o subsequente desenvolvimento dos requisitos. Nos diagramas de contexto, há as entidades externas; nos diagramas de casos de uso, os atores; e nos modelos de processos de negócio, as raias e *pools*.

As atividades de elicitação – que investigam – e as atividades de análise – que organizam a informação na elaboração ou atualização desses diagramas – se alternam. A elicitação a partir das manifestações do negócio anteriores ao desenvolvimento fornece insumos para a análise subsequente que produz ou atualiza um (ou mais) desses modelos. Ao fazer isso, descobrem-se novas necessidades de elicitação.

Como não poderia ser diferente, o escopo descrito na iniciação do desenvolvimento abstrai-se de uma série de decisões que ainda devem ser tomadas para haver a especificidade necessária à sequência do desenvolvimento (projeto, implementação, testes etc.). Saber que não se sabe alguma coisa é um bom ponto de partida para mudar essa situação.

Todos esses modelos permitem identificar as áreas de processos que devem ser abordadas pelo sistema em desenvolvimento. Elas ainda incorporam decisões que devem ser tomadas quanto a um escopo mais específico em

nível de tarefa e regras de negócio; e ao detalhamento sobre como o sistema deve se comportar no âmbito dessas tarefas.

Esse é o início da jornada por onde se começa o exercício das atividades de Engenharia de Requisitos a serem exploradas em detalhes nos Capítulos 7, 8 e 9.

6.7. Exercícios

1. Qual grupo de tarefas não pertence à Engenharia de Requisitos?
 - a) Elicitação de requisitos.
 - b) Testes de aceitação do usuário.
 - c) Gerência de requisitos.
 - d) Análise de requisitos.
2. O que caracteriza a Elicitação de Requisitos?
 - a) Avalia o impacto de mudanças.
 - b) Soluciona conflitos.
 - c) Especifica requisitos.
 - d) Levanta necessidades de partes interessadas.
3. O que caracteriza a Análise de Requisitos?
 - a) Gerencia questões que surgem ao longo do desenvolvimento dos requisitos.
 - b) Organiza e documenta requisitos.
 - c) Busca entender melhor o domínio do problema.
 - d) Identifica partes interessadas.
4. O que não caracteriza a Gerência de Requisitos?
 - a) Busca aprovação dos requisitos.
 - b) Trata as solicitações de mudanças.
 - c) Gera modelos do estado futuro.
 - d) Prioriza requisitos.
5. Por que o estudo de viabilidade pode facilitar o trabalho inicial da Engenharia de Requisitos?
6. A partir da especificação do estudo de caso presente no anexo, identifique os candidatos a partes interessadas a serem envolvidas no trabalho de requisitos. Assuma que você é o analista de requisitos da empresa contratada para executar o projeto.

7. A partir da especificação preliminar do estudo de caso do Anexo, elabore um rascunho do seu diagrama de contexto. Considere que seja um rascunho, pois você pode identificar dúvidas que precisariam ser respondidas antes de considerar o modelo finalizado. Registre essas dúvidas e avalie o quanto o processo de elaboração do diagrama o ajudou a perceber essas questões.

7. Elicitação de Requisitos

“O óbvio, Lóri, é a verdade mais difícil de se enxergar.”

Clarice Lispector (Uma Aprendizagem ou o Livro dos Prazeres) “Só os profetas enxergam o óbvio.”

Nelson Rodrigues Este capítulo aborda a elicitação de requisitos, definindo o que é, quais as tarefas envolvidas e como essas tarefas interagem entre si, tudo isso de forma a destacar o caráter proativo exigido do analista ao longo do trabalho e comentando como alcançar essa proatividade. O capítulo ainda apresenta de forma prática várias técnicas úteis à elicitação, apresentando pontos fortes, fracos e considerações sobre o seu uso.

7.1. Definindo a elicitação de requisitos

O termo “elicitação de requisitos” é um anglicismo, uma versão brasileira do termo em inglês *elicit requirements*, que se consolidou com o tempo entre as pessoas envolvidas com a disciplina de requisitos. Porém, muitas pessoas preferem o termo levantamento de requisitos. Outros autores usam denominações como: coleta de requisitos, descoberta de requisitos, extração de requisitos, obtenção de requisitos, captura de requisitos ou aquisição de requisitos.

Perceba que esses termos podem passar uma ideia um pouco distinta do que é o trabalho que se descreve neste capítulo. Coletar requisitos passa uma ideia de que os requisitos estão em um balcão e basta pegá-los. Pode ser, mas raras vezes isso corresponde à realidade. Adquirir requisitos passa a ideia de que envolve uma negociação, o que faz sentido algumas vezes.

Extraír requisitos passa a ideia de que se está garimpando a informação, o que de fato é necessário algumas vezes. Descobrir requisitos passa a ideia de que o trabalho é investigativo, o que também procede outras tantas vezes. Para agrupar todas as ideias associadas a esses vários termos, optou-se neste livro por usar a palavra “elicitación”.

E o que é, afinal, a elicitação? É um processo de aquisição de conhecimento, onde se aplicam técnicas para compreender melhor o negócio a ser impactado pelo projeto, para identificar partes interessadas e para identificar e refinar os tipos de requisitos que foram apresentados no Capítulo 5.

E qual o produto gerado pelo trabalho da elicitação? São as memórias de levantamento, que documentam o conhecimento adquirido e cujo formato varia em função das técnicas de elicitação usadas. Exemplos: notas de entrevistas, respostas de questionários, atas de reunião, gravações de áudio e vídeo *etc.*

Essas memórias de levantamento representam informação não estruturada. É necessário que essas informações passem por um tratamento para que possam ser utilizadas no desenvolvimento do software. Isso envolve: organizar, sintetizar, eliminar redundâncias, descobrir omissões e erros. O tratamento que se dá a essas informações é o que se chama análise de requisitos, que será assunto do capítulo seguinte. Em resumo, reunir todas as memórias de levantamento (exemplo: atas de entrevista) e entregar para a equipe desenvolver o software não funcionará.

É a análise de requisitos que permite avaliar se houve qualidade na elicitação. Na prática, é muito difícil que se esgote a elicitação em um passo único no projeto. O mais razoável é que, após algumas sessões de elicitação, tendo informação suficiente, já se inicie uma parte do trabalho de análise de requisitos. E esse trabalho irá orientar quais necessidades de informação devem ser buscadas, provocando um novo ciclo de elicitação-análise.

7.2. Atividades da elicitação

Para qualquer que seja a técnica empregada na elicitação de requisitos, destacam-se quatro atividades fundamentais: ➤

Preparação (ou planejamento).



Execução.



Documentação.



Confirmação.

7.2.1. Preparação para elicitação

O objetivo da preparação para elicitação é garantir que todos os recursos necessários estejam organizados e reservados para sua realização. Deve-se entender recursos como: agenda das pessoas a serem envolvidas, disponibilidade de salas, planejamento de viagens (se necessário), questões logísticas de maneira geral. A preparação é fundamental para garantir um bom resultado na elicitação. O critério de sucesso é satisfazer os objetivos de informação associados ao momento em que o projeto se encontra, sem desperdiçar recursos.

De acordo com Jones (2007), os resultados da elicitação ainda estão longe desses objetivos. Seu trabalho indica que raramente os requisitos iniciais são 50% ou mais completos e que o tempo dos envolvidos com reuniões e discussões está em quarto lugar dentre as atividades que mais consomem recursos em um projeto de software.

Os objetivos de informação citados podem ser resumidos em três grupos conforme o momento e dizem respeito a obter informações que permitam:



Obter acordo sobre o problema a ser resolvido (ou oportunidade a ser aproveitada), as partes interessadas inicialmente identificadas, as

restrições que se aplicam e a proposta ainda em alto nível para uma possível solução e quais suas características.



Convergir itens em aberto quanto à abrangência da solução em um escopo mais refinado, já identificando os requisitos associados aos objetivos de uma tarefa do usuário e incluindo decisões sobre os requisitos não funcionais: requer descer de processos mais abrangentes para tarefas mais específicas e subir de fragmentos de informação no sentido oposto. Objetivos relacionados à abrangência buscam chegar a conclusões como as declarações: ✓

“Os dados de fiscais serão consultados nos serviços *on-line*”.



“Eu quero manter o cadastro de fiscais no sistema”.



“Eu quero um relatório de obras sem responsável técnico”.



“Eu quero um relatório de obras sem placas”.



Estabelecer o consenso com o entendimento mais profundo sobre a funcionalidade do sistema e seus requisitos não funcionais: quando o que se busca é a profundidade de determinado requisito, a ênfase é a visão da parte interessada sobre o comportamento que se espera da solução e das restrições que se aplicam. É como se os resultados da elicitação contassem uma história onde, ao final, o objetivo da parte interessada é alcançado.

Este é também o momento em que as técnicas de elicitação devem ser selecionadas. Vários fatores influenciam nessa decisão: domínio do negócio, cultura empresarial, habilidades da equipe, recursos disponíveis. Não existe “a melhor” técnica de elicitação de maneira absoluta. Para uma determinada situação, uma técnica pode se encaixar melhor que outra.

Na prática, muitas vezes a elicitação ocorre com o uso de mais de uma técnica. O importante é que o analista esteja familiarizado com várias técnicas para que possa usar a mais adequada para uma determinada situação do projeto. A ideia é formar uma “caixa de ferramentas” e tirar o melhor proveito de cada uma nos momentos necessários.

Por exemplo, a preparação da elicitação referente à primeira categoria de objetivos citada pode resultar na seguinte tática: ➤

Os documentos com as manifestações que provocaram o projeto devem ser entendidos, ainda que o resultado seja mais questões em aberto do que respostas. A técnica de análise de documento é usada para esse propósito.



A “inteligência” construída a partir da análise de documentos gera uma pauta que deve ser utilizada na busca por respostas usando a técnica de entrevistas.



Termos do domínio do problema presentes nos documentos e que surgem nas entrevistas são insumos para a técnica de glossário.



As expectativas quanto à solução proposta junto aos funcionários são mais bem mapeadas usando a técnica de pesquisa (questionário).

Para qualquer necessidade de informação deve-se manter sempre em perspectiva na preparação para elicitação: ➤

Requisitos de negócio, escopo da solução: orientam o analista sobre quais informações serão elicitadas.



Lista de partes interessadas com papéis e responsabilidades: usada para identificar quais partes interessadas irão participar das atividades de elicitação (com quem interagir).

7.2.2. Execução da elicitação

Ao executar atividades de elicitação, o objetivo é levantar informações, de maneira proativa, junto às partes interessadas usando as técnicas selecionadas na preparação. Cada técnica tem uma metodologia própria. Os tópicos seguintes apresentam as técnicas mais comuns de elicitação e como aplicá-las em detalhes.

Muito importante durante a aplicação de qualquer uma delas é evitar desvio do escopo do projeto. Ao interagir com uma parte interessada para levantar suas necessidades, ela provavelmente trará à tona todos os problemas que vivencia. O projeto nem sempre tem o objetivo de resolver todos esses problemas. Portanto, nessa interação é importante conscientizá-la dos objetivos do projeto. Ainda assim, é possível que ela exponha necessidades que estão fora desses objetivos. Como analista, você poderia retrucar: “compreendi esta sua necessidade. Poderia me esclarecer, por favor, qual o vínculo dela com os objetivos do projeto?”.

Caso não haja qualquer vínculo, fica mais fácil orientá-la a direcionar essa necessidade para o projeto pertinente ou que verifique a viabilidade de criar um projeto para resolvê-la. Por isso a importância de uma clara definição dos requisitos de negócio. A rastreabilidade dos requisitos das partes interessadas para os requisitos de negócio é o que garante que o escopo não será desviado.

Ao longo da elicitação, fique atento também para capturar os atributos dos requisitos. O plano de gerenciamento de requisitos deve definir quais atributos devem ser capturados. Ele é elaborado pelo gerente de projetos ou já faz parte da definição do processo de desenvolvimento utilizado pela

empresa (neste caso, um ativo de processo organizacional). Exemplos de atributos de requisito podem ser: origem, prioridade, autor, proprietário, estado, risco *etc.* Este é um tema que será mais explorado no Capítulo 9.

E nessa captura dos requisitos e de seus atributos não conte apenas com sua memória, procure usar algum mecanismo de registro das informações (por exemplo: papel, gravador). Mas lembre-se antes de pedir a autorização das partes interessadas presentes no evento para fazer esse registro.

Uma das grandes e frequentes falhas no trabalho de elicitação é não identificar requisitos, e essa necessidade se manifestar mais à frente no projeto, gerando graves consequências. São os chamados “requisitos implícitos”. São desejos não manifestos pela parte interessada; seja por ela imaginar que isso já é de conhecimento do analista de requisitos, e por isso não é citado; ou por ser um desejo subconsciente que só se tornará consciente quando ela visualizar o produto pronto. De ambos os modos, a omissão de requisitos é uma falha grave do trabalho de elicitação.

A pergunta que surge diante desse cenário é: “mas como vou identificar esses requisitos se a parte interessada não falar?”.

Não se deve esperar que as partes interessadas digam tudo que é necessário voluntariamente. Se fosse assim, a função de analista de requisitos poderia ser extinta e substituída por um tomador de notas. O valor do bom analista se revela exatamente nessas situações. Conseguir identificar o que a parte interessada pensa que é óbvio e não precisa ser dito ou antecipar um desejo que ela ainda não percebeu são ações que evitam muito retrabalho à frente no projeto.

“E há alguma técnica para descobrir esses pontos não ditos?”. A resposta é sim, e há dois caminhos complementares nesse sentido. O primeiro é aprofundar o conhecimento no negócio da parte interessada.

Mesmo que o responsável pela elicitação seja um analista funcional, não se pode esperar que esteja em um mesmo nível de conhecimento em sua interação com as partes interessadas. Sempre haverá um desnível que torna possível ao analista não perceber o “óbvio” ou o “subconsciente”. Sem dúvida, contar com um profissional com maior conhecimento no domínio do problema diminui o potencial de requisitos permanecerem ocultos.

Contudo, formar esse conhecimento pode requerer anos de treinamento e experiência.

As restrições de tempo e custo em um projeto não permitem que um investimento similar seja feito e que o conhecimento do analista de requisitos se equipare ao da parte interessada. Contudo, deve haver tempo e recursos para que o analista obtenha ou melhore o seu conhecimento do negócio.

Daí a importância do segundo caminho: organizar os pedaços de informação coletados em diferentes visões com o objetivo de identificar lacunas. Isso é parte dos objetivos das atividades de análise de requisitos.

Enquanto a interação do analista com a parte interessada ocorrer com um desnível de conhecimento do negócio muito grande, ficará muito difícil para o analista perceber o “óbvio” ou o “inconsciente”. Ao melhorar seu conhecimento do negócio da parte interessada, o analista ganha uma perspectiva mais ampla e clara das necessidades do outro. O óbvio da parte interessada passa a ser óbvio também para o analista. Ajudá-lo nesse objetivo é parte da missão deste livro.

7.2.3. Documentação dos resultados da elicitação

As técnicas utilizadas durante a execução da elicitação, bem como a abordagem planejada, determinam o tipo de documentação. Há abordagens em que os produtos da elicitação são produtos finais, como histórias do usuário usadas em metodologias ágeis. Considerando abordagens em que os produtos da elicitação são requisitos intermediários, cujo propósito é registrar decisões tomadas sobre assuntos até então pendentes, a documentação produzida é denominada de memórias de levantamento. Esses documentos podem ser: atas de reunião, relatórios, registro de áudio ou vídeo, quadros com anotações *etc.*

O propósito de documentar resultados ainda intermediários e não partir direto para a análise de requisitos é para que: ➤

- a informação não se perca – afinal, a memória das pessoas é falível;

-

o conhecimento adquirido na elicitação possa ser compartilhado com os demais membros da equipe do projeto; ➤
outras pessoas possam dar sequência à análise de requisitos. Nem sempre o trabalho de requisitos é realizado por um só analista; ➤
seja possível confirmar, por meio dessa documentação, o seu entendimento com as partes interessadas. Lembre-se de que dificuldades de comunicação representam a maior parte dos problemas na Engenharia de Requisitos.

Além da documentação das informações colhidas ao realizar a elicitação, é importante registrar questões em aberto e dúvidas que surgiram durante esses eventos e que não puderam ser resolvidas para acompanhamento até a sua resolução. Este é o objetivo da técnica controle de questões abordada no Capítulo 9.

É muito comum entre alguns profissionais a ideia de que registrar o resultado de uma interação com a parte interessada para levantamento de informação é um processo burocrático. Muitos adotam o padrão de não registrar nada, confiando que as informações ficarão na memória até que sejam usadas, seja para gerar uma especificação ou o próprio programa diretamente. E pode ser que isso funcione muito bem em alguns contextos, mas certamente não funcionará para todos. O tópico sobre nível de detalhe da especificação do Capítulo 2 destaca fatores que podem favorecer uma comunicação informal e somente verbal.

Uma informação que se transmite verbalmente pode se perder e também está sujeita a um ruído significativo cada vez que é transmitida. O ponto é que o registro das informações é uma estratégia de gestão do conhecimento.

O que diferencia a História da Pré-História? O surgimento da escrita. Na pré-história, a única maneira do conhecimento ser compartilhado e perpetuado era via oral. A escrita acelerou a evolução dos povos que a dominaram. Portanto, desenvolver requisitos sem o registro de informação alguma significa trabalhar no método pré--histórico. O que é uma ironia, tendo em conta que software seria uma das manifestações mais recentes do avanço tecnológico da humanidade.

Durante os eventos de levantamento, é possível que a discussão dos assuntos seja fragmentada e um mesmo assunto tenha sido revisitado várias

vezes. Ainda assim, ao elaborar um documento de entendimento, deve-se organizar o registro das decisões tomadas e as informações relevantes coletadas por assunto. Afinal, seu propósito é garantir o entendimento comum dos pontos nele presentes e não cumprir o papel do registro de uma sessão do júri.

Mesmo quando se dá a gravação em vídeo ou áudio, é fundamental compilar esses pontos em um documento de entendimento com informações sobre: ➤

- destaques dos tópicos discutidos; ➤
- decisões tomadas; ➤
- questões não resolvidas; ➤
- ações acordadas e prazos limites; ➤
- pessoas responsáveis por cada ação; ➤
- data do próximo encontro, se houver.

7.2.4. Confirmação dos resultados da elicitação

Um dos autores certa vez saiu para jantar fora com sua esposa. No restaurante, foram atendidos na mesa por uma moça, que, após apresentar o cardápio, ouviu o pedido. Depois de certo tempo ela traz somente um dos pratos à mesa. Imaginando que o segundo prato chegaria em seguida, o casal aguardou. Após certa demora, o casal chama a atendente para perguntar quando o segundo prato seria trazido. Sua resposta foi: — Nossa, esqueci de pedir!

É possível que você, leitor, já tenha passado alguma vez por experiência semelhante; ou receber algo diferente que pediu. Por isso, em restaurantes mais organizados, todo garçom adota um protocolo muito simples: anotar cada item pedido e antes de sair da mesa confirmar o pedido inteiro.

Vários pontos de falha poderiam existir sem esse protocolo. Frequentemente, quando se atende uma mesa, mais de uma pessoa fala com o garçom. Também é muito comum uma pessoa pedir algo, logo depois mudar de ideia e pedir outra coisa. Sem a confirmação final do pedido, itens podem ficar esquecidos ou trocados. Do percurso da mesa até a cozinha para entregar o pedido, provavelmente o garçom será solicitado a atender

outras mesas. Sem anotar cada pedido, há que se ter uma memória prodigiosa para não esquecer nenhum pedido de nenhuma mesa e não trocar itens.

O problema é muito parecido com o do analista de requisitos. Suponha que o analista vá realizar a elicitação com entrevistas. Durante a entrevista muitas informações serão recebidas. Se essas informações não forem documentadas durante ou ao final da entrevista, o risco de perda de alguma informação é alto (assumindo que não há pré-requisito de memória prodigiosa para ser analista de requisitos). E, ainda que se documente tudo, há sempre algum ruído entre a emissão da informação e a compreensão da informação pelo outro. Documentar um entendimento errado do que o outro disse não tem valor algum. Por isso a necessidade de confirmação do que foi elicitado.

Falhas de comunicação não são exclusividade do trabalho de requisitos e estão presentes em diversas outras atividades humanas. Em várias delas, as consequências dessas falhas de comunicação podem ser muito mais graves do que as que ocorrem na Engenharia de Requisitos. Falhas de comunicação do controle de tráfego aéreo com um piloto de avião podem causar grandes tragédias (e já causaram).

Conforme a criticidade em determinadas profissões, desenvolvem-se estratégias e protocolos de comunicação mais elaborados para minimizar essas falhas, como na aviação. Para o trabalho de requisitos, não há a necessidade de algo tão elaborado. O protocolo para a elicitação apresentado na Figura 7.1 sobre como receber a informação, documentar e confirmar é simples e eficaz e é denominado “Protocolo do Garçom”.

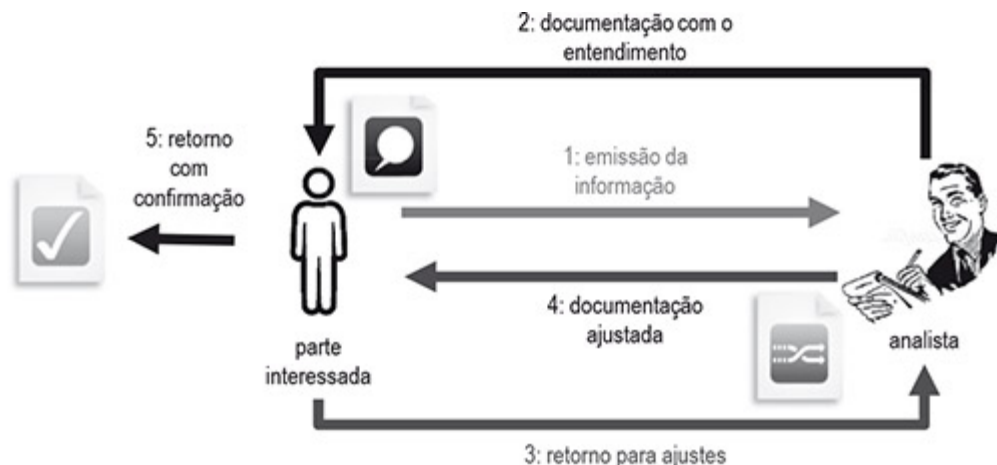


Figura 7.1. O protocolo do garçom, método simples para filtrar falhas de comunicação na elicitação.

Normalmente, em uma primeira versão da documentação há problemas de entendimento, e o documento retorna sem confirmação, com a indicação dos pontos equivocados ou ausentes. Após a realização dos ajustes, uma nova versão é encaminhada às partes interessadas. Essa dinâmica se repete até que as partes interessadas confirmem a informação refletindo o discutido nos eventos de levantamento e devolvam o documento para utilização como representação formal das suas necessidades.

Se a informação capturada na elicitação foi mal compreendida, então sem a confirmação sua documentação também é incorreta. Essa memória de levantamento defeituosa será utilizada na análise de requisitos e levará a requisitos incorretos a serem especificados, modelados e por fim construídos.

A confirmação é algo simples e rápido de se fazer e ajuda a evitar que esses defeitos de informação contaminem a elicitação. Ainda assim, muitas vezes os documentos enviados para confirmação ficam sem resposta. Uma solução no plano de cada evento de confirmação é determinar a data-alvo para receber os comentários complementando ou retificando as informações presentes na própria comunicação com a memória de levantamento elaborada. No plano do projeto como um todo, é possível estabelecer nas políticas de comunicação a eficácia plena da aceitação do teor integral das informações enviadas para confirmação caso a comunicação de retificações não seja feita em um prazo de até determinada quantidade de dias.

7.3. Técnica: análise de documentos

A dificuldade em conseguir acesso às partes interessadas é um dos problemas mais relatados pelos participantes dos treinamentos sobre Engenharia de Requisitos ministrados pelos autores. A reflexão sobre essa dificuldade levou à identificação de fatores que vão além da falta de disponibilidade de agenda dessas partes interessadas.

Durante esses treinamentos, são usados casos reais de projetos para diversos exercícios. Em um primeiro momento, e sem a apresentação dos objetivos de informação buscados na Engenharia de Requisitos, percebe-se pouco interesse de alguns participantes na análise da documentação onde se materializam as necessidades do negócio. Para esses participantes, ela é uma atividade tediosa e que entrega pouco resultado.

Essa percepção não se restringe ao campo de uma sala de aula, estando presente na atuação de vários profissionais engajados na Engenharia de Requisitos. A alegação para isso é a falta de respostas oferecidas pelos documentos (mal) analisados.

Será que, ao menos em um primeiro momento, o mais importante são as respostas? Na obra de Douglas Adams, “Guia do Mochileiro das Galáxias”, solicita-se a um computador chamado Pensador Profundo, designado para responder a qualquer pergunta, a resposta para a pergunta fundamental da vida, do universo e tudo o mais. Depois de sete milhões de anos ele responde: 42. Qual o valor dessa resposta sem saber qual a pergunta? Não é por acaso que um novo projeto teve de ser comissionado naquela história.

Portanto, perde-se uma oportunidade importante de o analista adquirir conteúdo sobre o contexto abordado ou de aumentar o nível de informação sobre o domínio do problema e sobre as possíveis soluções a serem adotadas e também identificar questões a serem abordadas junto às partes interessadas. Sempre há algo já documentado que deveria ser analisado primeiro para depois partir para outras técnicas de elicitação. E isso enriquece o processo de revelação dos requisitos.

Claro que não se espera que o analista de requisitos necessariamente se torne um analista funcional com conhecimento especializado em um determinado domínio de negócio ou processo de negócio; mas entre esse extremo e o outro, em que não se usa a documentação disponível para apoiar o processo de descoberta de requisitos, há uma enorme lacuna.

Este tópico trata do que é a análise de documentos e como ela deve ser utilizada para eliminar essa lacuna em diferentes momentos no processo de desenvolvimento dos requisitos.

7.3.1. O que é a análise de documentos

A análise de documentos é um meio de elicitar requisitos pelo estudo de documentação disponível sobre uma solução existente para identificação de informação relevante para o desenvolvimento de uma nova solução.

O termo “documentação” neste caso deve ter um significado mais amplo do que as especificações dos requisitos da solução e incluir documentos sobre o domínio do problema que permitam alcançar os objetivos de informação presentes em momentos iniciais e com importância estruturante para todo o ciclo de vida do projeto. São dois os objetivos de informação: ➤

Identificar as necessidades de negócio (oportunidades a aproveitar, problemas a resolver, objetivos a alcançar e métricas de sucesso).



Identificar, inicialmente, as partes interessadas chave e, em seguida, as demais (chave ou não).

Sommerville (2006) cita como instrumento para ajudar na descoberta de requisitos a interação com partes interessadas por meio de entrevistas e observações, podendo incluir cenários e protótipos. Pressman (2014), nesse particular, não entra em mais detalhes, apenas citando Sommerville. O motivo talvez seja o significado de requisito nessas importantes obras estar limitado à especificação da solução e à não utilização de uma estrutura de requisitos mais abrangente, onde se apresentam os requisitos de negócio e das partes interessadas.

O corpo de conhecimento de gerenciamento de projetos do PMI (*PMBOK® Guide*) descreve a análise de documentos como uma das ferramentas do gerenciamento de escopo do projeto, mais especificamente ao descrever o processo de **coleta de requisitos**.

Este processo é descrito como um meio para elicitar requisitos pela análise da documentação existente e para identificar informação relevante para os requisitos. Estabelece que haja uma faixa ampla de documentos que podem ser analisados, incluindo, mas não se limitando a: planos de negócio, literatura de marketing, acordos, pedidos de proposta (RFP), fluxos de processos atuais, modelos de dados lógicos, repositórios de regras de negócio, documentação de software de aplicação, processos de negócio,

relatórios, manuais, memorandos, documentação de interface, casos de uso, outras especificações de requisitos, registros de problemas ou questões em aberto (*issues*), políticas, procedimentos e normas como leis, códigos ou ordens *etc.*

7.3.2. Como realizar a análise de documentos

7.3.2.1. Preparação

Inicialmente devem-se determinar as necessidades de informação a serem respondidas por esta técnica. Esses objetivos dependem do momento em que se está no ciclo de vida do projeto. Por exemplo: ➤

Momentos preliminares, quando o principal interesse é entender melhor o domínio do problema e tornar as necessidades de negócio mais específicas: os objetivos estão relacionados a identificar e entender fluxos operacionais e a organização da empresa; transformar necessidades ainda vagas e genéricas em objetivos organizacionais a serem atingidos de forma que o seu sucesso possa ser validado.



Explorar o alcance da solução e dos componentes de software que a compõem: os objetivos estão relacionados a quais tarefas ou, se não houver informação suficiente para isso, quais macroprocessos devem ser explorados em maior profundidade.



Detalhar o escopo buscando maior aprofundamento de uma tarefa em particular: os objetivos estão relacionados à descoberta de quais regras de negócio são aplicáveis, quais dados devem ser informados pelo usuário, quais informações devem ser apresentadas pelo software, quais dados devem ser armazenados e quais devem ser recuperados.

A partir da determinação dessas necessidades de informação, devem ser elaboradas questões que buscam atender a essas necessidades, avaliar quais os documentos mais adequados para a análise e como disponibilizá-los.

Vale lembrar que nem sempre todos os documentos a serem analisados já estão identificados no início do projeto. À medida que o trabalho de elicitação flui, outros documentos podem ser identificados para serem objeto dessa análise.

7.3.2.2. Execução

A análise de documentos consiste então em estudar a documentação selecionada com o objetivo de responder às questões formuladas inicialmente. Durante esse processo, novas dúvidas podem surgir e talvez não sejam respondidas no próprio processo.

Neste caso, tais questões devem ser incorporadas ao controle de questões para que sejam tratadas posteriormente e eventualmente respondidas por outras técnicas de elicitação. É possível também que não se encontrem respostas para todas as questões formuladas inicialmente. Elas também deverão ser mantidas no controle de questões para resolução posterior.

7.3.2.3. Finalização

As respostas às questões iniciais devem ser documentadas em uma memória de levantamento (um relatório que consolida a análise efetuada), que deve ser submetida às partes interessadas especialistas no assunto e com autoridade para confirmar a sua validade. Obrigatoriamente esta memória de levantamento deve citar em suas referências os documentos analisados. Caso as questões iniciais estejam registradas no controle de questões, cabe associar os itens respondidos à memória de levantamento para manter a rastreabilidade.

Em um projeto o volume de documentos analisados pode ser grande. Por exemplo, todo documento que foi utilizado para fundamentar respostas às questões formuladas inicialmente deveria ser guardado no repositório do projeto, pois provavelmente outras consultas serão necessárias. Logo, os documentos coletados devem ser organizados de forma que possam ser recuperados mais facilmente, como na Tabela 7.1. Nela, o tipo indica a classificação para o documento usado na análise, como, por exemplo, um

termo de referência, um manual de procedimentos, um organograma. A localização indica onde o documento pode ser recuperado (preferencialmente em meio digital, como parte da biblioteca do projeto). O nome permite qualificar especificamente a que se refere aquele documento e a memória de levantamento o documento com os objetivos de informação específicos daquela análise e as respostas encontradas com o seu estado atual, indicando se já foram confirmadas ou não.

Tabela 7.1. Resumo da análise de documentos.

Tipo	Localização
Termo de referência	(svn)/projetos/2014/0120/CREASP
Nome	
EDITAL DO PREGÃO ELETRÔNICO 018/2009	
Memória de levantamento	
ML.01 – Resumo da análise do edital e questões desenvolvidas	

7.3.2.4. Vantagens

A principal vantagem da análise de documentos é que não se trata de um trabalho iniciado do zero, já que aproveita material existente para descobrir ou validar requisitos. Por isso é geralmente uma das primeiras técnicas de elicitação a serem aplicadas. Ela também complementa ou ajuda a planejar outras atividades de elicitação com outras técnicas (por exemplo, entrevistas). E mesmo quando uma área de negócio diz que não existe nada documentado para se analisar, as ferramentas de pesquisa da internet já facilitam bastante um nivelamento preliminar do domínio do problema. É também uma abordagem muito útil quando não há um especialista no assunto disponível para entrevistas. Indo além, há situações onde há informação que será encontrada somente nos documentos, pois nenhum indivíduo do público-alvo da elicitação pode conhecê-la. Por fim, informação documentada costuma ser mais objetiva que aquela obtida de pessoas. Não que informação subjetiva seja menos importante, mas fica mais fácil interpretá-la depois que se obtém uma base mais objetiva.

7.3.2.5. Desvantagens

Para processos novos é mais difícil ter documentação disponível. Sem dúvida que pode haver documentação sobre uma visão de futuro; contudo, deve-se tomar cuidado ao considerar se o documento em análise não se limita à perspectiva do estado atual das coisas (*as-is*).

É comum em muitas empresas que os níveis de maturidade da gestão de conhecimento sejam baixos e a documentação existente fique desatualizada ou mesmo não reflita informações válidas. Neste caso, a análise da documentação torna-se pouco útil ou pode introduzir informações errôneas nos requisitos.

Deve-se avaliar também o custo-benefício associado à análise de documentação muito extensa para necessidades pontuais de informação. Pode ser demorado demais ou tedioso. Neste caso, outra técnica de elicitação pode cumprir o objetivo de maneira mais eficiente.

7.3.2.6. Conclusão

A análise de documentos é uma técnica fundamental para ganhar conhecimento sobre o negócio que o software irá tratar e é o caminho natural, um dos primeiros passos, de preparação para atividades de elicitação com outras técnicas.

7.4. Técnica: glossário

O glossário é um importante e reconhecido instrumento para a gestão do conhecimento e um recurso valioso para a produção de software. Além de seu uso como um produto final, o processo para sua elaboração contribui bastante para a elicitação de requisitos. Trataremos da importância do glossário nessas duas dimensões (produto e processo), além de oferecer orientação prática sobre como elaborar e manter um glossário.

7.4.1. Introdução

Segue um pequeno caso ocorrido com um dos autores. Alguns anos atrás meu coordenador me pediu de última hora que participasse de uma reunião sobre o projeto que começava: — O pessoal está lá no segundo andar

discutindo sobre o projeto do CVV2. Queria que você acompanhasse para saber que impacto isso pode ter nos nossos sistemas. A reunião acabou de começar, corre lá.

Entrei na sala discretamente e sentei na cadeira mais próxima da entrada. Estavam presentes umas oito pessoas. A conversa estava adiantada, e o tal do CVV2 era citado a toda hora. Mas eu não tinha ideia do que era aquilo. Sabia o que era CVV, mas o CVV2 não. Tentei entender a conversa por alguns minutos até que percebi que não ia conseguir me situar. Estava com vergonha de interromper a discussão para fazer essa pergunta banal. Então aproveitei que a conversa se concentrou em um canto da mesa e perguntei em voz baixa no pé do ouvido de um colega: — Adriano, o que é CVV2?

No que ele responde, meio sem graça: — Também não sei!

Então ambos rimos e ele tomou a iniciativa de interromper a reunião para fazer a fatídica pergunta: “o que é CVV2?”.

Imagine executar um projeto sem que todos os envolvidos consigam entender o vocabulário de termos novos utilizado. Ou ainda: imagine que os envolvidos tenham entendimentos distintos sobre alguns termos usados. Acha difícil isso ocorrer? Pergunte o que significa “trem” para quem mora em Minas Gerais. Enquanto para todos os demais brasileiros a resposta talvez seja muito parecida com “veículo que anda sobre trilhos”, em Minas Gerais você descobrirá que “trem” significa tudo, inclusive o veículo que anda sobre trilhos!

“Ah, mas esse exemplo não tem nada a ver com a minha realidade”, você pode pensar. Então experimente o termo “processo”. Dentro da mesma empresa você irá perceber diferentes significados, dependendo da área que você abordar. Na área jurídica, na de qualidade, na de TI, na administrativa, todas usam este termo, porém com significados bem distintos.

Logo, problemas à vista! Torna-se fundamental então usar uma ferramenta para compartilhar esse conhecimento sobre os termos e unificar entendimentos. O glossário cumpre muito bem esse papel.

Como produto final, o glossário é algo simples: uma lista alfabética de termos de um determinado domínio de conhecimento com a sua definição.

Ainda que simples, é uma ferramenta poderosa na difusão do conhecimento entre todos envolvidos no projeto.

7.4.2. O que é o glossário

O glossário identifica e define termos-chave para o domínio do problema, capturando o vocabulário comum das partes interessadas. Este livro possui ao final um glossário dos termos da Engenharia de Requisitos e suas definições resumidas. Convidamos você leitor para que o consulte agora, olhe suas definições e perceba quantos termos você não conhecia ou já conhecia, mas com um significado às vezes distinto.

Antes de ser um produto específico da Engenharia de Requisitos, o glossário é um ativo da gerência do conhecimento. A gerência do conhecimento é um processo de negócio que formaliza a gestão e o uso dos ativos intelectuais de uma organização. Ela promove uma abordagem colaborativa e de integração para criação, captura, organização, acesso e uso de ativos de informação, incluindo o conhecimento tácito não capturado das pessoas.

Considerando-se a importância do glossário, seu uso deveria ser obrigatório em todos os projetos e este deveria estar acessível a todos da equipe de desenvolvimento e demais partes interessadas.

7.4.3. Onde encontrar

Muitas áreas de negócio têm os seus próprios glossários, o que é ótimo e pode facilitar muito o trabalho do analista de requisitos. No entanto, ainda que haja um glossário disponível, avalie desenvolver um específico para o projeto, por dois motivos: a)

Potencial de haver diferentes públicos usuários da informação.

b) Seu processo de atualização e respectivos direitos associados.

Indo além do projeto, cada sistema deveria possuir um glossário próprio. Além de complementar, por exemplo, o manual de usuário (caso exista), facilitará também os projetos de manutenção do mesmo sistema. Neste caso, pode não ser necessário criar um glossário para o projeto – usa-se o do

sistema ou aproveitam-se os termos que podem ser necessários para compor o glossário do projeto.

7.4.4. Quando começar

A elaboração do glossário deve ter início com as atividades de elicitação, quando se analisam o domínio do problema e as necessidades de negócio. Quanto mais cedo isso ocorrer, melhor. E deve ser atualizado de maneira contínua, conforme o analista explora os requisitos das partes interessadas e especifica os requisitos da solução.

À medida que as atividades de elicitação e análise progridem, o analista identifica novos termos candidatos a compor o glossário, define os já identificados e valida as definições que elaborou junto às partes interessadas com autoridade para isso. Eventualmente, também ajusta definições já feitas. Para que isso funcione é necessário que o glossário tenha um dono, um responsável por sua manutenção.

7.4.5. Como elaborar um glossário

Para identificar os termos candidatos ao glossário, fique atento aos termos:



- únicos para o domínio;
- com mais de uma definição;
- com definição local distinta do senso comum;
- com potencial de causar dificuldades de entendimento;
- técnicos do negócio;
- do tipo abreviações e siglas;
- que são sinônimos e antônimos.

Fique atento ao *feedback* da equipe do projeto – se ao longo do trabalho alguém tem dúvidas sobre um termo (ou mesmo o desconhece), então isso é uma pista de que este termo deveria compor o glossário (ou ser mais bem explicado, caso já possua uma definição).

A estrutura organizacional de uma empresa, com seus departamentos, diretorias, gerências, quase sempre é referenciada por siglas (GITI,

DEREH, GEFIN, DIMAP, SUPOPE etc.). Logo, as siglas das unidades organizacionais impactadas pelo projeto são boas candidatas ao glossário, considerando um cenário onde desenvolvimento de software é feito externamente. Outro mundo onde siglas são uma constante são os sistemas corporativos (HCH, SBT, GOT, SISREL, DCO etc.), portanto, os sistemas que terão interface com o projeto são também bons candidatos ao glossário.

Termos do senso comum não precisam estar no glossário, sob o risco de sobrecarregá-lo. Atente-se que o ideal é elaborar algo enxuto, prático e fácil de consultar. Não é o objetivo compor um dicionário exaustivo de termos.

O glossário deve ser atualizado conforme se avança no desenvolvimento dos requisitos – inicialmente, a partir das necessidades de negócio e do domínio do problema; em seguida, consolidando a abrangência da solução e explorando os detalhes associados àquele escopo.

Para facilitar o uso, faça a correlação entre os termos usando remissivas simples (“ver”, apenas remetendo ao termo que contém a definição, usado para sinônimos) e remissivas cruzadas (“ver também” remete a outros termos relacionados que podem ser de interesse do usuário).

7.4.6. Importância como produto

O uso do glossário melhora a comunicação interna (entre a equipe) e externa (com o cliente), pois diminui o potencial de divergência na interpretação dos termos usados ao longo do desenvolvimento do projeto e que poderiam causar sérios problemas de comunicação.

Por exemplo, o desenvolvimento das demais atividades de elicitação serão mais produtivas e efetivas quando o analista de requisitos conhece sua terminologia. Não serão necessárias interrupções a todo instante quando se discute determinado tema para explicar um termo já definido no glossário.

Além disso, ele simplifica a elaboração e manutenção de outros documentos de requisitos porque a definição dos termos usados não precisa (nem deveria) estar presente nos demais documentos. Isso torna a especificação de requisitos mais leve e fácil de manter.

7.4.7. Importância como processo

Os benefícios do glossário não estão restritos somente ao produto final. Seu processo de elaboração é importante na descoberta e elicitacão de diversas questões, também ajudando na organizaão das suas respostas. A ideia é que descobrir as perguntas e as pessoas que podem fornecer as respostas muitas vezes é mais importante do que descobrir as respostas para o limitado horizonte de perguntas conhecidas.

O filósofo Ludwig Wittgenstein teorizou que os limites de nossa linguagem são os limites de nosso mundo (MOURA, 2014). Se restrito pelos limites de sua linguagem, o analista de requisitos corre sérios riscos de falhar em sua missão de capturar e entender as necessidades do negócio e as desenvolver em um conjunto de requisitos que ofereçam uma solução para elas.

Por exemplo, quando se estuda um documento com as necessidades de negócio, surgem termos cujo significado preciso foge ao entendimento do analista de requisitos e de qualquer pessoa que não esteja inserida naquele domínio em particular.

Observe o trecho extraído do termo de referência no pregão 046/2010 do CREA-SP: *O escopo do Registro de Responsabilidade de Obras de Médio e Grande Porte deve compreender os seguintes requisitos gerais:*

1. *Obras com Registro Pendente;*
2. *Formulário On-Line de Registro de Responsabilidade;*
3. *Relatórios para Fiscalização;*
4. *Relatórios para Gestão da Fiscalização.*

O leitor pode saber o que significa “registro”, “fiscalização” e “gestão” de maneira geral; mas será que esse significado geral é suficiente para entender plenamente o seu significado no domínio do problema?

Nesse simples trecho surgem pelo menos cinco termos que devem ter os seus significados explorados mais profundamente com o objetivo de obter (e confirmar) um melhor entendimento sobre o domínio do problema. São eles: ➤

Registro de Responsabilidade.



Obras de Médio e Grande Porte.



Obra com Registro Pendente.



Fiscalização.



Gestão da Fiscalização.

Possivelmente, não será na análise do documento que se obterão as respostas sobre o que cada um desses termos especificamente significa. Mas ele será o ponto de partida para identificação de termos candidatos e, com isso, combustível (o que se deseja enfatizar) para outras atividades de elicitação.

Organizar um glossário pode ser visto predominantemente como uma atividade de análise, ainda que análise e elicitação andem sempre juntas. O produto de um tipo de trabalho realimenta o outro, em um processo evolutivo no qual os requisitos vão sendo descobertos (revelados). Por essa importância em gerar pauta para outras atividades de elicitação, destacamos isso como um benefício do processo de elaboração de glossário. Ele sempre deve estar associado à identificação de oportunidades de ampliar a linguagem dos envolvidos no desenvolvimento da solução de forma que não sejam desnecessariamente limitados por ela.

7.4.8. Conclusão

O glossário é um documento importante não só para o projeto de software. Toda organização preocupada com a gerência do conhecimento deveria ter um (ou mais) glossários sobre o seu negócio para permitir, por exemplo, que um novo funcionário tenha mais facilidade para se contextualizar, diminuindo a necessidade de treinamento e acompanhamento. A existência

de um glossário nesse âmbito facilita e muito a elicitação e a análise de requisitos.

7.5. Técnica: observação (etnografia)

Nem todas as partes interessadas conseguem se articular com clareza em uma entrevista sobre o trabalho pelo qual são responsáveis. Isso pode ocorrer mesmo quando o analista atua de maneira proativa e está bem preparado para a entrevista. São limitações de comunicação inerentes a muitas pessoas.

Nessas oportunidades, por que não assistir ao trabalho sendo realizado pela pessoa em vez de perguntar sobre ele? Esse é o objetivo da observação de campo ou etnografia.

7.5.1. O que é observação

A origem desta técnica está na antropologia; ainda assim, ela tem sido cada vez mais usada em áreas fora desse domínio. São áreas onde é reconhecida a importância de obter o entendimento das interações entre pessoas com outras pessoas, instituições, máquinas (e sistemas de software) ou seu ambiente. O entendimento dessas interações requer assimilar o que as pessoas conhecem e como esse conhecimento é criado, transmitido, distribuído e aplicado (BHARWANI, 2006); ou seja, é exatamente o objetivo que se busca ao elicitar os requisitos para uma solução de software.

A observação pode ser usada para entender requisitos sociais e organizacionais. O observador imerge no ambiente de trabalho onde a solução será usada observando o trabalho cotidiano e tomando notas das tarefas em execução nas quais as partes interessadas estão envolvidas.

Ela é então um meio de elicitar requisitos pela condução de uma avaliação do ambiente de trabalho das partes interessadas apropriadas, quando documentando detalhes sobre um processo existente ou quando se propõe melhorar ou modificar um processo atual (IIBA, 2009).

Uma variação desta técnica é a abordagem de aprendiz. Neste caso, o analista de requisitos atua como alguém novato que se integra à equipe que

realiza o trabalho a ser observado e chega com o objetivo de aprender como executá-lo. De acordo com o filósofo Aristóteles: “é fazendo que se aprende a fazer aquilo que se deve aprender a fazer”.

Um ponto positivo dessa variação é que já fica claro para as pessoas que realizam o trabalho que não há conhecimento implícito do “aprendiz”: tudo tem que ser explicado para que se consiga executar o trabalho corretamente.

Um cuidado a se tomar é que a observação de um fenômeno interfere nele. Se a interferência for muito significativa, ainda que os participantes não se manifestem nesse sentido, avalie interrompê-la, seja porque está atrapalhando o trabalho dos observados ou porque o trabalho passou a ser realizado de forma distinta da usual, o que frustra os objetivos da observação.

7.5.2. Como realizar a observação

7.5.2.1. Preparação

Deve-se sempre estabelecer quais são os objetivos da observação. A partir disso, selecionar o grupo de pessoas e a janela de tempo adequada. A seleção é feita com base na compatibilidade entre o tipo de necessidade de informação e as responsabilidades dos grupos em avaliação ou das janelas de tempo disponíveis. Para fins de elicitação de requisitos funcionais, o uso da observação de campo está associado a dois tipos de necessidade de informação: ➤

Escopo: identificar todas as tarefas e serviços existentes do usuário que se pretende informatizar, mas sabendo que o escopo ainda estará aberto a decisões sobre o que será parte da abrangência da solução.



Profundidade: descrever o comportamento da solução pela sua interação com o responsável por uma tarefa e as regras que se aplicam.

Com esses objetivos definidos, avalie a documentação disponível, como o organograma ou as informações sobre o fluxo operacional, para identificar

quem são as pessoas com as responsabilidades e competências necessárias para a observação de seu trabalho. Enfim, deve-se determinar quais: ➤

questões que se buscam responder durante ou após a observação. A experiência de observação em si permite identificar questões que inicialmente nem se percebiam como necessárias; ➤

usuários cujo trabalho deve ser observado, como, por exemplo, especialistas e novatos; ➤

atividades a observar; ➤

momentos para realizar a observação. Há os eventos previsíveis, os aleatórios, os sazonais, os esporádicos. Tente definir um período de observação que seja o mais rico possível em termos de informação a coletar.

Também se deve definir qual tipo de postura o observador assumirá: ➤

Passiva (ou invisível): o analista observa a parte interessada trabalhar, tomando notas, mas sem fazer perguntas ou interferir na rotina. Ele espera até que o processo todo tenha sido concluído antes de fazer qualquer pergunta.



Ativa (ou visível): enquanto o analista observa o processo atual e toma notas, ele pode dialogar com a parte interessada. Quando surge alguma questão (por que algo é feito de determinada forma?), ele pergunta imediatamente, ainda que interrompa a rotina do observado.

O ideal seria observar o processo mais de uma vez, para garantir que se entendeu como ele funciona e por que ele funciona dessa forma.

Toda observação também é uma oportunidade de melhoria do processo observado; esteja atento a isso. Se sua preocupação for apenas mapear um processo de trabalho para levar isso para sua especificação, corre-se o risco de o software automatizar um processo ineficiente ou defeituoso. Neste caso, o projeto que deveria entregar uma solução pode acabar entregando um meio mais eficiente de fazer as coisas erradas.

7.5.2.2. Execução

Para a execução da observação, o analista: ➤

- apresenta-se às pessoas que serão observadas; ➤
- assegura a essas pessoas que seu trabalho não será criticado. Esclarece que as informações resultantes servirão como insumo para a análise dos requisitos. A ideia é obter aceitação do observado; ➤
- comunica que sua presença é apenas para estudar seus processos e que evitará discutir soluções para os problemas que surgirem; ➤
- pode sugerir que os observados “pensem alto” enquanto trabalham, como uma forma de compartilharem suas intenções, desafios e preocupações.

Feito isso, dá-se o início à observação e ao registro de notas detalhadas. Se a abordagem de observação ativa é a escolhida, então deve-se perguntar questões exploratórias sobre o motivo pelo qual certos processos e tarefas são executados da maneira observada.

7.5.2.3. Finalização

Obtenha as respostas para as questões originalmente formuladas ou para aquelas que surgiram ao longo da observação. Elabore uma memória de levantamento documentando os achados em forma de notas e as forneça aos participantes assim que possível, para revisão e qualquer esclarecimento.

Quando observar várias pessoas, compile as notas em intervalos regulares para identificar pontos de vista comuns e diferentes entre elas. Reveja os achados com todo o grupo para garantir que os detalhes finais sejam representativos do todo.

7.5.3. Vantagens

A observação pode ser a melhor solução para obter uma visão prática e realista do negócio. Ela permite identificar fluxos de informações informais e a maneira como as pessoas realmente trabalham. E isso é muito importante, pois ajuda a descobrir requisitos implícitos (SOMMERVILLE, 2006).

Algumas pessoas têm dificuldade para se expressar, o que muitas vezes torna a entrevista infrutífera. Ou também há pessoas que não possuem a disponibilidade de tempo adequada para entrevistas. Nesses casos a observação pode ser uma alternativa interessante.

Quando conjugada com outras técnicas de elicitación, permite confirmar, complementar e confrontar os dados coletados, além de também planejar atividades de elicitación dessas outras técnicas.

É também uma ótima opção para identificar requisitos de usabilidade (BIAS, 2005). Ao perceber como as tarefas são executadas, a intensidade em que ocorrem, quais passos são mais demorados ou rápidos, fica mais fácil propor uma solução com uma interface mais amigável à realidade de trabalho das partes interessadas.

Outro ponto importante é que, ao empregar a observação, se demonstra mais atenção ao problema do cliente, pois o analista de requisitos se coloca no lugar do outro. O cliente perceberá que membros da equipe do projeto estão observando *in loco* os problemas enfrentados, qual a importância e criticidade do trabalho realizado e qual solução se adequaria melhor a essa realidade.

7.5.4. Desvantagens

O uso da observação se restringe a processos existentes. Quando o projeto implica implementar um novo processo de negócio ou produto inteiramente novo, não há muito o que observar.

Para trabalhos que envolvem alto nível de atividade intelectual, a observação se revela mais difícil. São processos que não são facilmente observáveis. Ainda que se decida utilizar a observação nesses casos, a abordagem deveria ser da observação ativa, o que pode acabar resultando em quase uma entrevista.

Comparada a outras técnicas de elicitación, é onerosa. Por exemplo, enquanto uma entrevista pode ser executada em uma ou duas horas e revelar muitas informações úteis, nesse mesmo tempo a observação não produzirá o mesmo volume de informação.

Como a observação opera em uma determinada janela de tempo, situações críticas e exceções podem não acontecer durante as sessões de observação. Por exemplo, a devolução de um produto pode ocorrer esporadicamente e em uma frequência baixa. É um evento aleatório e somente a sorte pode fazer com que ele surja durante a sessão de observação. Ou seja, para observar de forma completa, é necessário um tempo considerável, que geralmente não está disponível no projeto.

Por fim, nem todos se sentem confortáveis sendo observados, pode ser constrangedor. Por isso é importante que, ao abrir a sessão de observação, uma atenção especial seja dada à comunicação adequada ao grupo sobre o que será feito, como será feito e quais os objetivos. Ao comunicar-se de forma clara com o grupo, a resistência à observação pode ser diminuída. No entanto, se o constrangimento continuar a persistir, outra abordagem deverá ser buscada.

7.5.5. Conclusão

Apesar de ser onerosa em termos de tempo, é a técnica que consegue as informações mais fidedignas do contexto do negócio. Mesmo que o projeto não tenha disponibilidade de tempo para um uso mais intenso desta técnica, recomenda-se ao menos que se incluam algumas sessões de observação nos processos de negócio mais críticos afetados pelo projeto.

7.6. Técnica: entrevista

Um dos autores teve sua primeira experiência em conduzir uma entrevista ainda na faculdade. Ele entrou em um projeto de pesquisa em um grupo com mais dois colegas. O trabalho envolvia uma etapa de diagnóstico em uma empresa e os alunos seriam os responsáveis pelo levantamento das informações, que seria feito com entrevistas com diversas pessoas da empresa. Nunca haviam feito uma entrevista antes, o que gerou preocupação sobre como se prepararem para fazer o trabalho adequadamente. Então, foram pesquisar algum material que pudesse ajudar. Não havia Google na época, porém descobriram um livro que ajudou muito (KENDALL, 1992). Neste livro, a entrevista era uma seção de um capítulo,

com uma abordagem bem didática e várias dicas práticas (que serão abordadas também a seguir).

Uma dica que marcou muito era mais ou menos assim: “quando o entrevistado começar a falar, não abaixe a cabeça e comece a anotar tudo furiosamente”. A intenção da dica era preservar o contato visual com o entrevistado. Pois bem, depois de terem estudado o assunto, definiram que iam os três juntos para as entrevistas (que intimidador deve ter sido para o entrevistado!), cada um deles com uma cópia da pauta das perguntas. Enquanto um perguntasse, os outros tomariam as notas. Também se revezariam na função de perguntar. E o que aconteceu na primeira pergunta da primeira entrevista? Quando o entrevistado iniciou a resposta, os três abaixaram a cabeça ao mesmo tempo para tentar anotar tudo o que a pessoa começou a falar!

Apesar da inexperiência, o trabalho até que foi bem conduzido. Realizaram diversas entrevistas e foram aprendendo pouco a pouco o que deveriam melhorar na próxima. Algo que conseguiram fazer bem, e que é fundamental para a eficácia da entrevista, é a preparação do roteiro de perguntas. Descobriram que o uso do gravador, embora os liberasse da preocupação das anotações, dava um trabalho enorme depois para gerar a ata. Ao ouvir a gravação depois para produzir a ata, perceberam que o diálogo é muitas vezes ilógico e desconexo, porém nem sempre se percebe isso quando se está no meio da conversa! Um exemplo: uma pergunta era sobre a frota da empresa, e a resposta que a pessoa deu foi mais ou menos assim: “(...) temos também uma Kombi, que tem capacidade de carga de 1.000 kg. Só que está meio velhinha, então carregamos nela apenas 1.500 kg”.

Nenhum dos entrevistadores percebeu essa incongruência na resposta durante a entrevista; talvez porque estivessem preocupados demais anotando...

7.6.1. O que é a entrevista

É uma forma de diálogo, formal ou informal, entre duas ou mais pessoas, onde o entrevistador busca respostas para um conjunto de questões previamente planejadas e os entrevistados se apresentam como fontes de

informação. Um grande desafio para o entrevistador é desenvolver um ambiente de confiança e sintonia com o entrevistado, para que as informações consigam fluir bem.

O termo “entrevista” tem um sentido muito amplo, mas o enfoque aqui tem um sentido bem distinto de entrevistas que ocorrem em programas de entretenimento na TV. O objetivo do programa é entreter, e quase sempre o entrevistador assume o papel de protagonista, deixando o entrevistado em segundo plano. Se o resultado foi divertido, então o objetivo foi cumprido.

Para os projetos de software o objetivo é distinto: o entrevistado deve ter o papel principal e o entrevistador ser um mero facilitador da conversa, para que ela flua no sentido de responder às questões planejadas. Porém, não imagine por isso que o trabalho do entrevistador seja fácil. Ele deve ter uma habilidade de comunicação interpessoal muito bem desenvolvida para que consiga lidar com as distintas personalidades dos entrevistados e fazer a conversa ser fluida e objetiva.

Em geral, as pessoas entrevistadas são cooperativas e têm interesse no sucesso do projeto, pois irão usufruir seus benefícios. Portanto, o entrevistador quase sempre começa a entrevista com o jogo a seu favor. E deve trabalhar para deixar sempre esse ambiente favorável. Mas tem muita gente que desperdiça essa valiosa vantagem inicial.

7.6.2. A primeira impressão é a que fica

Quando o projeto envolve partes interessadas já conhecidas, ou com as quais já houve interação em outros projetos, é mais fácil se preparar para a postura que deve ser adotada nas entrevistas. Também se a experiência foi positiva, certamente as próximas entrevistas serão mais fáceis de conduzir.

No entanto, é frequente participar de projetos com partes interessadas que são pessoas desconhecidas. Neste caso, há que se ter muito cuidado no primeiro contato, para que uma gafe (uma fala mal colocada, um comentário inapropriado) não cause má impressão e jogue fora o espírito colaborativo com que o entrevistado chegou.

Este momento em que duas pessoas estranhas se encontram para um diálogo tem muita semelhança com uma paquera. O que funciona para uma

normalmente funciona para outra. E o que atrapalha em uma também costuma atrapalhar a outra. A finalidade de ambas é parecida: estabelecer rapidamente uma sintonia com o outro. O que vem depois disso, daí já é bem diferente.

Nos treinamentos promovidos pelos autores costumamos fazer uma reflexão com os participantes sobre os fatores que causam inibição no entrevistado, com base nas nossas experiências pessoais (seja de paquera ou de entrevista). Vamos à lista de alguns desses itens (sem ordem específica):



Julgar/criticar as informações emitidas pelo outro.

☐

Completar as frases do outro ou cortar sua fala.

☐

Ser arrogante ou deixar a impressão de que sabe mais do que o outro sobre o seu tema.

☐

Corrigir o outro, seja em alguma informação ou em sua gramática.

☐

Não demonstrar interesse nas informações recebidas, no outro ou no seu problema.

☐

Dar a solução antes de ouvir o problema.

☐

Falta de cortesia, simpatia, contato visual ou pontualidade.



Local, hora ou duração inadequada.



Cometer uma gafe (piada ou comentário inapropriado).



Errar na apresentação pessoal (formal/informal, higiene pessoal).



Linguajar inadequado (muito técnico, muito formal ou informal, uso de gírias/palavrões).



Demonstrar despreparo sobre o assunto a ser abordado.

É quase impossível esgotar essa lista, pois a capacidade do ser humano de cometer equívocos é quase ilimitada. O entrevistador deve ter muito tato ao lidar com pessoas, para não criar indevidamente barreiras com o entrevistado.

7.6.3. Diretrizes para o entrevistador

A seguir apresentam-se algumas diretrizes práticas para apoiar o entrevistador na condução da entrevista. São elas: Seja um bom ouvinte.

Vá de coração aberto; livre-se de preconceitos.

Busque fatos, mas também opiniões.

7.6.3.1. Seja um bom ouvinte

O entrevistado deveria falar mais que o entrevistador. Parece trivial, mas o inverso ocorre muito. Se você fala mais que o entrevistado, significa que está colocando barreiras para a informação fluir a partir dele. Desenvolva

sua escuta ativa. Isso significa demonstrar disponibilidade e interesse pelo que o outro fala. Veja algumas dicas: ➤

Demonstrar ao outro que você o escuta, por exemplo: contato visual, pequenos gestos de mão ou cabeça, postura que reflita interesse, expressões como “aham”, “ok”, “claro”, “entendi”.



Pedir que explique aquilo que não compreendeu.



Resumir: “(...) se entendi o que você disse, o processo então começa assim e termina assim, certo?”.



Interpretar o comportamento verbal e não verbal do outro.



Desenvolver uma sintonia (*rapport*) com o outro.



Gerar um clima emocional acolhedor.



Respeitar os silêncios naturais de uma conversa, ser paciente e respeitar o tempo do outro.

7.6.3.2. Vá de coração aberto; livre-se de preconceitos

A missão da entrevista é colher informações que ajudem no projeto e não necessariamente confirmar a opinião do entrevistador. O pior pecado é o entrevistador ir para a entrevista achando que já sabe as respostas. Invariavelmente, a entrevista termina confirmando o que ele gostaria de

confirmar, mas muitas vezes não porque as informações estejam corretas, e sim porque o entrevistador direcionou (talvez de forma inconsciente) o entrevistado a responder o que lhe convinha.

Despido de preconceitos, consegue-se compreender melhor a informação que vem do outro, e muitas vezes isso leva a rever pensamentos já consolidados. O preconceito direciona a atenção aos dados que corroboram com o seu pensamento e desviam a atenção do que vai contra ele. Muitas vezes isso também o induz a formular perguntas de forma tendenciosa, o que bloqueia uma resposta que a princípio seria de seu desagrado.

7.6.3.3. Busque os fatos, mas também opiniões

As pessoas gostam de participar da solução de um problema. Uma opinião pode revelar o problema principal. Só que muitas vezes o entrevistador despreza opiniões e dá atenção somente a fatos. E muitas vezes alguém já pensou a solução e só espera a oportunidade de compartilhá-la. Aqueles que sofrem os problemas que motivam o projeto certamente devem ter ideias interessantes para a solução que está sendo elaborada.

Vejamos um exemplo. O entrevistador conversa com alguém da área de fiscalização de obras tentando entender o processo de fiscalização: o que é, por que é necessário, quem está envolvido, como acontece, quantas obras são fiscalizadas no mês, qual o tempo médio da fiscalização. Enfim, ele pode abordar várias questões sobre fatos da fiscalização de obras. No entanto, se der abertura para o entrevistado manifestar sua opinião, há chance de descobrir coisas mais importantes, como: “bom, já que pediu minha opinião, eu acho que este processo nem precisaria existir. Já existe outra equipe que colhe os mesmos dados antes de nós”.

7.6.4. Como realizar a entrevista

7.6.4.1. Preparação

O primeiro passo é definir claramente o objetivo da entrevista e quais informações precisam ser buscadas. Em seguida, identificar quem são as pessoas adequadas para serem entrevistadas. Este ponto é muito crítico, pois se você seleciona a pessoa errada, a entrevista está perdida.

Muitas vezes a definição sobre a pessoa a ser entrevistada não é tomada pelo entrevistador. Por exemplo, seu objetivo é entender um processo de negócio de um departamento. Como o chefe do departamento é muito ocupado, ele designa outra pessoa para ser entrevistada. Durante o procedimento, você percebe que ela não é a pessoa ideal, pois em vários momentos as respostas são como: — Ah, isso eu não sei.

— Sobre este tema, melhor falar com fulano.

— Quem sabe mais disso é sicrano.

Para evitar situações assim, busque participar das decisões de designação de pessoas para as entrevistas. O gerente do seu projeto deve ajudá-lo nisso.

Curiosamente, uma das dificuldades com requisitos frequentemente relatadas por participantes dos treinamentos realizados pelos autores é a designação de pessoas com conhecimento insuficiente do tema para a participação de entrevistas.

Avalie entrevistar pessoas nos vários níveis organizacionais que serão afetados pelo projeto. A visão que cada um tem do mesmo processo de negócio pode ser distinta, e isso pode revelar informações importantes a tratar nos requisitos.

Tentar entrevistar várias pessoas de uma vez pode ser improdutivo. Uma única pessoa pode assumir a fala do grupo, ou a presença de uma pode inibir opiniões de outra. Por exemplo, um subordinado pode ficar constrangido em discordar ou responder algo na presença do chefe.

Estude bem o assunto a ser abordado. Analise a documentação existente, converse com outros membros da equipe que possam ter mais conhecimento que você no tema, pesquise. Uma pergunta idiota pode tirar toda a sua credibilidade perante o entrevistado. Depois será difícil conseguir sua cooperação em outras entrevistas. Assim como uma pergunta inteligente pode levar você a ganhar a admiração do entrevistado.

Ao estudar o assunto da entrevista, familiarize-se com o vocabulário do negócio e use-o na entrevista. Isso o ajudará a obter mais rapidamente uma sintonia com o entrevistado, afinal você está falando o seu idioma. O glossário é muito útil nesse sentido.

Defina a duração da entrevista. Às vezes isso não está sob seu controle, talvez seja o entrevistado que defina quanto tempo terá de disponibilidade. Neste caso, você adequa a entrevista para priorizar as questões e otimizar o rendimento da entrevista nesse tempo restrito. Caso você tenha liberdade de definir a duração da entrevista, evite que ela se estenda por mais que duas horas. Torna-se cansativo para ambos, o nível de atenção decresce. Se perceber que há muitas questões a tratar e que precisará de muito tempo, divida a entrevista em várias sessões de menor duração.

Muita atenção na definição do local da entrevista. Se for o local do entrevistado, avalie antes se há possibilidade de muitas interrupções e se a conversa transcorrerá em ambiente reservado ou compartilhado. Se a entrevista for realizada por videoconferência, certifique-se de que toda a infraestrutura estará pronta e disponível de ambos os lados no horário marcado.

Por fim, certifique-se de que a pessoa seja avisada da entrevista, do seu objetivo e dos assuntos a serem abordados, e também de qualquer outra informação que ela precise providenciar para levar à entrevista.

7.6.4.2. Preparação do roteiro de questões

O roteiro de questões é fundamental para conseguir melhor resultado na entrevista. Não necessariamente o roteiro irá ditar a sequência da entrevista – isso depende do formato selecionado. Mesmo que a entrevista seja conduzida em ordem diversa do roteiro, este ainda é útil para que, no fechamento, o entrevistador consiga verificar se tudo que deveria ser tratado foi feito.

Tão importante quanto orientar o entrevistado, a elaboração do roteiro é também uma forma de o entrevistador se preparar. Pensar na seleção ideal de perguntas, imaginar possíveis respostas; algumas respostas podem precisar de perguntas adicionais, tentar antecipá-las *etc.*

Algumas perguntas que farão parte do roteiro já são de conhecimento prévio do entrevistador. São questões que surgiram em outros momentos, talvez em outras entrevistas com outras pessoas, e que nesta entrevista serão mais bem tratadas. Neste caso, o **controle de questões** é uma ferramenta valiosa na elaboração do roteiro. O entrevistador pode avaliar todas as

questões que já foram detectadas e que ainda devem ser respondidas. Algumas delas podem ser ideais para abordar na entrevista que está sendo preparada.

A técnica do 5W + 2H ajuda a não esquecer nenhum aspecto relevante do assunto tratado. Seu nome deriva do acrônimo das perguntas em inglês abordadas: *what?* (o quê, de quê, para quê?), *who?* (quem, com quem, de quem?), *when?* (quando?), *where?* (onde, de onde, para onde?), *why?* (por quê?), *how?* (como?) e *how much?* (quanto?). Então, imagine que o interesse seja conhecer melhor um processo de negócio – por exemplo, reembolso de despesas de viagem. Se na entrevista conseguimos obter resposta a todas as dimensões do assunto abordadas pelo 5W + 2H, então dificilmente algo relevante ficou omissso.

Evite questões longas ou complexas. Em uma pergunta longa é provável que o entrevistado só responda a uma parte dela (ele não irá se lembrar da pergunta toda). Em uma pergunta complexa, ele talvez só responda a parte que conseguiu entender. Ou responda algo para um entendimento errado da pergunta. Logo, colocar uma questão como a seguinte é contraproducente: “Por que isso é assim, desde quando isso acontece, quem determinou a mudança e que prejuízos são causados?”. Melhor fracioná-la e simplificar.

Ainda que uma questão já tenha sido abordada em uma entrevista, pode ser interessante incluí-la para outro entrevistado. Respostas divergentes ajudam a antecipar a identificação de conflitos ou erros na informação dada pelo entrevistado. Por exemplo: nem sempre o que o chefe diz sobre um processo é o que de fato ocorre quando os subordinados o executam.

7.6.4.2.1. Tipos de questões As perguntas podem ser classificadas em dois tipos: abertas e fechadas.

As **perguntas abertas** não possuem respostas “certas”. A resposta é livre. São ideais para coletar opiniões ou em situações exploratórias. Exemplos: como funciona o processo de reembolso de despesas? Na sua visão, quais são os problemas deste processo?

Entre suas vantagens estão:



São mais interessantes para o entrevistado e ele fica mais à vontade.



Permitem que o entrevistador se adapte ao vocabulário do entrevistado.



Fornecem mais riqueza de detalhes.



Permitem maior espontaneidade do entrevistado.

Como desvantagens:



Algumas respostas podem resultar em muitos detalhes desnecessários.



Possibilidade de perda do controle da entrevista. O entrevistado pode falar muito e fazer desvios para assuntos fora do interesse.



Podem dar a impressão de que o entrevistador não está preparado.



Podem dificultar a comparação e interpretação de resultados.



Consumem mais tempo.

Já as **questões fechadas** induzem a respostas curtas e diretas. Têm respostas “fáceis”. Não há muito espaço para o entrevistado ficar divagando. Exemplos: quantas solicitações de reembolso de despesas ocorrem no mês? Qual o tempo médio gasto no processo todo? Quem é o responsável pelas aprovações?

Suas vantagens são:



Economiza tempo.



Permite comparar mais facilmente as respostas das entrevistas.



Facilita ao entrevistador manter o controle da entrevista.

As desvantagens da questão fechada são: ➤

Pode ser chata para o entrevistado.



Perde-se riqueza de detalhes na resposta.

7.6.4.3. Formato da entrevista

A entrevista pode ter um caráter formal ou informal. A entrevista informal pode ocorrer quando o entrevistador e o entrevistado possuem mais intimidade e o acesso ao entrevistado é mais fácil. As entrevistas podem acontecer sem a necessidade de todo um ritual prévio de contato, podendo ser em locais e horários mais flexíveis. É o caso do desenvolvimento interno do projeto, onde a equipe do projeto e os clientes são colegas de trabalho, muitas vezes trabalhando no mesmo local e já velhos conhecidos de outros projetos.

Além disso, a entrevista pode ser planejada para ocorrer de forma estruturada ou não. Na não estruturada não há uma definição da ordem em

que as questões serão abordadas. Conforme o desenrolar da entrevista, caminhos possíveis são avaliados e a sequência vai sendo estabelecida. O roteiro é um apoio para o entrevistador, mas não define a ordem das perguntas. Neste caso, a entrevista torna-se um diálogo mais natural. Porém, requer mais experiência do entrevistador para que seja bem conduzida e não se perca o controle. Vale ressaltar que, ainda que a sequência das questões não seja definida *a priori*, as questões devem ser definidas antecipadamente. Ou seja, planejamento é necessário. Não estruturada não quer dizer não planejada!

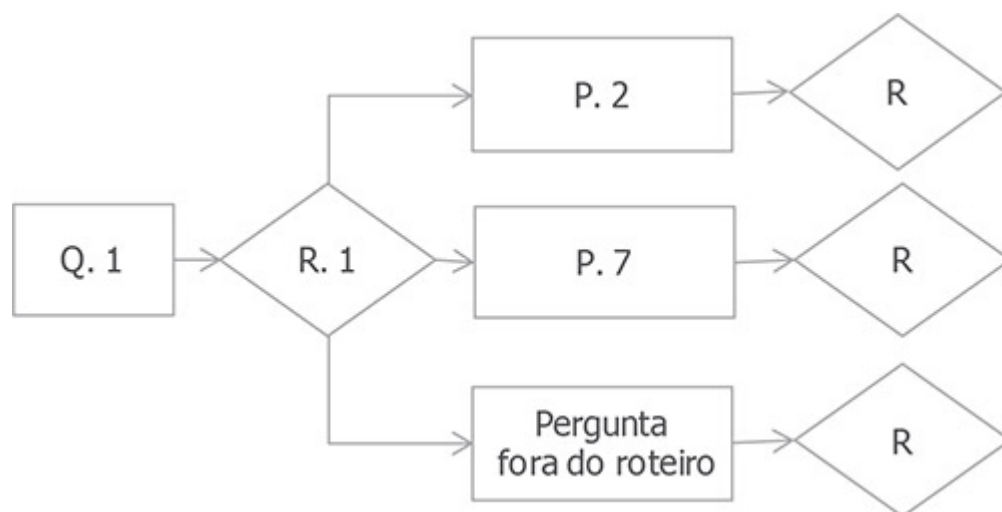


Figura 7.2. Nas entrevistas não estruturadas, o roteiro é apenas um guia para o entrevistador, mas não o obriga a seguir uma sequência prévia de perguntas.

Já as entrevistas estruturadas são conduzidas na sequência planejada do roteiro. É mais indicada quando o entrevistador não é muito experiente ou quando se deseja garantir que as mesmas perguntas serão feitas para todos os entrevistados.



Figura 7.3. Na entrevista estruturada, o roteiro define a sequência de perguntas ao longo da entrevista.

Existem três tipos básicos de estruturas para uma entrevista (vide Figura 7.4): ➤

Pirâmide: inicia com questões fechadas. À medida que a entrevista progride, questões abertas são colocadas. Útil para situações onde o entrevistado parece relutante em abordar um determinado assunto, para “aquecer” o entrevistado ou “quebrar o gelo”.



Funil: inicia com questões abertas. À medida que a entrevista avança, perguntas fechadas são feitas. Oferece um meio não ameaçador de começar a entrevista. Útil para situações em que o entrevistado precisa “desabafar” logo ou deseja ir direto ao ponto.



Diamante: combinação das anteriores. Inicia com questões fechadas, passa a questões abertas e fecha a entrevista novamente com questões fechadas. Em geral é a melhor forma de estruturar a entrevista. Contudo, tende a ser mais longa.

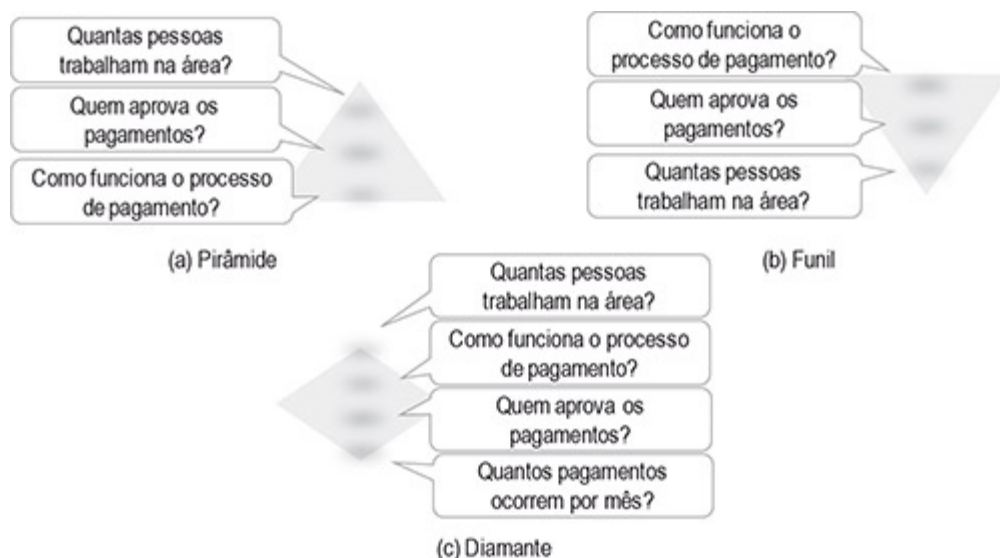


Figura 7.4. Três tipos básicos de estruturas para entrevista.

7.6.4.4. Forma de registro

Durante a entrevista, aspectos importantes devem ser registrados para que não se percam. Esse registro é importante não só para evitar esquecimento do entrevistador sobre o que foi conversado, como também para compartilhar o conhecimento adquirido na entrevista com o resto da equipe. A decisão de qual ferramenta usar para registro depende em parte do entrevistado e de que uso será feito desses dados após a entrevista.

Papel e caneta (ou lápis) costumam ser o ferramental mais eficaz e de melhor custo--benefício. O custo é irrisório, além de não travar nem acabar a bateria, como pode ocorrer se você for usar computador ou o *tablet* para as anotações. Um porém dessa abordagem é que causa interrupção do contato visual e ritmo do diálogo com o entrevistado. Mesmo que você tenha habilidade de escrever sem olhar para o papel, mantendo o olhar no entrevistado, a sua atenção à conversa não é a mesma, nem a velocidade de registro será tão veloz como a fala do outro. Isso pode ser solucionado com o apoio de um secretário, alguém diferente do entrevistador com a responsabilidade de tomar notas, liberando o entrevistador para deixar a conversa fluir melhor com o entrevistado.

O uso de um gravador ou câmera tem a vantagem de não causar a interrupção no diálogo e no contato visual, além de oferecer um registro fiel e completo do que o entrevistado disse. A gravação pode ser útil também para que outro membro da equipe possa se preparar para entrevistar a mesma pessoa posteriormente, se necessário.

No entanto, o uso de gravador ou câmera para registro da entrevista pode deixar o entrevistado desconfortável ou inibido. A decisão de usar essa ferramenta deve ter a concordância do entrevistado, e alguns optam para que não se use. Outro risco é o entrevistador ficar acomodado e menos atento às respostas. Neste caso, ele perderá a oportunidade de explorar questões novas imediatamente quando surgirem. Do contrário, só perceberá isso ao ouvir a gravação, mas não terá mais a pessoa à sua frente para dirimir imediatamente a dúvida.

Por fim, basear o registro somente na gravação irá tornar onerosa a geração da ata da entrevista. Por exemplo, se a entrevista durou uma hora, você

precisará de um tempo maior que esse para gerar a ata, pois ouvirá por partes, documentando à medida que a ouve.

Assumindo que a pessoa autorizou a gravação, torna-se desnecessário tomar notas? Claro que não! O gravador não registra tudo. Enquanto a pessoa fala, você vai processando na mente aquelas informações, estabelecendo relações com outras informações, lembrando-se de questões que devem ser abordadas com outras pessoas, enfim, sua mente borbulha. E o gravador não registra isso. Diversas questões que passam pela sua cabeça durante a entrevista precisam ser anotadas para que não sejam esquecidas, e não necessariamente serão faladas.

O ideal então é conseguir tomar notas dos pontos importantes ao longo da conversa, o que agiliza a produção da ata. A gravação pode ser usada então mais como cópia de segurança, para ouvir pontualmente a parte da entrevista onde você perdeu detalhes ou ficou confuso.

7.6.5. Execução

Ao executar a entrevista, seja pontual tanto para início quanto para fechamento – recomendação que deveria ser banal, mas que na cultura latina não costuma ser valorizada. O escritor Millôr Fernandes resumiu isso bem: “pontual é alguém que decidiu esperar muito”.

Para abrir a entrevista, não assuma que o outro já deve saber quem você é e qual seu objetivo. Portanto, apresente-se, explique como será a condução da sessão, sua duração, finalidade e o que será feito com as informações coletadas. Se você fez uma boa preparação, o entrevistado já deveria ter recebido essas informações no convite da entrevista. Mas mesmo assim custa pouco repassar isso na abertura.

Durante a entrevista:



escute mais que pergunte; ➤
mantenha contato visual e atenção à linguagem corporal do entrevistado e também à sua própria. A mensagem não verbal pode contradizer a fala do entrevistado; ➤

mantenha o foco nos objetivos e nas questões predefinidas. O roteiro ajuda nisso; ➤
registre as questões que surgirem e que não foram tratadas. Adicione-as depois ao **controle de questões**; ➤
verifique se a questão foi respondida de maneira completa; ➤
confirme o que foi entendido a partir das informações fornecidas.

Para o fechamento da entrevista: ➤

repasse o roteiro para verificar se todos os pontos foram tratados; ➤
verifique com o entrevistado se algum assunto relevante não foi abordado e se sabe de alguém mais que possa contribuir com o que foi discutido. É uma pergunta de rotina. Se você fez uma boa preparação, a resposta quase sempre será que ele não tem nada a acrescentar. No entanto, esta pode ser a oportunidade que o entrevistado estava esperando para abordar algo relevante e que não foi percebido no planejamento; ➤
resuma a sessão; ➤
lembre-o do próximo encontro, se houver, já dando orientações do que será tratado; ➤
agradeça ao entrevistado.

7.6.6. Finalização

Após o encerramento da entrevista, procure resumi-la o quanto antes, em uma ata ou relatório, preferencialmente imediatamente após o seu fechamento. Este é o documento que será usado para compartilhar a informação obtida na entrevista com outros da equipe e também para validar com a parte interessada se houve algum equívoco no entendimento das informações passadas. Somente a gravação do áudio não cumpre esse papel.

A demora reduz a qualidade dos dados documentados. Lembre-se de que sua mente estava borbulhando de ideias durante a entrevista. Essas ideias vão se perdendo se não forem documentadas.

Para agilizar a documentação, deixe previamente preparado um modelo da ata ou do relatório a ser produzido, de tal forma que seja simples passar a

limpo as notas.

Organize as informações de forma lógica; não necessariamente na ordem que elas surgiram na entrevista. Concentre-se nos apontamentos relevantes ao projeto. A documentação não é uma transcrição literal da conversa.

Após finalizar o documento, envie-o ao entrevistado para revisão e obtenção do seu “de acordo”.

7.6.6.1. Vantagens



Incentiva a estabelecer uma relação mais próxima com as partes interessadas.



Permite a observação do comportamento não verbal (quando presencial).



Permite confirmar de imediato a compreensão da informação.



Permite expressar opiniões de forma privada.



Novas questões podem ser tratadas imediatamente quando surgirem.



Permite discussões amplas e explicações sobre as perguntas e respostas (vantagem sobre questionário/pesquisa).



Se o entrevistador for astuto, permite a descoberta de requisitos que estão no subconsciente do entrevistado.

7.6.6.2. Desvantagens

□

Não é uma ferramenta ideal para se chegar ao consenso sobre requisitos.

□

Pode ser oneroso aplicá-la a um conjunto grande de partes interessadas.

□

Compromisso considerável de tempo e envolvimento dos participantes.

□

As entrevistas não estruturadas exigem um entrevistador mais experiente.

□

Há risco de inadvertidamente conduzir o entrevistado (método Faustão).

7.6.7. Conclusão

Pela sua simplicidade e praticidade, entrevista é uma das técnicas de elicitación mais utilizadas. O maior cuidado é o entrevistador conseguir prepará-la bem e manter o foco durante a sessão; caso contrário, a entrevista torna-se mais uma reunião. E reuniões costumam ser o maior foco de desperdício de tempo nas empresas.

7.7. Técnica: pesquisa/questionário

Creio que você, leitor, já deva ter participado, ou pelo menos tenha sido convidado a participar, de alguma pesquisa de satisfação com alguma empresa da qual tenha consumido produtos ou serviços. Pode ser um hotel no qual tenha se hospedado, uma companhia aérea pela qual tenha viajado, um banco no qual possui uma conta. Enfim, seria difícil enumerar todas as empresas que usam esta técnica.

Basicamente, essas pesquisas de satisfação envolvem responder a um questionário de avaliação do produto ou serviço que você comprou. Às vezes esse questionário é fornecido em papel. Mas o mais frequente talvez seja uma página na internet para você responder de forma *on-line*.

O objetivo é colher informações da sua experiência de consumo com a empresa. Isso poderia ser feito através de uma entrevista de alguém da empresa com você. Mas imagine o custo disso para uma empresa que realiza milhares de transações por dia com muitas pessoas distintas. Seria proibitivo. A entrevista poderia até ser mais eficaz na obtenção dessas informações, mas o custo-benefício de aplicá-la a todos os clientes em geral não é vantajoso.

Embora a pesquisa seja comum em várias empresas, pergunto: quantas vezes você ignorou tais questionários? Caso tenha tido alguma insatisfação com a empresa, é bem provável que tenha aproveitado a oportunidade para registrar sua reclamação. Mas se tudo ocorreu de forma satisfatória, seu ímpeto de respondê-la talvez seja menor. Bom, ao menos costuma ser assim com a maioria.

Este é um ponto interessante; se a maioria dos clientes não responde à pesquisa, seu objetivo de obter um *feedback* acaba ficando prejudicado. Ciente disso, algumas empresas criam estímulos para que o cliente responda: fornecem um brinde, dão direito a concorrer a um sorteio, dão pontos do programa de fidelidade *etc.* Isso eleva bastante o interesse em participar da pesquisa.

Certa vez um dos autores foi convidado por uma empresa para participar de uma pesquisa, e o “brinde” para estimular a participação era concorrer a um fim de semana em um *resort* badalado. A primeira página do formulário

solicitava os dados de qualificação como cliente, o que foi preenchido sem dificuldade e rapidamente seguiu-se para a próxima página. No entanto, surge de surpresa logo em seguida uma mensagem mais ou menos assim: “infelizmente seu perfil não se encaixa no público-alvo desta pesquisa” e ela foi abortada de imediato. Isso gerou certa irritação – afinal, se o perfil não estava no foco da pesquisa, por que então o convite? Claramente não houve um cuidado na seleção do público-alvo para o qual a pesquisa foi distribuída.

Em outra oportunidade, um dos autores participou de uma pesquisa, só que dessa vez para um trabalho de mestrado. Havia uma determinada pergunta do questionário com opções de múltipla escolha, na qual nenhuma delas se encaixava em sua realidade (nem mesmo aproximadamente), e não havia uma opção “outros” para marcar. Como a pergunta era obrigatória, foi respondida qualquer coisa para seguir adiante. Assim como ocorreu com um dos autores, é possível que isso tenha ocorrido com outros tantos que também participaram da pesquisa. Então já dá para imaginar que a pesquisa feita sobre essas respostas não teria muita validade.

7.7.1. O que é a pesquisa/questionário

A pesquisa consiste na aplicação de um questionário às partes interessadas e posterior análise das respostas. Difere da entrevista porque não há interação com os respondentes durante a resposta. Porém, tanto ajuda para complementar quanto para preparar uma entrevista.

É uma técnica que permite a rápida obtenção de informações quantitativas e qualitativas de um público-alvo numeroso. Quando aplicada a uma amostra representativa do público, permite representar as opiniões de toda a população.

É também interessante de se aplicar quando as partes interessadas não estão em um único local físico. Imagine um projeto de escala global, envolvendo pessoas de diversos países. Realizar entrevistas presenciais implicaria em custos de viagem significativos. Usar os recursos de videoconferência seria uma alternativa mais econômica. Mas e o fuso horário? Entrevistar pessoas localizadas em um fuso horário de 10, 12 horas de diferença implica em

alguém ter que trabalhar em um horário incômodo. O uso de questionários, neste caso, minimizaria a necessidade de entrevistas.

7.7.2. Como realizar a pesquisa

7.7.2.1. Preparação

Há muita coisa em comum na preparação de uma entrevista e de uma pesquisa. A primeira definição deve ser sobre o objetivo que se pretende atingir; que informações se desejam obter; e em seguida selecionar o público-alvo adequado.

Se você seleciona um público mais amplo que o necessário, isso pode implicar em desperdício de recursos do projeto. E talvez, o que seja pior, implicar em ter respostas na sua pesquisa de pessoas que não deveriam ser consideradas nessa análise.

Se você seleciona um público mais restrito que o necessário, informações importantes talvez não sejam coletadas das pessoas que deveriam ser ouvidas no projeto. A consequência disso pode ser requisitos omissos ou errados.

Se você não está seguro da abrangência correta do público-alvo, melhor errar na dose para mais, incluindo um público mais amplo. Acrescente perguntas ao questionário que lhe permitam reconhecer melhor esse público e assim poder ignorar as respostas de quem não deveria estar no público-alvo.

A preparação do questionário é muito similar à preparação do roteiro de perguntas de uma entrevista. Aliás, o roteiro da entrevista é um questionário! Uma preocupação adicional é elaborar perguntas da forma mais clara possível e, se necessário, instruções para as pessoas responderem ao questionário. Durante uma entrevista, se a pergunta não é compreendida, o entrevistador pode reformulá-la ou explicá-la para que o entrevistado responda de forma mais adequada. Em uma pesquisa, se a pergunta não foi bem compreendida, você receberá respostas de pouco ou nenhum valor.

Outro cuidado deve ser dado à forma e à ordem em que as questões são apresentadas, para que não influenciem o respondente. Ou seja, deve-se

usar linguagem neutra e uma organização que não induza a pessoa a uma resposta “desejada”.

O questionário pode conter dois tipos de perguntas: ➤

Questões com respostas limitadas: cada questão oferece um conjunto de respostas possíveis. Útil quando a faixa de respostas é bem compreendida previamente. São bem efetivas para obtenção de dados quantitativos para análises estatísticas. Exemplo: qual é o canal de comunicação com a empresa que você mais usa?

- a) Intranet.
- b) E-mail.
- c) Telefone.
- d) Redes sociais.



Questões com respostas livres: o respondente é livre para responder as questões como desejar. As respostas podem fornecer mais detalhes e maior variação na faixa de respostas possíveis. Existe mais dificuldade na consolidação das respostas. O exemplo pode ser a mesma pergunta da questão fechada, com a diferença de que a resposta pode ser livre, não há a intenção ou mesmo como prever quais valores possíveis para as respostas.

Mesmo com todo o cuidado na elaboração do questionário, é muito importante que ele seja testado antes de ser distribuído. Aplique-o a algumas pessoas do público-alvo acompanhando o preenchimento das respostas. Avalie as dificuldades e dúvidas que as pessoas tiveram. Reformule o que for necessário.

Contudo, de nada adianta ter um questionário muito bem elaborado se ele não chegar ao público-alvo. Então, defina o meio de distribuição mais adequado. Por que não por e-mail? Talvez esta seja de fato a melhor opção na maioria das vezes; mas nem sempre. Você tem o endereço de e-mail de todos do seu público-alvo? Esses endereços estão atualizados? Quantos receberão sua mensagem na caixa de spam?

É muito difícil conseguir que 100% do público-alvo responda à pesquisa. Então, avalie qual a taxa de resposta alvo necessária para cumprir os objetivos da pesquisa. Esta será a meta a se buscar quando a pesquisa for executada.

Para conseguir melhores taxas de resposta, avalie oferecer alguma recompensa para quem responder ou definir alguma penalidade para quem não responder.

Por fim, você deve definir uma data limite para recebimento dos questionários respondidos, para proceder à análise das respostas.

7.7.2.2. Execução

Se o planejamento da pesquisa foi bem feito, a execução é a parte mais fácil do trabalho. Basta distribuir o questionário com suas instruções para o público-alvo e monitorar o seu recebimento. Isso é importante para identificar e corrigir em tempo eventuais falhas na distribuição. A evolução da taxa de resposta também deve ser acompanhada, pois irá lhe indicar se é necessária alguma ação para melhorar essa taxa. Por exemplo: novo comunicado reforçando o pedido para a pesquisa, postergação da data de término *etc.*

7.7.2.3. Finalização

Encerrado o período da execução da pesquisa, procede-se com a análise e a consolidação das respostas. Isso leva a um relatório com os resultados relevantes identificados na pesquisa, que deve ser validado por alguma parte interessada com a autoridade necessária. A partir de então, o relatório pode ser utilizado para a análise de requisitos.

7.7.3. Vantagens



Técnica relativamente rápida e barata de ser aplicada.



Obtém de maneira mais fácil informações de um público numeroso.



Em geral não demanda tanto tempo dos respondentes como em uma entrevista.



Muito útil quando o público-alvo está disperso geograficamente.



Quando aplicado a uma amostra representativa das partes interessadas, permite representar as opiniões de toda a população.



As questões com respostas limitadas são muito efetivas para a geração de dados quantitativos para análise estatística.

7.7.4. Desvantagens



Se a taxa de resposta for baixa, o resultado pode não ser estatisticamente significativo.



Uma falha na elaboração do questionário pode levar a respostas em branco ou incorretas, ou também gerar interpretações distintas da mesma pergunta, o que pode levar a erros nos requisitos.



O uso de questões com respostas ilimitadas demanda mais esforço de análise das respostas, se o público for numeroso.

7.7.5. Conclusão

A pesquisa é uma ferramenta interessante para a caixa de ferramentas da elicitação. Apesar de rápida e barata, precisa ser muito bem planejada e executada, pois em geral a predisposição das pessoas para responder questionários é baixa. Logo, é difícil usar a pesquisa mais de uma vez ao longo do projeto e obter uma boa taxa de respostas.

7.8. Exercícios

1. Por que é importante documentar o resultado da elicitação? Por que não partir direto para a Análise de Requisitos?
2. A partir do estudo de caso do Anexo, procure destacar quais termos seriam candidatos a compor o glossário do projeto visando alinhar conhecimento do negócio entre a equipe do projeto e o cliente.
3. Com base na sua experiência com entrevistas passadas, liste os três maiores fatores na execução da entrevista que percebeu terem causado inibição no entrevistado. Se você ainda não realizou nenhuma entrevista, considere entrevistas realizados por outros e que você tenha presenciado. Ou considere sua experiência com paqueras.
4. Prepare uma entrevista com o dono do projeto do estudo de caso do Anexo. O objetivo é refinar o escopo da especificação preliminar deste estudo de caso. Ele é uma pessoa bastante ocupada. Diretrizes e dicas:



A realização da entrevista não pode exceder sessenta minutos.



Procure elencar até dez perguntas (mais que isso não é possível abordar no tempo proposto).



Para cada pergunta listada, procure imaginar que respostas poderiam ser dadas e elenque perguntas (continuação da original) para aprofundar melhor a resposta provocada pela primeira pergunta.



Organize as perguntas de forma a conduzir a entrevista por assuntos.



Avalie se há questões a abordar sobre os requisitos de negócio.



Evite perguntas que um subordinado poderia responder, otimize seu tempo.

8. Análise de Requisitos

“Noite, a amada. Noite, quando palavras se dissolvem e as coisas tornam-se vivas. Quando a destrutiva análise diária está completa, e tudo que é realmente importante se torna inteiro e razoável novamente. Quando o homem junta outra vez o seu ser fragmentado e cresce com a calma de uma árvore.”

Antoine de Saint-Exupéry Este capítulo descreve a análise de requisitos, as atividades que a compõem e os objetivos que se pretende alcançar com a sua realização, tendo em perspectiva a sua inter-relação com os demais processos da Engenharia de Requisitos. Os insumos para a análise são os resultados da elicitação. Os produtos da análise são os requisitos da solução e da transição, geralmente na forma de especificações com requisitos funcionais e não funcionais prontos para encaminhar para empacotamento em unidades coesas para o projeto do software. Também são resultados da análise hiatos de informação ou oportunidades de racionalização que retroalimentam novas atividades de elicitação; conflitos não passíveis de resolução com as informações disponíveis para análise são direcionados para a gerência de requisitos.

A análise de requisitos é apresentada não só com o objetivo de organizar requisitos, mas também como instrumento para descoberta de requisitos implícitos, ocultos e cuja descoberta tardia pode inviabilizar o projeto e tornar nulos os resultados para os investimentos já realizados.

8.1. Um problema: a descoberta tardia de que ainda falta muito

O produto de software não atende às necessidades de negócio. As partes interessadas descobrem isso apenas quando do teste de aceitação ou mesmo na tentativa de transição para o ambiente real (produção). Essa é uma das experiências mais frustrantes para todos os envolvidos. Isso também traz graves consequências para o projeto, que pode ser cancelado por não mais se justificar; e para o negócio, que despendeu recursos valiosos para permanecer com sua necessidade não atendida.

Projetos apresentam um fenômeno mais comum do que se pode imaginar: estar próximo à metade do caminho quando se acredita que esteja concluído (ou quase).

O fenômeno não é motivado necessariamente por uma mudança no escopo original, como poderia se pensar em um primeiro momento. O escopo original deve ser definido em termos de áreas funcionais (ou mesmo organizações) afetadas e macroprocessos de negócio que limitam a abrangência da solução nesse nível mais alto. Essa declaração de escopo não deve limitar quais sejam especificamente os requisitos da solução ou de transição. Por isso, a descoberta desses requisitos mais específicos é uma revelação própria do esforço de desenvolvimento de requisitos e não deve ser vista como uma mudança ou causa que explique ou justifique o fenômeno descrito.

Parece então um paradoxo, pela contradição implícita. O paradoxo se desfaz quando se esclarece qual o marco utilizado para avaliar os 50% concluídos e qual o outro marco para avaliar os 100% (ou, mais frequentemente, 90%). O último considera o atendimento ao escopo identificado, enquanto o outro considera um escopo necessário para atender plenamente ao propósito para o qual o projeto foi iniciado. A Figura 8.1 ilustra esse cenário.

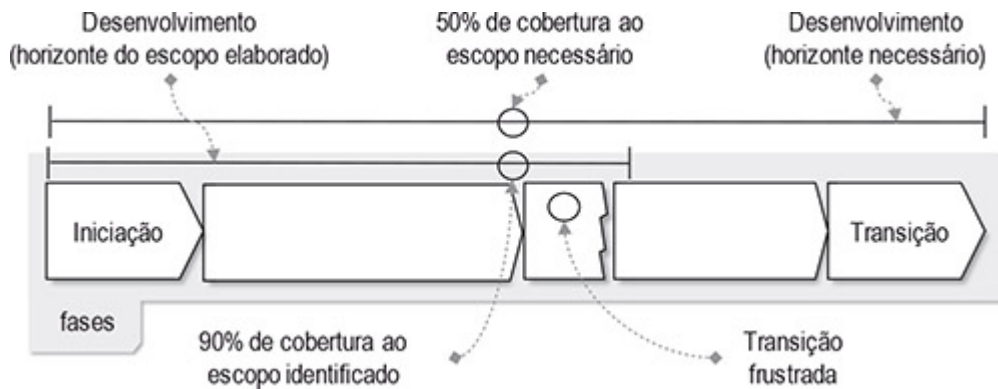


Figura 8.1. A dinâmica da crença em 90% concluídos quando ainda está bem distante do término.

A análise deve antecipar a descoberta do horizonte representado pelos 100% (ou ao menos os 80% mais críticos desse horizonte em um primeiro momento). Com isso, encontra uma bela oportunidade de evitar que o fenômeno descrito se multiplique. Deve-se ressaltar que a análise é parte da solução porque há ações necessárias que dependem de decisões em outros âmbitos, como, por exemplo, ciclos de *feedback* mais curtos entre o projeto e os usuários e com resultados intermediários mais tangíveis.

8.2. Visão geral da análise de requisitos

Se a elicitação de requisitos descobre as peças do quebra-cabeças, então a análise de requisitos procura montá-lo. O objetivo da análise de requisitos é aumentar o entendimento atual da informação, completá-la e melhorá-la. As informações descobertas com a elicitação das necessidades de negócio e dos requisitos das partes interessadas são seus insumos. A Figura 8.2 ilustra essa visão preliminar da análise de requisitos.

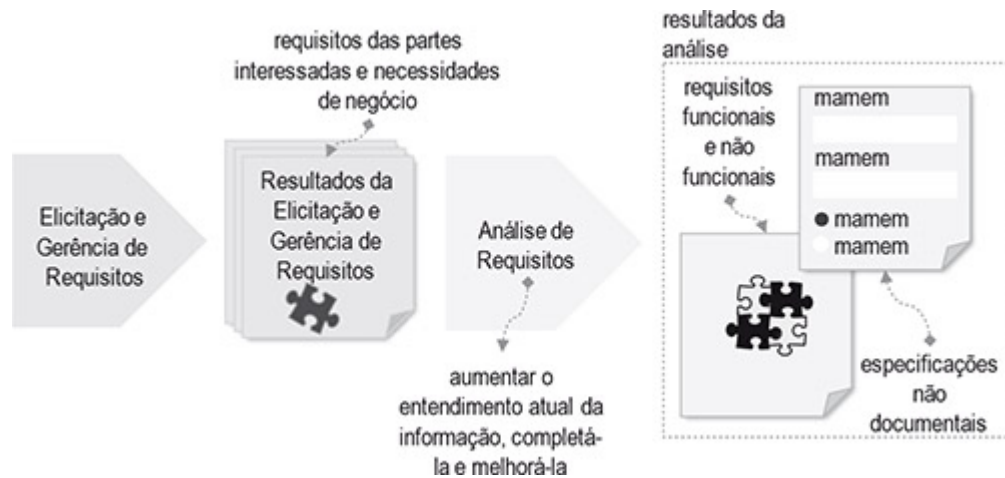


Figura 8.2. Uma visão resumida da análise de requisitos.

Essa visão preliminar é uma simplificação, pois coloca a análise sucedendo a elicitação, como que em um fluxo e sem *feedback*. Não costuma ser assim porque os produtos da elicitação são potencialmente omissos; ambíguos; redundantes; equivocados; ou incluem conflitos entre si.

Uma representação mais completa da análise deve incluir como parte de seus produtos pautas para novas sessões de elicitação a partir da descoberta dessas necessidades de informação ou decisão. Ela deve também posicionar a relação entre a análise e a elicitação como bidirecional. A Figura 8.3 faz isso.

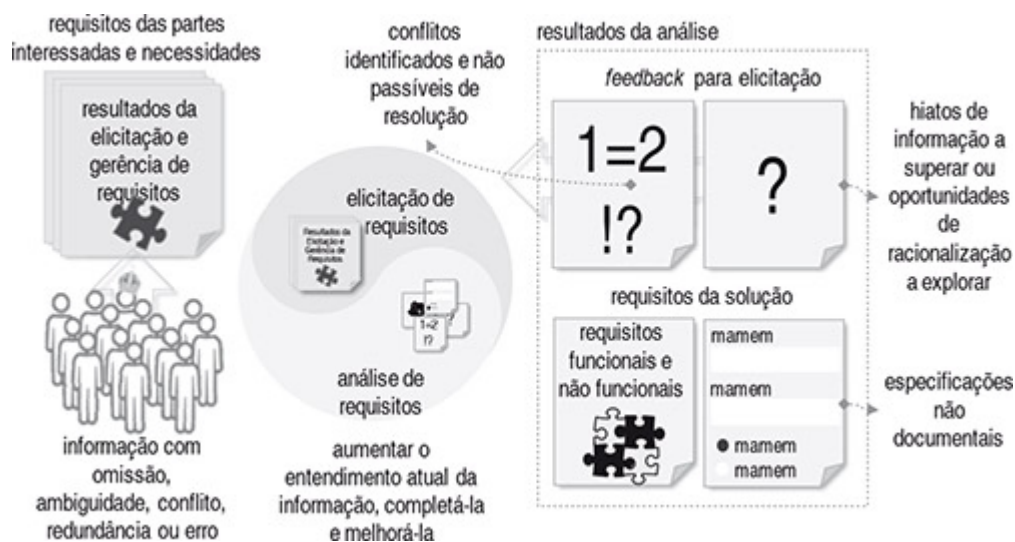


Figura 8.3. Dinâmica da interação entre elicitação e análise, onde os produtos de uma são insumos para a outra.

8.2.1. O que é análise de requisitos?

A análise de requisitos transforma a informação presente nos requisitos das partes interessadas e nas necessidades de negócio em diferentes processos expostos na Figura 8.4 de: ➤

exame da informação;



decomposição da informação; e ➤
síntese da informação.

O produto primário da análise de requisitos é a inteligência de negócio registrada em documentos e modelos com as especificações de requisitos funcionais e não funcionais para o software. Porém, a inexistência de especificações em documentos não implica na ausência de atividades de análise.

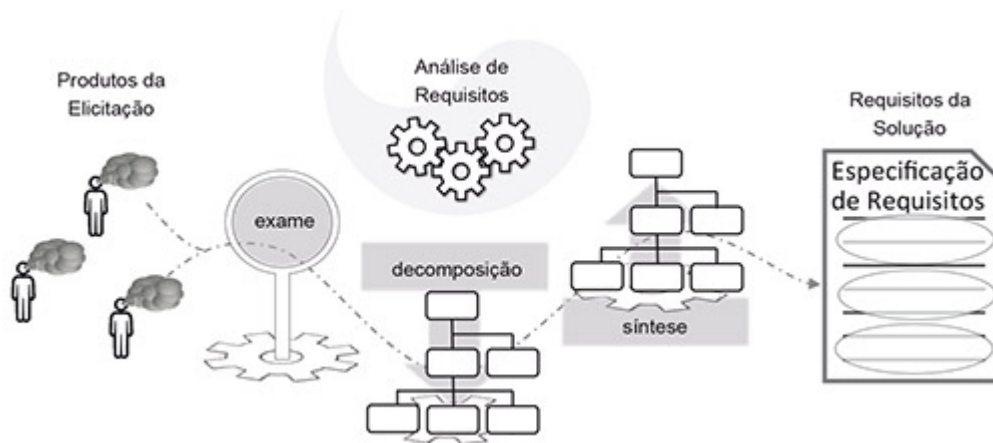


Figura 8.4. As habilidades básicas na análise de requisitos.

8.2.2. Por que análise de requisitos?

Há um grupo com 14 partes interessadas que fornecem requisitos com diferentes responsabilidades e autoridades nesse sentido. É razoável encaminhar para a implementação esses requisitos, com seus conflitos e oportunidades de racionalização ainda a serem revelados?

Bem, tudo é possível. Mas a qual custo com retrabalho que poderia ser evitado? Ainda assim, nesse cenário, haveria trabalho de análise de requisitos; contudo, pagando-se mais e esperando-se mais tempo. Algum retrabalho é inevitável em um processo de descoberta; é válido até questionar o uso do termo “retrabalho”. Não haver organização e disciplina em atividades de análise promove retrabalho além do necessário para o desenvolvimento dos requisitos.

A análise de requisitos tem por objetivo aumentar o entendimento atual da informação, completá-la e melhorá-la ao identificar omissão, ambiguidade, conflito, redundância ou erro nos requisitos.

O meio pelo qual a análise de requisitos cumpre esse objetivo é através da formulação de questionamentos a serem usados como importantes insumos para novas atividades de elicitação. Daí, pode-se dizer que há um fluxo de retroalimentação entre as atividades de elicitação e análise.

8.2.3. Como realizar a análise de requisitos?

A análise de requisitos é olhar mais de perto especificamente cada pedaço de informação revelado na elicitação e sobre como eles se relacionam entre si em conjunto. Envolve elaborar a informação de maneira progressiva e iterativa para descrever: ➤

O motivo para a mudança: que é a necessidade de negócio, o domínio do problema e um escopo inicial em alto nível.



A abrangência da solução (e da transição): que é o desenvolvimento de um escopo em termos específicos de seus requisitos funcionais e não funcionais.



A profundidade da solução (e da transição): que é o detalhamento com o passo a passo sobre esses requisitos no escopo.

Para tanto, a análise deve prover estrutura aos requisitos e às informações relacionadas. Ela consiste de diferentes tipos de atividade, a seguir relacionados, para o desenvolvimento da informação em direção a um conhecimento que pode ser aplicado para atingir os objetivos do negócio: ➤

Organizar a partir de exame, decomposição e síntese.



Modelar e usar modelos para refinar a informação.



Especificar para documentar os requisitos.



Verificar se o processo e os produtos observam boas práticas.



Validar se a solução satisfaz o cliente.

8.3. Organizar a partir do exame, decomposição e síntese

A elicitação de requisitos junto às partes interessadas resulta em documentos isolados (ou registros não documentais) com memórias de levantamento confirmadas, que devem ser examinadas durante a análise de requisitos. O analista deve considerar o maior contexto da mudança em que o software se insere. Nesse contexto, são tomadas decisões que: ➤

mantêm o fluxo operacional existente; e ➤

inovam em relação a um fluxo operacional existente.

Fluxo operacional deve ser entendido como a forma pela qual são realizadas as tarefas nos procedimentos formais e nas práticas estabelecidas na organização. A palavra-chave é **tarefa**.

8.3.1. Exame para identificar ou descrever tarefas

Os requisitos das partes interessadas são difusos, fragmentados e escondem muitas questões em aberto. Com o objetivo de facilitar a descoberta e a resolução dessas questões, definiu-se o conceito de nível de granularidade associado aos objetivos de um requisito.

Para ilustrar a descoberta dessas questões, considere dois requisitos levantados junto às partes interessadas para “Controlar entrada e saída de pessoas e veículos nas instalações portuárias” e “Validar o CPF do visitante na Receita Federal”.

O exame em discussão busca identificar requisitos como esses, que agregam vários objetivos do usuário. O trabalho ou qualquer tipo de comportamento que se deseja descrever, como manifestar que curtiu uma foto, pode ser descrito em uma infinidade de níveis de abstração. O mais material e que costuma ser compreendido até por trabalhadores braçais menos acostumados com conceitos abstratos é o da tarefa. Organizar os requisitos deve ter por objetivo nivelá-los para que possam ser tratados de maneira mais concreta e menos abstrata no nível de objetivo do usuário.

Por exemplo, um terceiro requisito “Listar todos os incidentes cadastrados em um período, segregados por área portuária” permite depreender especificamente uma tarefa; já os outros dois citados, não. Esses outros escondem questões como: ➤

Quais são as tarefas cuja realização deve ser alocada ao software em desenvolvimento para que se atinja o objetivo de controlar entrada e saída de pessoas e veículos nas instalações portuárias?

□

Quais são as tarefas em que se deve validar o CPF do visitante junto à Receita Federal?

Sobre a identificação dessas tarefas, sobre as respostas às perguntas citadas, qualquer projeto que inclui o desenvolvimento de software deve organizar os requisitos como comportamento que se deseja transferir para o software a partir de tarefas nas quais ele passará a se inserir.

Destaca-se que dificilmente se tem sucesso em identificar de maneira completa o escopo nesses termos já em momentos iniciais de um projeto.

8.3.1.1. Tarefas já existentes no fluxo operacional

O desenvolvimento do escopo nesses termos exige que decisões sejam tomadas pelas partes interessadas sobre quais tarefas devem ser absorvidas pelo software e em que grau ele deve absorvê-las. Trata-se de decisões que devem ser tomadas conforme a melhor adequação às necessidades de negócio e restrições do domínio do problema.

São decisões que implicam em:



Identificar tarefas: antes realizadas por um intermediário – que devem passar a ser realizadas diretamente por quem as origina ou para quem os seus produtos de informação se destinam. Por exemplo, quantas tarefas – antes realizadas por um caixa bancário – passam a ser feitas diretamente pelo cliente do banco quando movimenta seus recursos por meio de software (autoatendimento ou *internet banking*) para esse fim? Quantas tarefas – antes realizadas por um atendente em uma cooperativa de táxis – passam a ser executadas pelo próprio passageiro usando um aplicativo em seu celular?



Consolidar tarefas: até então definidas como tarefas diferentes. Com as facilidades oferecidas pela tecnologia da informação e comunicação, a separação em duas ou mais tarefas como reflexo de diferentes responsabilidades pode se revelar não mais necessária. Por exemplo, como parte da liberação de crédito prevista em um contrato de financiamento, há a atualização da planilha financeira com o registro do evento; novo cálculo para determinar valor de parcelas e encargos a vencer; e sua atualização em outras linhas da planilha financeira. Antes do projeto, havia duas tarefas diferentes no procedimento operacional abordado. Com a possibilidade de

transferir o processamento descrito para o software, as duas podem ser consolidadas em uma única.



Transferir parte do processamento de tarefas: ainda que mantenham a dependência de intermediários – o software, de maneira a garantir o correto intercâmbio de informações, e a observação das regras de negócios que se aplicam. Por exemplo, a avaliação de condições para determinar quais se aplicam ao informar um pedido que antes era de responsabilidade de uma pessoa e que passa a ser realizado pelo software, e, portanto, a tarefa em que isso acontece deve ser incluída como parte do escopo na análise dos requisitos.

8.3.1.2. Inovação no fluxo operacional

Enquanto os três casos expostos antes se referem a tarefas já presentes no fluxo operacional, há comportamento – proativo ou em resposta a um evento externo – que se estabelece como requisito para o software e que corresponde à inovação. São novas tarefas que talvez nem existissem se não houvesse a tecnologia da informação para suportá-las.

Por exemplo, originalmente, quando de uma renegociação das condições de um contrato de financiamento, era necessário celebrar um novo contrato estabelecendo os novos termos e condições aplicáveis e, então, com os recursos advindos desse novo contrato, quitar o anterior. O motivo dessa organização do trabalho é a complexidade associada a um procedimento único considerando as limitações de um indivíduo. Uma vez que essa complexidade esteja alocada como um requisito para o software, a situação é diferente; é possível criar uma tarefa única para esse fim.

8.3.1.3. Diferentes objetivos de informação

Sobre o nível de detalhe dos requisitos, dependendo do momento em que se encontra o projeto, a organização dos requisitos pode buscar descrever o escopo nos termos apresentados ou descrever o comportamento de cada tarefa em particular.

8.3.1.3.1. Descrever o escopo em termos específicos Não é viável examinar as memórias de levantamento para estabelecer o escopo sem que haja o desenvolvimento contínuo de registros documentais. Eles, pelo menos, devem relacionar as conclusões a partir da análise dos requisitos em uma lista de requisitos funcionais.

Os requisitos não funcionais devem também ser considerados ao organizar as informações no nível de tarefa quando se especifica um escopo, pois é possível haver diferentes requisitos não funcionais sobre um mesmo tema se o compararmos no plano geral e no plano de uma tarefa ou grupo de tarefas em especial.

Por exemplo, é possível que o tempo médio aceitável para uma transação interativa para o software em geral seja de até dez segundos. Para as transações de consulta ao catálogo de produtos, há uma exigência maior e o tempo de resposta médio não deve ultrapassar dois segundos.

Portanto, para o objetivo de qualificar o escopo, no mínimo deve haver uma especificação documentando o estoque de requisitos funcionais e os requisitos não funcionais.

8.3.1.3.2. Descrever o comportamento dos elementos constituintes do escopo Consolidar o escopo como resumido até este ponto é um objetivo intermediário. O outro objetivo de informação no desenvolvimento dos requisitos é entender, revelar e conceber o passo a passo de cada um dos requisitos no escopo e as regras de negócio que se aplicam; enfim, aprofundar o escopo.

Quando se tem esse objetivo, e dependendo da estratégia utilizada, é possível alcançá-lo sem a necessidade de especificações detalhadas. Técnicas como a prototipação podem se revelar adequadas e com isso exigir um trabalho menor examinando documentos. Determinar quando isso ocorre deve considerar as condicionantes apresentadas no Capítulo 2, ao expor o nível de detalhe dos requisitos.

8.3.2. Decomposição da informação

Por volta do término da década de 1970 até o final da década de 1990, a principal estratégia de desenvolvimento de software era baseada na

decomposição funcional. Quando se estabelece que a análise de requisitos é também um processo de decomposição, não se deve interpretar dessa forma.

Deve-se interpretar que as partes interessadas expressam os seus requisitos durante a elicitación em diferentes níveis. Alguns deles são expressos como macroprocessos que agregam vários objetivos do usuário. São os requisitos que correspondem aos objetivos agregadores e eles podem indicar que: ➤

- o software deve facilitar ou responder integralmente pela consecução de todos os objetivos agregados; ➤

- o software deve fazer isso apenas para alguns deles, o que indica uma questão implícita porque não estabelece quais sejam eles; ou ➤

- não se sabe nem mesmo quais sejam esses objetivos mais específicos, o que também indica uma questão implícita.

Como exemplo, o requisito para “Controlar entrada e saída de pessoas e veículos nas instalações portuárias” citado anteriormente.

A análise de requisitos deve identificar essas possibilidades como questões a serem respondidas, dúvidas. Uma vez elucidadas, o requisito deve ser decomposto em seus elementos constituintes e revelados quais desses elementos são requisitos para o software em desenvolvimento.

8.3.3. Síntese da informação

As informações levantadas ao longo da elicitación de requisitos estão também na forma de fragmentos compartilhados entre as partes interessadas e os analistas. Um fragmento de requisito tipicamente carrega consigo questões ainda em aberto e que devem ser exploradas. Eles correspondem aos requisitos no nível de objetivo de subfunção.

Por exemplo, o requisito “Apenas alunos que tenham pelo menos 75% de presença devem receber o certificado de participação”.

Esse fragmento esconde algumas questões como: quais seriam os requisitos funcionais em que isso se aplica? Poderíamos pensar: no requisito funcional que descreve (ou indica) a emissão do certificado! Contudo, não se pode afirmar com segurança que seja o único. Dependendo do caso, é possível haver um requisito funcional com as suas particularidades, em que o

próprio aluno emite seu certificado. Também é possível haver outro requisito funcional – parecido, mas diferente – voltado para a equipe de gestão de cursos que permita solicitar o certificado de participação de qualquer aluno, de qualquer curso, em qualquer época.

Durante a análise de requisitos, esses fragmentos devem ser avaliados como regras de negócio que devem ser observadas pelo software em desenvolvimento. Diferentes partes interessadas podem ter expressado uma mesma regra de negócio de maneiras diferentes. A informação pode ser complementar entre si ou apresentar contradição quando descreve uma mesma regra de negócio. Dessa análise são elaboradas questões para elicitación ou sintetizadas especificações descrevendo as regras de negócio já resolvidas.

As regras de negócio organizadas na análise de requisitos devem ser alocadas aos requisitos funcionais em que se aplicam. Muitas vezes, as partes interessadas “jogam soltas” as regras de negócio e o analista deve trabalhar para que sejam descobertos esses requisitos funcionais em que elas se aplicam por meio da elicitación de requisitos.

Esses fragmentos também podem indicar rituais compartilhados como parte de outros requisitos funcionais. Por exemplo, várias partes interessadas indicam haver um ritual comum em diferentes transações bancárias para um terminal de autoatendimento. O seu propósito é identificar de maneira positiva que o usuário seja realmente quem diz ser e, com isso, prevenir fraudes. Os responsáveis pelas diferentes transações sabem apenas informar que esse ritual deve ser realizado; alguns o descrevem de um jeito, outros de maneira diferente.

A análise revela a necessidade de descobrir quem é a autoridade sobre esse ritual comum, como ele funciona e quais devem ser as transações que o usam. Uma vez de posse dessas respostas, o analista deve “sintetizar” essa informação em uma especificação funcional própria ou como parte de outra especificação.

Os resultados da análise não se limitam à síntese dos requisitos a partir desses dois tipos de fragmentos. Durante a análise, quando se juntam as peças para identificar ou elaborar um requisito da solução ou de transição, é

bem possível descobrir que estão faltando algumas peças na caixa do quebra-cabeças! Os modelos são ótimas ferramentas para esse fim.

8.4. Modelar e usar modelos para refinar a informação

Este tópico apresenta a elaboração de modelos como um instrumento para refinar a informação e revelar os requisitos que resolvem a necessidade de negócio.

8.4.1. O que é modelagem?

Requisitos isolados não costumam ser complexos; são o relacionamento e a interdependência entre esses requisitos isolados que levam à complexidade. Um modelo é uma representação visual e descritiva para levar informação a uma audiência; em especial, com o objetivo de suportar a sua análise, entendimento e comunicação. O modelo busca representar informação de maneira estruturada para organizá-la e transmiti-la de forma mais amigável.

8.4.2. Por que modelagem?

Um conjunto de requisitos inclui várias perspectivas pelas quais pode ser analisado. Os modelos ajudam a visualizar e resumir essa complexidade em requisitos priorizando uma perspectiva. Concentram a informação sobre uma determinada perspectiva, uma faceta da realidade, abstraindo-se das demais. Eles fornecem contexto para transmitir informação via diferentes visões dos requisitos. Representar (ou entender) a realidade, portanto, exige diferentes tipos de modelos.

Elaborar ou utilizar modelos preexistentes durante a análise permite refinar a informação. Isso porque os modelos também permitem confirmar o conhecimento desenvolvido, identificar lacunas de informação e eliminar informação redundante ou contraditória mais facilmente.

8.4.3. Como realizar a modelagem?

Os modelos podem tanto ser insumos quanto produtos na análise de requisitos. Portanto, há potencial de uso de modelos já elaborados e disponíveis desde antes do projeto, assim como a produção de modelos como consequência da atividade de análise.

8.4.3.1. Modelos preexistentes

O analista deve determinar quais modelos existentes podem ser usados como ponto de partida e quais ele deve desenvolver para alcançar os objetivos da análise.

Por exemplo, se há um modelo de processo descrevendo a colaboração entre diferentes atividades em direção a um objetivo de negócio, então alocar os requisitos (que já devem estar desenvolvidos no nível de tarefa, como explicado anteriormente) como parte das atividades facilita concluir que determinada atividade seja: ➤

- manual, porque não há e não deve haver requisito alocado a ela; ➤
- informatizada ou automatizada, porém não há requisito alocado a ela porque isso é responsabilidade de outro projeto; ➤
- mais bem investigada, porque não há requisitos para ela, mas percebe-se que sem isso as necessidades de negócio não são plenamente atendidas.

8.4.3.2. Modelos a elaborar

Ainda que não haja modelos preexistentes a se estudar, uma das responsabilidades da análise de requisitos é decidir quais técnicas de modelagem são mais adequadas e quando deveriam ser usadas.

Nos capítulos anteriores, algumas técnicas de modelagem foram apresentadas, como o diagrama de contexto, o glossário e a declaração de problema. Neste, outras serão apresentadas, indicando seus pontos fortes e as circunstâncias em que melhor se aplicam.

As técnicas exploradas neste capítulo são:



Especificação via sentenças textuais.



Histórias do usuário.



Decomposição funcional.



Modelo de processo.



Modelo de domínio.



Caso de uso.

8.4.3.3. Tipos de modelo quanto à forma

Há diferentes tipos de modelos. Um ou mais formatos podem ser escolhidos conforme o tipo de informação a ser modelada e o objetivo da modelagem. Quanto à forma, eles podem ser classificados como: ➤

Matrizes: uma matriz é usada para modelar um requisito ou conjunto de requisitos que tem uma estrutura complexa, mas uniforme. Matrizes também são usadas para priorizar requisitos e registrar outros atributos.



Diagramas: um diagrama é uma representação visual, em geral composto de figuras, de um requisito ou conjunto de requisitos. Um diagrama é especialmente útil para descrever a complexidade mais facilmente por imagens do que por palavras. Os diagramas podem também ser usados para delimitar fronteiras, para criar hierarquias e

classificar itens em categorias, e para apresentar a estrutura dos dados e seus relacionamentos.

8.4.3.4. Tipos de modelo quanto à informação

Os modelos têm o papel de prover informação em diferentes perspectivas, abstraindo-se dos detalhes das demais com o objetivo de com isso diminuir a sua complexidade inerente. A Figura 8.5 ilustra essa função dos modelos.

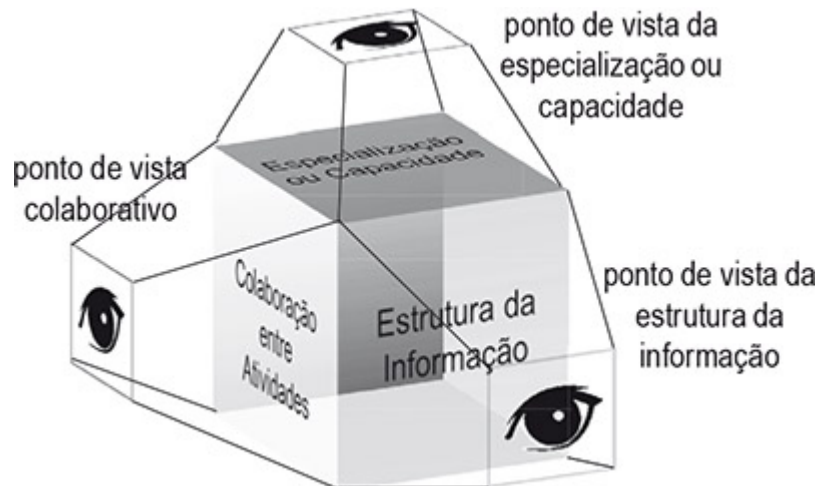


Figura 8.5. As diferentes visões representadas em modelos durante a análise.

Nesse sentido, do tipo de informação e do objetivo da modelagem, os modelos podem descrever os requisitos na perspectiva de: ➤

Colaboração: o assunto é a interação entre os requisitos, já associados aos objetivos de indivíduos realizando uma tarefa, posicionados em um plano no qual processos em diferentes níveis colaboram entre si na busca por um objetivo maior. São modelos que colocam em perspectiva a informação sobre o fluxo de atividades; modelos que representam uma sequência de ações, eventos ou caminhos que podem ser seguidos. Exemplos: modelo de processos, casos de uso e cenários, histórias do usuário, diagrama de sequência e diagrama de colaboração.

□

Especialização (ou capacidade): o assunto é a organização da informação com base em características e funções de uma

organização ou solução; são as afinidades entre diferentes responsabilidades que compartilham a necessidade por habilidades e conhecimentos comuns no desempenho de funções. Modelos deste tipo agrupam e dão estrutura aos requisitos conforme o seu papel no desempenho dessas funções. Exemplo: decomposição funcional e prototipação. Modelos com informações que representam organizações, grupos de pessoas, papéis e seus relacionamentos dentro de uma organização maior e para com a solução são especializações desse assunto. Por exemplo, a modelagem organizacional em um organograma; matrizes com papéis e permissões; e relação de partes interessadas.



Estrutura da informação: o assunto é a organização da informação em grupos coesos descrevendo conceitos sobre os quais há requisitos de armazenamento ou recuperação. Exemplo: dicionário de dados, glossário, modelo de domínio, diagrama de estados.

Ao exercitar a criação dessas diferentes perspectivas sobre os requisitos, o analista perceberá com mais facilidade pontos falhos da sua especificação.

8.4.3.5. Seleção de modelos

É comum haver múltiplas escolhas válidas e é bastante improvável que todos os modelos citados sejam usados em um mesmo projeto. Certamente mais de um tipo será usado na maior parte dos projetos. A escolha deve privilegiar a identificação de oportunidades para aperfeiçoar as operações do negócio. Alguns exemplos comuns de oportunidades que o analista possivelmente identificará incluem: ➤

Automatizar ou simplificar o trabalho realizado: tarefas relativamente simples, nas quais decisões são tomadas com base em regras estritas e inflexíveis, são candidatas principais a automação.



Melhorar o acesso à informação: fornecer maior quantidade de informação para a equipe que interage diretamente ou indiretamente com clientes, reduzindo assim a necessidade de especialistas. Os que tomam decisões podem não exigir esse nível de detalhe, mas deveriam estar conscientes de onde e a partir de quem eles podem obtê-lo se exigido. Normalmente, os que tomam decisões necessitam que lhes sejam fornecidos dados adquiridos e usados por pessoal operacional com a sua relevância e significado.



Reduzir a complexidade das interfaces: as interfaces são necessárias sempre que o trabalho é transferido entre sistemas ou pessoas. Por meio da redução de sua complexidade, pode-se aperfeiçoar o entendimento.



Aumentar a consistência do comportamento observável: diferentes profissionais podem manejar casos similares de maneiras bem diferentes, promovendo a insatisfação e frustração de clientes.



Eliminar redundância de informação: grupos de diferentes partes interessadas podem compartilhar necessidades em comum que podem ser atendidas com uma única solução, reduzindo assim o custo de implementação.

Como um exemplo de critério de seleção de modelos, um dos autores definiu uma política de qualidade na qual todo conceito de negócio armazenado pelo software que passasse por três ou mais estados conforme se transforma ao longo do processo de negócio deveria ter um diagrama de estados elaborado e limitado ao domínio do problema abordado. Cada transição de estados deveria, então, ser associada a um ou mais requisitos funcionais; caso não estivesse, deveria haver a confirmação junto à parte

interessada com autoridade para tal de que a transição não é um requisito no escopo do software em desenvolvimento.

A Figura 8.6 ilustra a implementação da estratégia associada à política de qualidade descrita em um cenário que tem como requisito de armazenamento os dados sobre o conceito Pessoa. Há quatro transições sem qualquer requisito associado. O que isso quer dizer?

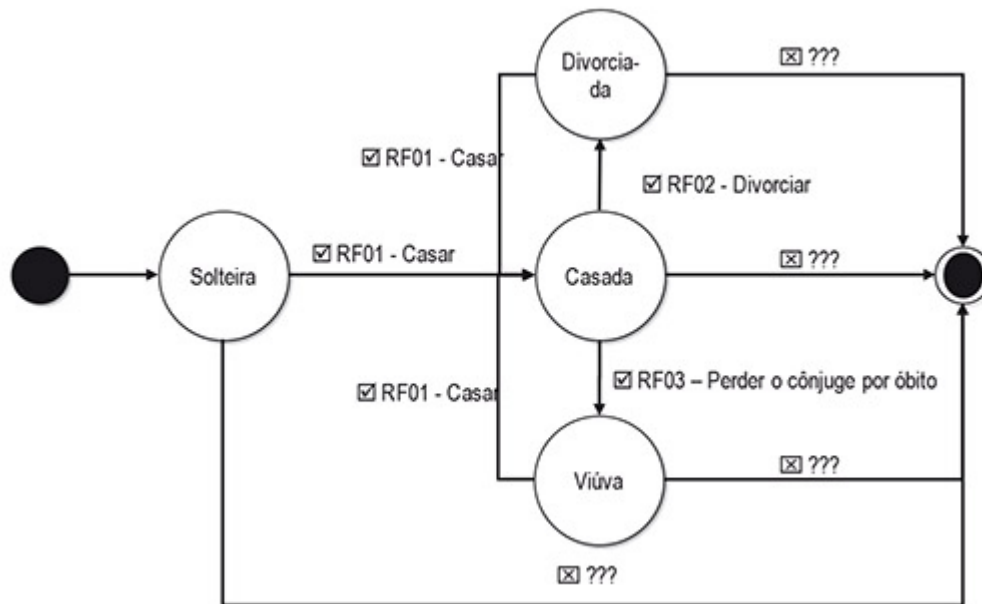


Figura 8.6. Exemplo de um diagrama de estados ilustrando as transições e os requisitos para o software que os abordam.

Provocar esse questionamento é ser proativo. A combinação de dois modelos – especificação em sentenças textuais e diagrama de estados, como no exemplo – permite descobrir obstáculos ao atendimento das necessidades de negócio com o investimento mínimo; em sua ausência, níveis de retrabalho nas disciplinas seguintes acumulam-se exponencialmente.

Observe que o modelo não deve ser um fim em si mesmo. Na empresa em que essa política foi implementada, anteriormente se exigia como um “entregável” o diagrama de estados. Centenas de folhas de papel, horas de esforço para elaborá-las, tramitar, validar e arquivar sem qualquer propósito para os vários elementos cujos únicos estados eram: ativo e inativo.

Fatores humanos, restrições organizacionais e a metodologia são fatores que devem ser considerados ao selecionar qual técnica de modelagem utilizar.

Eles são explorados a seguir.

8.4.3.6. Seleção de modelos e as restrições organizacionais

Um exemplo de restrição organizacional é não haver pessoal com habilidade em determinada técnica de modelagem e não haver recursos para adquirir essa capacidade para um projeto.

O uso de modelos gráficos tem maior eficácia quando associado a ferramentas de software, porém há muitos casos onde uma simples folha em branco cumpre o papel de elaborar um esboço. Contudo, tais ferramentas são fundamentais para a atualização de novas versões do modelo, ao passo em que se recebe retroalimentação a partir de versões intermediárias elaboradas.

Outro exemplo de restrição organizacional são os padrões sobre quais ferramentas de software utilizar na modelagem. Hoje esse é o principal fator a considerar, pois há muita variedade e disponibilidade de produtos para modelagem que vão do *freeware* a licenças de uso corporativo. Mesmo que haja aplicabilidade de dois determinados modelos em um cenário, é possível que restrições de tempo ou orçamento exijam que se priorize um em detrimento do outro. Essa priorização deve considerar outros fatores na seleção dos modelos utilizados: ➤

Timing: momentos iniciais privilegiam modelos que descrevem as necessidades de negócio, o domínio do problema e o escopo em nível geral; momentos intermediários na Engenharia de Requisitos privilegiam a definição do escopo de maneira específica; e momentos finais privilegiam o detalhamento dos itens no escopo. Por exemplo, um diagrama de casos de uso pode ser usado em momentos preliminares e com o objetivo de delimitar o escopo da solução em desenvolvimento. Da mesma forma, um diagrama de contexto cumpre o mesmo propósito. Considerando o nível de informação disponível nesse ponto do desenvolvimento, o diagrama de contexto representa a informação sem a necessidade de representar macroprocessos como se fossem casos de uso – quando não são.



Nível de abstração: conforme o nível de abstração dos requisitos – se eles são mais gerais ou mais específicos –, devem ser privilegiados modelos que acomodem o respectivo nível de abstração. Por exemplo, deve-se priorizar o uso de especificações em sentenças textuais enquanto os requisitos se desenvolvem ao longo da dialética de elicitação e análise quando há várias partes interessadas interagindo entre si e a equipe de desenvolvimento, ainda que seja possível elaborar histórias do usuário com o mesmo propósito.



Cobertura: não concentrar energias em modelos que enfocam uma determinada perspectiva em detrimento de outra que deixaria de ser abordada. Por exemplo, por que investir na elaboração de um número de modelos descrevendo uma perspectiva colaborativa e uma perspectiva da estrutura da informação quando não há modelo algum explorando a dimensão da especialização?

8.4.3.7. Seleção de modelos e fatores humanos

Um exemplo de fator humano é a maneira pela qual as pessoas adquirem informação sobre um assunto. Há pessoas cujo entendimento de um tema exige apoio visual, outras recebem melhor a informação textual. De maneira similar, há pessoas com maior facilidade de compreender conceitos abstratos, enquanto outras conseguem apenas entender conceitos abstratos quando materializados em objetos concretos.

A Tabela 8.1 ilustra o esquema de classificação de estilos cognitivos. Há um questionário desenvolvido para avaliar onde uma pessoa se enquadra nesse esquema de classificação denominado “Questionário Honey-Alonso de Estilos de Aprendizagem” (ALONSO, 2002).

Tabela 8.1. O esquema de classificação de estilos cognitivos.

Teórico	Reflexivo
• Aprendem melhor quando as coisas são apresentadas como parte de um sistema,	• Aprendem melhor com novas experiências, mas sem estar

modelo, teoria ou conceito. • Gostam de analisar e sintetizar. • Se algo é lógico, então é bom.	diretamente envolvidos nelas. • Reúnem dados e os analisam com determinação para emitir conclusões. • Observam a atuação dos demais, escutam, mas não intervêm até tomarem pé da situação.
Pragmático • Ponto forte é a aplicação prática das ideias. • Descobrem o aspecto positivo das novas ideias e aproveitam a primeira oportunidade para experimentá-las. • Tendem a ser impacientes quanto há pessoas que teorizam.	Ativo • Envolvem-se totalmente e sem preconceitos em novas experiências. • Aproveitam o momento presente e se deixam levar pelos acontecimentos. • Tendem a se entusiasmar com o novo e a agir primeiro e pensar depois nas consequências.

8.4.3.8. Metodologia

Por fim, as diferentes metodologias de desenvolvimento de sistemas têm objetivos comuns: ➤

Retenção do conhecimento: o esforço de desenvolvimento de software produz itens como programas de computador, roteiros de configuração, bancos de dados *etc.* necessários à sua execução em um ambiente operacional físico. Produtos intermediários resultantes do trabalho intelectual se perderiam se não fossem registrados. Então, o papel da metodologia no âmbito da gestão do conhecimento é no mínimo reter esse trabalho para que este não se perca.



Melhoria contínua: o desenvolvimento de software é o resultado do encadeamento de várias atividades com objetivos específicos que não necessariamente devem seguir a mesma sequência e ter uma única forma de realização. A metodologia descreve práticas comprovadas e torna seu uso institucional, de forma que todos tenham uma orientação uniforme que capture os casos de sucesso à medida que a metodologia evolui.



Garantia de qualidade: a metodologia serve de modelo de referência para a verificação da conformidade na garantia de qualidade. A avaliação da qualidade de um produto depende de suas especificações. Ela também depende da qualidade do processo que orienta a sua produção. Essas avaliações, tanto do produto quanto do processo, necessitam de referências para o certo e para o errado. A metodologia cumpre este papel do processo, de maneira similar ao papel que as especificações que a metodologia define servem como referencial para avaliar a qualidade dos produtos.



Direcionamento do trabalho: a metodologia viabiliza a comunicação entre diferentes especialistas e organizações. O desenvolvimento de sistemas na atualidade é resultado de diversas especializações funcionais verticais, inclusive podendo envolver várias organizações. A metodologia é o que provê os meios para a integração horizontal entre os produtos de trabalho intermediários de cada especialização em direção ao software disponível, funcional, em um ambiente operacional físico.



Fator normalizador: a metodologia permite estabelecer, negociar e avaliar de maneira homogênea níveis de serviços de diferentes organizações internas e externas. Dessa forma, é possível planejar e monitorar o desempenho, ainda que haja uma variedade de organizações e especialidades interagindo. A metodologia cumpre o papel, nesse contexto, de uniformizar o trabalho.

Em função de um ou mais desses objetivos, a seleção de modelos sofre impactos que podem limitar o horizonte de escolhas.

8.5. Especificar para documentar os requisitos

A modelagem e a especificação são coisas diferentes, pois a primeira coloca ênfase no desenvolvimento (revelação) dos requisitos e na sua comunicação com as partes interessadas no negócio. A segunda coloca ênfase na transmissão da informação para a equipe de desenvolvimento; ainda que ambas as atividades exijam a produção de informação compreensível por todas as partes.

8.5.1. O que é especificação?

A especificação de requisitos documenta os diferentes tipos de requisitos. O formato da documentação resultante da especificação de requisitos depende da organização, das necessidades do projeto e do ciclo de vida utilizado.

Por exemplo, as histórias do usuário serão os documentos que descrevem o estoque de requisitos a ser entregue quando se utiliza uma abordagem ágil de desenvolvimento; por sua vez, se uma abordagem derivada do Processo Unificado for utilizada, então as listas de requisitos cumprem esse papel.

A especificação de requisitos não precisa necessariamente cobrir todo o escopo, mas pode representar a informação disponível em determinado momento no tempo.

8.5.2. Por que especificação?

O PMI (2015) fornece uma relação bastante completa para especificar requisitos. Ela está transcrita a seguir: ➤

- Estabelece uma linha de base para: ✓
 - validar as necessidades das partes interessadas; ✓
 - definir a solução para as necessidades de negócio; ✓
 - a evolução da solução.



Fornece insumo para:



a equipe de projeto (*design*), os desenvolvedores, analistas de testes e garantia de qualidade; ✓

o manual do usuário e outros tipos de documentação, sejam eles impressos ou embarcados.



Fornece detalhamento de suporte para: ✓

acordos contratuais, quando aplicáveis (por exemplo, os requisitos são insumos centrais para a declaração de trabalho ou para uma solicitação de proposta); ✓

auditoria em indústrias reguladas e projetos de alto risco que têm como exigência seus requisitos documentados.



Viabiliza reutilizar informação por outros times de projeto que necessitam de um entendimento dos requisitos enquanto o projeto está em processo de implementação ou posteriormente.

O PMI também registra, de maneira muito apropriada, que, apesar da importância de documentar os requisitos, deve-se manter em mente que esta é apenas uma de várias técnicas para garantir o consenso entre todas as partes interessadas quanto ao comportamento esperado da solução (como explorado anteriormente) e que a documentação não deveria substituir a comunicação e a colaboração.

8.5.3. Quando elaborar uma especificação?

Há vários momentos em que se faz necessário elaborar documentos com especificações de requisitos. Dependendo do momento em que se esteja, há diferentes necessidades de informação, cujas respostas devem ser capturadas em especificações de requisitos. O esquema apresentado no

Capítulo 6 descreve um processo genérico com três marcos associados e que é útil para discutir a especificação de requisitos conforme esses momentos.

Descrevem-se as especificações relativas aos objetivos de informação que devem ser alcançados até o primeiro marco: a qualificação do domínio do problema, as necessidades de negócio e um escopo em termos de áreas funcionais, organizações afetadas e macroprocessos afetados; possivelmente uma lista de características para a solução ainda muito geral e pouco específica.

Os próximos tópicos abordam ações para atingir os outros objetivos de: ➤

- qualificar especificamente quais são as tarefas transferidas para o software; e ➤

- descrever o comportamento que se espera do software no que tange a essas tarefas.

8.5.4. Como elaborar uma especificação?

Nesse sentido, o mínimo do mínimo que se necessita especificar são os requisitos identificados no nível do objetivo do usuário, expressos em sentenças textuais. Mesmo no desenvolvimento usando abordagens ágeis e que priorizam a colaboração, há necessidade de especificações associadas a esse mínimo. Os adeptos do Scrum, por exemplo, têm no *Product Backlog* a especificação de histórias do usuário que compõem o escopo inicial para o desenvolvimento.

Essas sentenças textuais devem:



- expressar um e apenas um requisito por vez; ➤

- evitar cláusulas condicionais complexas; ➤

- usar uma terminologia consistente; ➤

- expressar requisitos em uma sentença com verbo em voz ativa; ➤

- indicar claramente o que ou quem é responsável pela ação realizada pelo requisito.

Para requisitos associados à manutenção de dados cadastrais – os denominados CRUD (*Create, Read, Update, Delete*) –, a prática comum é usar o verbo “manter”. Por exemplo, “Manter Produto” (ou “Manter Cadastro de Produto”) é uma sentença que remete a incluir, consultar, atualizar e excluir um registro de produto.

As sentenças não devem assumir que o leitor tenha um conhecimento prévio do domínio.

Por exemplo, a sentença da Tabela 8.2 expressa vários requisitos e inclui cláusulas condicionais, além de não incluir um verbo que caracterize a ação principal realizada.

Tabela 8.2. Como não especificar requisitos por sentenças textuais.

Identificador: RF001	Requisito: Controle de Portaria
Para realizar o controle de entrada e saída na portaria deve ser realizado o cadastro do visitante através dos seguintes atributos: Nome, Registro Geral e Imagem. Caso a visita tenha sido liberada pelo condomínio, então podem ser registradas a data e a hora em que o visitante acessou a portaria. Ao sair do condomínio, devem ser registradas a data e a hora em que o visitante saiu, mas só deve ser permitido para visitante com registro de entrada e sem registro de saída.	

Os mesmos requisitos deveriam ser reestruturados conforme a Tabela 8.3.

Tabela 8.3. Resultado da reformulação das sentenças textuais.

Identificador: RF001	Requisito: Manter Cadastro de Visitante
O condômino cadastra o visitante registrando Nome e Registro Geral.	
Identificador: RF002	Requisito: Liberar Acesso de Visitante
O condômino cadastra a liberação de acesso selecionando um visitante já cadastrado e indica o período (data e hora de início e data e hora de fim) para realizar a visita.	
Identificador: RF003	Requisito: Registrar Entrada de Visitante
O porteiro ou guarda verifica se a documentação apresentada pelo visitante confere com os dados apresentados na liberação de acesso (Nome do Visitante e Registro Geral) e registra a data e a hora em que o visitante entrou no condomínio.	

A representação das condicionantes que se aplicam em um nível superior, no encadeamento dos processos em que os requisitos se inserem, deve ser feita por meio da modelagem de processos, por exemplo.

O detalhamento dos requisitos identificados nas sentenças textuais pode ser realizado por meio de casos de uso e especificações de regras de negócio ou mesmo outros métodos não documentais, como a prototipação. As especificações de mudança/manutenção seguem a mesma lógica.

Deve-se ressaltar que alcançar o marco de consolidação do escopo não implica em esgotar a identificação de todos os requisitos, assim como alcançar o marco do detalhamento dos requisitos não implica em esgotar a descrição do passo a passo e das regras de negócio que se apliquem a esses requisitos.

Ou seja, ainda que o marco de consolidação do escopo seja alcançado, é possível que ainda haja itens de importância secundária em cinza – escondendo vários requisitos funcionais e não funcionais a descobrir como dentro do escopo (preto) ou fora (branco) em requisitos como “emitir relatórios gerenciais”. Ainda que o projeto tenha ultrapassado o marco citado, novas especificações indicando quais requisitos funcionais farão parte do escopo serão elaboradas como histórias do usuário ou elementos em uma lista de requisitos mais refinada.

Embora isso não seja recomendável, há casos em que as especificações de requisitos são elaboradas após o projeto. Sua utilidade é reduzida. Porém, em uma perspectiva de evolução do produto, essas especificações cumprirão o papel de reter conhecimento produzido no projeto, facilitando as manutenções futuras.

8.6. Verificação de requisitos

Por mais cuidado que se tome ao elaborar uma especificação de requisitos, o seu próprio autor nunca será uma pessoa isenta para avaliar se o trabalho foi bem feito. Erros quase sempre estarão presentes (lembre-se de que não existe especificação perfeita), e quem elaborou o trabalho costuma ter mais dificuldade para perceber essas falhas. Por isso a avaliação da especificação de requisitos por uma terceira pessoa ajuda a filtrar problemas e a melhorar

a qualidade do trabalho de requisitos. Este é o papel das atividades de verificação e validação de requisitos.

8.6.1. O que é verificação?

Verificar requisitos é comparar os produtos da modelagem e da especificação de requisitos com modelos de referência e adotar ações para apontar as não conformidades para que estas possam ser justificadas ou corrigidas.

A diferença entre a verificação e a validação de requisitos pode ser resumida da seguinte forma: ➤

Verificação de requisitos: as especificações atendem ao padrão?
Suas atividades não exigem a participação do cliente.



Validação de requisitos: as especificações atendem ao cliente?
Suas atividades exigem a participação do cliente.

8.6.2. Quem realiza a verificação?

O próprio autor da especificação não deveria ser o responsável pela verificação do seu trabalho, pois sua análise quase sempre é enviesada. A verificação de requisitos é uma atividade essencialmente interna à equipe de projeto. Há organizações com profissionais especializados nessa atividade, eventualmente parte de um grupo denominado de Garantia de Qualidade de Software (*Software Quality Assurance* – SQA). O alcance das atividades é mais amplo porque são verificados também produtos de outras disciplinas, não só de requisitos.

8.6.3. Por que verificação?

A verificação de requisitos busca garantir que as especificações de requisitos e seus modelos atendam ao padrão de qualidade para que possam ser utilizados de forma eficaz nas atividades seguintes do projeto. Ela

também cumpre o papel de preparar os requisitos para a validação. Tentar validar os requisitos sem antes verificá-los pode implicar em desviar a atenção da validação para questões que a verificação poderia ter tratado antes.

8.6.4. Quando realizar a verificação?

A verificação tem início quando um conjunto de especificações é considerado feito e os responsáveis pela gerência de requisitos decidem juntá-lo em um pacote de requisitos para encaminhá-lo para desenvolvimento. Trata-se de uma avaliação final do pacote de requisitos, antes de proceder com a validação junto às partes interessadas.

8.6.5. Como realizar a verificação?

A verificação acontece por meio da inspeção dos resultados da modelagem e especificação usando modelos de referência internos ou externos; e pela comparação entre diferentes elementos da documentação associada.

O CMMI apresenta a verificação com objetivos específicos a serem alcançados. Destaca-se um subconjunto em uma visão de atividades para explorar como verificar requisitos. Essas atividades estão divididas em dois grupos, como a seguir: ➤

Preparar para verificação:

✓

Selecionar produtos de trabalho para verificação.

✓

Estabelecer critérios e procedimentos de verificação.

✓

Estabelecer ambiente de verificação.

□

Verificar produtos de trabalho selecionados: ✓
Executar a verificação.

✓

Analisar resultados da verificação.

Das duas primeiras atividades, recomenda-se criar no plano corporativo diretrizes aplicáveis a todos os tipos de projetos desenvolvidos. A cada projeto cabe reavaliar esses dois pontos para identificar oportunidades de adequação. A Figura 8.7 ilustra a organização dessas atividades como descrito.

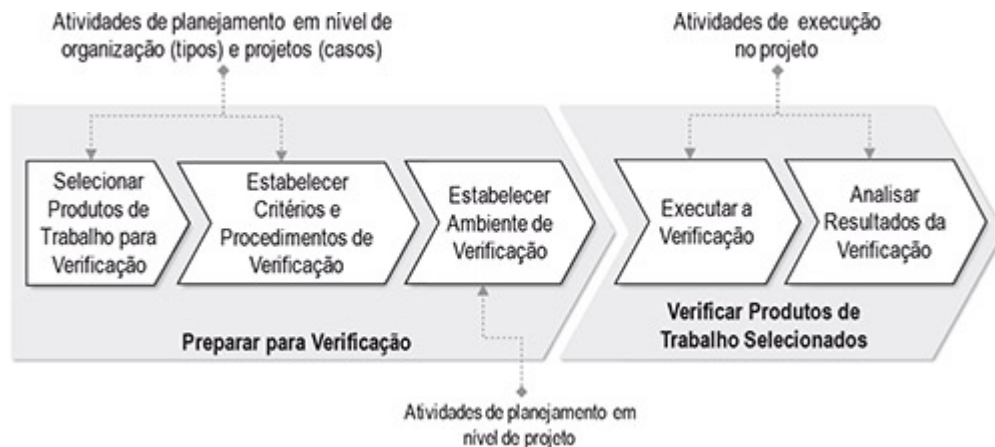


Figura 8.7. A dinâmica do trabalho de verificação de requisitos.

A atividade de **Selecionar Produtos de Trabalho para Verificação** permite a identificação de tipos de produtos de trabalho para verificação (quando executada no plano da organização) ou casos de produtos de trabalho (quando executada no plano de um projeto), métodos a serem usados para executar a verificação e as exigências que devem ser satisfeitas para cada produto de trabalho selecionado.

A atividade de **Estabelecer Ambiente de Verificação** permite a determinação do ambiente a ser usado para dar sequência à verificação. Seus produtos respondem à questão: o que é necessário para que seja realizada a verificação conforme os outros planos?

A atividade de **Estabelecer Critérios e Procedimentos de Verificação** permite o desenvolvimento de procedimentos e critérios que devem estar alinhados com os produtos de trabalho, métodos e características do ambiente de verificação. As atividades em **Verificar Produtos de Trabalho Selecionados** conduzem a verificação de acordo com os métodos, procedimentos e critérios definidos.

8.6.5.1. Verificação de um modelo em especial e sua integração com os demais

A seguir apresenta-se orientação prática para facilitar a verificação no nível de projeto ou corporativo. A intenção não é descrever um processo, mas apresentar os elementos que permitam a organização das atividades citadas. Deve-se verificar se cada modelo/especificação: ➤

Não apresenta elemento omissos em outros modelos. Compare cada modelo, parte do pacote em verificação, com os outros modelos disponíveis no projeto. Procure por elementos que são mencionados no modelo ora em verificação e que estão omissos em modelos que exploram outras perspectivas. Por exemplo, em um diagrama de atividades há etapa de emissão de certificados; contudo, não há requisito funcional ou caso de uso algum para este fim.



Usa uma terminologia para um mesmo conceito de negócio de maneira consistente em todos os modelos. O glossário, cujo uso se recomenda como obrigatório, é um excelente instrumento para esse fim. O modelo de domínio é outro recurso que cumpre o mesmo objetivo. O uso de uma terminologia inconsistente pode indicar um erro ou a necessidade de elicitação adicional para identificar requisitos, até então, omissos. Por exemplo, no requisito funcional “RF032 – Registrar Avaliação do Pedido” é utilizado o termo “Pedido”. Já no requisito “RF012 – Encaminhar Solicitação para Produção” utiliza-se “Solicitação”. Ambos os requisitos se referem ao mesmo assunto? Ou uma coisa é um pedido e outra coisa é uma solicitação? O glossário deveria dar essa resposta. Deve-se resolver

todas as discrepâncias, corrigindo a terminologia ou ajustando os modelos conforme necessário.



As necessidades de informação para o tipo de modelo em análise estão atendidas de maneira completa. Por exemplo, há uma especificação para Modelo e Notação de Processo de Negócio (*Business Process Model and Notation* – BPMN) adotada para descrever a dimensão colaborativa dos requisitos. Ou então, para o mesmo fim, existe o Diagrama de Atividades como definido na Linguagem Unificada de Modelagem (*Unified Modeling Language* – UML). Um deles é o modelo de referência na organização ou projeto. Então, verificar um modelo colaborativo é avaliar se ele observa o padrão e não omite informação definida como obrigatória por ele.



Há requisitos ausentes identificados pela comparação do modelo em análise com os resultados da elicitação (memórias de levantamento validadas). O item de verificação e a diretriz técnica associada, a seguir transcritos, ilustram critérios de verificação sobre a avaliação do escopo: *Item de Verificação 42. O Quadro de Requisitos Funcionais (Escopo) está COMPLETO? (02.DT.48).*

02.DT.48 – *Verificar se o Quadro de Requisitos Funcionais (Escopo) está completo.*

O Quadro de Requisitos Funcionais (Escopo) está CORRETO quando todos os elementos têm fundamentos para sua identificação nos documentos de memória do levantamento que devem permitir identificar as tarefas e os serviços do usuário transferidas para o software a partir de manifestações espontâneas dos envolvidos, ou como resultado da aplicação de técnicas de elicitação por parte da equipe de desenvolvimento. Está COMPLETO quando o mesmo se verifica em sentido inverso.



Todos os eventos externos para os quais o software deve prover uma resposta a partir de seu processamento ou resultados necessários que devem ser produzidos pelo software (independentemente de algum evento externo) devem ser identificados e descritos em todas as suas variações conforme o nível de detalhe definido para o projeto. A Figura 8.8 ilustra eventos para os quais o software deve prover resposta.

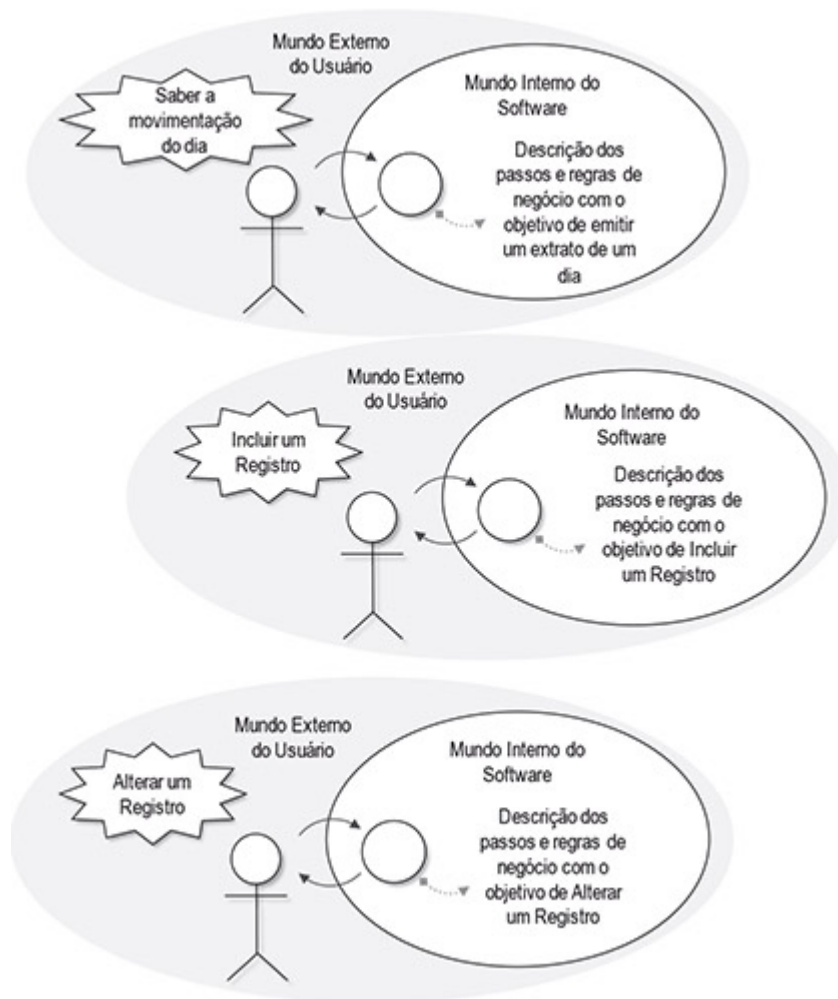


Figura 8.8. Exemplos de eventos externos e independentes para os quais o software deve prover respostas.

Na literatura sobre verificação de software, percebe-se uma ênfase na “sintaxe” e é comum encontrar exemplos como: *O nome do caso de uso inicia com um verbo no infinitivo, começando com letra maiúscula?*

Havendo mais de uma palavra, estas também começam com letra maiúscula (exceto preposições)?

Observar uma sintaxe é relevante, mas vive-se uma época em que fazer rápido é mais importante do que fazer “certo” quando o “errado” for violar regras como as citadas. Ainda mais se for o caso de colocar uma preposição iniciando com letra maiúscula!

8.6.5.2. Verificação da definição do escopo na lista de requisitos funcionais

Quando o objetivo é verificar se a especificação de requisitos cumpre o propósito de definir o escopo de forma ampla e inequívoca, certifique-se de que cada requisito funcional é: ➤

identificado no nível de objetivo do usuário. O requisito no nível agregador que não permita a identificação de todas as tarefas contidas deve ser rejeitado. O requisito no nível de subfunção que não cumpre o propósito de tratar expectativas críticas das partes interessadas também deve ser rejeitado. Ainda que se permitam essas exceções, isso deve se manter dentro de uma regra 80/20 (no máximo 20% dos requisitos nos níveis agregador e subfunção); ➤
denominado a partir de um verbo em voz ativa em uma frase que representa o objetivo do usuário. Nomes genéricos não estabelecem as expectativas do leitor nem proveem um ponto focal ao qual as pessoas possam referenciar de forma conveniente.

8.6.5.3. Verificação da descrição do requisito funcional – o detalhamento do escopo

A verificação associada ao detalhamento do escopo deve sempre estar associada ao nível de detalhe definido para as especificações no projeto ou produto. Certifique-se de que cada requisito funcional seja descrito de maneira que: ➤

haja um cenário com a história de sucesso em direção ao objetivo do usuário sem a consideração de possíveis falhas e, adiante, haja fragmentos de histórias que apresentem quais condições alternativas poderiam acontecer; ➤

sejam capturadas todas as falhas e alternativas que devem ser manipuladas. Um requisito funcional pode ter vários fluxos alternativos. Desconsiderar alguns desses fluxos significa que o desenvolvedor não entenderá apropriadamente o comportamento exigido para o software e aumentam as chances de que o software seja entregue deficiente. Preste atenção em especial em lógicas de desdobramento comum – nenhum registro encontrado, um e apenas um encontrado ou mais de um encontrado. Por exemplo, o requisito funcional “RF032 – Registrar Avaliação do Pedido” corresponde ao processamento esperado pelo software em resposta a um evento externo – a decisão sobre a avaliação que um faturista fez do pedido. Esta única decisão pode envolver diferentes caminhos, como ilustrado na Figura 8.9;

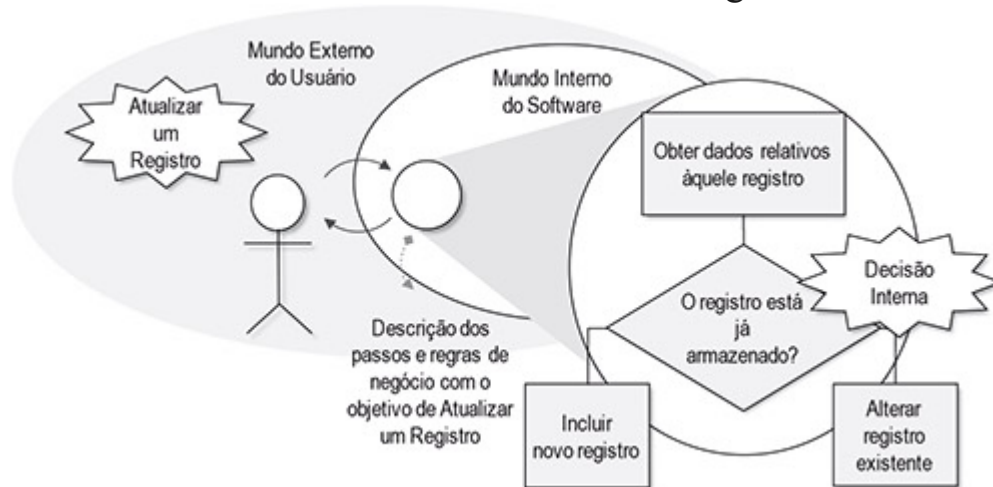


Figura 8.9. Diferentes caminhos em um mesmo requisito funcional.

□

não haja cenários que incluam em seu corpo requisitos não funcionais. Eles podem facilmente promover a desordem e prejudicar a clareza da informação. É necessário haver anexos com informação sobre os requisitos não funcionais que se aplicam; ➤ o leitor seja capaz de seguir facilmente o caminho por um cenário em particular no qual está interessado – caso contrário, ele possivelmente ficará frustrado ou perderá informação importante; ➤

a terminologia usada ao expressar o requisito seja compreensível para todas as partes interessadas e consistentes com os termos usados dentro da organização. Especificações de requisitos que são complicadas demais para leitores não técnicos, ou imprecisas para os desenvolvedores, são deficientes e potencializam sistemas pobremente construídos, inadequados. Os requisitos devem ser escritos de forma legível o suficiente para que as partes interessadas se deem ao trabalho de ler e avaliar, e preciso o suficiente para que os desenvolvedores entendam o que estão construindo. Utilize exemplos onde for apropriado para clarificar e fortalecer a compreensão; ➤

não haja duas ou mais condições descritas em separado que tenham o mesmo efeito quando se aplicam; ➤

não haja passos excessivamente grandes ou excessivamente pequenos, ou que obscureçam seu objetivo e sejam difíceis de ler e compreender. A descrição de cada cenário deveria possuir de três a nove passos. Idealmente, eles estão todos em níveis similares, um nível de abstração abaixo do nível de objetivo do usuário; ➤

cada passo indique claramente qual ator (em geral, um usuário e o próprio software) está executando uma ação e o que o ator deve conseguir. Ambos os leitores e desenvolvedores se confundem sobre o comportamento do sistema se esses dois pontos não estiverem claros; ➤

cada cenário conte uma história direta e simples, sem desdobramentos. Elimine de um cenário passos que não avançam o ator e simplifique passagens que distraiam o leitor da progressão da história; crie um fragmento descrevendo alternativas; ➤

não inclua detalhes de implementação, neutros em relação à tecnologia, pois fazer diferente aumenta a complexidade e obscurece o seu objetivo. Deve-se isolar o leitor de detalhes específicos de tecnologia, deixando-os livres para colocar foco no comportamento essencial do sistema. Isso não implica em dizer que não é necessário se preocupar com essa dimensão, apenas que deve ser descrita em separado.

8.6.5.4. Técnicas úteis à verificação

As técnicas citadas a seguir auxiliam direta ou indiretamente na verificação de requisitos. Optamos por explicá-las em partes específicas do livro, pois algumas têm utilidade em vários momentos da Engenharia de Requisitos. Portanto, aqui vamos apenas citá-las comentando no que podem ser úteis à verificação de requisitos. Vejamos: ➤

Controle de questões: se existem questões pendentes (ou em aberto) no controle de questões, significa que o trabalho de elicitação ainda não foi concluído para o projeto todo. E possivelmente podem indicar que há lacunas em requisitos em uma iteração específica. Consequentemente, o trabalho de análise também não se concluiu. E, logicamente, a especificação de requisitos não pode ser considerada completa.



Glossário: favorece o uso de um vocabulário consistente na especificação e familiar às partes interessadas.



Matriz de rastreabilidade: ajuda a verificar se existem requisitos de mais alto nível que ainda não foram tratados, além de detectar requisitos que estão sobrando, ou seja, que não deveriam fazer parte do projeto.



Checklists (listas de verificação): definem uma abordagem padrão para a identificação de problemas na especificação, apontando itens a serem seguidos pelo responsável pela verificação e pelo próprio autor da especificação.



Revisão (ou inspeção): usa uma visão externa ao autor da especificação para avaliar se a especificação consegue transmitir a mensagem pretendida por ele.



Geração de casos de teste: os casos de teste são importantes para a verificação do produto de software que será entregue, mas sua elaboração tem como efeito colateral benéfico a identificação de falhas na especificação, por exemplo, identificando requisitos não testáveis ou incompletos. Logo, a elaboração de casos de teste funciona como uma revisão da especificação.



Medição de pontos de função: o resultado da medição não tem muita utilidade para a verificação de requisitos, e sim para planejamento, acompanhamento e controle do projeto (VAZQUEZ, 2013). Porém, o processo de medição traz como efeito colateral benéfico a identificação de diversas falhas na especificação de requisitos. Isso porque o processo de medição nada mais é que também uma revisão estruturada da especificação dos requisitos (DEKKERS, 2001). A experiência dos autores mostra que essa abordagem é muito útil para detectar falhas na especificação de requisitos de: consistência, clareza e completude.

8.7. Validação de requisitos

A validação de requisitos é um trabalho de garantia de qualidade na Engenharia de Requisitos que busca assegurar que todos os requisitos especificados estejam alinhados com os requisitos de negócio. Ou seja, procurar garantir que todas as necessidades de negócio das partes interessadas no escopo do projeto serão satisfeitas. A finalidade é garantir que a especificação defina o produto certo a ser desenvolvido pelo projeto. Em resumo, avalia se o software satisfaz o cliente. Contextualizando a validação de requisitos nos processos de gerenciamento de projetos descrito no *PMBOK® Guide*, ela seria uma atividade componente do processo Validar o Escopo.

As técnicas citadas a seguir auxiliam direta ou indiretamente na validação de requisitos. Optamos por explicá-las em partes específicas do livro, pois algumas têm utilidade em vários momentos da Engenharia de Requisitos. Portanto, aqui vamos apenas citá-las e comentar em que podem ser úteis à validação de requisitos. Vejamos: ➤

Protótipos: talvez seja a maneira mais eficaz de validar requisitos. Ao apresentar uma proposta concreta do que será entregue como produto, a parte interessada consegue avaliar com mais interesse e atenção a solução proposta. É onde provavelmente se obtém um *feedback* mais rico do quão correta é a solução proposta.



Checklists (listas de verificação): definem uma abordagem padrão para a identificação de problemas da especificação, apontando itens de verificação a serem seguidos pelo responsável pela verificação. Embora seja uma técnica mais diretamente relacionada à verificação de requisitos, pode também ser útil para orientar o trabalho de validação pelo cliente. O que muda na abordagem de *checklist* de verificação para o de validação é que provavelmente as listas de itens a verificar são distintas nos dois casos. Além do fato de que, na verificação, quem executa a lista é a equipe do projeto e na validação são as próprias partes interessadas chave.



Inspeção (ou revisão): usa uma visão externa ao autor da especificação para avaliar se a especificação consegue transmitir a mensagem pretendida por ele. É uma técnica que se aplica tanto à verificação quanto à validação. A diferença é que na verificação o revisor é da equipe do projeto e na validação o revisor é uma parte interessada chave.

8.8. Técnica: histórias do usuário

Uma história do usuário é uma breve declaração que descreve algo que o sistema deve fazer para o usuário. É um tipo de especificação de requisitos adotado por muitas equipes de projetos “ágeis”. Como, por exemplo, na história do usuário representada na Figura 8.10.

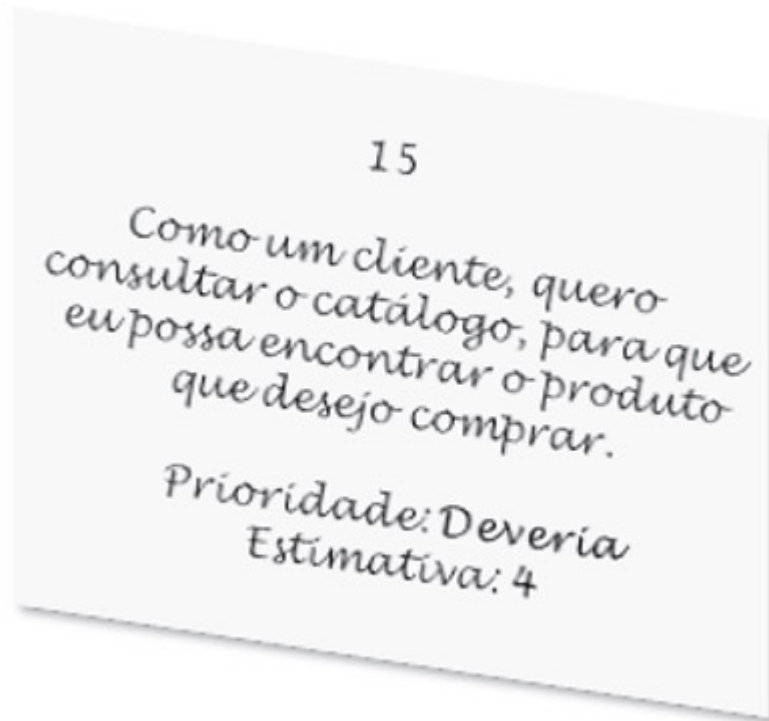


Figura 8.10. Exemplo de história de usuário: “como um cliente, quero consultar o catálogo, para que eu possa encontrar o produto que desejo comprar”.

Considerando a estrutura de tipos para classificação de requisitos apresentada, a história do usuário se posiciona como parte dos requisitos da solução, especificamente um requisito funcional, inicialmente no nível de objetivo do usuário. Coloca-se a história do usuário apenas inicialmente nesse nível, pois ela pode eventualmente ser subdividida como uma decisão de projeto para que sua implementação caiba em uma iteração.

Ela se restringe a definir o escopo sem entrar no detalhamento do passo a passo ou das regras de negócio que se aplicam à tarefa do software. Os detalhes do comportamento do sistema são desenvolvidos por meio de interações entre a equipe de desenvolvimento e o dono do produto; pela definição de um critério de aceitação. Isso não muda o seu papel como uma representação dos requisitos da solução. Contudo, o processo pelo qual a história do usuário é identificada, elaborada e mantida difere do processo

tipicamente associado a outras especificações que ocupam espaço similar, como listas de requisitos ou diagramas de casos de uso.

O uso de histórias do usuário foi introduzido como uma unidade de funcionalidade da *Extreme Programming* (XP). O progresso é demonstrado pela entrega de código testado e integrado que implementa uma história do usuário. Uma história do usuário deveria ser compreensível e de valor na perspectiva dos clientes, passível de testes pelos desenvolvedores e pequena o suficiente para que os programadores possam construí-la em uma iteração (tipicamente, entre duas e quatro semanas em projetos ágeis).

Hoje o método é parte integrante de outras abordagens, como o Scrum, na arena da gerência de projetos. Enquanto no XP promove-se a elaboração pelo próprio usuário, no Scrum quem as elabora é o dono do produto (*Product Owner*) a partir de informações elicitadas junto a outras partes interessadas.

8.8.1. Por que histórias do usuário?

Ao especificar requisitos por meio de histórias do usuário, cria-se um ambiente de propriedade das partes interessadas, que estabelecem prioridades e alocam recursos de forma incremental e interativa.

Porque a história do usuário não é uma documentação detalhada (e não é este seu objetivo), seu uso força a colaboração entre os membros da equipe para que a história possa ser compreendida por todos. Pelo mesmo motivo, exige pouco esforço em sua manutenção e exige que o valor entregue seja claramente articulado. Apesar desses benefícios, ela pode não ser adequada para ambientes mais burocráticos ou quando o nível de documentação do projeto é imposto.

O propósito da simples retenção do conhecimento é um dos fatores invocados como justificativa nessa imposição. Esse tipo de coisa apenas se justificaria se houvesse a intenção de investir tempo e recursos na manutenção de documentação detalhada atualizada. Caso contrário, ela ficará obsoleta. Ou seja, se não houver intenção de investir tempo e recursos nessa atualização, desperdiçam-se recursos em um detalhamento que basicamente cumpre o papel de um arquivo morto.

Um cuidado que deve estar presente quando se trabalha com histórias do usuário é a observação de requisitos não funcionais. A história do usuário não aborda explicitamente como documentar requisitos não funcionais.

8.8.2. Como elaborar uma história do usuário?

O estilo para redação de uma história do usuário é livre. Não há um modelo de referência que estipule um formato padronizado para sua elaboração. Contudo, a história do usuário deve responder a três perguntas: ➤

Quem se beneficia? Resposta indica a parte interessada que se beneficia da história do usuário (ator).



O que se quer? Resposta deve prover uma visão alto nível da funcionalidade para o usuário (descrição).



Qual é o benefício? Resposta indica o valor de negócio que a história proporciona (por quê).

As histórias do usuário devem ser escritas no espaço equivalente a um cartão; isso porque dessa forma limita-se a sua extensão, obrigando a produção de um texto claro e objetivo. Muitas empresas utilizam modelos pré-concebidos e há autores que recomendam escrever histórias do usuário no formato: Como (*papel*), eu quero (*algo*) para (*me beneficiar*).

8.8.2.1. INVEST

Um critério de qualidade para uma história do usuário é definida pelo acrônimo INVEST, descrito na Tabela 8.4.

Tabela 8.4. Critérios de qualidade INVEST para elaboração de histórias do usuário.

Letra	Significado	Descrição
I	Independente (<i>Independent</i>)	A observação do nível de objetivo do usuário ao identificar uma história do usuário garante que ela seja independente. Não importa como a história do usuário se encadeia com outras em um nível mais

		alto. Isoladamente, ela pode ser desenvolvida, testada e até mesmo entregue.
N	Negociável (<i>Negotiable</i>)	Em abordagens com maior orientação ao planejamento, o papel de uma lista de requisitos ou diagrama de casos de uso é limitar o escopo da solução em um nível que torne explícito o que deve ser transferido ao software; que sirva como um contrato do que especificamente está incluído no projeto, considerando seu prazo e investimentos associados. Em abordagens com maior orientação à mudança, isso é diferente. O papel das histórias do usuário deve ser promover a negociação sobre o escopo de determinada iteração de desenvolvimento. As histórias do usuário devem manifestar objetivos de pequena granularidade que permitam a estimativa do investimento na entrega do software correspondente sem perder uma visão dos resultados associados. De posse desses dois parâmetros – resultados e investimentos –, é possível negociar trocas, escolher entre as limitações de tempo da próxima iteração e, dentre as várias histórias pendentes, escolher aquelas que serão alocadas para a próxima iteração.
V	Valioso (<i>Valuable</i>)	Software é como uma cebola; mas não porque nos faz chorar. O núcleo dessa cebola encosta no hardware, e sucessivas camadas com serviços específicos vão se acumulando até o limite externo dessa cebola: o usuário. A intenção é que determinado comportamento compartilhado esteja disponível em um único lugar e sem redundâncias desnecessárias. O “valioso” nesse contexto indica que uma história do usuário se refere à entrega de software que corresponda a um resultado na visão desse usuário e não em uma perspectiva técnica que descreve como serviços compartilhados são estruturados em diferentes camadas de uma infraestrutura comum de suporte. A história do usuário, portanto, não deve se referir a uma sub-rotina, um componente reutilizável.
E	Estimável (<i>Estimable</i>)	Estimar o esforço para desenvolver um produto de software como um todo de maneira direta – sem qualquer parâmetro – é um desafio quase impossível. O mesmo não deve acontecer com uma história do usuário. Soluções para estimativas como dias ideais (<i>ideal days</i>) ou pontos de história (<i>story points</i>) têm por objetivo atribuir um peso relativo entre as histórias. Essas referências então são utilizadas para determinar quais são as histórias do usuário que irão compor a próxima iteração, conforme a sua prioridade.
S	Pequeno	As histórias do usuário deveriam ser pequenas o suficiente de modo a

	(Small)	serem concluídas em uma interação. No desenvolvimento ágil, histórias do usuário originalmente estabelecidas no nível do objetivo do usuário podem ser decompostas em histórias de menor nível de maneira que estas possam atender a esse objetivo, apesar de, em um nível de granularidade mais fino, ainda observar os demais.
T	Testável (Testable)	A história do usuário deve ser passível de teste e, portanto, deve evitar terminologia que não seja clara.

8.8.2.2. CCC

Outro critério de qualidade para orientar na elaboração de histórias do usuário é a sua aderência a três características (CCC): ➤

Cartão: a história do usuário é pequena o suficiente para caber em um cartão.



Conversação: ela consegue promover a comunicação entre o usuário e o time, proporcionando um entendimento comum da funcionalidade a ser entregue.



Confirmação: associada à história do usuário, consta qual o comportamento esperado para confirmar o seu escopo. Também conhecido como plano de teste, escrito no verso do cartão.

Cada história do usuário deve ter em algum momento provas de validação associadas, permitindo que o desenvolvedor e, mais tarde, o cliente possam verificar se a história está completa. Como não se tem uma formulação completa dos requisitos funcionais e não funcionais, a falta de provas de validação abre a possibilidade de discussões longas e não construtivas no momento da entrega do produto.

Alguns exemplos de histórias do usuário são: ➤

Como operador do hotel, eu quero estabelecer taxas ótimas para os quartos no meu hotel para maximizar meus ingressos.

□

Como vice-presidente de marketing e vendas, eu quero rever o desempenho histórico de vendas, para identificar regiões geográficas e produtos de melhor desempenho.

□

Como cliente, quero consultar o catálogo, para que eu possa encontrar o produto que desejo comprar.

□

Como um cliente, quero que o produto seja acompanhado com o valor de desconto para compra à vista, para que eu tenha a visibilidade da diferença monetária do produto à vista ou a prazo.

□

Como um cliente, quero que os produtos selecionados para compra fiquem armazenados em um carrinho de compras, para que eu possa visualizar todos os meus produtos e o preço total.

8.8.2.3. Dividindo histórias do usuário

Antes de uma história do usuário estar pronta para ser agendada para execução em uma próxima iteração, ela deve ser “pequena o suficiente” para poder ser concluída dentro da iteração.

No entanto, muitas histórias do usuário começam maiores do que isso. Dividir uma história do usuário consiste em quebrá-la em partes menores, preservando a propriedade de que cada história do usuário deve ter, separadamente, valor de negócio mensurável.

Dean Leffingwell (2011) define dez padrões para dividir uma história do usuário, conforme apresentado na tabela a seguir.

Tabela 8.5. Padrões para simplificar dividindo histórias do usuário.

1. Etapas de fluxo de trabalho: procure por etapas no fluxo de trabalho que encadeiam diferentes papéis juntos ou diferentes funções que podem ser feitas de forma independente.

Como um gerenciador de conteúdo, posso publicar uma notícia para o site corporativo.	... eu posso publicar uma notícia diretamente para o site corporativo. ... eu posso publicar uma notícia com avaliação do editor. ... eu posso publicar uma notícia com revisão legal. ... eu posso ver uma notícia em um site de teste. ... eu posso publicar uma notícia do site de preparação para a produção.
2. Variações de regras de negócio: divisão das histórias para lidar com a complexidade das regras de negócio.	
Como usuário, eu posso procurar voos com datas flexíveis.	... eu posso encontrar voos para uma semana específica. ... eu posso encontrar voos entre datas específicas.
3. Maior esforço: a história pode ser dividida em várias partes para que o esforço recaia em implementar a primeira história. Adicionar mais recursos deveria ser relativamente trivial.	
Como usuário, posso pagar com cartão Visa, American Express, MasterCard.	... posso pagar com um tipo de cartão. ... posso pagar com todos os tipos de cartões.
4. Simples/complexo: procure por histórias gerais que escondem a complexidade. Quando a definição dos critérios de aceitação revela variadas maneiras de abordar isso, então você pode dividir ao longo dessa variância.	
Como um candidato a empréstimo, eu quero calcular meus pagamentos de hipoteca.	... calcular os pagamentos manualmente. ... utilizar um modelo de planilha <i>on-line</i> usar uma calculadora <i>on-line</i> .
5. As variações de dados: escolha objetos de dados que podem ter variações com base em funções e ações.	
Como tutor de cursos <i>on-line</i> , posso criar conteúdo.	... em espanhol. ... em inglês. ... em português.
6. Métodos de entrada de dados: algumas vezes a complexidade se encontra na interface de usuário e não na própria funcionalidade. O correto seria então dividir a história para criá-la com a interface mais simples e depois criar uma interface mais complexa.	

Como usuário, posso agendar data para viagem.	... fazendo uso de uma entrada única de dados. ... com uma interface de calendário.
7. Adiar qualidades do sistema: procure oportunidades para adiar o trabalho. Dividir a história entre o que eu preciso fazer para que funcione e o que eu preciso fazer para que seja mais rápido.	
Como usuário, eu posso pesquisar voos.	... a resposta é imediata. ... a resposta leva cinco segundos.
8. Operações (exemplo: Criar, Ler, Atualizar, Excluir – CRUD): concentre-se ao longo de operações (pense métodos de alto nível ou operações do tipo CRUD).	
Como dono de um supermercado, eu posso administrar produtos.	... eu posso adicionar produtos. ... eu posso atualizar dados de produtos. ... eu posso excluir produtos.
9. Cenários de casos de uso: se temos quaisquer cenários de casos de uso existentes para o sistema, podemos escrever e dividir histórias de acordo com os cenários individuais desses casos de uso.	
10. Quebre para fora um ponto: muitas vezes as histórias não são necessariamente complexas, mas apenas possuem muitas incógnitas. Transforme possíveis critérios de aceitação em perguntas e busque as respostas.	
Transforme “Como um vendedor, eu quero recolher o pagamento com PayPal, pois, como é universalmente aceito, vai aumentar minha receita” em... “Como funciona o PayPal?”	... como é que eu sei que tenho um pagamento bem-sucedido? ... como eu sei quando um pagamento não foi bem-sucedido e quais são as opções que eu posso apresentar para o comprador?

8.8.2.4. Épicos

Enquanto uma história do usuário no nível do objetivo do usuário pode ser objeto de subdivisão para que as histórias resultantes sejam pequenas o suficiente para que possam individualmente ser acomodadas a uma arquitetura de software e implementadas dentro de uma iteração, há manifestações do usuário que correspondem a objetivos agregadores. São os chamados “épicos”. Um exemplo de épico seria: *Como um operador de hotel, eu quero definir a taxa ideal para quartos no meu hotel.*

Observam-se vários objetivos do usuário nessa declaração. Então, pode-se dividir esse épico em diferentes histórias, como: a)

Como um operador de hotel, eu quero definir a taxa ideal para quartos com base em preços do ano anterior.

- b) Como um operador de hotel, eu quero definir a taxa ideal para quartos com base em quanto hotéis comparáveis ao meu estão cobrando.

8.8.2.5. Temas

Outro conceito relevante é tema. Um tema é uma coleção de histórias do usuário relacionadas. Por exemplo, histórias do usuário que lidam com o processo de registro de cursos para estudantes.

8.9. Técnica: modelagem de processos

O objetivo deste tópico não é ensinar modelagem de processos. É apresentar a informação básica para orientar o analista na leitura e interpretação de uma representação de processos existente ou para organizar requisitos alocando-os em processos de negócio em diferentes níveis.

8.9.1. O que é um processo?

Processo é um conjunto de atividades interdependentes, ordenadas no tempo e espaço de forma encadeada, que ocorrem como resposta a eventos e que possui um objetivo, início, fim, entradas e saídas bem definidos. Essas atividades são geralmente entre diferentes funções – tipicamente, representadas por unidades organizacionais, torres ou verticais dentro de uma mesma organização – ou entre organizações que trabalham juntas para criar um produto ou serviço final. As atividades são apresentadas no contexto da sua relação entre si para proporcionar uma visão da sequência e do fluxo. Essa visão inclui um conjunto definido de atividades ou outras ações realizadas por pessoas, sistemas ou uma combinação dos dois e tem um ou mais resultados que podem levar ao fim do processo ou a uma entrega de resultados para outro processo.

8.9.2. O que é modelagem de processo?

De acordo com o Guia para o Gerenciamento de Processos de Negócio (CBOK®), da Associação de Profissionais de Processos de Negócio (ABPMP), a modelagem de processos de negócio é o conjunto de atividades envolvidas na criação de representações de processos de negócio existentes ou propostos. Ela pode prover uma perspectiva ponta a ponta ou uma porção dos processos primários, de suporte ou de gerenciamento. O propósito da modelagem é criar uma representação do funcionamento do processo de maneira completa e precisa. Por esse motivo, o nível de detalhamento e o tipo específico de representação do processo têm como base o que é esperado da iniciativa de modelagem. Um diagrama simples pode ser suficiente em alguns casos, enquanto um modelo completo e detalhado pode ser necessário em outros.

Não se deve confundir modelagem de processo de negócio com gerência de processo de negócio. Ambos têm um acrônimo em inglês de BPM; contudo, este termo é aplicado para o último: *Business Process Management*.

Praticamente todas as organizações criam descrições de seus principais processos em diferentes níveis de detalhe. Descrevem, por exemplo, como enviar as remessas de um cliente ou como gerar os relatórios financeiros de fechamento de mês. Em muitos casos, vão além de descrições textuais, incluindo modelos gráficos, que normalmente são fluxogramas (ou uma de suas muitas derivações) que determinam de que forma as atividades estão inter-relacionadas.

A modelagem de processos de negócio permite criar uma abstração de como funciona um negócio, pois fornece o entendimento de como são realizadas as diversas atividades contidas em cada processo. Na modelagem de processos, informações e documentos são utilizados pelos analistas, gerando um fluxo de como as atividades são realizadas, desde seu início até alcançar o objetivo do processo.

8.9.3. Diagrama, mapa e modelo de processos

Os termos “diagrama de processo”, “mapa de processo” e “modelo de processos” são muitas vezes utilizados de forma intercambiável ou como

sinônimos. Contudo, diagramas, mapas e modelos têm diferentes propósitos e aplicações. Na prática, diagrama, mapa e modelo são diferentes estágios do desenvolvimento do processo, cada qual agregando mais informação e utilidade para entendimento, análise e desenho de processos: ➤

Um **diagrama** retrata os principais elementos de um fluxo de processo, mas omite detalhes menores de entendimento dos fluxos de trabalho. Uma analogia pode ser feita com um diagrama simples que pode ser utilizado para demonstrar a rota até um local de armazenagem; ele pode retratar coisas como marcos geográficos e distâncias de forma simplificada ou exagerada, mas ainda assim serve para ajudar a encontrar o armazém. De maneira similar, um diagrama de processo ajuda rapidamente a identificar e entender as principais atividades do processo.



Um **mapa** fornece uma visão abrangente dos principais componentes do processo e apresenta maior precisão do que um diagrama. Tenderá a agregar maior detalhe acerca do processo e de alguns dos relacionamentos mais importantes com outros elementos, tais como atores, eventos e resultados.



Um **modelo** implica na representação de um determinado estado do negócio (atual ou futuro) e dos respectivos recursos envolvidos, tais como pessoas, informação, instalações, automação, finanças e insumos. Como é utilizado para representar com mais precisão o funcionamento daquilo que está sendo modelado, requer mais dados acerca do processo e dos fatores que afetam seu comportamento.

Para que os modelos de processo cumpram esse propósito, recomenda-se a utilização de padrões. A *Object Management Group* (OMG) estabelece o diagrama de atividades como parte da *Universal Modeling Language* (UML) e que pode ser usado também para modelagem de processos de negócio. A mesma organização também define um padrão especificamente para esse fim denominado *Business Process Model and Notation* (BPMN).

8.9.4. Por que modelagem de processos?

Uma vez criados, modelos de processo são documentos operacionais valiosos, que podem orientar funcionários nos passos a serem seguidos nos processos, garantindo que estes sejam desempenhados de forma padronizada e possibilitando que todas as partes interessadas tenham o mesmo conhecimento dos processos. Documentar processos permite que técnicas analíticas sejam aplicadas, manualmente ou por meio de software de análise de modelos, para detectar deficiências e congestionamentos nos processos e simular processos melhorados antes que estes sejam postos em prática.

Dentre alguns dos principais objetivos da modelagem de processos, podemos citar: ➤

Entender a estrutura e a dinâmica das áreas da organização.



Entender os problemas atuais da organização e identificar potenciais melhorias.



Assegurar que todos tenham entendimento comum da organização.



Servir como insumo para a Engenharia de Requisitos.



Auxiliar a identificação de competências.

8.9.5. Como realizar a modelagem de processos?

Modelos de processos podem ser criados manualmente ou com o uso de ferramentas específicas de diagramação; as mais sofisticadas são chamadas de *Business Process Management Suites* (BPMS). Um BPMS é um

conjunto de ferramentas automatizadas que proveem suporte ao BPM. Possibilita a modelagem, a execução (simulada ou não), o controle e o monitoramento dos processos de forma automatizada. Define a arquitetura e a infraestrutura tecnológica necessárias para a modelagem do negócio, a execução em produção dos fluxos de trabalho, a aplicação de regras de negócio, a utilização de dados corporativos, a simulação de cenários e a operação de outras aplicações do ambiente BPMS.

Na modelagem BPMN são construídos os diagramas de fluxo de trabalho (*workflow*), cada um contendo as atividades realizadas. Recomenda-se usar um verbo no infinitivo, seguido de um nome e algum detalhe, se necessário. Por exemplo, “enviar correspondência”. Todos os modelos devem estar ligados em um fluxo de como são realizadas as atividades. As etapas de cada processo devem ser descritas de forma que qualquer pessoa que leia o modelo consiga entender. O recomendado é que seja feito da forma mais simples e clara possível. Raias são utilizadas para que seja possível identificar qual ator realizou determinada atividade e quando houve a mudança para outro ator.

Os elementos da modelagem de processos de negócios são: ➤

Evento: acontecimento que inicia a execução (inicial), afeta o comportamento do processo (intermediário) ou o conclui (final).



Atividades: conjunto de ações realizadas.



Atores: responsáveis pelas atividades.



Entradas/saídas: insumos necessários para o processo ser executado na entrada e nas saídas geradas ao final do processo.



Regras: restrições que causam dependências entre atividades.

8.9.5.1. Representações de processos de negócio e a Engenharia de Requisitos

No contexto da Engenharia de Requisitos, quaisquer das representações citadas cumprem os propósitos abordados neste livro. A ideia é elevar-se a um nível superior ao de uma tarefa. Colocar em perspectiva diferentes níveis de objetivos de negócio para mais facilmente entender ou confirmar como os objetivos associados às diferentes tarefas se encadeiam nessa direção.

A modelagem de processo representa de maneira simples a dimensão colaborativa da solução. Isso faz com que todas as partes interessadas compartilhem a mesma visão (geralmente cada uma só conhece o seu papel no processo). Com isso, aumenta-se significativamente o potencial do conjunto de requisitos para a solução proposta estar alinhada ao negócio, cumprir o propósito de atender às suas necessidades e alcançar os resultados desejados.

Em função das responsabilidades do analista de requisitos, ele não utiliza a modelagem de processos para redesenhar o fluxo operacional de uma organização. Contudo, as diferentes representações de processos são instrumentos úteis para que sejam identificadas oportunidades de racionalização e eliminação de redundância ou informações em conflito. Apontar tais oportunidades e conflitos deve ser um dos objetivos da análise de requisitos, e a modelagem de processos tem papel central nisso.

Algumas outras aplicações práticas da modelagem de processos: ➤

Preparação para elicitação: um modelo de processo já existente permite descobrir quais papéis são desempenhados nos macroprocessos incluídos no escopo. Isso é relevante para descobrir as responsabilidades de partes interessadas já identificadas ou para descobrir a necessidade de identificar novas partes interessadas que desempenhem papéis. Também permite depreender um entendimento sobre como o seu trabalho é realizado, quais são os objetivos de nível mais alto e que produtos ou serviços são gerados.



Execução da elicitação: mapas, diagramas ou modelos de processo podem ser aproveitados para complementar entrevistas – atividades realizadas prioritariamente aos pares – em eventos de elicitação em grupo onde a representação escolhida é utilizada como fio condutor para confirmar ou retificar as informações, identificar e tentar resolver junto ao grupo conflitos presentes como ilustrado na Figura 8.11.



Modelagem de requisitos: ainda que não esteja disponível uma representação de processos, montar um fluxo descrevendo o processo a partir dos requisitos é uma experiência que traz resultados positivos similares.



Validação de requisitos: um modelo de processo é mais adequado ao uso de protótipos porque permite simular com precisão o papel dos requisitos identificados como parte da solução mais abrangente. Observe que não se deve associar a validação ao tempo de testes de aceitação do usuário ao final de projeto. Ela deve ser associada ao ponto em que há protótipos abordando uma área de processo e na qual se viabilize uma dinâmica similar à apresentada na Figura 8.11, mas substituindo requisitos por protótipos e uma experiência mais próxima ao uso.

Seja para descobrir requisitos ou para validar a sua eficácia, contribuindo para atender às necessidades de negócio, a modelagem de processos permite confirmar se uma atividade realmente deve se manter manual, se realmente parte ou todo o trabalho é responsabilidade de outro sistema, ou se houve falha na elicitação.

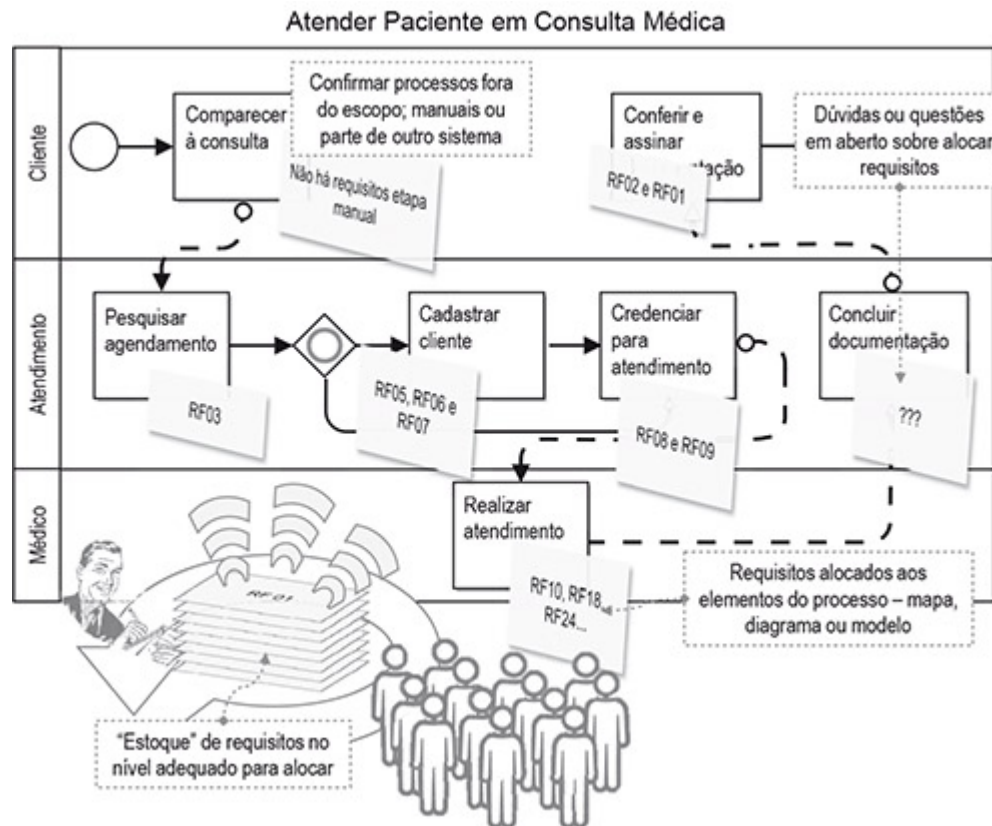


Figura 8.11. O modelo de processos e suas aplicações na Engenharia de Requisitos.

8.10. Técnica: decomposição funcional

Em termos práticos, qualquer coisa de maior complexidade é passível de ser decomposta em seus elementos constituintes mais simples. Por sua vez, esses elementos também podem ser decompostos. Essa dinâmica em sucessivas vezes constrói uma hierarquia com diferentes níveis de decomposição funcional. Isso se aplica aos requisitos, que também podem ser levantados e especificados em diferentes níveis de expansão.

Toda hierarquia requer ao menos dois elementos, um superior e outro subordinado a ele. Um elemento não deve ser subordinado a mais de um elemento superior. Um elemento superior deve ter ao menos um elemento subordinado, não havendo limite máximo para a quantidade de elementos subordinados. Em termos práticos, há limitações. Por exemplo, a “Lei de Miller” (MILLER, 1956) descreve a limitação da capacidade da mente humana em processar informação. Ela coloca que a quantidade de objetos

que alguém consegue manter em memória de maneira simultânea está em sete, mais ou menos dois. Os elementos subordinados, ainda que únicos, compartilham aspectos em comum de tal forma que haja afinidade suficiente entre os elementos pares para estarem subordinados ao mesmo elemento superior. A Figura 8.12 representa esses conceitos e ilustra a sua inter-relação.

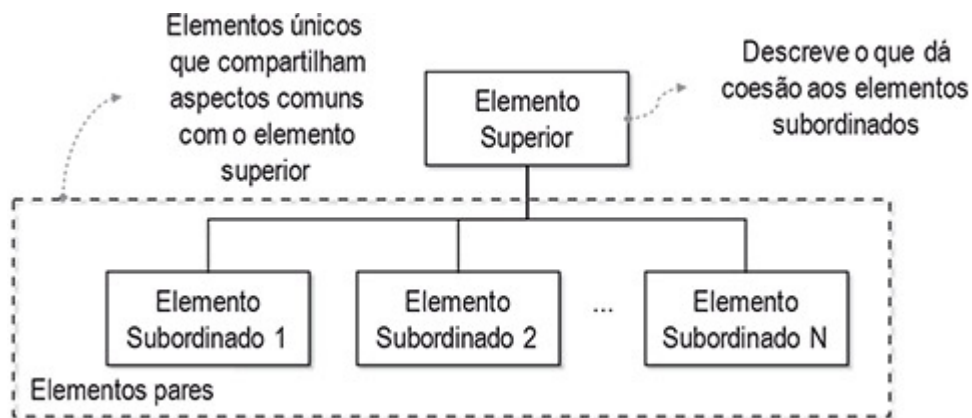


Figura 8.12. A hierarquia resultante da decomposição funcional, seus elementos e inter-relação.

Os aspectos comuns no caso dos requisitos organizados em uma hierarquia estão relacionados à sua especialização qualificada pelo elemento superior.

Por exemplo, durante a elicitação de requisitos para o desenvolvimento do PowerPoint identificam-se funcionalidades de inserir uma forma, definir o seu estilo, escolher como será o seu preenchimento, escolher como será o seu contorno, aplicar a formatação de outra forma já existente *etc.* Todas essas tarefas subordinam-se a ações sobre a forma (o elemento “Formatar – Ferramentas de Desenho”, na Figura 8.13).

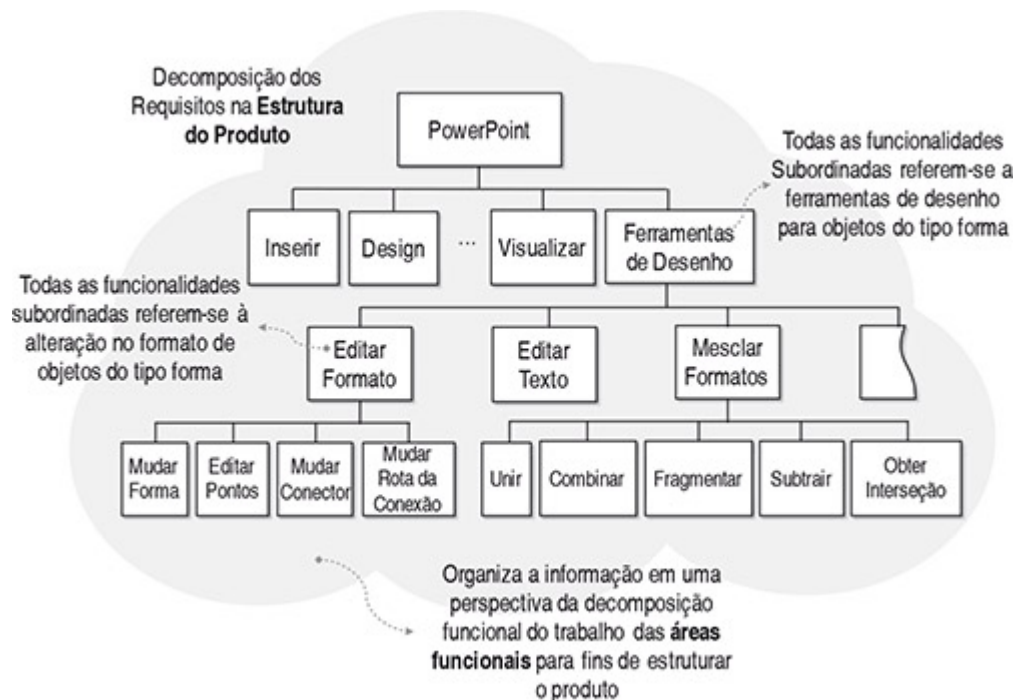


Figura 8.13. Uma visão de decomposição funcional do PowerPoint, destacando macrofuncionalidades que definem as características comuns das funcionalidades subordinadas.

Um elemento análogo à decomposição funcional e muito usado em planos de projeto é a estrutura analítica de projeto (EAP ou *Work Breakdown Structure* – WBS), cujo objetivo é lidar com vários elementos de menor complexidade em vez de um único elemento de maior complexidade e eliminar incertezas quanto ao que mais especificamente deve ser feito. A diferença é que a EAP estabelece uma hierarquia de atividades do projeto (ou pacotes de trabalho). Já a decomposição funcional dos requisitos estabelece uma hierarquia de atividades até o nível das tarefas do usuário que serão oferecidas pelo produto de software. A Figura 8.14 coloca essas duas visões em perspectiva.

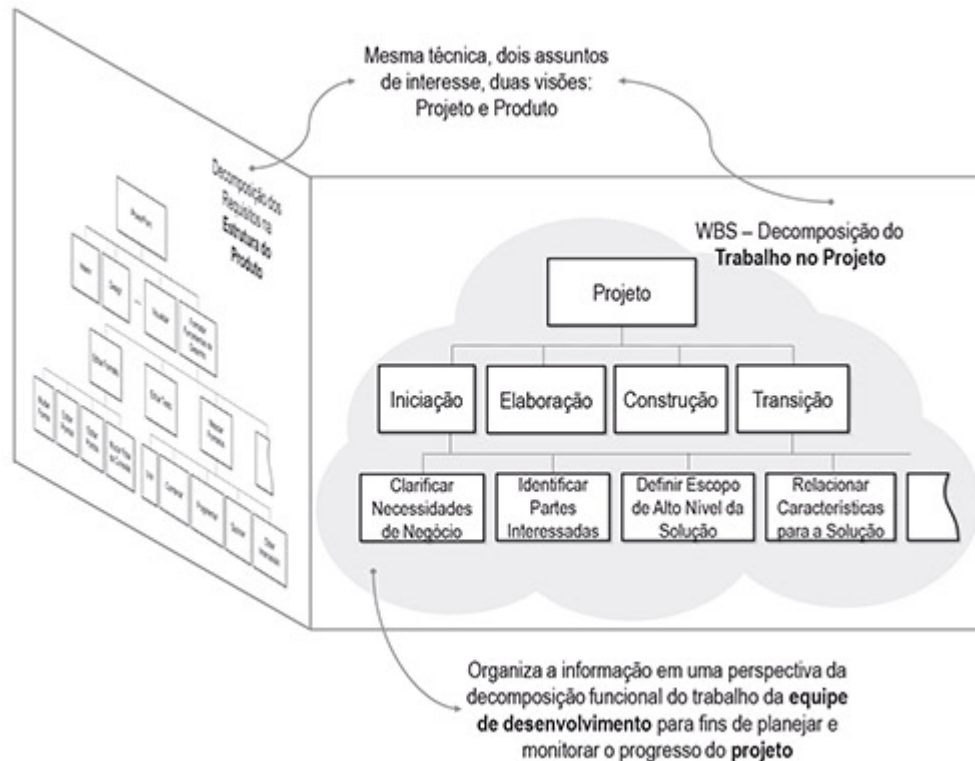


Figura 8.14. A mesma técnica, dois assuntos diferentes estruturando o produto e o projeto.

8.10.1. Como aplicar a decomposição funcional?

A decomposição funcional na análise de requisitos é um meio para organizar e dar estrutura aos requisitos; não uma estratégia para o seu desenvolvimento.

O nível apropriado de decomposição funcional define onde, por que e como parar a decomposição do assunto de forma a atender aos objetivos da análise. Como o nível de tarefa é o único que pode ser identificado de maneira inequívoca, recomenda-se este patamar como ponto no qual há entendimento e detalhe suficientes para que seja possível usar os resultados da decomposição funcional em outras tarefas, como: a verificação de requisitos, a validação de requisitos e a especificação de requisitos.

8.10.2. Por que aplicar a decomposição funcional?

O ponto de partida para a elaboração da especificação são as necessidades de negócio. A partir delas as informações solicitadas são refinadas junto às partes interessadas de forma a descobrir as tarefas ocultas em requisitos funcionais agregadores.

Portanto, o uso dos conceitos associados à decomposição funcional está muito mais próximo a uma “composição funcional”, onde o estoque de requisitos funcionais (já refinados e identificados no nível de tarefa, no nível de objetivo do usuário) é agregado em uma hierarquia com diferentes níveis de forma a facilitar outras atividades. A Figura 8.15 ilustra essa dinâmica.

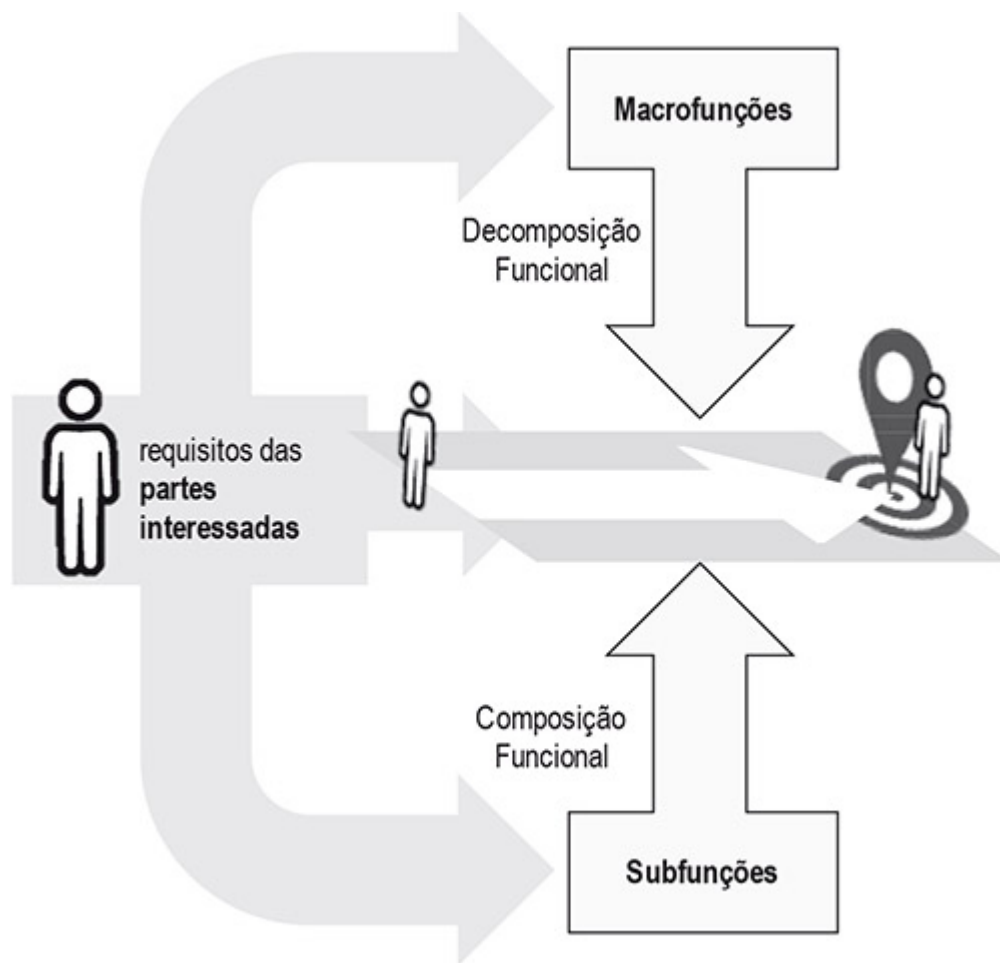


Figura 8.15. A dinâmica de composição e decomposição da informação.

As informações advindas dessa dinâmica de composição e decomposição funcional têm um papel importante na elicitação. Eventos de elicitação junto a grupos têm uma dinâmica diferente de uma entrevista. Não costuma

ser produtivo realizar um *workshop* de requisitos sem que na preparação prévia haja um fio condutor para apresentar as descobertas já realizadas. O papel dos requisitos organizados em uma hierarquia de decomposição funcional é solicitar retroalimentação sobre áreas que precisam de mais aprofundamento.

Outro benefício da decomposição funcional é evidenciar de maneira visual e de fácil percepção partes “fracas” do escopo. Ou seja, a figura final da decomposição, quando apresentada de forma não harmônica (por exemplo, partes muito mais profundas que outras), pode indicar que não foi dada a devida atenção a essas partes.

8.11. Técnica: modelagem de domínio

A modelagem de domínio é uma extensão da elaboração do glossário. Ela busca identificar conceitos do negócio sobre os quais há necessidade de recuperação ou armazenamento de dados e as suas inter-relações com outros conceitos no domínio do problema.

8.11.1. Como realizar a modelagem de domínio?

O modelo de domínio deve ser elaborado a partir do entendimento dos elementos que compõem o domínio do problema. Isso é feito por meio da identificação dos conceitos de negócio nos quais dados são consultados ou armazenados pelo software.

É indicado começar a modelagem de domínio em momentos iniciais do desenvolvimento ou mesmo antes dele. Nesses momentos, ainda não se sabe quais são os requisitos funcionais e quais deles serão alocados ao software em desenvolvimento. Portanto, inicialmente, a modelagem de domínio não deve se limitar aos requisitos com a visão da solução, pois eles ainda não estão completamente desenvolvidos.

Versões iniciais do modelo de domínio devem ultrapassar um limite ainda incerto; portanto, elas não devem excluir conceitos de negócio com potencial de alocação ao escopo final dos produtos em desenvolvimento, dependendo de decisões ainda em aberto do projeto.

Os principais elementos de um modelo de domínio são os conceitos de negócio e os relacionamentos entre eles. Ambos são descritos a seguir.

8.11.2. Conceito de negócio

Um conceito de negócio pode ser qualquer coisa que tenha um significado para o negócio. É uma construção que representa como uma unidade coesa um grupo de dados relativos a um mesmo assunto. As necessidades de informação associadas ao funcionamento do negócio é o que promove essa unidade. Essa coesão é denominada dependência funcional.

Por exemplo, um sistema informatizado para suporte à gestão da contratação de serviços de TI desde o seu planejamento até a sua avaliação e encerramento aderente à IN-04 do (MPOG, 2014). A norma estabelece: *Subseção I – Da instituição da Equipe de Planejamento da Contratação*
Art. 11. A fase de Planejamento da Contratação terá início com o recebimento pela Área de Tecnologia da Informação do Documento de Oficialização da Demanda – DOD, a cargo da Área Requisitante da Solução, para instituição da Equipe de Planejamento da Contratação, que conterà no mínimo: I – necessidade da contratação, considerando os objetivos estratégicos e as necessidades corporativas da instituição, bem como o seu alinhamento ao PDTI; II – explicitação da motivação e demonstrativo de resultados a serem alcançados com a contratação da Solução de Tecnologia da Informação; III – indicação da fonte dos recursos para a contratação; e IV – indicação do Integrante Requisitante para composição da Equipe de Planejamento da Contratação.

Em momentos iniciais do desenvolvimento, todos os substantivos e nomes deverbais (mais detalhes adiante) devem ser avaliados como candidatos a conceitos de negócio para compor um modelo de domínio. Esses tipos de termos estão sublinhados no texto exemplo.

Os substantivos possivelmente representam objetos de interesse, como documentos, pessoas, estruturas organizacionais, parâmetros operacionais ou estruturas físicas. Esses objetos de interesse possuem estrutura, comportamento ou inter-relações como qualidades.

A solução pode necessitar armazenar ou recuperar informação que descreve essas qualidades. O exercício da Engenharia de Requisitos transformará esse “pode” em um “deve” conforme cada caso.

Nome deverbal é a forma nominal de um verbo – por exemplo, *receber* é o verbo e *recebimento* é sua forma nominal. Possivelmente representam eventos ou manifestações que também possuem as mesmas qualidades citadas e o mesmo potencial para armazenamento ou recuperação pelo software.

Deve-se filtrar logo de início os candidatos que não refletem um conceito de negócio. Por exemplo, na expressão “explicitação da motivação” o autor apenas quer indicar um único conceito, “motivação”, e não sugere necessidade de armazenar dados sobre um evento que torna a motivação explícita. É um caso similar à “descrição da motivação”, que não indica um marco no fluxo operacional especificamente para descrever a motivação.

Já na expressão “Indicação do integrante”, há um conceito de negócio com informações sobre um rito (“a indicação”) com várias necessidades de informação possíveis, como: ➤

Quando da indicação?

☐

Indicação de quem?

☐

Qual o objetivo da indicação?

☐

Por quanto tempo é a indicação?

Há também outro conceito de negócio sobre “o integrante”, com necessidades de informação como: ➤

Quem é ele?

☐

Como se pode entrar em contato com ele?

Para a modelagem de domínio, o interesse são conceitos de negócio que agreguem um grupo de dados funcionalmente dependente. Então, a “explicitação da motivação” nem cabe representar no modelo como um conceito de negócio, pois não há qualquer informação que dependa desse conceito. Ela é parte de um grupo de dados sobre o conceito de “Documento de Oficialização de Demanda”. Na dúvida, confirme com as partes interessadas.

Algum conhecimento sobre o domínio do problema pode eliminar, ou ao menos diminuir, a prioridade dos candidatos que, já se sabe, estarão fora do escopo da solução e estão presentes apenas para dar contexto. Ainda assim, lembre-se de que você não é a autoridade no assunto e confirme com a parte interessada.

Quais requisitos de armazenamento serão derivados desses conceitos de negócio são decisões ainda pendentes ao longo do desenvolvimento dos requisitos. O analista deve investigar quais desses conceitos de negócio devem ter dados mantidos ou recuperados pelo software para que os objetivos de negócio que motivaram o projeto sejam alcançados. Essa informação é alcançada por meio de atividades de elicitação.

Uma primeira versão de um modelo de domínio poderia ser como a ilustrada na Figura 8.16. Sua elaboração promove o desenvolvimento de pauta para atividades de elicitação, por exemplo: ➤

Sobre a motivação e os resultados: há alguma informação além de sua descrição? Há padrão com categorias complementares à sua descrição específica para a contratação? Há realmente uma descrição específica para contratação ou apenas categorias nas quais cada contratação é enquadrada?

□

Sobre os objetivos estratégicos, o PDTI, as necessidades corporativas, as fontes de recursos: são todos resultados de decisões anteriores à contratação? Há registro prévio com as informações necessárias sobre esses conceitos no negócio? Esses

conceitos de negócio devem ser mantidos independentemente da contratação? O que se precisa saber especificamente sobre cada um deles?



Sobre a indicação da fonte de recursos, a instituição da equipe de planejamento, a indicação do integrante requisitante: existe um evento no negócio relativo a esses eventos na contratação e sobre o qual se registram informações como o responsável por ele? Por força de qual instrumento decisório ele aconteceu? Qual o limite em sua utilização etc.? Quando se colocam esses eventos, referem-se apenas a uma descrição que designe recursos humanos e materiais para a contratação?

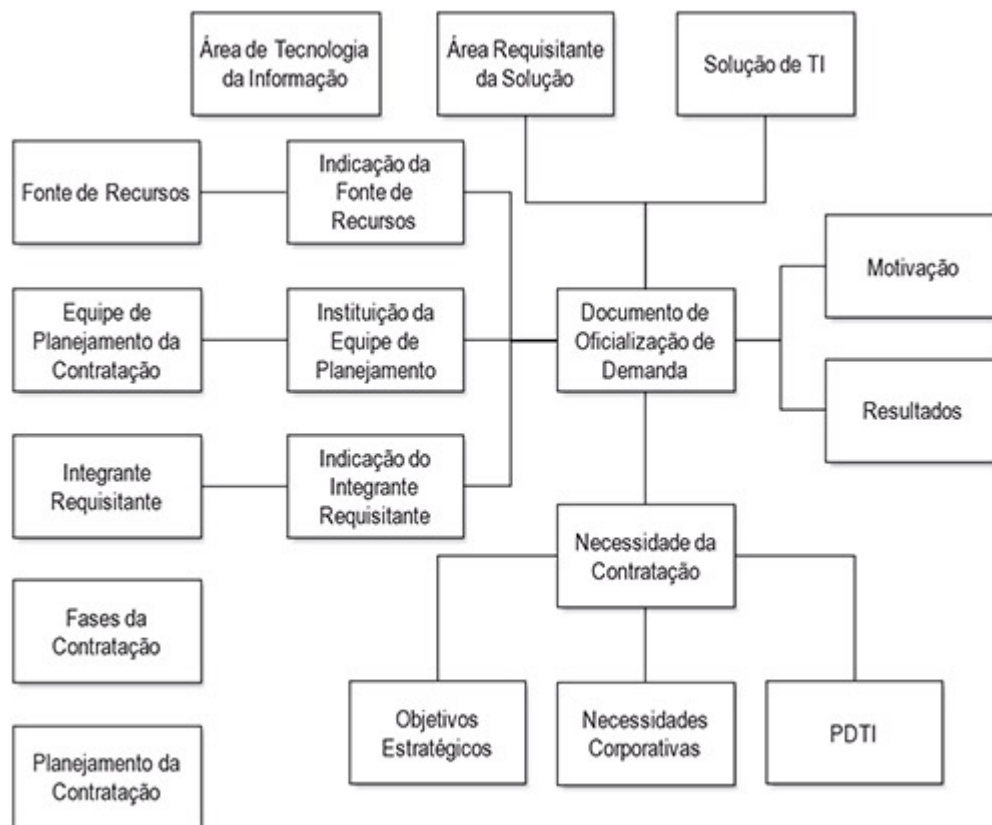


Figura 8.16. Versão inicial do modelo de domínio para o trecho apresentado da IN-04.

Conforme são obtidas respostas às perguntas, o modelo vai sendo aperfeiçoado. Por exemplo, se as respostas indicarem que: ➤

não há nada além de uma descrição para registrar no Documento de Oficialização de Demandas a sua motivação, os resultados esperados e as necessidades corporativas, esses elementos devem ser removidos como conceitos de negócio do modelo; ➤

os objetivos estratégicos têm uma série de informações associadas; essas informações são mantidas pela Secretaria de Planejamento; e o Documento de Oficialização de Demanda apenas precisa referenciar uma Ação Programática no Planejamento Estratégico; deve-se manter no modelo de domínio um conceito de negócio referente a essas informações.

As necessidades de informação típicas, que motivam requisitos de armazenamento sobre os conceitos de negócio identificados, são relacionadas à descrição de: ➤

Sua estrutura: qual a estrutura de um Documento de Oficialização de Demanda? Que informações ele contém?



Seu comportamento: quem elabora o Documento de Oficialização de Demanda? Qual o seu ciclo de vida? Quais etapas desse ciclo de vida devem ter tarefas total ou parcialmente transferidas para o software?



Suas inter-relações: há relação entre o Documento de Oficialização de Demanda e a Área Requisitante da Solução? Que outras relações relevantes existem?

A modelagem de domínio permite organizar os requisitos das partes interessadas e os requisitos de negócio em requisitos passíveis de serem verificados. A meta deve ser esgotar o escopo da solução proposta em termos de sua capacidade para: ➤

capturar, validar e complementar dados recebidos do ambiente externo sobre esses conceitos de negócio; ➤
consultar a situação de conceitos de negócio internos ou externos à organização; ➤
fornecer informações e resultados a partir dos dados armazenados sobre esses conceitos de negócio.

Não devem ser consideradas apenas as necessidades de pessoas ao investigar e organizar as necessidades de informação sobre conceitos de negócio. Devem ser levadas em conta também as necessidades de integração da solução em desenvolvimento com outras soluções, equipamentos ou qualquer tipo de entidade que consuma ou forneça informações para a solução proposta.

8.11.3. *Checklist* para organizar o modelo de domínio

São assuntos que devem ser usados como referência para identificar os conceitos de negócio: ➤

Documentos tramitados e cujos dados são informados, transformados, armazenados e distribuídos em seu teor bruto ou com informações derivadas a partir deles.



Pessoas representando os diferentes papéis que necessitam interagir com o software.



Eventos que demarcam diferentes pontos do processo de negócio.



Manifestações que provocam a ação e o acompanhamento dessa ação por meio da solução em desenvolvimento.



Estruturas organizacionais com suas alçadas e competências no âmbito do software. Por exemplo: filiais, departamentos, gerências, lojas, agências, diretorias.



Parâmetros operacionais que limitam e estabelecem políticas que devem ser respeitadas pelo software. Praticamente todos os sistemas possuem parâmetros.



Estruturas físicas com a sua descrição e qualificação, como prédios, salas, plantas, fábricas, produtos, depósitos, localizações, equipamentos *etc.*

8.11.4. Relacionamentos entre conceitos

Os relacionamentos entre os conceitos de negócio são informações relevantes porque a análise dos requisitos das partes interessadas normalmente explica as regras de alguns relacionamentos e deixam outros em aberto.

Nem sempre essas lacunas de informação são facilmente percebidas de outra forma. A atualização do modelo de domínio facilita identificá-los de maneira a promover a elaboração de questões para superá-los.

Por exemplo, um mesmo objetivo estratégico pode ter nenhuma contratação para que seja alcançado, uma ou várias. Uma contratação deve estar associada ao menos a um objetivo estratégico, mas pode também estar associada a vários.

8.11.5. Representações para o modelo de domínio

Uma representação útil para indicar o relacionamento entre diferentes conceitos de negócio é utilizada na modelagem de dados; destacando que o objetivo não é produzir um modelo de dados para implementar em um banco de dados ou mesmo um modelo lógico observando regras de

normalização. O objetivo é produzir um modelo de dados conceitual que descreva o domínio do problema e que, durante a sua elaboração, o analista possa mais facilmente descobrir o que ainda não se sabe.

Um modelo de classes no nível de conceitual também pode ser utilizado com o objetivo de representar um modelo de domínio.

A Figura 8.17 ilustra a representação de “pés de galinha” e a representação textual das regras que definem as quantidades mínimas e máximas entre casos de conceitos de negócio. Por exemplo, uma mãe biológica deve ter ao menos um filho (caso contrário, não seria uma mãe) e pode ter vários (a mulher com maior número de filhos de que se tem notícia teve 69 filhos). Cardinalidade é o nome que se dá ao que essas regras descrevem.

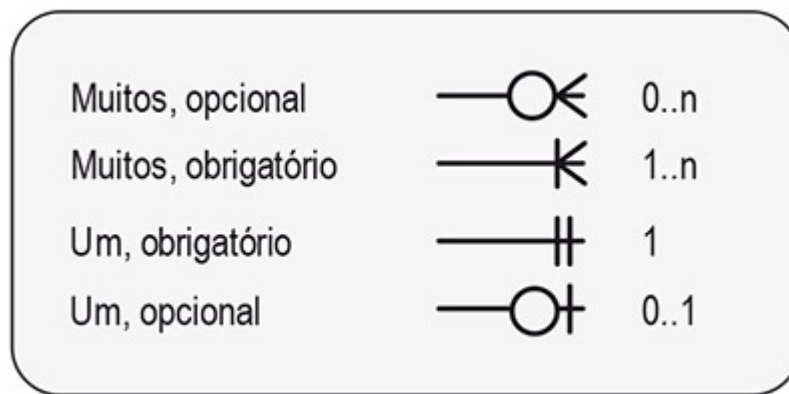


Figura 8.17. A representação “pés de galinha” e a representação da cardinalidade para os relacionamentos entre os conceitos de negócio.

Por exemplo, após mais investigação, descobriu-se mais informação sobre os relacionamentos entre os conceitos de negócio e as regras que definem a sua estrutura; com a nova informação, o modelo de domínio evoluiu. A Figura 8.18 ilustra como ficou uma parte do modelo sob a luz das novas informações.

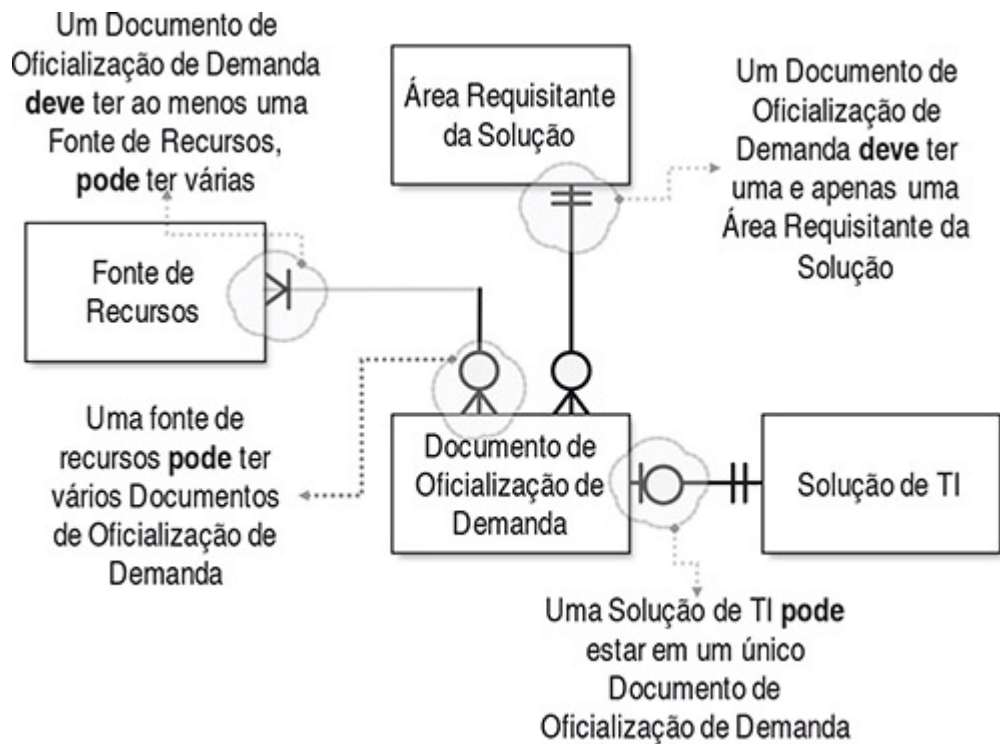


Figura 8.18. Fragmento do modelo de domínio com a representação de regras que explicam a relação entre os diferentes conceitos de negócio.

8.11.6. Por que realizar modelagem de domínio?

A modelagem de domínio fornece uma visão estrutural dos requisitos. Isso complementa a visão colaborativa fornecida pela modelagem de processo e as visões em diferentes especialidades fornecidas pela decomposição funcional.

O objetivo da modelagem de domínio é organizar a informação obtida na elicitação de requisitos em conceitos de negócio e descrever as regras que determinam o relacionamento entre eles.

Priorize, na utilização em atividades em grupo junto às partes interessadas, produtos da modelagem de processos ou da decomposição funcional. Contudo, a mesma dinâmica utilizada na alocação de requisitos aos processos ou na organização dos requisitos em uma hierarquia pode ser aplicada ao modelo de domínio com os mesmos objetivos e resultados.

Não é objetivo da modelagem de domínio no âmbito da disciplina de requisitos produzir um modelo de dados que obedeça a regras de normalização. Nem tampouco identificar e descrever todos os atributos e os

dados dependentes dos conceitos de negócio identificados. Há atividades em outras disciplinas para tratar desses objetivos.

8.12. Técnica: modelagem de casos de uso

Todo software deve ter um comportamento em resposta a eventos promovidos por seus usuários ou promover eventos que façam esses usuários agirem. Uma solução para especificar o comportamento associado a essa funcionalidade é por meio da descrição de casos de uso do software pelos seus usuários, denominados atores na terminologia de quem define seu padrão, a UML.

8.12.1. O que é um caso de uso?

Um caso de uso é um conjunto de passos que descreve um cenário principal e possíveis cenários alternativos para um ator alcançar um objetivo com o uso do sistema.

Como todo requisito funcional, o caso de uso não entra no mérito de como o comportamento descrito será mapeado nos componentes em uma arquitetura de hardware e software, ou como este será implementado em uma linguagem de programação. Portanto, ele descreve o que o software deve fazer e não como ele deve fazê-lo (em uma perspectiva de *design* ou implementação).

A especificação de requisitos funcionais que utiliza casos de uso é composta por: ➤

- diagrama de casos de uso;



- descrição dos atores;



- especificação dos casos de uso.

8.12.2. Diagrama de casos de uso

O diagrama de casos de uso demonstra de forma gráfica quais funcionalidades do software atenderão a quais usuários específicos. Permite a identificação do caso de uso como uma unidade de função para o software em análise. Representa graficamente os casos de uso, os papéis que os usuários desempenham (denominados atores por esse motivo) e a inter-relação desses elementos. A Figura 8.19 ilustra um diagrama de casos de uso e seus elementos básicos são descritos a seguir: ➤

Ator: representa uma pessoa (ou um grupo de pessoas) que desempenha um papel ao interagir com o software. Pode ser também qualquer “coisa” que interage com o software com a finalidade de cumprir um objetivo. Exemplos desses outros atores são outros sistemas ou dispositivos. Seu símbolo é um boneco palito.



Caso de uso: representa uma funcionalidade que atende a um ou mais requisitos do cliente. Como nome, sugere-se usar um verbo no infinitivo com um complemento; por exemplo, “incluir cliente”. Seu símbolo é uma elipse com seu nome no interior. Os casos de uso podem opcionalmente estar envolvidos por um retângulo que representa os limites do sistema.



Relacionamento: um ator interage com um caso de uso e isso é representado por um relacionamento. Os casos de uso também podem se relacionar entre si. Seu símbolo é uma linha ou uma seta.

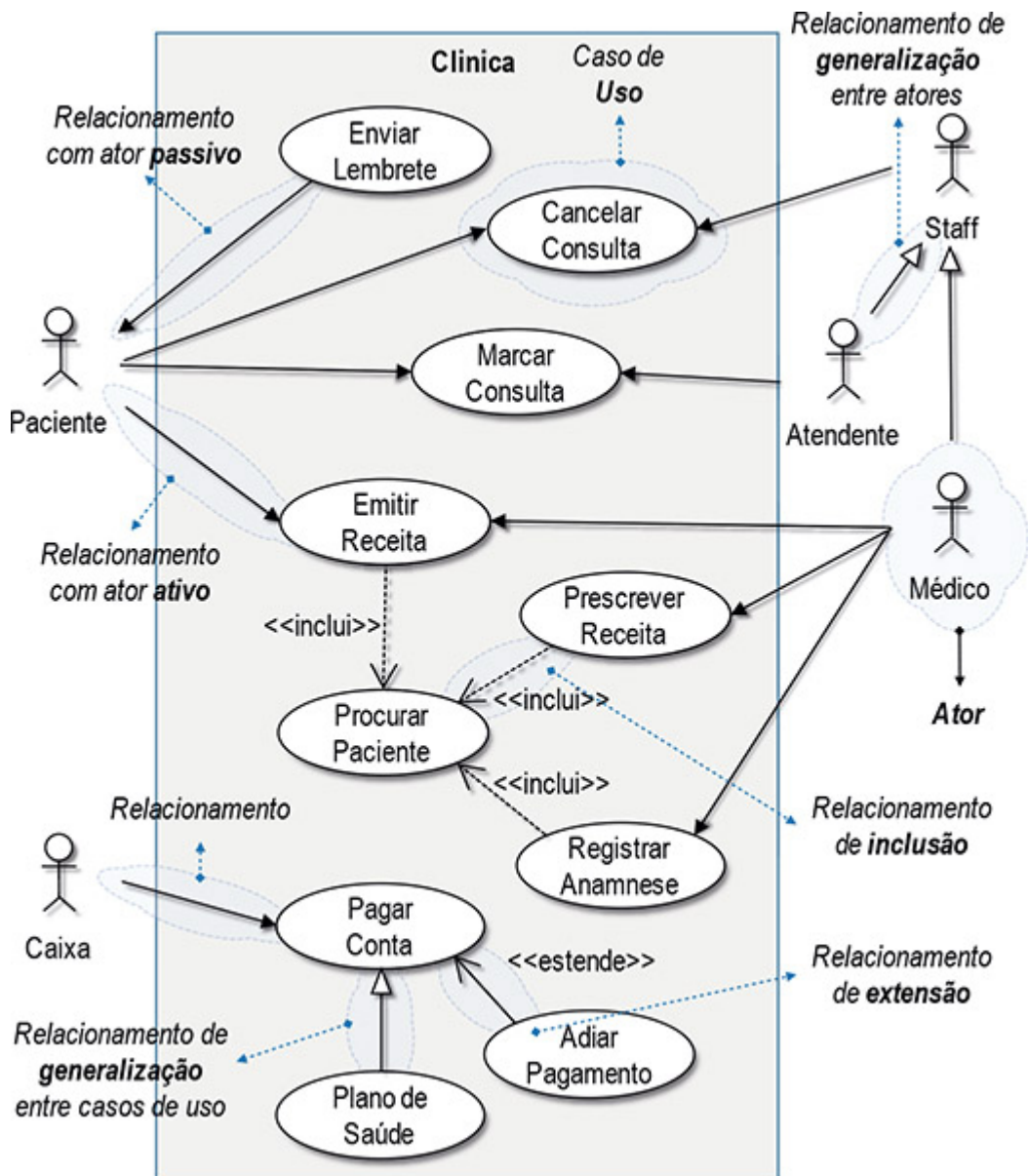


Figura 8.19. Diagrama de casos de uso e seus elementos.

8.12.2.1. Associação entre um ator e o caso de uso

A associação entre um ator e um caso de uso é denominada relacionamento de comunicação. Nessa relação, distinguem-se: ➤

Ator ativo: quando este inicia (ou dispara) a execução do caso de uso. A seta no relacionamento de comunicação aponta para o caso de uso, como no diagrama da Figura 8.19, onde o Paciente inicia o caso de uso “Marcar Consulta”. Quando a linha do relacionamento não possui seta, convencionam-se que o ator é ativo.



Ator passivo: quando o caso de uso não é iniciado pelo ator e ele reage a uma provocação do software que inicia a comunicação. A seta no relacionamento de comunicação aponta para o ator. Exemplos comuns são notificações e alarmes, como no diagrama da Figura 8.19, onde o Paciente é instado a agir pelo caso de uso “Enviar Lembrete”.

8.12.2.2. Outros tipos de relacionamento

Existem tipos especiais de relacionamentos entre os elementos de um diagrama de casos de uso. São eles: ➤

Generalização ou especialização.



Extensão.



Inclusão.

8.12.2.2.1. Generalização ou especialização A condição para identificar um caso de uso de generalização é haver um comportamento e uma finalidade comuns compartilhados por diferentes casos de uso que se especializam em um comportamento específico. O caso de uso de generalização tem o objetivo de prover uma descrição unificada desse comportamento comum em vez de tê-lo descrito várias vezes. Ao estabelecer um relacionamento entre um caso de uso de comportamento genérico e outro que se especializa nesse comportamento, diz-se que este último herda o comportamento descrito pelo primeiro.

No diagrama esse tipo de relacionamento é representado por uma seta de ponta fechada partindo do caso de uso de especialização em direção ao caso de uso de generalização.

8.12.2.2.2. Extensão O caso de uso estendido representa um comportamento opcional que o caso de uso que o estende possui.

Por exemplo, o comportamento associado ao adiar um pagamento é compartilhado como parte de um cenário alternativo de diferentes casos de uso; então ele é identificado e descrito como um caso de uso em si e um relacionamento de extensão é estabelecido entre ele e os demais estendidos.

No diagrama, um relacionamento de extensão é representado como uma seta pontilhada que se origina no caso de uso de extensão em direção ao caso de uso base.

8.12.2.2.3. Inclusão Se há uma sequência de passos comuns e de observação obrigatória em diferentes cenários, então ela deve ser descrita como uma unidade em um caso de uso de inclusão especificamente para esse fim. O caso de uso incluído representa um comportamento obrigatório que o caso de uso que o inclui deve possuir.

Por exemplo, tanto para marcar consulta quanto para pedir remédio há uma série de passos comuns para procurar o registro do paciente.

No diagrama, o relacionamento de inclusão é representado com uma seta pontilhada que se origina no caso de uso base em direção ao caso de uso incluído.

Os relacionamentos de extensão e inclusão só podem ser usados entre casos de uso. A generalização ou especialização pode ser usada entre dois casos de uso ou entre dois atores, mas não entre um ator e um caso de uso.

8.12.3. Especificação dos casos de uso

Sobre o detalhamento do caso de uso, cada um deve ter o seu comportamento descrito em um documento, como parte de um documento, ou em uma ferramenta de gerência de requisitos. A UML não estabelece um padrão para esse fim. Deve-se estabelecer um modelo que melhor atenda às características do contexto em que o método será utilizado.

Independentemente de um modelo em particular, a especificação deve atender às necessidades de informação relacionadas a seguir: ➤

Nome do caso de uso.



Breve descrição com o resumo lógico do comportamento do caso de uso.



Os atores que interagem com o software em análise.



As pré-condições necessárias ao início do caso de uso e as pós-condições esperadas ao seu término.



A sequência de passos que descreve o fluxo principal com o intercâmbio de informação entre o usuário e o software e que descreve os requisitos de armazenamento associados.



Diferentes sequências de passos organizadas em cenários que descrevem fluxos alternativos e de exceção.



A descrição das regras de negócio que se aplicam, ou, preferencialmente, a referência às regras de negócio descritas em um espaço especificamente para esse fim (já que as regras de negócio são potencialmente compartilhadas, pois se aplicam a diferentes passos de diferentes cenários e casos de uso).

8.12.3.1. Cenários

A especificação de um caso de uso é, basicamente, a descrição dos cenários que o compõem. Um cenário corresponde a diferentes passos que se desdobram a partir de um evento que inicia o caso de uso e das condições que afetam como ele deve se comportar.

A descrição de um cenário explora:



como e quando um caso de uso começa; ➤
quando o caso de uso interage com os atores e quais dados eles trocam entre si; ➤
quando o caso de uso referencia ou mantém dados; ➤
como e quando o caso de uso termina.

A descrição de um cenário não aborda aspectos da interface gráfica com o usuário, plataforma de hardware ou software e qualquer requisito não funcional.

O termo “fluxo” também é usado para designar um cenário. Uma diferença entre “fluxo” e “cenário” é que os cenários são subdivididos em dois tipos, enquanto os fluxos são subdivididos em três tipos. São eles: ➤

Principal: descrição que reflete um único cenário para atingir o objetivo do caso de uso, sem qualquer consideração com possíveis falhas. É o “caminho feliz”, curso típico de eventos, fluxo normal, fluxo básico.



Alternativo: cumpre o papel de complementar o cenário principal com fragmentos de fluxos que apresentam o comportamento esperado sob condições alternativas que podem ocorrer no decorrer do cenário principal.



Exceção: no caso de uso do termo “fluxo”, há um tipo de cenário alternativo denominado fluxo de exceção especificamente para descrever o comportamento associado às condições de exceção.

8.12.3.2. Como identificar um caso de uso?

A identificação de um caso de uso está intimamente relacionada às decisões sobre quais tarefas do usuário devem ser transferidas para o software. Cada uma delas deve ser mapeada para um caso de uso e os diferentes cenários

associados descritos. Ao elaborar essa descrição, identificam-se oportunidades de melhorar a qualidade da especificação por meio de novos casos de uso, agora de inclusão ou extensão, conforme o que já foi discutido.

8.12.3.3. Como descrever um caso de uso?

A descrição do caso de uso deve prever os cenários relativos à avaliação das condições que se apliquem; porém, sem descrever as regras de negócio associadas. Isso deve ser especificado à parte. Se uma ferramenta de gerência de requisitos é utilizada, essa separação se torna irrelevante, já que a especificação do caso de uso passa a ser um simples relatório e não o repositório centralizado da informação.

8.12.4. Por que casos de uso?

Com a informação organizada em um modelo funcional baseado em casos de uso, é fornecida informação para trabalhar em três áreas muito importantes nos projetos: ➤

Definição de requisitos: geralmente novos requisitos geram novos casos de uso conforme o sistema vai sendo analisado e modelado.



Comunicação com os clientes: por ser simples, sua compreensão não exige conhecimentos técnicos. O cliente pode entender facilmente tanto o diagrama quanto a especificação, o que facilita a comunicação da equipe técnica com os clientes.



Geração de casos de teste: a junção de todos os cenários para um caso de uso pode sugerir uma bateria de testes para cada cenário.

8.13. Técnica: listas de verificação

O uso de lista de verificação ou *checklist* tem grande apelo como ferramenta de segurança no trabalho. Um caso que ilustra isso surgiu durante a fase final de avaliação das aeronaves especificadas pelo exército dos EUA em 18 de julho de 1934. Três fábricas tinham apresentado aeronaves para testes. Um problema foi encontrado e uma das aeronaves parou de funcionar durante a subida, explodindo em chamas no momento do impacto. Uma investigação foi feita e descobriu-se que a causa do problema foi um erro da pilotagem. O copiloto havia se esquecido de liberar a trava do elevador antes da decolagem. Uma vez no ar, o piloto percebeu o que estava acontecendo e tentou alcançar a maçaneta de bloqueio do elevador, mas já era tarde demais.

Para que problemas como esse não se repetissem, pilotos se uniram e concluíram que era necessária alguma maneira de ter certeza de que tudo foi feito. Quatro listas de verificação foram desenvolvidas: decolagem, voo, antes do pouso e após o desembarque. Os pilotos perceberam que o problema aconteceu devido ao sistema de aviação ser complexo para a memória de um ser humano. As listas de verificação para o piloto e o copiloto garantiram que nada seria esquecido. Com as listas de verificação, um planejamento cuidadoso e rigoroso treinamento, a aeronave conseguiu voar 1.800.000 milhas sem um acidente sequer.

A ideia de lista de verificação do piloto foi bem aceita e mais listas foram desenvolvidas para outros membros da tripulação. Assim, o uso de listas de verificação passou a ser disseminada na aviação.

No contexto da Engenharia de Requisitos, o uso dessas listas nas atividades de verificação e validação é também muito útil.

8.13.1. O que são listas de verificação?

As equipes de projeto ou os responsáveis pelas políticas gerais de qualidade em um nível mais elevado na organização determinam critérios para uso como referência de qualidade no processo de desenvolvimento de software. Eles indicam os elementos críticos e definem as condições que devem se confirmar ao se avaliar uma especificação ou modelo. Os tipos de especificação e modelos, e as condições-alvo associadas, refletem lições aprendidas com o sucesso – ou o fracasso – em projetos passados e com

boas práticas de mercado. O critério de qualidade indica quais as condições para o elemento ser avaliado como correto ou errado, como adequado para ser usado em outras oportunidades ou que requeira maior atenção ainda em sua elaboração.

Por exemplo, um documento de visão é uma especificação que descreve o domínio do problema, as necessidades de negócio, o escopo da solução em termos de seus macroprocessos constituintes e de seu ambiente externo qualificado pelas organizações, pessoas e outros agentes que interagem com o software como parte das responsabilidades associadas ao papel que desempenham. A definição desses elementos deve fazer com que o documento de visão cumpra adequadamente o seu propósito. E essa definição pode ter sido fruto de experiência em projetos internos ou boas práticas consolidadas no mercado. Por exemplo, um elemento crítico nessa especificação é a caracterização do papel que os agentes citados desempenham. E um critério de qualidade para verificar isso é haver, organizada na especificação, informação que descreva o conjunto de habilidades e conhecimentos necessários, indicando também responsabilidades e relacionamentos com outros agentes que desempenham outros papéis.

Uma lista de verificação organiza a referência descrita anteriormente na forma de perguntas ou assertivas encadeadas sequencialmente. Com base nelas, produtos de trabalho individuais podem ser avaliados pela comparação da condição em que se encontram com os critérios de qualidade presentes no modelo de referência.

Cada tipo de produto de trabalho da Engenharia de Requisitos deveria ter uma lista de verificação associada.

8.13.1.1. Como usar listas de verificação?

As listas de verificação são usadas durante revisões técnicas formais.

Tipo de produto de trabalho: **Diagrama de Contexto** Elemento: **requisitos de armazenamento externos sobre um conceito de negócio 02.DT.30 – Verificar se a identificação do conceito de negócio está correta** A identificação de um conceito de negócio sobre o qual se recuperam ou armazenam dados é CORRETA quando: - o elemento identificado observa a definição de conceito de negócio (02.DT.29); - há fundamentos para sua identificação nos documentos com a memória da elicitação que devem permitir identificar as necessidades

de armazenamento do usuário a partir de manifestações espontâneas dos envolvidos, ou como resultado da aplicação de técnicas de elicitação por parte da equipe de desenvolvimento; - há fundamentos nas especificações com os requisitos funcionais (se apropriado, conforme a fase).

02.DT.32 – Verificar se o conceito de negócio é externo à solução proposta Um conceito de negócio é EXTERNO À SOLUÇÃO PROPOSTA quando: - não há requisito funcional para a manutenção de dados sobre o conceito de negócio no âmbito da solução proposta; e - há necessariamente e apenas passos ou regras de negócio em requisitos funcionais que recuperam dados sobre o conceito de negócio. Não se trata especificamente de requisitos funcionais descrevendo como a solução deve responder a um evento externo, nem requisitos funcionais descrevendo o envio de dados para integração com outras soluções.

02.DT.33 – Verificar se o conceito de negócio é necessário O conceito de negócio é NECESSÁRIO quando: - estabelece a capacidade de o negócio precisar de resoluções internas; ou - permite conformidade do negócio a exigências, interfaces ou padrões externos. O que evidencia essas condições é o campo “Necessidades Abordadas (N.XX)” ser preenchido com todos os itens do Quadro de Necessidades e Oportunidades que o justificam e apenas esses itens.

02.DT.34 – Verificar se o Quadro de Requisitos de Armazenamento (Externos) está completo O Quadro de Requisitos de Armazenamento – Externos à Solução Proposta – está COMPLETO quando todos os conceitos de negócios a seguir qualificados estejam relacionados e apenas eles: - CORRETOS (02.DT.30); e - EXTERNOS À SOLUÇÃO PROPOSTA (02.DT.32).

Quadro 8.1. Exemplo completo de política de verificação.

Os tipos de produto de trabalho deveriam estar organizados em modelos para facilitar o desenvolvimento. A Figura 8.20 ilustra um fragmento do modelo que aplica a lista de verificação usada como exemplo. Mas atenção: o uso de modelos como o apresentado, sem o entendimento adequado de seu propósito e do que as informações nele presentes representam, traz mais prejuízos que vantagens.

Quadro de Conceitos de Negócio (Assuntos)

Nome

CN.01

Necessidades de Negócio (N.XX)

Fundamentos (ML.XX)

Necessidade de manutenção em outra aplicação?	Em caso positivo, quais aplicações e OOSS
Não ☐ Sim ☐	

Figura 8.20. Fragmento com modelo descrevendo as necessidades de informação.

8.13.2. Por que listas de verificação?

A motivação para usar listas de verificação é facilitar o controle da qualidade para os produtos de trabalho entregues pelo projeto, mais especificamente: ➤

Garantir que as especificações sejam desenvolvidas e verificadas de maneira uniforme.

☐

Descobrir inconsistências entre diferentes modelos.

☐

Encontrar especificações supérfluas para o atendimento dos requisitos de negócio.

☐

Evitar que especificações incompletas avancem no projeto.

☐

Desenvolver projetos mais gerenciáveis.

As listas de verificação ajudam a identificar o que foi feito e em que qualidade, de forma objetiva. Sem o apoio de uma lista de verificação, o trabalho dependeria da experiência e do critério usado por cada um responsável por verificar, o que pode significar uma verificação não padronizada. Um mesmo artefato verificado de maneira *ad hoc* por diferentes pessoas pode levar a resultados distintos, dependendo de quem

verifica. Com a lista de verificação, até o membro menos experiente da equipe pode executar atividades de verificação.

A lista de verificação deveria ser elaborada aproveitando a experiência dos profissionais mais antigos, com olhar mais criterioso e com vivência em vários problemas que poderiam ser detectados pela lista. À medida que problemas forem detectados no projeto, deve-se revisar a lista de verificação para avaliar se algum item pode ser acrescentado ou modificado, para filtrar previamente problemas similares em futuros projetos.

Logo, a lista de verificação ajuda também a diminuir o potencial de falhas em projetos, economiza tempo nas atividades de verificação e permite o compartilhamento de conhecimento e experiência entre os membros da equipe.

8.14. Técnica: prototipação

Estabelecer a proposta de interface gráfica do software com o usuário é uma responsabilidade da disciplina de Análise e Projeto (arquitetura). Um termo de uso mais abrangente denominado “UX” (*User Experience*) tem sido cada vez mais usado e se consolida como especialização profissional, considerando também a integração com elementos de hardware e serviços. A prototipação é fundamental no desenvolvimento da UX de um produto de software... mas não é esse uso que se discute aqui. O enfoque é para a validação e descoberta de requisitos.

A análise de requisitos é responsável por dar orientação na identificação e definição dos elementos lógicos da interface com o usuário – os campos com as informações que devem ser fornecidas pelos usuários, o que deve ser apresentado ao usuário, a estrutura de ambos quanto a tamanho, tipos e regras de validação ou formatação que compõem esses elementos lógicos.

De maneira similar às decisões sobre questões de arquitetura de alto risco, que devem ser abordadas em paralelo ao desenvolvimento do software, é possível haver paralelismo entre a realização da Engenharia de Requisitos e a UX. Enquanto se detalham os requisitos funcionais, tomam-se decisões sobre a UX.

No primeiro caso, quando se discutem as decisões sobre questões de alto risco, utiliza-se o termo “deve” e, no segundo, sobre a experiência do usuário associada ao detalhamento de cada requisito, se estabelece uma possibilidade. Se essas questões de arquitetura de alto risco não forem abordadas logo no início do projeto, então todo o trabalho subsequente é comprometido, inclusive o da Engenharia de Requisitos. Essas decisões estabelecem limites entre o que será especificado como um requisito funcional ou não funcional durante a análise de requisitos e direciona o desenvolvimento subsequente de diversas especialidades. Por exemplo: ➤

Requisitos: o comportamento relativo ao armazenamento de dados com uma trilha de auditoria (*log*) ser especificado como passos em diferentes especificações funcionais ou estabelecido uma única vez em uma especificação complementar de forma geral depende de decisões quanto à arquitetura de software a ser utilizada e os seus componentes de infraestrutura com serviços comuns compartilhados.



Análise e projeto: a alocação de determinado comportamento especificado nos requisitos em uma unidade de software (programa, classe em Java, pacote PL/SQL etc.) depende de decisões prévias sobre a arquitetura.



Implementação: a linguagem de programação depende dessas decisões preliminares de arquitetura.

Já quando se discutem decisões sobre a experiência do usuário associada a cada requisito funcional, trata-se de uma escolha que depende da estratégia de desenvolvimento. É viável que todos os elementos lógicos da interface com o usuário sejam identificados e descritos para, depois, se decidir quanto à representação da informação como um campo de texto, um ícone, uma árvore, o modo para a entrada e validação de dados, quantos formulários deverão suportar o requisito funcional *etc.* A Engenharia de

Requisitos deve estabelecer os requisitos não funcionais da solução; porém, sem entrar nesse mérito.

Da mesma forma, é viável tomar ambas as decisões – sobre os elementos lógicos e a UX – em um mesmo momento, seja por um único perfil profissional que tenha as habilidades associadas à Engenharia de Requisitos e à UX ou por dois perfis profissionais diferentes trabalhando em conjunto.

Um fato que deve ser considerado é que a volatilidade sobre as decisões de UX é maior que a volatilidade associada às necessidades de informação. Trabalhar desenvolvendo os dois concomitantemente requer bastante disciplina para não negligenciar um deles.

8.14.1. O que é prototipação?

A prototipação é uma técnica que busca simular para o usuário o funcionamento dos seus requisitos antes que o produto final esteja pronto. É um processo iterativo onde se geram versões iniciais de protótipos, e por meio deles o usuário poderá analisar se os seus requisitos estão sendo atendidos e até mesmo descobrir novos requisitos. Também é utilizada para melhorar a experiência do usuário, avaliar opções de *design* e montar uma base para o desenvolvimento final do produto.

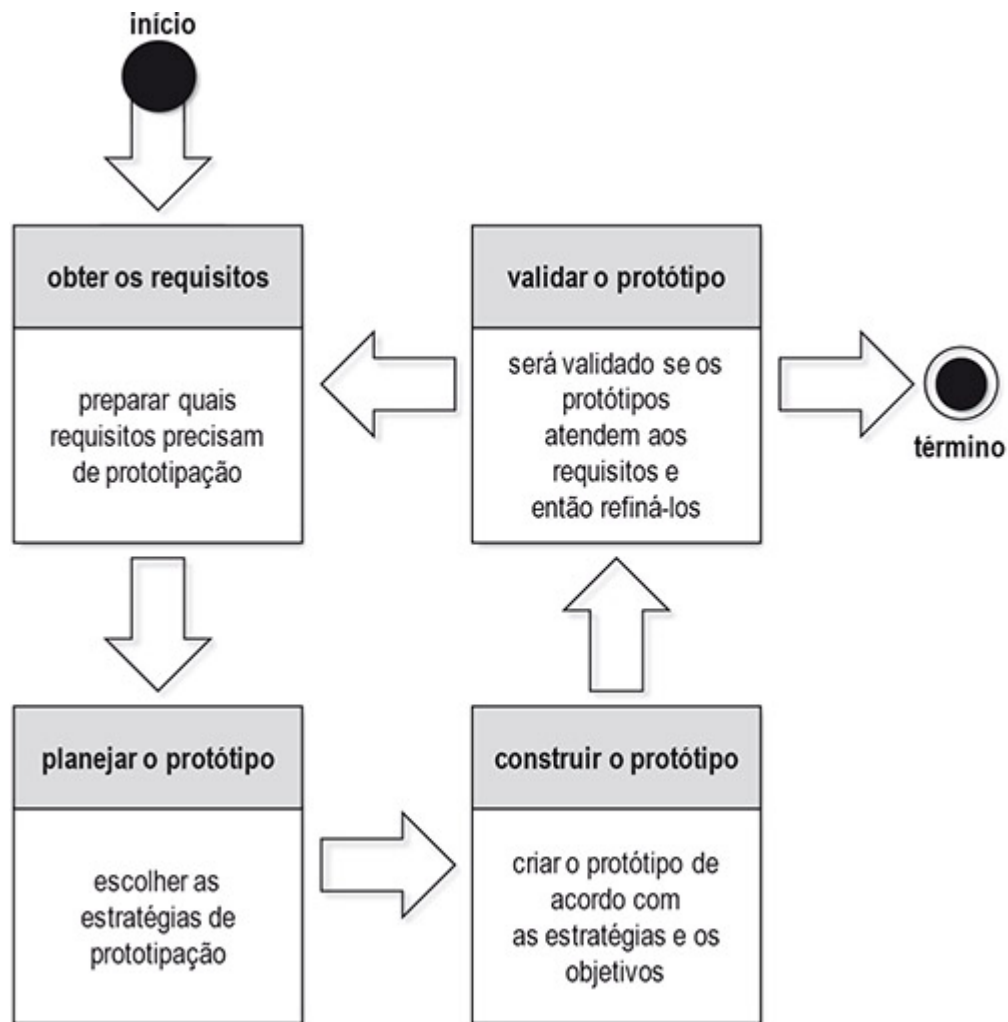


Figura 8.21. Ciclo da prototipação.

8.14.2. Como aplicar a prototipação?

O ciclo de uma prototipação começa na obtenção dos requisitos, passa pelo planejamento de um protótipo, onde são decididas qual técnica e qual abordagem de prototipação utilizar, para então construir um protótipo. Após a construção do protótipo, o usuário e o desenvolvedor irão utilizá-lo para refinar os requisitos. Feito isso, o processo se inicia novamente para uma nova prototipação, até que os requisitos todos estejam maduros para o desenvolvimento.

Diferentes técnicas para a prototipação são apresentadas adiante. A escolha de quais abordagens utilizar deve ser feita com cuidado, condizendo com o objetivo da prototipação, pois estas influenciarão na produtividade e no

custo do seu projeto. Escolher uma abordagem errada para o seu objetivo pode prejudicar o desenvolvimento do seu projeto em vez de ajudá-lo.

Por exemplo, se o objetivo for refinar requisitos funcionais, então não convém elaborar um protótipo com a estética do produto final, incluindo outros elementos além dos elementos lógicos necessários para o objetivo citado. Exceções à regra seriam os casos em que sem esse artifício não se consegue estabelecer a comunicação com uma parte interessada que não tem a capacidade de abstração mínima para entender uma interface apenas com os elementos lógicos, não permitindo o avanço sem a inclusão de outros elementos de usabilidade.

Feita a prototipação, o desenvolvedor e o cliente devem se reunir e avaliar se os protótipos construídos estão de acordo com os requisitos. O *feedback* com essas lacunas é o produto de maior valor da prototipação. Ele servirá de insumo para que os requisitos possam ser mais refinados. Se na primeira sessão de validação do protótipo não forem identificados problemas em requisitos, desconfie e avalie uma nova abordagem na prototipação para chegar ao resultado esperado. Ou avalie se a pessoa designada é a correta para validar o protótipo.

Terminando toda a análise da prototipação, os requisitos devem ser melhorados conforme o *feedback* gerado e, caso necessário, deve-se iniciar o processo de prototipação novamente, de forma que novos problemas sejam encontrados e novas soluções sejam estabelecidas. A cada prototipação, a ideia é que o nível de detalhe e a precisão dos protótipos aumentem até que todas as incertezas dos requisitos sejam solucionadas, para então poder ser construído o produto final.

8.14.2.1. Prototipação de baixa fidelidade × alta fidelidade

A prototipação é classificada como sendo de baixa fidelidade ou de alta fidelidade conforme o nível de aparência do protótipo em comparação ao produto final.

8.14.2.1.1. Prototipação de baixa fidelidade A prototipação de baixa fidelidade é a mais simples em termos de esforço, custo e tempo investido em sua elaboração. É feita em momentos iniciais do projeto. Os protótipos

de baixa fidelidade geralmente são desenhados na forma de esboços e sem muitos detalhes; não há preocupação prioritária com características estéticas. Se houve algo nesse sentido, então os resultados sobre esses elementos não devem ser tratados como uma linha de base para a gerência de mudanças, pois o objetivo está na descoberta dos elementos lógicos da interface com o usuário. Os protótipos criados nesse tipo de prototipação são também denominados *mockups* ou *wireframes*.

Esse tipo de prototipação impede que o usuário confunda o protótipo com um produto final. Esse estilo de protótipo influencia o interesse do usuário; o foco está apenas nas funcionalidades do sistema, entendendo que o protótipo é apenas um esboço que pode ser alterado com facilidade. Isso é positivo, pois não há risco de desvio da atenção do usuário para a parte estética. Uma desvantagem na falta de detalhes é a dificuldade que algumas pessoas têm de abstrair dos atributos adicionais aos elementos lógicos. A Figura 8.22 ilustra um protótipo de baixa fidelidade.

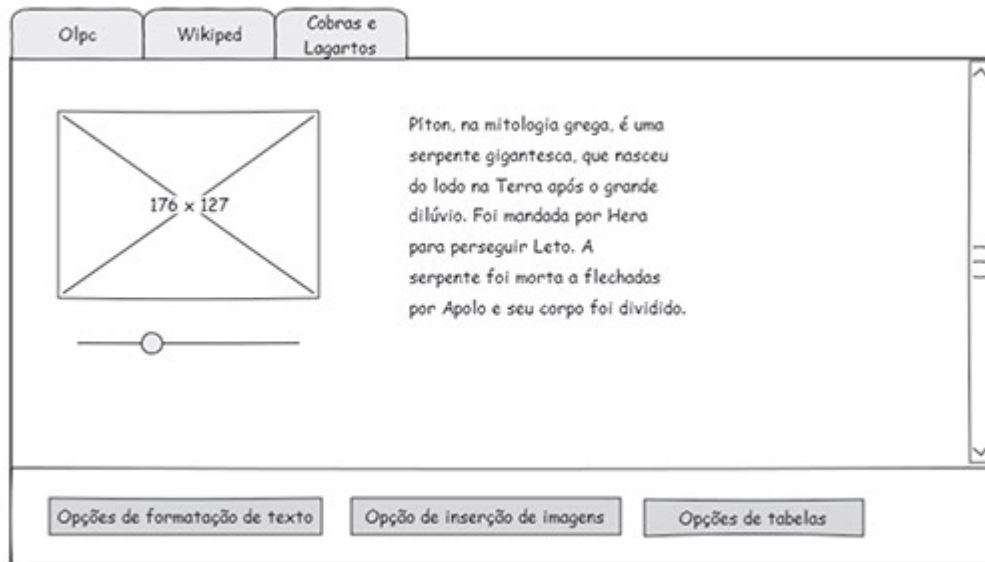


Figura 8.22. Exemplo de protótipo de baixa fidelidade.

8.14.2.1.2. Prototipação de alta fidelidade A prototipação de alta fidelidade apresenta um nível alto de detalhes do ponto de vista da aparência. O usuário deve ser capaz de enxergar do modo mais realista possível como será o produto final. Esses protótipos podem ser construídos usando a própria ferramenta de desenvolvimento de software ou ferramentas de prototipação que permitem simular a experiência de uso do

software sem que haja dados reais disponíveis ou código-fonte programado para esse fim. Essas últimas ferramentas, normalmente, têm o seu comportamento especificado por meio de configuração e, eventualmente, por *scripts* em linguagens de alto nível.

A prototipação de alta fidelidade é ótima para descobrir novos requisitos de usabilidade e outros requisitos não funcionais, porém o usuário pode se distrair e não validar as funcionalidades e seus elementos lógicos. Há o risco de o usuário criar expectativas ao pensar que o protótipo será (ou é) o produto final do software. É uma boa técnica para a prototipação de produtos destinados ao mercado em massa e como ferramenta para estabelecer os requisitos de UX para o produto. Uma desvantagem desta abordagem é que se demora mais para elaborar um protótipo de alta fidelidade do que um de baixa fidelidade.

8.14.2.2. Prototipação horizontal × vertical

Elaborar um protótipo completo para um sistema grande pode levar bastante tempo, e a demora diminui os benefícios da prototipação. Para evitar isso, é comum optar por elaborar protótipos parciais, buscando oferecer ou uma visão ampla (mas rasa) do produto ou uma visão profunda (mas específica) de uma parte do produto. Escolher entre essas duas abordagens requer avaliar o propósito que se deseja alcançar com a prototipação: consolidar o escopo ou detalhar os elementos já presentes no escopo.

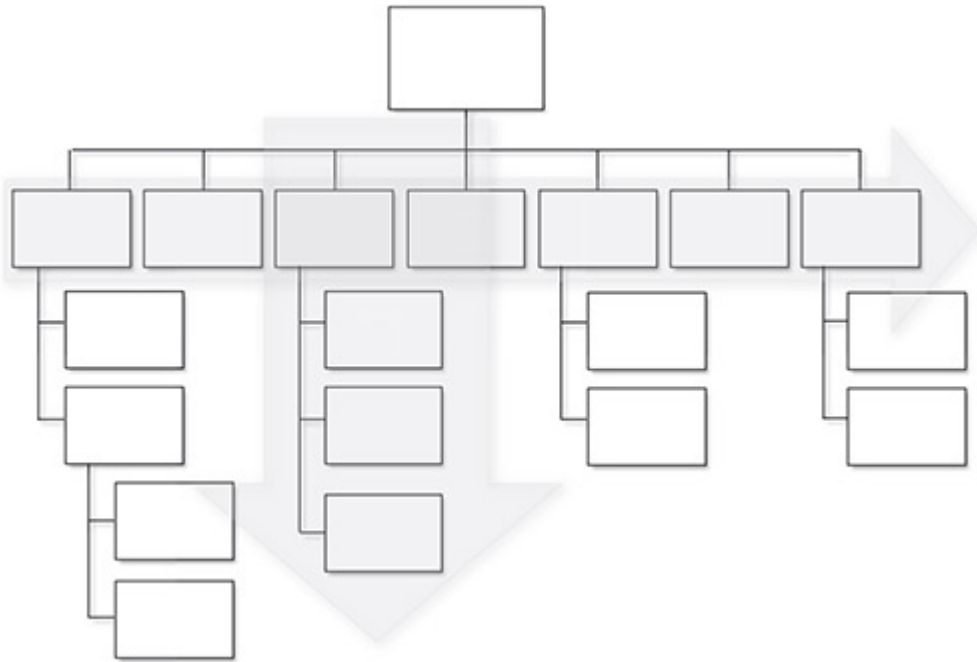


Figura 8.23. Protótipo horizontal x vertical.

8.14.2.2.1. Prototipação horizontal A prototipação horizontal aborda os requisitos do usuário em menor profundidade e busca cobrir de forma ampla várias funcionalidades, porém não se aprofunda no seu funcionamento. Por entrar em menor nível de detalhe, um maior conjunto de funcionalidades pode ser abordado em uma única sessão. Por abordar a abrangência para uma solução, a prototipação horizontal é adequada para as etapas iniciais do projeto. A vantagem desta abordagem é mostrar ao usuário as revelações que o desenvolvimento produziu até o momento sobre o funcionamento geral do sistema.

8.14.2.2.2. Prototipação vertical A prototipação vertical tem por objetivo criar protótipos que exploram as funcionalidades em maior profundidade e permitem explorar um menor número de funcionalidades em uma mesma sessão de prototipação, em comparação aos protótipos mais simplórios de um conjunto maior de funcionalidades na prototipação horizontal.

Esses protótipos são mais adequados em momentos adiantados do desenvolvimento e podem ajudar no refinamento e na avaliação dos requisitos do usuário. A vantagem desta abordagem é demonstrar requisitos específicos e aprofundar o entendimento dos detalhes de cada funcionalidade.

8.14.2.3. Prototipação descartável x evolutiva

Quanto ao ciclo de vida, os protótipos podem ser classificados em descartáveis ou evolutivos.

8.14.2.3.1. Prototipação evolutiva A prototipação evolutiva é a abordagem na qual os protótipos serão evoluídos até se tornarem prontos para compor a versão final do software. São elaborados com a própria ferramenta de desenvolvimento de software. Este é o caso onde o protótipo construído irá sofrer constante evolução até se tornar o resultado final do desenvolvimento. Portanto, ao se desenvolver o protótipo, está também se desenvolvendo o software. Seu ciclo de vida não se encerra com o término das atividades de prototipação. É a abordagem preferida para os projetos desenvolvidos com a metodologia “ágil”. No entanto, a prototipação evolutiva pode demandar a antecipação de questões relativas a disciplinas de *design* e construção, para que se tenha uma arquitetura do software preparada para suportar as sucessivas evoluções do protótipo até o produto final. Isso pode provocar atraso para obter a primeira versão do protótipo. Quando já existe previamente uma arquitetura definida para se usar, então pode ser rápido gerar a primeira versão do protótipo.

8.14.2.3.2. Prototipação descartável O protótipo construído desta forma, após cumprir seu objetivo, será descartado. Ao contrário da prototipação evolutiva, os protótipos não serão itens de configuração para a versão final do software. Os protótipos descartáveis geralmente são feitos com ferramentas próprias para desenho de protótipos. Seu ciclo de vida se encerra depois que se deixa de prototipar no projeto. Em geral são mais rápidos e baratos de produzir.

8.14.2.4. Riscos e cuidados

Ao realizar a prototipação, é importante definir bem com o cliente o objetivo pelo qual o protótipo foi criado. Se esse objetivo não for definido adequadamente, a prototipação pode ter impacto negativo. Seguem alguns cuidados a serem tomados ao realizar a prototipação: ➤

Certifique-se de que os usuários e os desenvolvedores não fiquem muito presos ao protótipo. Se o protótipo estiver bem detalhado e os objetivos com a prototipação não estão claros, então o usuário pode

criar a expectativa de que o sistema já está pronto ou que ficará exatamente igual ao protótipo, quando, dependendo dos objetivos da prototipação, ainda há decisões a serem tomadas sobre a interface de usuário.



Certifique-se de que, quando o usuário aprovar um protótipo de baixa fidelidade para validar os requisitos, os desenvolvedores não considerem o protótipo como o resultado final que descreve toda a UX prevista para o produto. O trabalho para descobrir esses elementos é escopo de atividade desses desenvolvedores, e eles não devem implementar uma tela sem considerar boas práticas de UX só porque o cliente aprovou um protótipo de baixa fidelidade com o objetivo de identificar os elementos lógicos da interface com o usuário.



Certifique-se de que, em uma sessão de prototipação para validar requisitos funcionais, o usuário não se atenha à UX, pois se isso acontecer, tempo de projeto será desperdiçado e serão criadas expectativas erradas. Uma solução é usar um protótipo de baixa fidelidade; assim o cliente tende a acreditar que o sistema final não terá essa aparência rústica ou má usabilidade.



Certifique-se de esclarecer que os tempos instantâneos obtidos na experiência de uso de um protótipo não devem ser comparados aos produtos finais. Como um protótipo é apenas uma ferramenta de simulação de como o sistema ficará quando pronto, não possuindo assim processamento real associado a ele, deve-se ter o cuidado para que o usuário não compare a velocidade no desempenho de um protótipo com o desempenho do produto final.

8.14.2.5. Protótipo x caso de uso

Um protótipo não possui especificação das funcionalidades apresentadas. Sem o conhecimento dos requisitos, quem olhar o protótipo poderá não saber o que as funcionalidades apresentadas fazem. Por isso, a combinação entre protótipo e especificações de casos de uso é muito interessante. O caso de uso documentará o fluxo e o funcionamento das funcionalidades, e estas serão demonstradas pelo protótipo. A junção protótipo e caso de uso também é uma ótima forma de documentar seu sistema para retenção do conhecimento sem a necessidade de aprender a ler alguma linguagem de programação e interpretar o fracionamento dos requisitos em diferentes itens de configuração do software.

8.14.3. Por que prototipação?

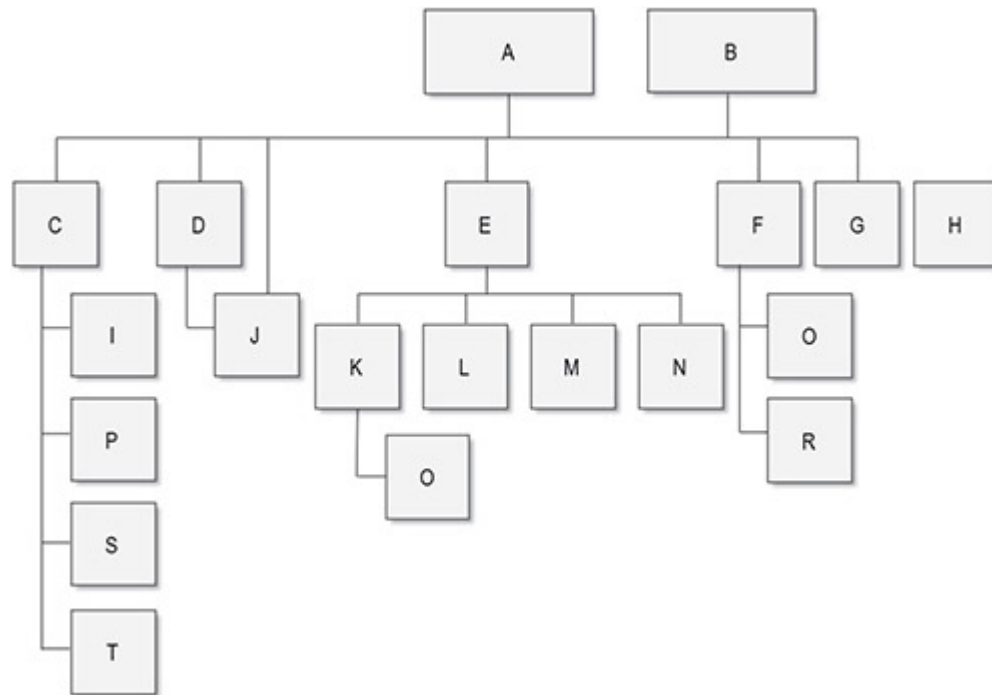
A prototipação tem como objetivo principal diminuir os riscos do projeto, permitindo a descoberta de problemas de requisitos em etapas iniciais que talvez fossem difíceis de identificar com outras técnicas.

A validação de um protótipo expõe mal-entendidos entre os envolvidos no projeto, assim como traz à tona os requisitos implícitos (ou “óbvios”). No primeiro caso, é útil tanto para identificar problemas de comunicação entre partes interessadas e equipe de requisitos quanto para ajudar a solucionar conflitos de visão entre as partes interessadas sobre qual melhor opção de solução para o produto. No segundo caso, dos requisitos implícitos, ao perceber durante a validação do protótipo que este não possui uma característica esperada, mas que não foi solicitada de forma explícita, o usuário irá apontar e reclamar.

Quanto mais cedo se introduz o protótipo no projeto, maior o potencial de benefício que pode ser obtido. Se a prototipação está sendo feita em etapas avançadas do projeto ou se está consumindo muito tempo, é sinal de que algo não está bem e os benefícios potenciais da prototipação podem se perder. Embora seja uma prática comum em muitas empresas, é frequente cometer o “pecado” de deixar para prototipar no projeto de forma tardia.

8.15. Exercícios

1. Qual é o objetivo da Análise de Requisitos?
2. A figura a seguir representa a decomposição funcional de um sistema genérico. Verifique se a decomposição foi feita corretamente. Caso encontre problemas, liste-os.



3. Ao fazer a decomposição dos requisitos em um modelo de processos, foram encontradas algumas atividades sem nenhum requisito alocado. O que isso significa?
4. Utilizando o estudo de caso do Anexo como referência, identifique os seus requisitos de armazenamento (conceitos de negócio). Registre também as dúvidas que surgirem nesse processo. Estas demandarão novas atividades de elicitação. Você pode usar a lista a seguir para ajudá-lo nessa identificação. Procure buscar por conceitos de negócio relacionados a:
 - a) documentos tramitados e cujos dados são informados, transformados, armazenados e distribuídos em seu teor bruto e/ou com informações derivadas a partir deles;
 - b) pessoas representando os diferentes papéis que precisam interagir;
 - c)

eventos que demarcam diferentes pontos no processo de negócio sendo abordado ou diferentes resultados entregues em marcos;

d)

manifestações que provocam a ação e o acompanhamento dessa ação; e)

estruturas organizacionais com suas alçadas e competências; f)

parâmetros operacionais que limitam e estabelecem políticas; g)

estruturas físicas com a sua descrição e qualificação, como prédios, plantas, fábricas, produtos, depósitos, localizações, equipamentos *etc.*

8. Utilizando o estudo de caso do Anexo como referência, elabore um diagrama de casos de uso que represente o escopo deste projeto. Registre também as dúvidas que surgirem nesse processo. Estas demandarão novas atividades de elicitação.
9. Utilizando o estudo de caso do Anexo como referência, especifique duas histórias do usuário. Elabore-as de acordo com o formato sugerido na lição: “como (papel), eu quero (algo) para (me beneficiar)”.

9. Gerência de Requisitos

“Nada é permanente, exceto a mudança.”

Heráclito Muita gente acredita que, para obter uma melhoria na gerência de requisitos do processo de software de sua empresa, há uma dependência fundamental de alguma ferramenta de gestão de requisitos. Será que essa crença é válida? Os autores presenciaram casos de empresas que gastaram (não investiram, ressalte-se) um bom dinheiro na compra de licenças de ferramentas dessa natureza, sem que isso resultasse em qualquer benefício perceptível. Seriam esses casos exceções?

A resposta será tratada ao final do capítulo. Mas, para começar a sua busca, é necessário antes recorrer a uma definição do que é a gerência de requisitos, apresentada a seguir.

9.1. Gerência de requisitos no CMMI-DEV

Para esta definição será utilizada uma referência madura e conhecida mundialmente, que é o CMMI-DEV, o modelo norte-americano de maturidade para o desenvolvimento de software (o modelo brasileiro MPS.BR é equivalente nesta definição). Este modelo define uma área de processo chamada Gerência de Requisitos (REQM) que tem por objetivo gerenciar os requisitos dos produtos do projeto e dos seus componentes e garantir o alinhamento entre os requisitos, os planos do projeto e os produtos de trabalho do projeto.

Para alcançar tais objetivos, são definidas práticas específicas a serem cumpridas:

1. **Entender os requisitos:** faz com que os requisitos sejam entendidos, avaliados e aceitos junto aos fornecedores de requisitos (partes interessadas), utilizando critérios objetivos. Para buscar o consenso entre as várias partes interessadas é comum precisar também solucionar conflitos.
2. **Obter comprometimento sobre os requisitos:** busca garantir que a equipe do projeto está comprometida em trabalhar na versão mais atual dos requisitos aprovados.
3. **Gerenciar mudanças:** garante que um processo formal de gestão de mudanças sobre os requisitos é aplicado.
4. **Manter rastreabilidade bidirecional entre requisitos e seus produtos:** a rastreabilidade ajuda na gestão do escopo e na avaliação de impacto de mudanças.
5. **Garantir alinhamento entre trabalho do projeto e requisitos:** revisões em planos e produtos de trabalho do projeto são realizadas visando identificar e corrigir inconsistências quanto aos requisitos.

Além das práticas apresentadas pelo CMMI-DEV, a gerência de requisitos envolve também: ➤

- priorizar um conjunto de requisitos antes da próxima iteração ou fase; ➤
- gerenciar questões surgidas ao longo das atividades do projeto.

9.1.1. Ciclo de vida da gerência de requisitos

De acordo com PMI (2014), apenas 20% das organizações relatam alta maturidade nas suas práticas da Engenharia de Requisitos. Isso significa que ainda há muito a se evoluir no mercado em geral quanto à difusão das melhores práticas. E uma delas tem relação com o ciclo de vida do requisito.

Um equívoco comum é achar que o requisito é útil apenas durante o projeto, e que, uma vez que este seja terminado, o objetivo do requisito foi

cumprido e este não é mais relevante. Essa crença leva muitas organizações a tratar o requisito como “vivo” e, portanto, gerenciável somente durante o projeto em que foi criado.

Antes de prosseguir com o raciocínio, convém esclarecer o conceito de projeto e produto, pois algumas vezes os autores encontraram empresas que não conseguiam distinguir entre ambos. O projeto tem um caráter temporário e visa entregar um produto ao seu final (um software novo ou melhorado). Seu ciclo de vida é mais curto que o do produto. O produto entregue pelo projeto será usado pela organização durante certo tempo. A duração do uso do produto determina o seu ciclo de vida. Ao longo dessa vida o produto de software passa por um projeto de desenvolvimento (que o cria) e pode passar por vários projetos de manutenção.

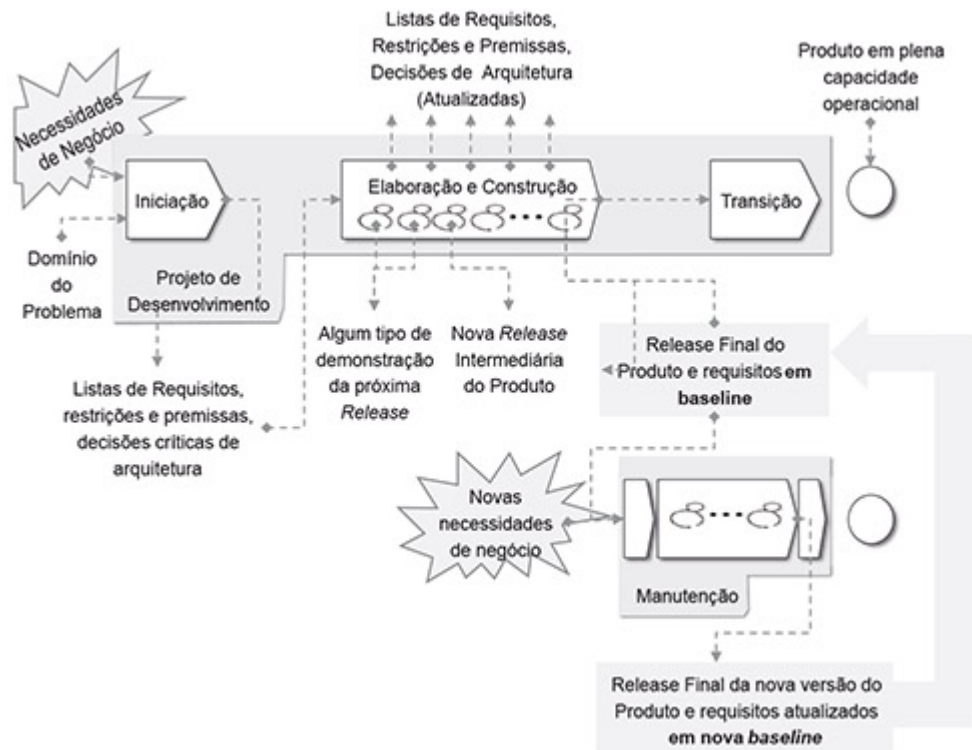


Figura 9.1. O requisito como um ativo.

Não é tão comum encontrar uma organização que implementa boas práticas da gestão de requisitos durante o projeto. E quando isso ocorre em um projeto de desenvolvimento de software, significa que o sistema que nasceu possui uma boa documentação inicial (“boa” neste caso não significa necessariamente documentação extensa). No entanto, se o requisito for

gerido só durante o projeto, com o seu término boa parte da documentação gerada vai ficando obsoleta após sucessivas manutenções. Quanto mais detalhada a documentação, mais rápida a sua obsolescência. Depois de algum tempo, este sistema se torna mais um dos sistemas legados da organização, que são difíceis de manter e poucos sabem bem o que faz.

Portanto, a gestão de requisitos não deveria terminar com o projeto. Requisitos continuam proporcionando valor durante toda a vida do software. O tempo de vida do requisito do produto deveria ser, no mínimo, igual ao tempo de vida do próprio produto.

Gerenciar requisitos é gerenciar o conhecimento adquirido sobre o software. Manter os requisitos ao longo da vida do produto pode facilitar: ➤

- o trabalho de manutenção do próprio software; ➤

- a análise do impacto de novas mudanças propostas para o negócio no software; ➤

- o apoio a outras atividades, como a formação de pessoas, governança corporativa e aderência a padrões.

Talvez pareça utópico falar do ciclo de vida do requisito coincidindo com o do produto, quando boa parte do mercado mal consegue gerir bem o ciclo de vida do requisito durante o projeto. A evolução na maturidade é gradual. Mas isso já ocorre em algumas organizações, principalmente aquelas que têm o produto de software como o seu negócio.

9.2. Plano de gerenciamento de requisitos

Como deve ser feita a gerência de requisitos em um projeto? Quem responde a essa questão é o plano de gerenciamento de requisitos. Este plano pode ser elaborado pela equipe de gestão de projetos durante o planejamento, ou, o mais comum, este plano está definido na metodologia de desenvolvimento de software adotada pela organização e que todos os projetos deveriam seguir.

O plano de gerenciamento de requisitos pode definir: ➤

- como é o processo de controle de mudança de requisitos, inclusive determinando quais partes interessadas: aprovam as mudanças, são

consultadas ou informadas sobre a mudança ou não precisam ser envolvidas no processo; ➤

quais atributos dos requisitos serão capturados. Não é apenas a definição do requisito que é relevante; há atributos que são fundamentais para que se possa fazer a gestão adequada dos requisitos. Exemplos de atributos: referência, autor, complexidade, proprietários, prioridade, risco, origem, estabilidade, situação; ➤

o nível de rastreabilidade dos requisitos; ➤

o processo de priorização de requisitos; ➤

o repositório para os requisitos e ferramentas a serem utilizadas.

9.2.1. Quem é responsável pela gerência de requisitos?

A elaboração do plano de requisitos é responsabilidade ou do gerente de projetos ou do responsável pela metodologia de desenvolvimento de software usada pela empresa. Já a execução deste plano não há uma regra, mas, falando de forma geral, boa parte fica a cargo da equipe de gerenciamento de projetos – sendo que o analista de requisitos exerce um papel de apoio em muitas dessas atividades.

9.3. Gestão de mudanças

Todo projeto necessita de um processo formal de gestão de mudanças para que não se perca o controle. Na perspectiva da gestão de projetos, a gestão de mudanças é mais ampla, mas neste livro o que interessa é discutir de forma específica a gestão de mudanças de requisitos.

A gestão de mudanças é o processo responsável por avaliar todas as solicitações de mudanças, eventualmente aprová-las ou rejeitá-las e, em caso de aprovação, promovê-las. Este processo ocorre durante todo o projeto, pois as mudanças não têm hora marcada para chegar.

As mudanças podem ser solicitadas por qualquer parte interessada. Às vezes um pedido de mudança pode ocorrer de forma verbal, mas é fundamental que seja formalizado por escrito e incorporado ao sistema de gestão do projeto. Todo pedido é então avaliado por um responsável (muito

comum que seja o gerente do projeto ou seu patrocinador) que irá decidir pela sua aprovação ou rejeição. No entanto, o *PMBOK® Guide* cria a figura do Comitê de Controle da Mudança, responsável por essa avaliação. Para projetos pequenos, esta figura é representada por uma pessoa; para projetos maiores, há um grupo de pessoas com essa responsabilidade. A Figura 9.2 ilustra esses papéis.

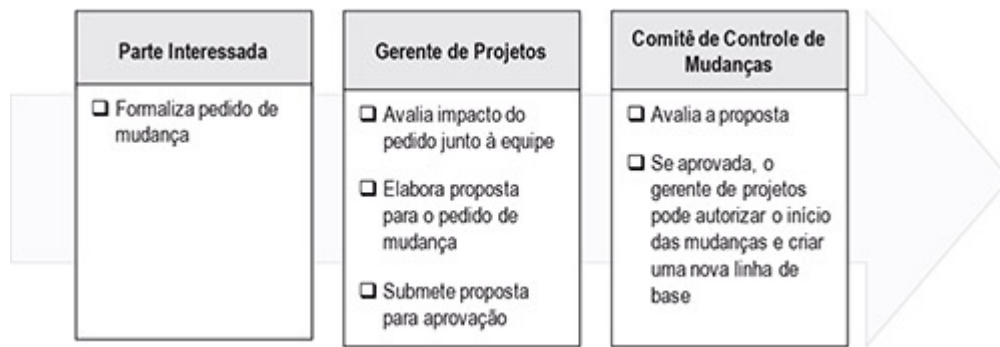


Figura 9.2. Papéis no processo de gestão da mudança.

Para que uma solicitação de mudança possa ser avaliada para aprovação ou rejeição, antes é necessário realizar uma avaliação de seu impacto sobre o projeto. Mesmo que essa solicitação de mudança seja sobre um único requisito, ela pode gerar impacto em outros requisitos relacionados. Essa avaliação de impacto precisa usar atributos do requisito como fonte, autor, proprietário e prioridade. A rastreabilidade de requisitos também pode ajudar a descobrir mais rapidamente requisitos relacionados ao que se deseja mudar.

As solicitações de mudança aprovadas demandarão novas atividades de elicitação e análise de requisitos. Isso gerará, portanto, uma nova versão da especificação de requisitos, que deverá ser aprovada pelas partes interessadas para ser usada como nova linha de base do escopo do projeto.

9.4. Obter aprovação sobre os requisitos

Obter a aprovação dos requisitos pode ser uma das tarefas mais desafiadoras da gerência de requisitos. Seguir com o trabalho do desenvolvimento de software sem a aprovação dos requisitos é trabalhar sem firmar um contrato com o cliente. O objetivo desta tarefa é garantir que

as partes interessadas que possuem autoridade entenderam e aceitaram os requisitos.

Essa aprovação pode ser sobre a especificação de requisitos final ou também para produtos intermediários da Engenharia de Requisitos, como as memórias de levantamento produzidas na elicitação de requisitos. O processo pode ser conduzido para que cada parte interessada aprove individualmente os requisitos, ou pode ser realizado de maneira a obter uma aprovação coletiva.

O desafio maior nesta tarefa é conseguir obter consenso entre as distintas partes interessadas. É comum que um projeto de software tenha várias partes interessadas – e quanto maior este número, maior a possibilidade de conflitos de interesse. Logo, para conseguir a aprovação dos requisitos, todos os conflitos devem ser resolvidos antes.

Durante a apresentação dos requisitos para a aprovação, várias questões podem ser levantadas pelas partes interessadas, que podem provocar a necessidade de novas atividades de elicitação e análise. Nesse caso, a sessão de aprovação provavelmente termina sem a aprovação (ou aprovação com ressalvas) e com uma lista de questões a serem resolvidas sobre os requisitos. É indicado que a sessão tenha um registro da resolução: qual a decisão tomada, a razão da decisão e as partes envolvidas presentes.

Após a aprovação, os requisitos devem ser mantidos sob uma linha de base. E qualquer alteração nesta linha de base deve ser feita através do processo de controle de mudanças.

9.4.1. Como apresentar os requisitos para aprovação?

A forma de apresentação dos requisitos às partes interessadas visando obter a aprovação pode variar conforme vários fatores: ➤

Cultura organizacional (muito formal e burocrática ou informal).



Cliente interno ou externo.



Nível de envolvimento das partes interessadas durante a elaboração dos requisitos.



O estilo de apresentação preferido pela parte interessada.



Quão detalhadamente os requisitos precisam ser comunicados.



Quais informações são importantes comunicar para quais interessados.



Tamanho do projeto.



Número de partes interessadas.

Em um projeto pequeno, para um cliente interno com o qual já se desenvolveu uma relação de confiança por conta de outros projetos já executados, com o qual se tem mais intimidade e que tem uma participação intensa no projeto, pode-se realizar uma apresentação informal dos requisitos. Isso é mais rápido e não envolve tantos detalhes. E considerando que as duas partes desenvolveram uma boa comunicação, os detalhes não são relevantes neste momento. A aprovação pode ocorrer também de maneira informal – por exemplo, verbalmente ou por e-mail.

Porém, a maioria dos projetos demanda uma apresentação formal dos requisitos para a aprovação. Essa formalidade pode envolver desde uma simples reunião até uma série de eventos com o uso de auditórios. Por exemplo, há alguns anos o Banco Central do Brasil desenvolveu o Sistema de Pagamentos Brasileiro (SPB). Foi um projeto que impactou todo o sistema bancário do país. E cada banco designou pelo menos um

representante para participar do projeto. Além disso, o projeto impactou várias áreas do próprio Banco Central e também de outros órgãos do governo. Nesse tipo de contexto, não seria possível concluir todo o processo de aprovação em uma única reunião. Considerando também que as iniciativas governamentais mais impactantes na sociedade têm que envolver audiências públicas, o leque de partes interessadas se abre para todo cidadão brasileiro. Logo, esse tipo de situação irá exigir um esforço significativo de tempo, logística e planejamento até que se consiga obter uma versão aprovada dos requisitos.

9.5. Técnica: controle de questões

O trabalho de requisitos começa em geral com poucas questões a serem respondidas e de caráter mais geral. Isso faz sentido quando se conhece pouco do problema e/ou da solução. À medida que a definição da solução vai sendo aprofundada, muitas questões vão surgindo, e em geral de caráter mais específico. E nem sempre todas as questões que surgem podem ser respondidas de imediato. Às vezes a resposta deve ser buscada com uma parte interessada distinta da que está sendo entrevistada, por exemplo. Se a questão for esquecida sem resposta, há grande chance de se introduzir uma falha na especificação. Logo, há que se ter um controle para que nenhuma questão fique sem tratamento.

Considerando que:



a memória humana é falível; ➤

o trabalho de requisitos em um projeto se estende frequentemente por semanas; ➤

nem sempre o trabalho de requisitos é realizado por uma única pessoa; ...então se torna necessário um controle sobre as questões que não pode ser feito “de cabeça” por uma pessoa ou em um caderninho. Por isso a técnica de controle de questões.

9.5.1. O que é o controle de questões

É uma técnica com o objetivo de rastrear, gerenciar e resolver questões e outras preocupações que precisam ser controladas até sua resolução durante o trabalho de requisitos. Também conhecido como controle de itens e, em inglês, *issue tracking* ou *item tracking*.

As questões quase sempre são dúvidas que precisam ser sanadas, mas podem também representar: conflitos a serem resolvidos, preocupações que devem ser discutidas, defeitos a corrigir na especificação, premissas a serem confirmadas.

É, portanto, um método organizado para controlar a resolução de questões, garantindo que nenhuma questão seja negligenciada ou perdida. Ajuda a manter o foco nos problemas que ainda não foram resolvidos e também centraliza e compartilha as questões entre a equipe de requisitos.

A questão deve ser gerenciada até que seja resolvida ou que seja determinado que nenhuma ação será tomada. Uma revisão regular dessas questões por todos os interessados garante visibilidade do seu encaminhamento e priorização sobre as questões mais relevantes ou urgentes a tratar. Caso as questões não possam ser solucionadas em um período de tempo razoável, pode ser necessário escalar a questão na hierarquia da organização.

A Tabela 9.1 exemplifica de forma resumida um controle de questões para o estudo de caso presente no Anexo. Ao longo dos exercícios dos capítulos anteriores, é possível que você, leitor, tenha percebido questões que precisariam ser sanadas sobre o projeto deste estudo de caso. Ao longo da leitura dos capítulos talvez algumas questões tenham sido esquecidas.

Porém, se essas questões tivessem sido registradas formalmente no controle de questões, é possível que até agora mais de uma dúzia de questões já tivessem sido identificadas. Isso ajudaria a planejar melhor as sessões de elicitação a fazer com o cliente, se o projeto fosse real.

Tabela 9.1. Exemplo de controle de questões simples para o estudo de caso.

ID	Questão	Autor	Data	Responsável	Situação
1	O que é SUPOPE?	Guilherme	03/08/2015	Guilherme	Resolvida

2	Quem representa os fiscais?	Carlos	03/08/2015	Carlos	Resolvida
3	Levantar volume de obras fiscalizadas/mês	Guilherme	10/08/2015	Carlos	Pendente
4	Levantar quantidade atual de fiscais	Guilherme	10/08/2015	Carlos	Pendente
5	O <i>call center</i> usará o SRRO?	Carlos	11/08/2015	Guilherme	Resolvida
6	Como os profissionais fiscalizados serão autenticados no sistema?	João	13/08/2015		Pendente

Por questões de espaço, optou-se por apresentar um exemplo limitado. Porém, para projetos maiores, outros atributos seriam necessários para o controle. No tópico seguinte isso será mais bem explorado.

9.5.2. Elementos do controle de questões

Dependendo da necessidade do projeto, o controle de questões pode ter os seguintes elementos: ➤

Descrição: uma definição clara e concisa do item identificado.



Descoberto por: pessoa que identificou o item.



Data de identificação da questão.



Impacto: as possíveis consequências, caso o item não seja resolvido até a data limite. O impacto pode ser avaliado, por exemplo, com base no cronograma, custo ou escopo.



Prioridade: determinar a prioridade do item, com base na avaliação das partes interessadas. A escala de prioridade deveria estar definida no plano de gestão de requisitos.



Data limite: até quando um item deve ser solucionado para evitar as consequências.



Dono: membro da equipe que é designado para gerenciar o item até o seu fechamento. O dono não deve ser obrigatoriamente a pessoa que identificou o item, ou as mesmas pessoas que têm ações designadas para solucionar o item.



Situação: a situação atual do item. Exemplos de situações que podem ser usadas incluem: aberto, designado, resolvido e cancelado. A situação atual do item deve ser comunicada para todas as partes interessadas relevantes.



Ação necessária para solucionar: detalhes de quais ações devem ser tomadas para solucionar o item. Pode ser mais de uma.



Responsável pela ação: pessoa designada para tomar uma ação específica para solucionar a questão.



Data de conclusão da ação: pode ser uma data futura estimada ou uma data passada real, caso o item esteja encerrado.



Resultado: os resultados da resolução.

Nem todos os projetos demandarão os mesmos atributos para que se possa fazer o controle de questões de forma eficaz. Os atributos devem existir para ajudar a cumprir o propósito do controle de questões, que é garantir a resolução da questão em tempo hábil para eliminar possíveis impactos negativos.

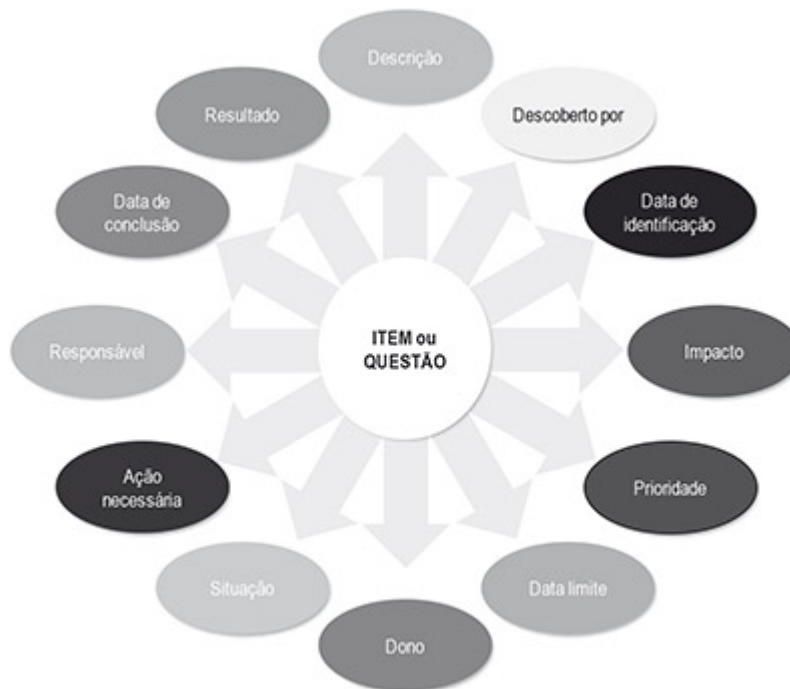


Figura 9.3. Elementos possíveis para o controle de questões.

9.6. Rastreabilidade de requisitos

Na mitologia grega há o mito do Fio de Ariadne, que descreve um grande labirinto onde habitava o Minotauro, que devorava pessoas mandadas em sacrifício. Até que Teseu resolveu enfrentar o monstro e conseguiu sair do labirinto por intermédio de um novelo de linha que Ariadne lhe deu e que permitiu a ele deixar um rastro do caminho que percorreu.

Esse mito é útil para entender o motivo e a importância de numerar as informações que descrevem o domínio do problema, assim como de sistematizar o seu registro.

O trabalho de chegar à solução a partir do domínio do problema é intrincado e propício a provocar a desorientação naqueles que o percorrem; a mesma descrição pode ser dada a um labirinto. Na interação com as partes interessadas, é muito fácil perder de vista as necessidades de negócio. Manter registros sobre a relação entre essas necessidades de negócio e os requisitos das partes interessadas tem objetivo similar ao de Teseu: marcar o caminho de volta.

Ao rastrear quais necessidades estão sendo abordadas conforme se refina a solução durante o desenvolvimento, é possível: ➤

- gerenciar o escopo do projeto visando entregar o que é necessário;



- verificar que todos os requisitos da solução são atendidos pela implementação; ➤

- verificar que a aplicação faz apenas aquilo a que se propõe; ➤

- verificar a consistência entre os requisitos em seus diferentes níveis de refinamento.

Adicionalmente, usando o mesmo princípio ao longo de todo o ciclo de vida do requisito, é possível mais facilmente: ➤

- avaliar o impacto de mudanças nos requisitos; ➤

- avaliar o impacto de uma falha de um teste sobre os requisitos (isto é, se o teste falha, o requisito pode não ser satisfeito); ➤

- gerenciar mudanças.

Software está presente de forma cada vez mais intensa na realidade humana, ajudando a controlar operações tão complexas quanto uma missão espacial e procedimentos médicos de alta complexidade, como uma radioterapia.

Dada a criticidade na qual o software está inserido em muitos casos, a exigência de qualidade e principalmente confiabilidade passa a ser cada vez maior. Não é exagero dizer que software controla vidas humanas. Portanto, é importante utilizar técnicas mais profissionais para desenvolvimento e manutenção desse tipo de software. E a rastreabilidade é um importante instrumento para ajudar nesse objetivo. Quanto maior o projeto ou mais crítico for o software, maior será a importância da rastreabilidade.

9.6.1. O que é a rastreabilidade de requisitos

A rastreabilidade dos requisitos é o processo de identificar e documentar os elos (ou vínculos) que envolvem um determinado requisito, para que seja possível rastrear sua origem, os artefatos derivados e os demais requisitos relacionados.

9.6.2. Benefícios da rastreabilidade

A rastreabilidade ajuda a manter uma coerência entre os requisitos desejados do sistema e os artefatos construídos ao longo do desenvolvimento de software. Também é uma maneira de entender o relacionamento entre os requisitos, componentes da arquitetura e programas da implementação, ajudando ao analista a perceber quais elementos do projeto satisfazem os requisitos.

A rastreabilidade ajuda a garantir que o software está em conformidade com os requisitos e auxilia na gestão do escopo, da mudança, de riscos, tempo, custo e comunicação. É também utilizada para detectar funcionalidades ausentes ou para verificar se existe alguma funcionalidade implementada que não é suportada por nenhum requisito (requisito supérfluo).

Em resumo, a rastreabilidade permite: ➤

- analisar o impacto de forma rápida e simples (especificação modificável), o que ajuda a estimar mais facilmente variações em cronogramas e em custos do projeto; ➤

- descobrir inconsistências e lacunas nos requisitos (ajuda a chegar a uma especificação completa), ou seja, saber se os requisitos de mais alto nível são tratados pelos de mais baixo nível; ➤

- verificar se a solução faz apenas aquilo a que se propõe (especificação correta); ➤

- ajuda na gestão de riscos: requisitos com muitas relações têm mais riscos; ➤

- visualizar a alocação de requisitos a componentes de software; ➤

- visualizar requisitos nos processos de testes; ➤

- corrigir defeitos através da identificação do componente de origem do erro.

9.6.3. Tipos de rastreabilidade

A capacidade de rastrear um requisito até seus refinamentos ou produtos é definida como rastrear para frente (*forwards*), e a de rastrear um requisito refinado ou produto até sua origem é definida como rastrear para trás (*backwards*). Essas duas capacidades, também conhecidas como rastreabilidade bidirecional, devem estar presentes em todos os tipos de rastreabilidade, para que seja possível cumprir os seus objetivos. Um processo de rastreabilidade é falho se não executa uma das duas capacidades.

Há dois tipos de rastreabilidade:



Rastreabilidade horizontal e vertical.



Pré e pós-rastreabilidade.

9.6.3.1. Rastreabilidade horizontal

A rastreabilidade horizontal consiste em estabelecer vínculos entre as diferentes versões de requisitos ou artefatos em uma determinada fase do ciclo de vida do projeto, conforme Figura 9.4. Ou também estabelecer vínculos entre requisitos e artefatos que estão em um mesmo nível.

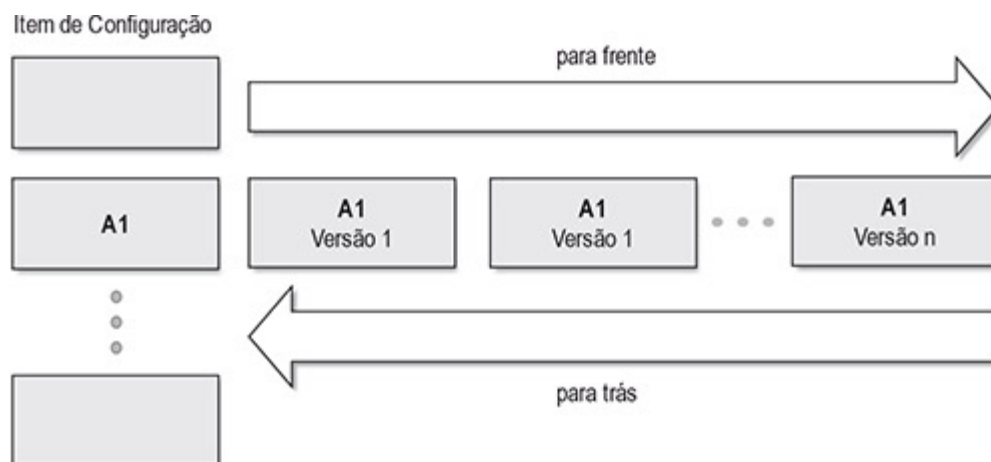


Figura 9.4. Exemplo de rastreabilidade horizontal entre versões do mesmo artefato.

9.6.3.2. Rastreabilidade vertical

Já a rastreabilidade vertical relaciona os requisitos ou artefatos produzidos nas distintas fases ao longo do ciclo de vida do projeto. Por exemplo, parte-se de uma origem comum – a solicitação do cliente – que pode listar vários requisitos. Em um primeiro momento elabora-se um documento de visão para o projeto, onde estão relacionados os requisitos especificados do software com os requisitos presentes na demanda original. Em um segundo momento há uma especificação mais detalhada dos requisitos, que poderiam ser casos de uso, e cada elemento especificado deve se relacionar a uma parte específica do documento de visão ou ao requisito da demanda original. O mesmo se aplicaria às telas do protótipo e aos casos de testes.

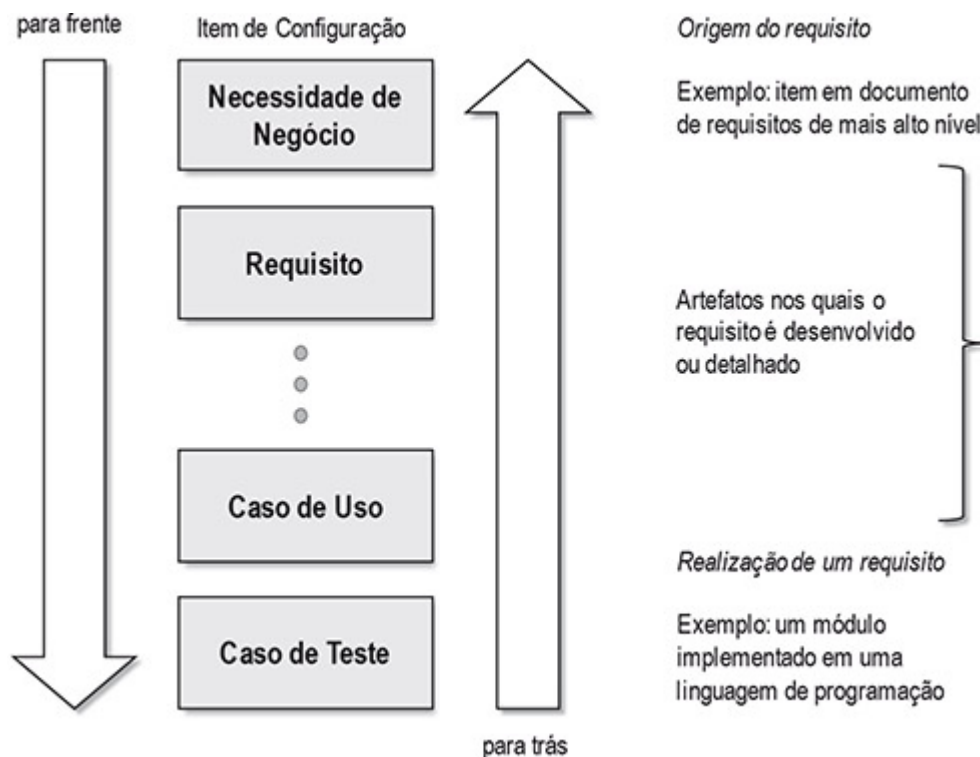


Figura 9.5. Exemplo de rastreabilidade vertical entre requisitos/artefatos produzidos em distintas fases do projeto.

9.6.3.3. Pré e pós-rastreabilidade

A pré-rastreabilidade está concentrada no ciclo de vida dos requisitos antes de serem incluídos na especificação de requisitos. Ela permite identificar a origem de cada requisito e também quais se originaram de uma determinada fonte (padrões organizacionais, clientes, usuários, normas, por exemplo). Qualquer mudança nessas fontes precisa ser atualizada na especificação de requisitos, e toda mudança na especificação deve ser realizada com referência a essas fontes. Isso requer visibilidade desses inter-relacionamentos, às vezes sutis.

A pós-rastreabilidade está concentrada no ciclo de vida dos requisitos depois de sua especificação de requisitos. Permite identificar quais componentes do software implementam um determinado requisito – por exemplo: elementos da arquitetura, programas-fonte. Também possibilita saber quais requisitos estão associados a um componente do software. Para conseguir fazer isso é necessário um ponto de referência: a linha de base da especificação de requisitos. Através dela serão relacionadas várias das disciplinas seguintes do projeto. Mudanças na linha de base precisarão ser propagadas nesses artefatos.

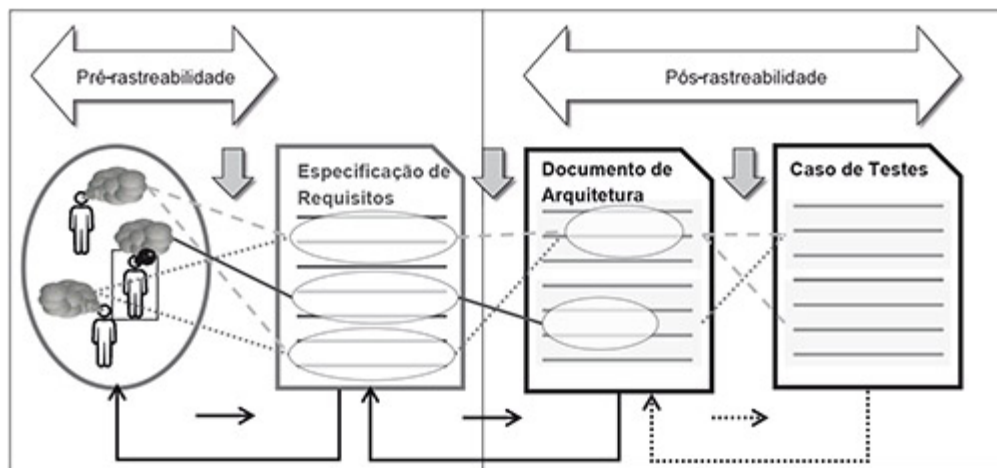


Figura 9.6. Ilustração da pré e pós-rastreabilidade de requisitos.

9.6.4. Matriz de rastreabilidade

Esta é uma ferramenta que facilita a visualização da rastreabilidade documentada entre requisitos e outros objetos (ou requisitos entre si). Nela, colocam-se os objetos rastreados nos eixos de uma tabela e marcam-se os pontos de interseção.

Se o projeto possuir poucos requisitos ou se a rastreabilidade for limitada aos requisitos de alto nível, uma tabela ou planilha geralmente resolve. Agora, se o projeto envolver um grande número de requisitos, é recomendado utilizar uma ferramenta especializada, pois o esforço de manutenção da matriz em uma planilha acaba tornando-se inviável.

9.7. Priorizar requisitos

Priorizar significa atribuir um valor de importância relativa entre os requisitos. O objetivo é maximizar o valor entregue pelo projeto, fazendo com que as coisas mais importantes sejam tratadas em primeiro lugar. É uma atividade que está sob a responsabilidade do gerente de projeto, porém o analista de requisitos participa da priorização, facilitando o processo. A decisão sobre o que é mais importante deve vir do cliente; o analista de requisitos ajuda esclarecendo os requisitos ou informando das consequências de alguma escolha. Por exemplo, não adianta o cliente dizer que o relatório de vendas é a primeira coisa a ser entregue no projeto. Cabe ao analista de requisitos esclarecer que há uma dependência deste requisito em relação aos requisitos que alimentam o sistema com dados de venda.

A priorização é algo contínuo e muda ao longo do desenvolvimento do projeto, assim como os próprios requisitos podem mudar. A grande dificuldade com a priorização, como visto no Capítulo 4, não está nas técnicas que podem ser usadas, e sim na tomada das decisões necessárias. Priorizar é fazer escolhas e eventualmente deixar algo para trás. Por isso muitos clientes tentam dizer que tudo (ou quase tudo) é prioritário, só que isso não é priorizar. Se todos os requisitos têm o mesmo valor de priorização, não há prioridade. Outros se omitem dessas decisões e deixam essas escolhas a cargo da equipe do projeto.

A priorização também pode ser influenciada de forma intencional ou não pela equipe de desenvolvimento, que pode superestimar a dificuldade ou complexidade da implementação de certos requisitos, para que a priorização lhe seja mais conveniente.

Vários critérios podem ser usados para a priorização. A Tabela 9.2 enumera os mais comuns.

Tabela 9.2. Critérios comuns para priorização de requisitos.

Critérios de priorização	Como priorizar os requisitos?	Quando utilizar?
Valor de negócio (benefício)	Com base na análise de custo-benefício.	Projetos incrementais ou que possuem limitações orçamentárias.
Risco (técnico ou de negócio)	Com base nos maiores riscos de falha para o projeto.	Maximizar a probabilidade de sucesso do projeto.
Custo	Com base nos custos de implementar os requisitos.	As partes interessadas podem mudar prioridades depois que entendem os custos.
Perdas	Com base nas perdas ocasionadas por não implementar o requisito.	Quando políticas e regulamentações são impostas à organização ou risco de perda de oportunidade para concorrência.
Dependência com outros requisitos	Com base na dependência que requisitos com alto valor agregado possuem de requisitos com baixo valor agregado.	Projetos incrementais ou que possuem limitações orçamentárias.
Estabilidade	Com base no consenso (mais úteis ou valiosos) obtido pelas partes interessadas.	Quando há conflitos ou indefinição de requisitos de partes interessadas.
Sensibilidade temporal	Com base na sensibilidade de tempo.	Quando há janelas de oportunidade para o negócio que devem ser aproveitadas (sazonalidade ou situações específicas – por exemplo, legais).

Várias técnicas podem ser usadas para a priorização. Algumas delas são comentadas a seguir.

9.7.1. Técnica: *timeboxing/budgeting*

Neste caso requisitos são priorizados com base na alocação de recursos fixos. *Timeboxing* prioriza os requisitos que a equipe é capaz de implementar em um período de tempo fixo (em geral, semanas), enquanto

budgeting prioriza dentro de um orçamento periódico fixo (em geral, mensal).

Há três estratégias básicas para ocupar essas “caixas” fixas de recursos: ➤

Tudo dentro: começa com todos os requisitos elegíveis na “caixa”, com duração ou custo atribuído. Em seguida é iniciada a remoção dos requisitos menos importantes até que se consiga satisfazer os limites de prazo ou orçamento. Ou seja, até que se consiga “fechar a caixa”.



Tudo fora: começa com a “caixa” vazia e é iniciada a adição de requisitos, com duração ou custo atribuídos, até que os limites de prazo ou orçamento sejam alcançados.



Seletivo: são identificados os requisitos de alta prioridade dentro de todo o escopo e em seguida é exercitada a adição ou remoção de requisitos até que se encontre uma combinação que cumpra com os limites do prazo ou orçamento.

Um exemplo muito comum de priorização por *timeboxing* é usado na metodologia Scrum, muito usada em projetos “ágeis”. No ciclo de desenvolvimento iterativo-incremental do Scrum, cada iteração tem duração fixa no projeto, definida entre duas e quatro semanas. Durante o planejamento da próxima iteração, a equipe busca definir quais requisitos serão implementados, selecionando um subconjunto dos requisitos ainda não implementados do projeto que seja possível de implementar nessa janela fixa de tempo. O ciclo se repete ao final de cada iteração, até que todo o escopo tenha sido desenvolvido.

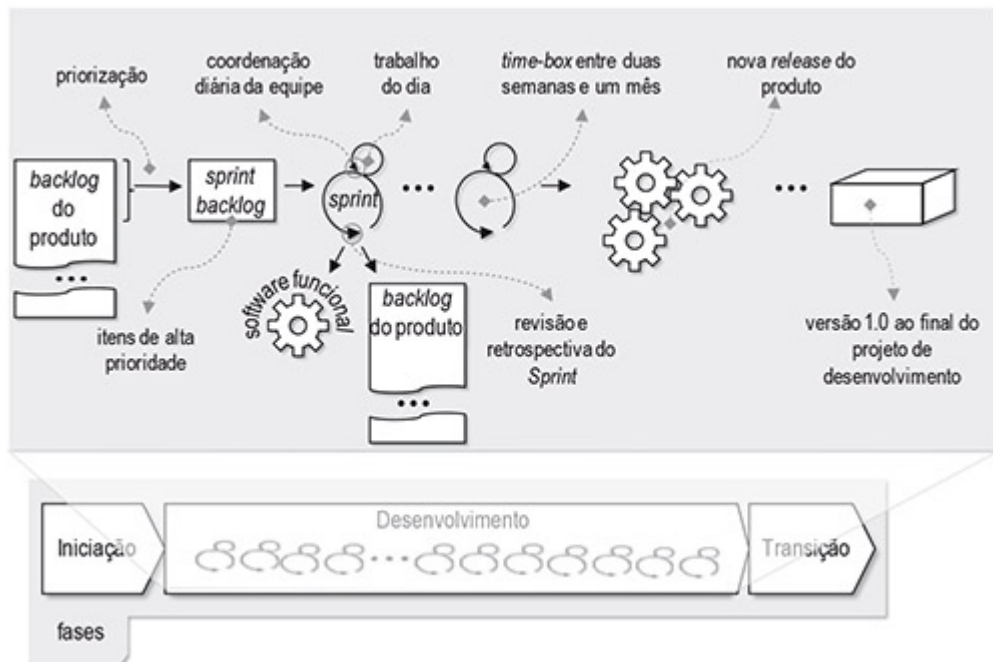


Figura 9.7. Dinâmica básica do processo Scrum.

9.7.2. Técnica: votação

A priorização por votação tem um nome autoexplicativo. Consiste em alocar uma quantidade fixa de recursos (dinheiro, votos ou outros elementos) para que cada participante da sessão de votação os distribua entre os requisitos propostos. Aqueles requisitos que receberem a maior quantidade de recursos serão os priorizados.

A princípio, parece uma abordagem estranha de se usar para priorizar requisitos com as áreas de negócio. E de fato não é comum usar esta técnica quando se fala de desenvolvimento de software para uso interno da organização. Porém, para empresas que desenvolvem software como um produto de mercado, com uma base significativa de clientes, é frequente usar a votação para priorização.

Para essas empresas, é comum receber dos clientes várias solicitações de melhoria do produto ao longo do tempo. E quem desenvolve “software-produto” normalmente planeja entrega periódica de novas versões ao mercado, normalmente de seis a 24 meses.

Portanto, ao planejar uma próxima versão do produto, é comum a empresa deparar com várias boas ideias sugeridas pelos clientes. Porém, há

limitações de recursos para desenvolver todas elas para a próxima versão. Então elabora-se uma pesquisa entre os clientes para saber quais das novas características têm mais valor para eles.

9.7.3. Técnica: análise Moscow

A análise Moscow é uma técnica que consiste em atribuir aos requisitos de um software quatro valores possíveis. O seu nome é derivado de um acrônimo em inglês para esses quatro valores, conforme a Figura 9.8.

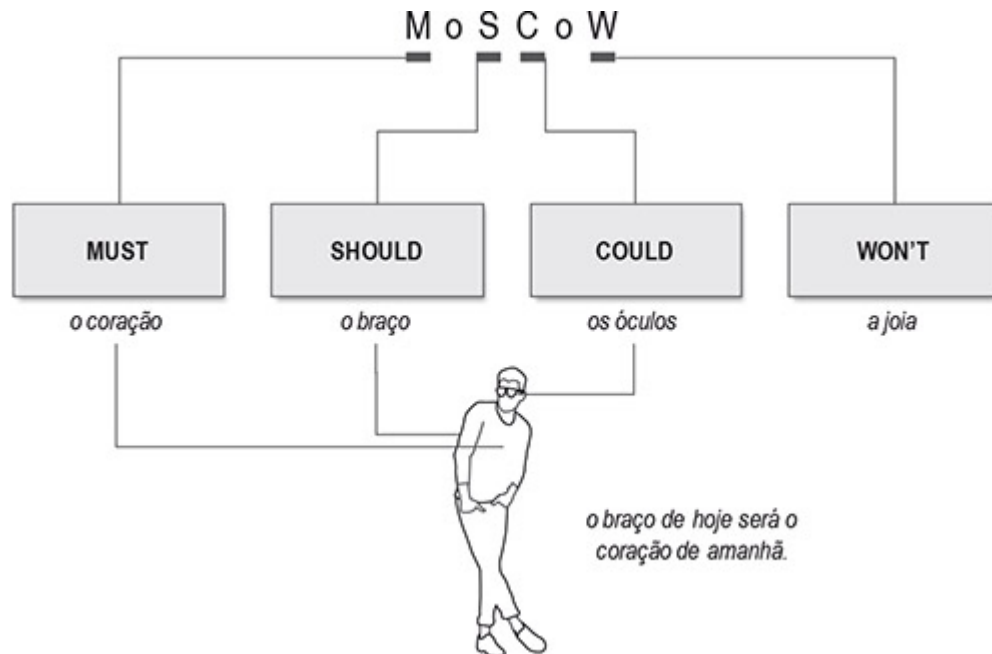


Figura 9.8. Valores possíveis para um requisito na priorização pela análise Moscow.

Must – Minimum Usable SubseT (deve ter): requisito obrigatório para o projeto. Esta categoria descreve o conjunto mínimo de requisitos que é essencial ao sistema a ser desenvolvido. Caso esse requisito não seja satisfeito no projeto, este não será um sucesso. Ou seja, é um requisito de importância máxima para que o projeto exista; sem ele o sistema não tem valor para o cliente.

Seguem algumas situações nas quais um requisito *must* pode estar envolvido: ➤

O sistema não poderá ser entregue na data acordada sem esse requisito.



Se esse requisito não for implementado, não há propósito em entregar o sistema.



O sistema não cumpre normas legais se esse requisito não for implementado.



O sistema se torna inseguro se esse requisito não for implementado.



Não será possível entregar o caso de negócio sem esse requisito.

Should (deveria ter): requisito importante, mas não vital para um projeto. Esta categoria descreve o requisito que é importante para o sistema a ser desenvolvido. Esse requisito é de alta prioridade – caso ele não seja cumprido, pode afetar a satisfação do cliente; no entanto, o projeto continuará sendo viável. Sem ele o projeto talvez não entregue a solução mais eficiente ou ideal, mas ainda resolverá o problema que motivou o projeto.

Could (poderia ter): requisito desejável, mas não necessário para um projeto. Esta categoria descreve o requisito que é desejável no sistema a ser desenvolvido. Esse requisito somente será incluído caso o tempo e os recursos disponíveis permitam que isso aconteça. São os requisitos mais prováveis de serem eliminados do escopo caso a meta de prazo ou custo do projeto esteja em risco. Esse requisito não vai influenciar diretamente no sucesso do projeto, mas pode influenciar na satisfação do cliente. Uma forma de diferenciar o *could* de *should* é o nível de insatisfação que o seu não atendimento provocará. No *should* este efeito será mais intenso.

Won't have this time (não terá agora): requisito dispensável no momento, mas que pode ser incluído em outros ciclos do projeto ou em outros projetos. Não se indica que um requisito do tipo *won't* seja excluído da

especificação, pois futuramente sua importância pode ser reavaliada e ele pode se tornar importante. Isso ajuda também a gerenciar as expectativas das partes interessadas, pois já se indica que a necessidade foi identificada; apenas que não tem a prioridade de ser atendida de imediato.

No limite, todo requisito que não é *must* representa uma reserva de contingência do escopo.

Esteja atento: esses valores podem mudar durante o projeto para um mesmo requisito. Se o próprio requisito pode mudar durante o projeto, por que não sua prioridade? A priorização é algo frequente no projeto e pode mudar de acordo com as necessidades do cliente.

9.7.3.1. Diretrizes para a priorização Moscow

De nada adianta classificar todos os requisitos de um projeto como *must*. Se todos os requisitos têm a mesma prioridade, não há priorização. Uma diretriz apresentada por DSDM (2014), ilustrada na Figura 9.9, é procurar fazer com que requisitos *must* e *should* representem não mais do que 80% do esforço do projeto, sendo que os requisitos *must* não devem exceder 60% do esforço do projeto. Restam então os requisitos categorizados como *should* e *could*, aproximadamente 20% do esforço total do projeto para cada um.

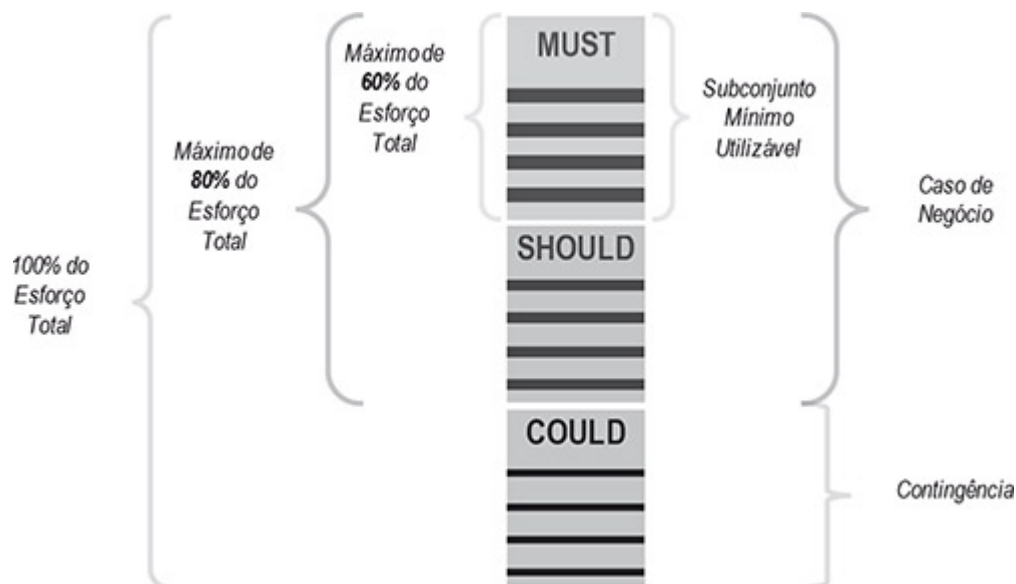


Figura 9.9. Diretriz para a priorização do projeto pela análise Moscow.

9.8. Gerência de requisitos depende de ferramenta?

Observando todos os temas tratados sobre a gerência de requisitos, pode-se perceber que a maioria não depende de ferramenta de software específica para ocorrer. São práticas que dependem apenas de organização, disciplina e vontade corporativa para serem executadas.

Dessas práticas, a única na qual uma ferramenta especializada ajudará de maneira significativa é a manutenção da rastreabilidade. Embora a rastreabilidade possa ser mantida manualmente, para projetos com muitos requisitos e um nível de rastreabilidade alto, tal atividade executada de forma manual torna-se impraticável.

Certamente as ferramentas de gerência de requisitos têm utilidade, e não só para manter a rastreabilidade. Porém, o que se deseja enfatizar é que é um equívoco acreditar que o primeiro passo na intenção de melhorar a gerência de requisitos tenha que ser dado na direção da aquisição de ferramenta. Várias ações (e mais eficazes) devem ser tomadas antes de se pensar em uma ferramenta específica.

9.9. Exercícios

1. Quais são os impactos negativos de não obter a aprovação da especificação de requisitos?
2. Marque a opção incorreta relativa ao controle de questões: a)
Cada analista de requisitos deve manter de forma individual seu controle de questões do projeto.
b) Garante que as questões sejam capturadas, rastreadas e resolvidas.
c) Pode ser implementado em uma planilha eletrônica.
d) Necessita de alocação de um responsável à questão para que ela seja solucionada em tempo hábil.
3. Por que é importante rastrear requisitos?
- 4.

Na análise Moscow, um requisito classificado como *won't* (dispensável) pode ser descartado de um projeto de software? Justifique sua resposta.

5. O que não é correto afirmar sobre a gerência de requisitos?
- a) Busca identificar partes interessadas e seus requisitos.
 - b) Gerencia as mudanças de requisitos ao longo do projeto.
 - c) Soluciona conflitos sobre os requisitos.
 - d) Prioriza requisitos.

Bibliografia

ADOLPH, S.; BRAMBLE, P.; COCKBURN, A.; POLS, A. **Patterns for Effective Use Cases**. (The Agile Software Development Series). Boston, MA: Addison-Wesley Professional, 2003.

AGILE MODELING. **User stories**: an agile introduction. Disponível em: <<http://www.agilemodeling.com/artifacts/userStory.htm#Themes>>. Acesso em: 28 abr. 2016.

ALONSO, C. M.; GALLEGO, D. J.; HONEY, P. **Los estilos de aprendizaje**: procedimientos de diagnóstico y mejora. Madrid: Mensajero, 2002.

ASSOCIATION OF BUSINESS PROCESS MANAGEMENT PROFESSIONALS. **Guia de Gerenciamento de Processos de Negócio**: corpo comum de conhecimento (CBOK). Versão 3.0. ABPMP, 2013.

BECK, K.; BEEDLE, M.; VAN BENNEKUM, A.; COCKBURN, A.; CUNNINGHAM, W.; FOWLER, M. *et al.* **Manifesto Ágil**. 2001. Disponível em: <<http://www.manifestoagil.com.br/>>. Acesso em: 28 abr. 2016.

BHARWANI, S. Understanding Complex Behavior and Decision Making Using Ethnographic Knowledge Elicitation Tools (KnETs). **Social Science Computer Review**, v. 24, n. 78, 2006.

BIAS, R. G.; MAYHEW, D. J.; KAUFMANN, M. **Cost-justifying Usability**: an update for an internet age. 2nd. ed. Morgan Kaufmann Publishers: San Francisco, CA: 2005.

BOEHM, B. W.; ABTS, C.; BROWN, A. W.; CHULANI, S.; CLARK, B. K.; HOROWITZ, E. *et al.* **Software Cost Estimation with COCOMO II**. New Jersey, NJ: Prentice Hall, 2000.

- BOEHM, B.; BASILI, V. Software Defect Reduction – Top 10 List. **IEEE Computer**, jan. 2001.
- BRASIL. MPOG – Ministério do Planejamento, Orçamento e Gestão. **Sistema de Administração de Recursos de Tecnologia da Informação e Informática (SISP)**. Instrução Normativa nº 4 (IN04). 11 set. 2014. Disponível em: <<http://www.governoeletronico.gov.br/biblioteca/arquivos/instrucao-normativa-nb0-4-de-11-de-setembro-de-2014-compilada/download>>. Acesso em: 28 abr. 2016.
- BROOKS, F. **The Mythical Man-Month**: essays on software engineering. Anniversary Edition. Boston, MA: Addison-Wesley, 1995.
- CASTRO, J. R.; CALAZANS, A. T. S.; PALDÊS, R. A.; GUIMARÃES, F. A. **Engenharia de requisitos**: um enfoque prático na construção de software orientado ao negócio. Florianópolis: Bookess, 2014.
- CATHO. **Cargo de Analista de Requisitos**. Disponível em: <<http://www.catho.com.br/profissoes/analista-de-requisitos>>. Acesso em: 28 abr. 2016.
- COCKBURN, A. **Writing Effective Use Cases**. Boston, MA: Addison-Wesley, 2000.
- COHN, M. User Stories, Epics and Themes. **Mountain Goat Software**, Oct. 24, 2011. Disponível em: <<http://www.mountaingoatsoftware.com/blog/stories-epics-and-themes>>. Acesso em: 28 abr. 2016.
- COMMON SOFTWARE MEASUREMENT INTERNATIONAL CONSORTIUM. **COSMIC Measurement Manual**: versão 4.0.1. Cosmic, 2015.
- CREA-SP. **Termo de referência do pregão 46/2010**. CREA-SP, 2010.
- CROSBY, P. **Quality is free**: the art of making quality certain – how to manage quality – so that it becomes a source of profit. New York, NY: McGraw-Hill, 1979.
- CRUZ, F. **Scrum e guia PMBOK unidos no gerenciamento de projetos**. Rio de Janeiro: Brasport, 2013.

- DEKKERS, C.; AGUIAR, M. Applying function point analysis to requirements completeness. **Crosstalk, The Journal of Defense Software Engineering**, Ogden, UT, fev. 2001.
- DEMARCO, T. **Structured Analysis and System Specification**. New Jersey, NJ: Prentice-Hall, 1979.
- DEMING, W. E. **Out of the Crisis**. Boston, MA: The MIT Press. 1994.
- DEPARTMENT OF DEFENSE. **Military Standard: Defense System Software Development**, DoD-Std-2167, June 4th, 1985.
- DRUCKER, P. F. **The Practice of Management**. New York, NY: HarperBusiness, 2006.
- DSDM. **The DSDM Agile Project Framework**. 2014. Disponível em: <<https://www.dsdm.org/content/moscow-prioritisation>>. Acesso em: 28 abr. 2016.
- DUARTE, G. **Dicionário de Administração e Negócios**. Fortaleza: KBR, 2011.
- DUBEY, S. S. **IT Strategy and Management**. New Delhi: PHI Learning Pvt., 2009.
- EIE – ENGINEERING IS ELEMENTARY. **The Engineering Design Process**. Disponível em: <<http://www.eie.org/overview/engineering-design-process>>. Acesso em: 28 abr. 2016.
- GALEOTE, S. **Conceitos e importância da prototipação de requisitos**. 2012. Disponível em: <<http://www.galeote.com.br/blog/2012/11/conceitos-e-importancia-da-prototipao-de-requisitos/>>. Acesso em: 28 abr. 2016.
- GANE, C.; SARSON, T. **Structured systems analysis: tools and techniques**. New Jersey, NJ: Prentice-Hall, 1979.
- GARTNER GROUP. **IT Glossary: Knowledge Management (KM)**. Disponível em: <<http://www.gartner.com/it-glossary/km-knowledge-management>>. Acesso em: 28 abr. 2016.
- GARTNER GROUP. **IT Key Metrics Data 2010: key applications measures – life cycle distribution**. 2010.

GENGIVIR, E. C. **Um modelo para rastreabilidade de requisitos de software baseado em generalização de elos e atributos.** Tese de Doutorado, São José dos Campos, INPE, 2009.

GOLDSTEIN, H. **Who killed the Virtual Case File?** IEEE Spectrum. 2005. Disponível em: <<http://spectrum.ieee.org/computing/software/who-killed-the-virtual-case-file>>. Acesso em: 10 maio 2016.

HELJO, S. **Reflections on Kanban versus Scrum development.** June 23, 2011. Disponível em: <<http://samuliheljo.com/blog/reflections-on-kanban-vs-scrum-development/>>. Acesso em: 28 abr. 2016.

IEEE. **About SWEBOOK.** IEEE Computer Society, 2004. Disponível em: <<https://www.computer.org/web/swebok>>. Acesso em: 28 abr. 2016.

IEEE. IEEE Std. 830–1998: IEEE Recommended Practice for Software Requirements Specifications. New York, NY: IEEE Computer Society, 1998.

INTERNATIONAL FUNCTION POINT USERS GROUP. **Function Point Counting Practices Manual.** Release 4.3.1. New Jersey, NJ: IFPUG, 2010.

INTERNATIONAL INSTITUTE OF BUSINESS ANALYSIS. **A Guide to the Business Analysis Body of Knowledge® (BABOK® Guide).** Versão 2.0. Toronto: IIBA, 2009.

INTERNATIONAL INSTITUTE OF BUSINESS ANALYSIS. **A Guide to the Business Analysis Body of Knowledge® (BABOK® Guide).** Versão 3.0, Toronto: IIBA, 2015.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. ISO/IEC/IEEE 24765: systems and software engineering – Vocabulary. Geneva: ISO/IEC/IEEE, 2010.

INTERNATIONAL REQUIREMENTS ENGINEERING BOARD. **A Glossary of Requirements Engineering Terminology:** versão 1.6. IREB, 2014

ITAIPU BINACIONAL. **Edital para contratação de Serviços técnicos para desenvolvimento e manutenção de sistemas nos ambientes tecnológicos utilizados pela Diretoria de Coordenação.** Tomada de preços nº 1548/2011. Disponível em:

<<http://fattocs.com/files/pt/editais/Itaipu/Itaipu-1548-2011.ZIP>>. Acesso em: 10 maio 2016.

JONES, C. **Estimating Software Costs**: bringing realism to estimating. 2. ed. Osborne: McGraw Hill, 2007.

JONES, C. Software Defects Origins and Removal Methods. **Namcook Analytics**, 2013. Disponível em: <<http://namcookanalytics.com/software-defect-origins-and-removal-methods>>. Acesso em: 28 abr. 2016.

JONES, C.; BONSIGNOUR, O. **The Economics of Software Quality**. Boston, MA: Addison-Wesley, 2012.

JURAN, J. M. **Quality Control Handbook**. 4th. ed. New York, NY: McGraw-Hill, 1988.

KENDALL, K. E.; KENDALL, J. E. **Systems Analysis and Design**. New Jersey, NJ: Prentice Hall, 1992.

LEFFINGWELL, D. **Agile Software Requirements**: lean requirements practices for teams, programs, and the enterprise. Boston, MA: Addison-Wesley, 2011.

LEFFINGWELL, D. Calculating the Return on Investment from More Effective Requirements Management. **American Programmer**, v. 10, n. 4, p. 13-16, 1997.

LIONS, J. L. **Ariane 5**: Flight 501 Failure. Report by the Inquiry Board. Disponível em: <<http://sunnyday.mit.edu/accidents/Ariane5accidentreport.html>>. Acesso em: 28 abr. 2016.

LLOYD, R. Metric mishap caused loss of NASA orbiter. CNN, Sep. 30, 1999. Disponível em: <<http://edition.cnn.com/TECH/space/9909/30/mars.metric.02/index.html>>. Acesso em: 28 abr. 2016.

MILLER, G. A. The magical number seven, plus or minus two: Some limits on our capacity for processing information. **Psychological Review**, v. 63, n. 2, p. 81-97, 1956.

OBJECT MANAGEMENT GROUP. **Business Process Model and Notation (BPMN)**. Versão 2.0. Needham, MA: OMG, 2010.

- PEREIRA, J. R. R. **Desenvolvimento de um software para métricas em rastreabilidade de requisitos de software**. Cornélio Procópio: UFTPR, 2011.
- PIGNEUR, Y.; OSTERWALDER, A. **Business Model Generation**. New York, NY: John Wiley & Sons, 2010.
- PMO INFORMÁTICA. **Historias de usuario: 30 ejemplos**. 1º jun. 2015. Disponível em: <<http://www.pmoinformatica.com/2015/05/historias-de-usuario-ejemplos.html>>. Acesso em: 28 abr. 2016.
- POHL, K.; RUPP, C. **Requirements Engineering Fundamentals**. Santa Barbara, CA: Rocky Nook Computing, 2011.
- PRESSMAN, R. S.; MAXIM, B. **Software Engineering: a practitioner's approach**. 8. ed. New York, NY: McGraw-Hill, 2014.
- PROJECT MANAGEMENT INSTITUTE. **A Guide to the Project Management Body of Knowledge: PMBOK® Guide**. 5th. ed. Newtown Square, PA: PMI, 2013.
- PROJECT MANAGEMENT INSTITUTE. **Business Analysis for Practitioners: a practice guide**. Newtown Square, PA: PMI, 2015.
- PROJECT MANAGEMENT INSTITUTE. **PMI's Pulse of the Profession: Requirements Management – A Core Competency for Project and Program Success**. Newtown Square, PA: PMI, 2014.
- SEI – SOFTWARE ENGINEERING INSTITUTE. **CMMI® for Development**. Versão 1.3. Pittsburgh, PA: Carnegie Mellon University, 2010.
- SHUJA, A. K.; KREBS, J. **IBM Rational Unified Process Reference and Certification Guide: solution designer**. Boston, MA: IBM Press Book, 2007.
- SHUJA, A. K.; KREBS, J. **IBM RUP Reference and Certification Guide: solution designer**. Boston, MA: IBM Press Book, 2008.
- SOFTEX. **Guia de Implementação – Parte 1: fundamentação para implementação do nível G do MR-MPS-SW**. Campinas: SOFTEX, 2012.
- SOMMERVILLE, I. **Software Engineering**. 8. ed. Boston, MA: Pearson Education, 2006.

TRIBUNAL DE CONTAS DA UNIÃO. **Acórdão nº 2314/2013**. TCU. Plenário. 2013. Disponível em: <<https://contas.tcu.gov.br/juris/Web/Juris/ConsultarTextual2/Jurisprudencia.faces?colegiado=PLENARIO&numeroAcordao=2314&anoAcordao=2013>>. Acesso em: 28 abr. 2016.

TRIBUNAL SUPERIOR ELEITORAL. **Glossário Eleitoral Brasileiro**. TSE. Disponível em: <<http://www.tse.jus.br/eleitor/glossario/glossario-eleitoral>>. Acesso em: 28 abr. 2016.

VAZQUEZ, C. Métricas 2014: integração do desenvolvimento ágil com a governança corporativa de TI usando métricas funcionais. **Fatto Consultoria e Sistemas**, 18 nov. 2014. Disponível em: <http://fattocs.com/pt/?p=2019&option=com_wordpress&Itemid=414>. Acesso em: 28 abr. 2016.

VAZQUEZ, C.; SIMÕES, G.; ALBERT, R. **Análise de Pontos de Função, Medição, Estimativas e Gerenciamento de Projetos de Software**. São Paulo: Érica, 2013.

WIEGERS, K. E. **More About Software Requirements**: thorny issues and practical advice. Washington, DC: Microsoft Press, 2006.

WIKIPÉDIA. **Cluster (spacecraft)**. Disponível em: <https://en.wikipedia.org/wiki/Cluster_%28spacecraft%29>. Acesso em: 28 abr. 2016.

WIKIPÉDIA. **Engenharia**. Disponível em: <<https://pt.wikipedia.org/wiki/Engenharia>>. Acesso em: 28 abr. 2016.

WIKIPÉDIA. **List of software bugs**. Disponível em: <http://en.wikipedia.org/wiki/List_of_software_bugs>. Acesso em 28 abr. 2016.

WIKIPÉDIA. **Mars Climate Orbiter**. Disponível em: <http://en.wikipedia.org/wiki/Mars_Climate_Orbiter>. Acesso em: 28 abr. 2016.

WIKIPÉDIA. **Virtual Case File**. Disponível em: <http://en.wikipedia.org/wiki/Virtual_Case_File>. Acesso em: 28 abr. 2016.

Glossário

A

Ágil: abordagem para desenvolvimento de projetos que busca seguir diretrizes e princípios do Manifesto Ágil (BECK et al., 2001).

Análise de pontos de função: método padrão para medir as funcionalidades de um software sob a perspectiva do usuário. Mede o desenvolvimento e a manutenção de software de forma independente da tecnologia utilizada para sua implementação.

Análise de documentos: técnica de elicitação de requisitos que estuda a documentação disponível sobre uma solução existente para identificação de informação relevante para o desenvolvimento de uma nova solução.

Análise de requisitos: atividade da Engenharia de Requisitos responsável pela documentação, organização, modelagem, classificação em grupos coerentes, verificação e validação dos requisitos.

Aplicação: conjunto coeso e automatizado de funções, procedimentos e dados que dão sustentação a um objetivo de negócio. Também chamado de sistema ou sistema de informação.

Artefato: qualquer item criado como parte da definição, manutenção ou utilização de um processo de desenvolvimento ou manutenção de sistemas de informação. Inclui, entre outros, descrições de processo, planos, procedimentos, especificações, desenhos de arquitetura, projeto detalhado, código-fonte e documentação para o usuário. Artefatos podem ou não ser entregues a um cliente ou usuário final.

Ator: elemento da técnica caso de uso que representa uma pessoa (ou representante de um grupo de pessoas) que desempenha um papel ou “coisa” (outros softwares, dispositivos) que interage com a aplicação, com a finalidade de cumprir um trabalho significativo. Veja Usuário.

B

BABOK®: *Business Analysis Body of Knowledge*. Padrão e guia para a análise de negócio do *International Institute of Business Analysis* (IIBA).

Backlog: lista de requisitos ainda não atendidos.

Baseline: uma configuração estável de artefatos de requisitos revisados e aprovados que é usada para fins de comparação e que somente pode ser alterada através de um processo formal de controle de mudanças.

Bug: ver Defeito.

C

Caso de negócio: estudo de viabilidade documentado que valida o custo-benefício de uma iniciativa de uma organização. Em geral, é o ponto de partida para criar um novo projeto.

Caso de teste: conjunto de condições usadas para teste de software. Deve especificar a saída esperada e os resultados esperados do processamento.

Caso de uso: elemento que modela um requisito funcional do ponto de vista da interação entre usuário (ou ator) e o sistema. Sua especificação faz isso por meio da descrição de um conjunto de passos organizados em cenários que visam alcançar um objetivo do usuário. Não deve conter termos técnicos da área de desenvolvimento ou descrever como o sistema será construído; deve conter apenas a linguagem do usuário. Tipicamente, um sistema terá vários casos de uso, cada um abordando objetivos individuais distintos a serem atingidos. Definido como parte da UML e componente central do RUP.

Checklist: ver Lista de verificação.

Cliente: refere-se ao cliente dos processos de Engenharia de Requisitos. Tipo especial de parte interessada com autoridade para tomar decisões sobre requisitos: solicitar, modificar, aprovar ou rejeitar.

COCOMO II: *CO*nstructive *CO*st *MO*del é um modelo de estimativa paramétrico que envolve o uso de equações matemáticas para estimar esforço, prazo e tamanho da equipe em projetos de software. É baseado em pesquisa e dados históricos. Usa como entrada a quantidade de linhas de

código (ou pontos de função) e a avaliação de outros aspectos chamados de *cost drivers*.

Conceito de negócio: ver Entidade.

Confiabilidade: capacidade de um sistema de realizar e manter seu funcionamento em circunstâncias de rotina, bem como em situações hostis e inesperadas.

Controle de questões: técnica da gestão de requisitos que define uma abordagem para rastrear, gerenciar e resolver questões e outras preocupações que precisam ser controladas para resolução durante o trabalho de requisitos. Conhecido em inglês por *issue log* ou *issue tracking*.

D

Defeito: um problema em um software que, se não corrigido, pode provocar sua falha, a geração de resultados incorretos ou comportamento não desejado. A ausência de funcionalidade que foi especificada e solicitada também é considerada um defeito. Também chamado de erro, falha e não conformidade.

Decomposição funcional: técnica usada para decompor uma solução (ou um problema) em funcionalidades ou partes menores. Sua meta é garantir que as partes sejam separadas da forma mais independente possível para que o todo possa ser mais facilmente compreendido (ou solucionado).

Desempenho: caracteriza o tanto de trabalho útil desempenhado por um sistema comparado ao tempo ou aos recursos usados. Dependendo do contexto, pode envolver menor tempo de resposta, alta vazão (maior taxa de processamento), baixo uso de recursos computacionais e alta disponibilidade do sistema.

Diagrama de atividades: representação dos fluxos operacionais, geralmente pela modelagem de etapas sequenciais em um processo, que encadeiam diferentes atividades em uma visão colaborativa, mais abrangente. Posiciona as atividades em diferentes raias (*lanes*), indicando os responsáveis por sua execução, e descreve o fluxo de controle de uma atividade para outra. Definido como parte da UML. Veja também Modelo de processo.

Diagrama de caso de uso: ilustra graficamente os casos de uso suportados por um sistema, os atores que interagem com estes e os relacionamentos entre os casos de uso e os atores.

Diagrama de contexto: representa todo o sistema como um único processo e é composto por fluxos de dados que mostram as interfaces entre o sistema e as entidades externas. O diagrama é uma forma de representar o sistema e sua relação com o ambiente. Permite identificar os limites dos processos, as áreas envolvidas com o processo e os relacionamentos com outros processos e elementos externos à empresa (por exemplo: clientes e fornecedores).

Diagrama de fluxo de dados: representa graficamente o fluxo de informações em um sistema, mostrando os dados que entram e saem, de onde os dados virão, para onde irão e onde serão armazenados. Pode ter vários níveis de detalhamento e ser composto pelos seguintes elementos: processos, fluxos de dados, depósitos de dados e entidades externas (criadores e consumidores de dados).

Documento de visão: contém a visão que as partes interessadas têm do sistema a ser desenvolvido, em termos das necessidades e características mais importantes. Por conter uma descrição dos requisitos centrais pretendidos, proporciona a base para requisitos mais detalhados. O documento de visão captura restrições de *design* e requisitos de alto nível para que o cliente possa compreender o sistema que será desenvolvido. Seu objetivo é fornecer uma visão ampla do produto que se pretende desenvolver, sem se aprofundar em detalhes. É um diagrama de fluxo de dados no seu mais alto nível (ou nível zero).

Domínio: área funcional em processo de análise. Ela pode corresponder às fronteiras de uma organização ou unidade organizacional, assim como partes interessadas chave fora daquelas fronteiras e suas interações com os recursos compreendidos nelas.

E

Elicitação de requisitos: atividade da Engenharia de Requisitos que seleciona e executa atividades para identificar e entender: os domínios envolvidos, requisitos de negócio, partes interessadas e seus requisitos, a

solução e requisitos de transição. Vai além da simples coleta de requisitos porque identifica de maneira proativa requisitos adicionais não explicitamente fornecidos. Gera como produtos memórias de levantamento.

Engenharia: ciência, a arte e a profissão de adquirir e de aplicar os conhecimentos matemáticos, técnicos e científicos na criação, no aperfeiçoamento e na implementação de utilidades, tais como materiais, estruturas, máquinas, aparelhos, sistemas ou processos, que realizem uma determinada função ou objetivo (WIKIPÉDIA, 2015).

Engenharia de software: (1) aplicação sistemática de conhecimento tecnológico e científico, métodos e experiência ao projeto, implementação, teste e documentação de software; (2) aplicação de uma abordagem sistemática e disciplinada quantificável ao desenvolvimento, à operação e à manutenção de software; ou seja, a aplicação de engenharia ao software (ISO/IEC/IEEE 24765).

Engenharia de requisitos: disciplina da engenharia de software que consiste no uso sistemático e repetitivo de técnicas para cobrir atividades de obtenção, documentação e manutenção de um conjunto de requisitos para software que atendam aos objetivos de negócio e sejam de qualidade.

Entidade: principal objeto de dados sobre o qual uma informação é coletada. Pode ser uma informação de pessoa, lugar, coisa ou evento. Pode ter uma ou várias ocorrências (instâncias). É um conceito de fundamental relevância para o usuário, sobre o qual uma coleção de fatos é armazenada. Uma associação entre entidades que contêm atributos é por si mesma uma entidade.

Entrevista: técnica de elicitação que consiste em uma forma de diálogo entre duas ou mais pessoas, onde uma delas (o entrevistador) busca respostas para um conjunto de questões previamente planejadas e a outra (entrevistado) se apresenta como fonte de informação.

Especificação de requisitos: documentação dos requisitos com a declaração oficial das necessidades de desenvolvimento para o software. É um contrato entre usuários e desenvolvedores. Especifica o que o software deve fazer; contudo, não diz como deve ser feito (decisões sob a responsabilidade de outra disciplina). A especificação de requisitos pode

conter outras informações, como restrições prévias ao espaço das soluções e premissas assumidas. Pode envolver modelos textuais e gráficos.

Estimativa: avaliação quantitativa ou resultado provável. Geralmente aplicada a custos, recursos, esforços e durações de projetos, é normalmente precedida de um modificador (ou seja, preliminar, conceitual, de viabilidade, de ordem de grandeza, definitiva). Deve sempre incluir uma indicação do seu nível de exatidão (por exemplo, +/- x%).

Estudo de viabilidade: conjunto de tarefas responsável por analisar situações do negócio que envolvem problemas a serem resolvidos ou oportunidades a serem aproveitadas. Em geral, é o ponto de partida para criar um projeto.

Etnografia: técnica para elicitar requisitos pela observação do ambiente de trabalho das partes interessadas. O observador imerge no ambiente de trabalho onde a solução será usada observando o trabalho cotidiano e tomando notas das tarefas em execução nas quais as partes interessadas estão envolvidas.

F

Fluxograma: diagrama que representa o passo a passo de um fluxo de trabalho, processo ou algoritmo. Os passos são representados por caixas e sua sequência é representada por setas que conectam essas caixas. Passos de decisão são representados por losangos.

Funcionalidade: veja Requisito funcional.

FURPS: padrão de classificação de requisitos definido no RUP. Possui uma extensão, o FURPS+. Acrônimo de: *Functionality* (funcionalidade), *Usability* (usabilidade), *Reliability* (confiabilidade), *Performance* (desempenho) e *Supportability* (suportabilidade).

G

Gestão de requisitos: também conhecida como gerência de requisitos. Parte da Engenharia de Requisitos, que agrega atividades para tratar mudanças, gerenciar conflitos, gerenciar questões, priorizar, obter aprovação das partes interessadas e comprometimento destas últimas e da equipe do projeto com a especificação de requisitos.

Glossário: lista alfabética de termos de um determinado domínio de conhecimento com a sua respectiva definição.

Gold plating: prática de adicionar a um sistema requisitos que não foram solicitados pelos usuários porque a equipe do projeto acha que o sistema ficará melhor assim. É considerada uma má prática de gestão de projetos e pode introduzir riscos no projeto.

H

História de usuário: breve descrição das funcionalidades que os usuários necessitam de uma solução para atender aos objetivos de negócio. Expressa normalmente em um parágrafo, com o seguinte formato: “como (papel), eu quero (algo) para (me beneficiar).”.

I

IEEE: organização sem fins lucrativos de profissionais interessados no avanço da tecnologia. Originalmente o seu nome vem do acrônimo de *Institute of Electrical and Electronics Engineers*, porém seu escopo de interesse e atuação se expandiu para muito além da área original.

Inspeção: técnica de qualidade que examina a especificação de requisitos visando: identificar não conformidade a padrões (verificação de requisitos) ou assegurar que esta soluciona o problema (validação de requisitos). Veja também Lista de verificação.

INVEST: acrônimo de uma lista de verificação para avaliar a qualidade de uma história de usuário. Se a história não cumprir um desses critérios, a equipe pode querer adequá-la ou mesmo considerar escrevê-la novamente. Significa: [I]ndependente, [N]egociável, [V]alorosa, [E]stimável, [S]ize (tamanho apropriado) e [T]estável.

K

Kanban: termo de origem japonesa que significa literalmente “cartão” ou “sinalização”. É um conceito relacionado com a utilização de cartões (*post-its* e outros) para indicar o andamento dos fluxos de produção em empresas de fabricação em série. Nesses cartões são colocadas indicações sobre uma

determinada tarefa – por exemplo, “para executar”, “em andamento” ou “finalizado”.

L

Linha de base: ver *Baseline*.

Lista de verificação: técnica de controle da qualidade. Ela pode incluir um conjunto de elementos que revisores usam para verificação e validação de requisitos. Eles podem ser especificamente desenvolvidos para capturar questões de preocupação para o projeto. Compensa as limitações de atenção e memória humana e ajuda a garantir a consistência e integridade na realização de uma tarefa.

M

Manutenção adaptativa: modificação de um produto de software, realizada após sua entrega, para mantê-lo funcional em um ambiente modificado ou em mudança. A manutenção adaptativa fornece as melhorias necessárias para acomodar mudanças no ambiente no qual o produto de software deve operar. Essas mudanças devem ser feitas para manter o ritmo com o ambiente em mudança. Por exemplo, o sistema operacional poderia ser atualizado e algumas mudanças podem ser feitas para acomodar o novo sistema operacional.

Manutenção corretiva: modificação reativa de um produto de software executada depois da entrega para corrigir problemas identificados. A modificação corrige os produtos de software para satisfazer os requisitos.

Manutenção perfectiva (ou preventiva): modificação de um produto de software depois da entrega para detectar e corrigir falhas latentes antes que elas se manifestem como falhas. Manutenção preventiva provê melhorias para usuários, melhorias de documentação de programas e registros para melhorar o desempenho, a facilidade de manutenção ou outros atributos do software. Compare com manutenção adaptativa; manutenção corretiva.

Matriz de rastreabilidade: ver Rastreabilidade.

Medição funcional: ver Análise de pontos de função.

Memória de levantamento: registro com os resultados de uma atividade de elicitação de requisitos. Podem ser notas de entrevistas, respostas de

questionários, atas de reunião, gravações de áudio e vídeo. Registros originalmente não documentais devem ser documentados para posterior confirmação.

Modelo de dados: representação gráfica dos dados persistentes do sistema e o relacionamento entre eles. Pode ter um ponto de vista conceitual, lógico ou físico.

Modelo de domínio: diagrama que representa conceitos de negócio relevantes para um domínio, relações entre esses conceitos e atributos usados para descrevê-los.

Modelo de processo: modelo de alto nível para processos de negócio. Destaca papéis, tarefas, sequência e relação entre eles.

MoSCoW: Técnica de priorização que consiste em atribuir quatro valores possíveis a um requisito. O nome da técnica deriva do acrônimo desses valores em inglês: *must* (obrigatório), *should* (importante), *could* (desejável) e *won't* (dispensável).

N

Necessidade de negócio: ver Requisito de negócio.

Nível de objetivo agregador: nível no qual um requisito funcional é descrito com a abrangência de processos de negócio de alto nível. Resume um conjunto de tarefas de um ou mais usuários.

Nível de objetivo de subfunção: nível no qual um requisito funcional é descrito com a abrangência de passos e/ou regras de uma ou mais tarefas desempenhadas pelo usuário.

Nível de objetivo do usuário: nível no qual um requisito funcional é descrito com a abrangência de uma única tarefa sob a responsabilidade de um único indivíduo em um momento que tem tudo o que precisa no tempo para que a tarefa seja feita.

O

Observação: ver Etnografia.

P

Parte interessada: grupo ou pessoa que tem interesses que podem ser afetados por uma iniciativa de mudança ou tem influência sobre ela.

Pesquisa (ou questionário): técnica de elicitação que consiste na formulação de um conjunto de perguntas em um questionário a ser enviado para uma ou mais partes interessadas responderem e devolverem preenchido para posterior avaliação dessas respostas.

PMBOK® Guide: *Project Management Body of Knowledge*. Padrão e guia para o gerenciamento de projetos elaborado pelo PMI – *Project Management Institute*.

Pontos de função: ver Análise de pontos de função.

Premissa: informação que se acredita ser verdade, mas que ainda não foi confirmada. Pode afetar todos os aspectos de um projeto e apresenta certo grau de risco caso não se confirme como verdadeira.

Priorização: atividade da gestão de requisitos de atribuir a cada requisito do projeto um valor de importância relativa (baseado em um ou mais critérios). A finalidade é assegurar que o esforço será focado nos requisitos mais críticos, reduzindo riscos do projeto ou potencializando valor entregue.

Prototipação: processo iterativo de gerar versões iniciais de um sistema futuro com o qual o usuário pode realizar verificações e experimentos, com o intuito de avaliar algumas de suas características antes que o sistema seja construído. Sua finalidade é reduzir os riscos do projeto, permitindo a descoberta de falhas nos requisitos em suas etapas iniciais.

Protótipo: ver Prototipação.

Q

Qualidade: grau em que um conjunto de características inerentes atende aos requisitos.

R

Rastreabilidade: atividade da gestão de requisitos que consiste em estabelecer relações entre requisitos, suas origens e produtos derivados. Isso

torna a especificação mais modificável, mais fácil de verificar se está correta e completa, além de facilitar a análise de impacto das mudanças.

Regra de negócio: são leis que regem o negócio. Incluem: políticas corporativas, legislação pertinente, regulamentações governamentais, padrões de mercado. Não estão ligadas à solução (software), mas ao domínio de negócio onde a solução será utilizada. Existem independentemente do software, do processo de negócio estar automatizado ou não.

Requisito: (1) condição ou capacidade necessária por um usuário para resolver um problema ou alcançar um objetivo; (2) condição ou capacidade que deve ser atingida ou possuída por um sistema ou componente de um sistema para satisfazer um contrato, padrão, especificação ou outro documento formalmente imposto; (3) representação documentada de uma condição ou capacidade como em (1) ou (2); (4) condição ou capacidade que deve ser alcançada ou possuída por um sistema, produto, serviço, resultado ou componente para satisfazer um contrato, padrão, especificação ou outro documento formalmente imposto. Requisitos incluem as necessidades quantificadas e documentadas, desejos e expectativas do patrocinador, clientes e outras partes interessadas (ISO/IEC/IEEE 24765).

Requisito da parte interessada: são declarações das necessidades de uma parte interessada em particular ou de uma classe de partes interessadas, que descrevem suas necessidades e como elas vão interagir com a solução. Servem como uma ponte entre as necessidades de negócio e as várias categorias de requisitos da solução.

Requisito de negócio: declarações de mais alto nível de objetivos, metas ou necessidades da organização. Eles descrevem as razões pelas quais um projeto foi iniciado, as metas que o projeto deve atingir e as métricas que serão utilizadas para medir o seu sucesso.

Requisito de solução: descrevem as características da solução (produto do projeto) que satisfaçam os requisitos de negócio e os requisitos das partes interessadas. Surgem do trabalho de análise de requisitos.

Requisito de transição: descrevem requisitos para facilitar a transição do estado atual da organização (*as is*) para um estado futuro desejado (*to be*) e que não serão mais necessários uma vez concluída a transição. São

diferenciados dos outros tipos de requisitos porque são sempre temporários por natureza e porque não podem ser desenvolvidos até que ambas, solução existente e solução nova, estejam definidas. Tipicamente, cobrem conversão de dados a partir de sistemas existentes, lacunas em habilidades que devem ser abordadas e outras mudanças relacionadas para alcançar o estado futuro desejado.

Requisito funcional: descreve o que o software faz em termos de tarefas e serviços. Inclui, mas não está limitado a: ➤

- transferência de dados (por exemplo, incluir dados do cliente); ➤
- transformação de dados (por exemplo, calcular juros bancários); ➤
- armazenamento de dados (por exemplo, armazenar dados de cliente); ➤
- recuperar dados (por exemplo, listar funcionários atuais).

Requisito inverso: declaração como uma proposição negativa indicando o que o sistema ou projeto não irá fazer ou entregar. Também conhecido por: “não escopo”, “escopo negativo”, “requisito negativo”, “limites do projeto”, “exclusões do escopo”.

Requisito não funcional: descreve condições que não se relacionam diretamente com o comportamento ou funcionalidade da solução, mas, em vez disso, descreve condições ambientais sob as quais a solução deve se manter efetiva ou qualidades que o sistema deve possuir. Eles são também conhecidos como requisitos de qualidade do software ou requisitos suplementares. Podem incluir aspectos relacionados (mas não limitados) a: ➤

- qualidade (por exemplo, usabilidade, confiabilidade, eficiência e portabilidade); ➤
- organização (por exemplo, locais de operação, hardware alvo e conformidade com normas); ➤
- ambiente (por exemplo, interoperabilidade, privacidade e segurança); ➤
- implementação (por exemplo, linguagem de desenvolvimento e sistema operacional).

Requerimento: veja Requisito.

Restrição: limitação às possíveis soluções em análise. Pode ser de origem de negócio (ex.: prazo, orçamento) ou técnica (ex.: plataforma, protocolo de comunicação). Em geral, nasce junto com o projeto e é imutável.

Restrição de negócio: restrições de caráter organizacional. Por exemplo: orçamento, prazo, número de recursos disponíveis, habilidades da equipe de projeto e das demais partes interessadas.

Restrição técnica: restrições de caráter tecnológico. Exemplos: (a) as decisões já tomadas sobre linguagens de desenvolvimento, plataformas de hardware/software e software de aplicação devem ser usadas; (b) limites quanto à utilização de recursos, tamanho de mensagens e sincronização, espaço exigido para armazenamento do software, quantidade e tamanho para arquivos, registros e elementos de dados; ou (c) qualquer padrão de arquitetura empresarial que deve ser seguido.

Revisão: ver Inspeção.

RUP: *Rational Unified Process*, processo de desenvolvimento de software iterativo e incremental criado pela *Rational*, posteriormente adquirida pela IBM. Usa a abordagem da orientação a objetos em sua concepção e é projetado e documentado utilizando a notação UML.

S

Scope creep: aumento descontrolado do escopo de um projeto sem ajustes no planejamento de tempo, custo e recursos.

Scrum: processo de desenvolvimento iterativo e incremental para projetos de software, caracterizado por ciclos de entregas curtos (2-4 semanas) e equipes pequenas (3-9 pessoas).

SMART: método para avaliar a validade de objetivos, que devem ser SMART: S indica eSpecífico – descreve algo que apresenta um resultado observável; M indica Mensurável – são resultados passíveis de acompanhamento e medição; A indica Alcançável – as necessidades de negócio consideram a viabilidade do investimento; R indica Relevante – estão alinhadas com a visão, missão e objetivos-chave da organização; T indica Tempestivo – os objetivos definidos têm uma janela de tempo definida que é consistente com as oportunidades ou problemas associados

SRS: *Software Requirements Specifications*, ver Especificação de requisitos.

Solução: conjunto de mudanças (muitas vezes implementadas por software, o foco deste livro) à situação atual de uma organização que são feitas de forma a habilitá-las a atender a uma necessidade de negócio, resolver problemas ou aproveitar oportunidades.

Stakeholder: ver Parte interessada.

Suportabilidade: abrange aspectos de extensibilidade, adaptabilidade, manutenibilidade, compatibilidade, configurabilidade, escalabilidade, instalabilidade, localizabilidade e testabilidade do software.

U

UAT: *User Acceptance Test*, o teste de um sistema ou unidade funcional geralmente realizado pelo comprador após a instalação, com a participação do vendedor para garantir que os requisitos contratuais foram atendidos (ISO/IEC 2382-20:1990, *Information technology – Vocabulary – Part 20: System development*. 20.05.07).

UML: *Unified Modeling Language*. É uma linguagem de modelagem aberta que permite que desenvolvedores visualizem os produtos de seu trabalho em diagramas padronizados. Junto com uma notação gráfica, a UML também especifica significados. É uma notação independente de processos. Não é uma metodologia de desenvolvimento, o que significa que ela não diz o que fazer primeiro e em seguida ou como projetar um sistema. Ela auxilia a visualizar seu desenho e a comunicação entre objetos.

Usabilidade: define a facilidade com que as pessoas podem empregar uma ferramenta ou objeto a fim de realizar uma tarefa específica. Na computação, normalmente se refere à simplicidade e facilidade com que uma interface, um programa de computador ou um website pode ser utilizado.

Usuário: qualquer pessoa ou coisa que interage com o software a qualquer momento.

V

Validação de requisitos: trabalho de qualidade na Engenharia de Requisitos que busca assegurar que todos os requisitos estão alinhados com os requisitos de negócio. Ou seja, que todas as necessidades de negócio do cliente no escopo do projeto serão satisfeitas. A finalidade é garantir que a especificação define o produto certo a ser desenvolvido pelo projeto.

Verificação de requisitos: trabalho de qualidade na Engenharia de Requisitos que busca garantir que as especificações de requisitos e seus modelos atendam aos padrões estabelecidos, para que então possam ser utilizados de forma eficaz nas atividades seguintes do projeto.

Votação: técnica de priorização de requisitos.

Anexo. Estudo Preliminar SRRO – Sistema de Registro de Responsabilidade de Obras[\[1\]](#)

A.1. Objetivo

Este estudo tem por objeto fundamentar a contratação de empresa especializada em desenvolvimento de sistemas para desenvolvimento do projeto do Sistema para Registro de Responsabilidade de Obras de Médio e Grande Porte. O Sistema deverá ser desenvolvido com a tecnologia ASP.NET e a Linguagem de Programação C#. A empresa será responsável por todas as fases do desenvolvimento: levantamento de requisitos, análise do sistema, desenvolvimento, implantação e acompanhamento da fase de produção no período de garantia, conforme especificações do documento de visão.

A.2. Justificativa

Obter agilidade na coleta de informações de obras de médio e grande porte e com isso tornar a fiscalização mais analítica e proativa, aumentando a eficiência do sistema.

A.3. Prazo de entrega

O prazo de entrega será em conformidade com item “3 – Cronograma das Atividades de Projeto do Documento de Visão”, apresentado a seguir.

A.4. Valor contratado

Valor contratado: R\$ 95.500,00 (noventa e cinco mil e quinhentos reais).

A.5. Responsáveis pelo projeto

Unidade de Compras do Departamento de Suprimentos com as especificações elaboradas pela Unidade de Desenvolvimento do Departamento de Informática.

A.6. Objeto

A contratação de empresa especializada em desenvolvimento de sistemas para desenvolvimento do Projeto de Sistema para Registro de Responsabilidade de Obras de Médio e Grande Porte. O sistema deverá ser desenvolvido com a tecnologia ASP.NET e a Linguagem de Programação C#.

A empresa será responsável por todas as fases do desenvolvimento: levantamento de requisitos, análise do sistema, desenvolvimento, implantação e acompanhamento da fase de produção no período de garantia, conforme especificações constantes no documento de visão.

A.7. Quantidade estimada de pontos de função

Estimamos o sistema em 78 pontos de função, porém consideramos que na fase de análise possam ser identificadas novas funcionalidade dentro do escopo de projeto, e para estas funcionalidades reservamos 22 pontos de função, totalizando 100 pontos de função para este projeto.

A.8. Recebimento

A CONTRATADA deverá entregar toda a Documentação de Análise elaborada durante a fase de levantamento de requisitos, todos dos artefatos gerados durante a fase de análise e todo o código-fonte devidamente comentado.

A CONTRATADA deverá fornecer seis meses de garantia e suporte ao sistema que será implantado.

A.9. Documento de Visão

1) Contextualização

Visão geral do sistema

O Sistema de Registro Responsabilidade de Obras de Edificações de Médio e Grande Porte tem por seu objetivo fundamental mudar o foco da ação operacional da fiscalização, de forma que esta atividade seja mais analítica e menos operacional.

Observando modelos de sucesso no setor público, a exemplo da Receita Federal, este projeto visa estabelecer critérios onde o profissional fiscalizado presta contas ao governo do estado e este analisa se essas informações estão em conformidade com as relativas atribuições de cada área de atividade técnica observada por este Órgão Público.

Cenário atual

Oportunidade	Problema Enfrentado
Agilidade na coleta de informações de obras de médio e grande porte.	Hoje o fiscal depara com situações que, muitas vezes, tornam impraticável a coleta eficiente de dados, devido a uma infinidade de empresas geridas em um contrato de grande porte.
Fiscalização mais analítica e menos operativa.	Com este sistema a fiscalização entra em um novo patamar de atividade. Hoje a principal dificuldade é conseguir operacionalizar uma ação eficiente com a grande quantidade de formulários manuscritos, para preencher, com grandes volumes de dados.
Aumentar a	É visível que o grande volume de trabalho para as fiscalizações exige um

eficiência do sistema.	do sistema de fiscalização mais eficiente, para otimizar o contingente de fiscais, mantendo o mesmo número de recursos com aproveitamento ótimo.
------------------------	--

Perspectivas do produto

O escopo do Registro de Responsabilidade de Obras de Médio e Grande Porte deve compreender os seguintes requisitos gerais:

1. Obras com Registro Pendente.
2. Formulário *On-Line* de Registro de Responsabilidade.
3. Relatórios Para Fiscalização.
4. Relatórios para a Gestão da Fiscalização.

Interfaces

Interface com o sistema de SERVIÇOS *ON-LINE* (para consultar dados de ART).

Plataforma de desenvolvimento

O Sistema dever ser concebido na plataforma web, sendo desenvolvido com ASP.NET e linguagem C# onde os dados serão geridos e armazenados no IBM DB2 9.5.

2) Requisitos

A seguir estão descritos os requisitos funcionais e os demais requisitos que devem ser satisfeitos para que o sistema atenda aos seus objetivos propostos.

Requisitos funcionais

Obras com Registro Pendente Listagem das obras que não foram declaradas.

Lista de obras pendentes

Funcionalidades Listar Obras com o Registro de Responsabilidade Pendente: ➤

Permite ao profissional fazer uma pesquisa para visualizar quais obras são passíveis de emissão de registro de responsabilidade.



O sistema deverá consultar as Anotações de Responsabilidade Técnicas com a metragem (m²) que caracteriza uma obra de médio ou grande porte e listar todas que não tiverem sido declaradas a partir da data de início do novo processo de fiscalização.



Os parâmetros de metragem e data deverão ser definidos pelo gestor responsável da SUPOPE.



O profissional deverá ter a opção de ser direcionado através do item escolhido para o formulário de registro.

Formulário de registro

Funcionalidades O formulário deverá carregar os dados da ART da empresa gestora do empreendimento.

O profissional declarará as seguintes informações: ➤

Local da obra.



Dados constantes do projeto aprovado.



Número do processo.



Número do alvará.



Placa afixada (s/n).



Placa das firmas e/ou profissionais subcontratados, com endereços e seus dados.

Usar como referência o formulário de papel utilizado hoje pela fiscalização que é Relatório de Fiscalização de Obras de Edificações de Médio e Grande Porte.

Relatórios para fiscalização

Funcionalidades ➤

O sistema deverá identificar o fiscal que fará a consulta ao relatório.



O sistema deverá listar as obras com possíveis irregularidades com base na lotação do fiscal, cruzando com as informações de CEP.



O fiscal poderá alterar a localização de obras com irregularidades em potencial através de um filtro por cidades.



Gerar relatórios analíticos apontando possíveis atividades de fiscalização para: ✓

Obras que deveriam ser declaradas e passaram do prazo limite de declaração, tendo por base a data da criação da ART principal do empreendimento.



Obras com empresas que não emitiram a ART.



Obras com empresas sem registro.



Empresas com registro divergente.



Empresas com número de profissional com CPF e número de registro divergente com o sistema.



Obras sem responsável técnico.



Obras sem placas.

Relatório para gestão da fiscalização

Funcionalidades ➤

Estatística de ações realizadas a partir do próprio sistema.



Geração de Mapas das Obras no Google Maps, a partir do CEP.

3) Cronograma das atividades de projeto

EDT	TAREFA	PRAZO
1	Projeto registro de responsabilidade	120 dias
1.1	Plano de gerenciamento do projeto	5 dias
1.2	Levantamento de requisitos	28 dias

1.2.1	Entendimento do documento de visão	10 dias
1.2.2	Refinamento dos requisitos levantados	10 dias
1.2.3	Avaliação de novos requisitos	5 dias
1.2.4	Fechamento do planejamento dos <i>releases</i>	3 dias
1.2.5	<i>Design</i> do sistema	10 dias
1.3	Análise	20 dias
1.3.1	<i>Design</i> do sistema	20 dias
1.4	Construção	40 dias
1.5	Testes	15 dias
1.6	Implantação	2 dias
1.7	Divulgação e treinamento	10 dias

4) Da divulgação e treinamento

O trabalho de elaboração do manual do usuário e treinamento será de responsabilidade da empresa contratada para realizar o projeto.

Para divulgação mais lúdica e intuitiva, deve ser elaborado um vídeo tutorial e disponibilizá-lo em sites especializados em *streaming*, onde estaremos colocando o link de acesso em nossos meios de interação *on-line* com o público.

O vídeo deve abordar todas as funcionalidades do Registro de Responsabilidade de forma prática, clara e explicativa.

O treinamento deverá ser dado para a equipe do *call center*, para que possamos esclarecer dúvidas do público através deste canal.

5) Da análise de custos

Com base nos dados levantados no presente estudo, obtivemos o número de 100 pontos de função (PF). Se considerarmos uma produtividade de 4 PF/h, este projeto ficaria pronto em quatro meses, como prevê o cronograma do item 3.

[1] Adaptação de (CREA-SP, 2010).